

**ВЫСШЕЕ
ОБРАЗОВАНИЕ**

В. В. Подбельский

Программирование

Базовый курс C#

Учебник

УМО ВО
РЕКОМЕНДУЕТ

 **Юрайт**
ИЗДАТЕЛЬСТВО
biblio-online.ru

В. В. Подбельский

ПРОГРАММИРОВАНИЕ БАЗОВЫЙ КУРС C#

УЧЕБНИК ДЛЯ ВУЗОВ

Рекомендовано Учебно-методическим отделом высшего образования в качестве учебника для студентов высших учебных заведений, обучающихся по инженерно-техническим направлениям

**Книга доступна на образовательной платформе «Юрайт» urait.ru,
а также в мобильном приложении «Юрайт.Библиотека»**

Москва • Юрайт • 2022

УДК 004.43(075.8)

ББК 32.973.2я73

П44

Автор:

Подбельский Вадим Валериевич — профессор, доктор технических наук, профессор департамента программной инженерии факультета компьютерных наук Национального исследовательского университета «Высшая школа экономики».

Подбельский, В. В.

П44 Программирование. Базовый курс C# : учебник для вузов / В. В. Подбельский. — Москва : Издательство Юрайт, 2022. — 369 с. — (Высшее образование). — Текст : непосредственный.

ISBN 978-5-534-10616-9

На основе последних версий языка программирования C# и платформы .NET Framework изложены основные концепции и механизмы современного программирования. Методика изложения и тщательно отобранные примеры позволяют освоить не только синтаксис и семантику языка C#, но и изучить фундаментальные принципы процедурного, объектного, объектно-ориентированного и обобщенного программирования. Контрольные вопросы позволяют читателю использовать книгу для самообразования. Предлагаемая к опубликованию рукопись учебника готовится на основе программы учебной дисциплины «Программирование» для направления подготовки бакалавров «Программная инженерия».

Соответствует актуальным требованиям федерального государственного образовательного стандарта высшего образования.

Книга предназначена для студентов, обучающихся по специальностям, связанных с IT-технологиями, а также для программистов, желающих освоить программирование на C# и освоить перспективные средства платформы .NET Framework.

УДК 004.43(075.8)

ББК 32.973.2я73

Все права защищены. Никакая часть данной книги не может быть воспроизведена в какой бы то ни было форме без письменного разрешения владельцев авторских прав.

ISBN 978-5-534-10616-9

© Подбельский В. В., 2019

© ООО «Издательство Юрайт», 2022

Оглавление

Предисловие	7
Глава 1. Объектная ориентация языка C#	12
1.1. Типы, классы, объекты.....	12
1.2. Программа на C#	14
1.3. Пространство имен	17
1.4. Создание консольного приложения	19
<i>Контрольные вопросы и задания.....</i>	<i>22</i>
Глава 2. Типы в языке C#	24
2.1. Типы ссылок и типы значений.....	24
2.2. Классификация типов C#	26
2.3. Простые (базовые) типы. Константы-литералы	27
2.4. Объявления переменных и констант базовых типов.....	30
<i>Контрольные вопросы и задания.....</i>	<i>33</i>
Глава 3. Операции и целочисленные выражения	34
3.1. Операции языка C#	34
3.2. Операции присваивания и оператор присваивания	37
3.3. Операции инкремента (++) и декремента (--)	38
3.4. Выражения с арифметическими операциями.....	39
3.5. Поразрядные (побитовые) операции	40
3.6. Переполнения при операциях с целыми	43
<i>Контрольные вопросы и задания.....</i>	<i>47</i>
Глава 4. Выражения с операндами базовых типов.....	48
4.1. Автоматическое и явное приведение арифметических типов	48
4.2. Особые ситуации в арифметических выражениях.....	50
4.3. Логический тип и логические выражения	52
4.4. Выражения с символьными операндами	55
4.5. Тернарная (условная) операция	57
<i>Контрольные вопросы и задания.....</i>	<i>59</i>
Глава 5. Типы C# и типы платформы .NET Framework	60
5.1. Платформа .NET Framework и спецификация CTS	60
5.2. Простые (базовые) типы C# как типы CTS	62
5.3. Специфические методы и поля простых типов	65
<i>Контрольные вопросы и задания.....</i>	<i>70</i>
Глава 6. Операторы	71
6.1. Общие сведения об операторах	71

6.2. Метки и оператор безусловного перехода.....	72
6.3. Условный оператор	73
6.4. Операторы цикла	74
6.5. Операторы передачи управления	80
6.6. Переключатель	83
<i>Контрольные вопросы и задания.....</i>	<i>88</i>
Глава 7. Массивы C#	90
7.1. Одномерные массивы.....	90
7.2. Массивы как наследники класса Array	96
7.3. Виды массивов и массивы многомерные	101
7.4. Массивы массивов и «непрямоугольные» массивы.....	104
7.5. Массивы массивов и поверхностное копирование	107
<i>Контрольные вопросы и задания.....</i>	<i>110</i>
Глава 8. Строки — объекты класса string.....	112
8.1. Строковые литералы	112
8.2. Строковые объекты и ссылки типа string	113
8.3. Операции над строками.....	114
8.4. Некоторые методы и свойства класса string	117
8.5. Форматирование	119
8.6. Форматирование в ToString() и интерполяция строк	122
8.7. Применение строк в переключателях	124
8.8. Массивы строк.....	124
8.9. Сравнение строк.....	126
8.10. Преобразования с участием строкового типа	128
8.11. Аргументы метода Main()	130
8.12. неизменяемость объектов класса string.....	132
<i>Контрольные вопросы и задания.....</i>	<i>133</i>
Глава 9. Методы C#	135
9.1. Методы-процедуры и методы-функции	135
9.2. Методы локальные и сжатые до выражений.....	139
9.3. Соотношение параметров и аргументов	141
9.4. Параметры с типами ссылок.....	148
9.5. Методы с переменным числом аргументов.....	152
9.6. Перегрузка методов.....	155
9.7. Рекурсивные методы	156
9.8. Применение метода Array.Sort().....	160
9.9. Кортежи и ссылки в методах.....	162
<i>Контрольные вопросы и задания.....</i>	<i>166</i>
Глава 10. Класс как совокупность статических членов	168
10.1. Статические члены класса.....	168
10.2. Поля классов (статические поля).....	170
10.3. Статические константы.....	173
10.4. Статические методы.....	175
10.5. Статический конструктор и статический класс	177
<i>Контрольные вопросы и задания.....</i>	<i>180</i>

Глава 11. Классы как типы	182
11.1. Объявление класса	182
11.2. Поля объектов	183
11.3. Объявления методов объектов	186
11.4. Пример класса и его объектов	188
11.5. Ссылка this	191
11.6. Конструкторы объектов класса.....	195
11.7. Деструкторы и инициализаторы объектов.....	201
<i>Контрольные вопросы и задания.....</i>	<i>204</i>
Глава 12. Средства взаимодействия с объектами	205
12.1. Принцип инкапсуляции и свойства классов	205
12.2. Автореализуемые свойства и свойства, сжатые до выражений	211
12.3. Индексаторы.....	214
12.4. Расширяющие методы и деконструкторы.....	219
<i>Контрольные вопросы и задания.....</i>	<i>225</i>
Глава 13. Включение, вложение и наследование классов	226
13.1. Включение объектов классов.....	226
13.2. Вложение классов.....	230
13.3. Наследование классов	232
13.4. Доступность членов класса при наследовании	235
13.5. Методы при наследовании.....	239
13.6. Абстрактные методы и абстрактные классы	245
13.7. Опечатанные классы и члены классов	247
13.8. Применение абстрактных классов	247
<i>Контрольные вопросы и задания.....</i>	<i>249</i>
Глава 14. Интерфейсы	251
14.1. Два вида наследования в ООП	251
14.2. Объявления интерфейсов	252
14.3. Реализация интерфейсов	254
14.4. Интерфейс как тип	259
14.5. Интерфейсы и наследование	264
<i>Контрольные вопросы и задания.....</i>	<i>270</i>
Глава 15. Перечисления и структуры	271
15.1. Перечисления	271
15.2. Базовый класс перечислений.....	276
15.3. Структуры	279
15.4. Упаковка и распаковка.....	284
15.5. Реализация структурами интерфейсов	288
<i>Контрольные вопросы и задания.....</i>	<i>292</i>
Глава 16. Исключения	294
16.1. О механизме исключений	294
16.2. Системные исключения и их обработка.....	296
16.3. Свойства исключений	299
16.4. Исключения в арифметических выражениях	301

16.5. Генерация исключений	303
16.6. Пользовательские классы исключений	307
<i>Контрольные вопросы и задания.....</i>	<i>308</i>
Глава 17. Делегаты и события.....	310
17.1. Синтаксис делегатов.....	310
17.2. Массивы делегатов	314
17.3. Многоадресные экземпляры делегатов	316
17.4. Делегаты и обратные вызовы.....	318
17.5. Анонимные методы и лямбда-выражения.....	321
17.6. События	329
<i>Контрольные вопросы и задания.....</i>	<i>336</i>
Глава 18. Обобщения.....	337
18.1. Обобщения как средство абстракции.....	337
18.2. Декларации обобщенных классов	339
18.3. Ограничения типизирующих параметров	341
18.4. Обобщенные структуры	346
18.5. Обобщенные интерфейсы.....	349
18.6. Обобщенные методы.....	351
18.7. Обобщенные делегаты	354
<i>Контрольные вопросы и задания.....</i>	<i>358</i>
Предметный указатель	359
Литература и электронные ресурсы.....	366
Новинки издательства «Юрайт» по дисциплине	
«Программирование» и смежным дисциплинам.....	368

Предисловие

Книга предназначена для студентов, изучающих программирование, а также для читателей, желающих самостоятельно освоить язык программирования С#. В отличие от достаточно многочисленных руководств по С# данная книга посвящена именно тем основам языка, без знания которых невозможно обойтись при его практическом применении и изучении справочных материалов.

В фундаментальных современных справочниках и руководствах по языку С# обязательно приводят сведения об усовершенствованиях и изменениях, последовательно появлявшихся в его версиях.

В версии 1.0 (2002 г.) С# был очень похож на Java, т. е. это был «простой объектно-ориентированный язык общего назначения», но с событиями, свойствами, делегатами (и, конечно, с классами и структурами).

В версии 2.0 С# (2005 г.) появились обобщения, разделяемые типы, анонимные методы, итераторы.

В версию 3.0 С# (2007 г.) были включены автореализуемые свойства, анонимные типы, лямбда-выражения, методы расширения, неявно типизированные переменные, разделяемые методы, технология LINQ, инициализаторы объектов и коллекций. Включение в версию 3.0 перечисленных средств постепенно превращало С# в гибридный язык, пригодный не только для процедурного и объектно-ориентированного программирования, но и позволяющий применять методологию функционального программирования.

В версию 4.0 С# (2010 г.) была включена динамическая привязка, именованные аргументы и классы для поддержки асинхронного программирования.

В версии 5.0 С# (2012 г.) в язык С# были внедрены удобные средства асинхронного программирования.

В версии 6.0 С# (2015 г.) усилия разработчиков языка были направлены на повышение выразительных возможностей языка. Были добавлены фильтры исключений, инициализаторы свойств, интерполяция строк, сжатые до выражения свойства.

В версии 7.0 С# (2017 г.) появились переменные, объявляемые как аргументы с модификатором **out**, кортежи и деконструкторы, локальные методы, методы, сжатые до выражений, сжатые до выражений аксессоры свойств и индексаторов, пустые аргументы для параметров с модификатором **out**, возвращаемые значения в виде ссылок с модификатором **ref**.

Сведения об эволюции языка C# и росте его выразительных возможностей мы привели для того, чтобы обратить внимание читателя на тот факт, что в настоящее время в языке C# начали появляться «атавизмы» — устаревшие средства, которые можно использовать, которые поддерживаются компиляторами, но для которых в современной версии языка есть эффективные замены. Например, из разбираемого в данной книге таким атавизмом являются анонимные методы, заменой которых служат лямбда-выражения. Вторым примером может служить класс `Tuple`, вместо которого сегодня применяют структуру `ValueTuple`, и т. д.

Вторая цель перечисления версий языка C# и развития его средств состояла сейчас в том, чтобы обратить внимание читателя на трудность изучения программирования на C# по справочникам и многочисленным примерам, доступным сегодня в электронном виде. Связано это с существованием разных уровней усвоения C#. Такой язык программирования, как C#, нельзя изучать «линейно», условно говоря, «от аксиом к теоремам, задачам и выводам», поэтому изложение материала (языка C# и программирования на C#) будет проходить «по спирали». К некоторым понятиям, использованным в той или иной иллюстративной программе с краткими пояснениями, в следующих главах обращаются вновь, постепенно полностью объясняя их.

Но насколько глубоко и подробно следует изучать синтаксис и конструкции языка, чтобы начать программировать? Вопрос почти философский. Его задавали и пытались на него ответить программисты и авторы алгоритмических языков уже много раз. Попытки минимизировать количество выразительных средств языка процедурного программирования привели к выводу, что для разработки программ (без учета обращений к внешним устройствам хранения и предоставления информации) достаточно иметь оператор присваивания, условный оператор и оператор безусловного перехода. Но названный минимальный набор операторов будет достаточен только в том случае, если для представления обрабатываемых данных будут использоваться только константы и переменные тех типов, для которых в языке фиксирован конкретный набор операций. В языке C# существуют не только константы и переменные. В нем имеется много конструкций для представления данных. Раз так, то необходимо изучать арсенал средств автоматизации кодирования, учитывающих названное многообразие структур для представления данных. Именно это даст возможность профессионально программировать на C#.

Какие трудности при этом существуют? Если обратиться к эволюции языков программирования, то увидим, что вначале языки назывались алгоритмическими и создавались в основном для автоматизации математических вычислений (классический пример Fortran и Algol 60). Такие языки включали (кроме названного минимального набора операторов) средства представления массивов, операторы для организации циклов и возможности создания подпрограмм и функций.

Не пытаясь проследить дальнейшее развитие языков программирования, остановимся на С#. Его коренное отличие от многих предшествующих языков – явное деление типов на два вида: типы ссылок и типы значений. К математике и к вычислительным методам это деление имеет весьма отдаленное отношение. Его назначение — повысить эффективность выполнения программ за счет учета аппаратных и архитектурных особенностей той среды, в которой выполняется программа после ее компиляции. Указав на эту особенность языка С#, отметим, что она создает трудности при его изучении. Если в классическом процедурном языке для интуитивного понимания того, что такое переменная, было достаточно представления о переменной из школьного курса алгебры, то в языке С# этого недостаточно. Явное отделение типов значений от ссылочных типов приводит к необходимости разбирать вопрос о том, что такое ссылка и чем отличается нулевая ссылка от переменной с типом значений, имеющей значение нуль. Эти вопросы будут подробно рассмотрены в книге, а сейчас кратко остановимся на содержании глав.

В книге 18 глав. Глава 1 дает общее представление о структуре простейшей программы на языке С#. Главы 2—6 знакомят читателя с такими базовыми понятиями процедурного программирования, как константы, переменные, выражения, операторы. Однако процедурный подход к созданию программ на языке С# с неизбежностью приводит к применению тех или иных классов и объектов. Даже традиционные для языков программирования переменные базовых типов в языке С# являются «проекциями» на классы и структуры из .NET Framework. Платформа .NET Framework и особенности базовых типов языка С#, реализованных средствами ее библиотечных классов, описаны в главе 5.

Главы 7 и 8 посвящены массивам и строкам. Для массивов и строк языка С# приходится различать объекты и ссылки на них. Тем самым читатель с необходимостью приходит к пониманию назначения конструкторов и особенностям применения операции **new**.

В главе 9 рассмотрены синтаксис и семантика методов языка С#, все виды параметров и особенности применения ссылок в качестве параметров. Подробно описаны: перегрузка методов, рекурсивные методы, локальные методы и методы с переменным числом аргументов.

Глава 10, описывающая классы как контейнеры их статических членов, завершает изложение традиционного для процедурных языков подхода к программированию. Набор определенных пользователем классов с их статическими данными и методами позволяет решать в процедурном стиле практически любые задачи программирования. Используя средства глав 2—10, читатель может перевести на С# программу с языков С, Паскаль или Фортран. Как таковое объектно-ориентированное программирование начинается с определения пользовательских классов (глава 11). Глава 12 продолжает эту тему и посвящена концепции инкапсуляции, т. е. изучению средств создания классов, объекты которых скрывают свою внутреннюю структуру.

Глава 13 посвящена отношениям между классами (и их объектами). Особое внимание уделено наследованию, абстрактным классам и виртуальным членам.

Язык C# дает возможность программисту вводить свои (пользовательские) типы не только с помощью классов, но и с помощью структур, перечислений, интерфейсов и делегатов. Эти средства рассматриваются в главах 14, 15 и 17. В главе 14 рассматриваются интерфейсы, обеспечивающие множественное наследование и создание разных классов, объекты которых обладают единой функциональностью. Структуры и перечисления в языке C# — это типы значений. Именно поэтому в главе 15 рассмотрены операции упаковки и распаковки.

В главе 16 подробно рассмотрен механизм исключений — их генерация и обработка. Описанных возможностей библиотечных классов исключений обычно достаточно для решения типовых задач защиты программ от ошибок.

В .Net Framework делегаты тесно связаны с обработкой событий. Оба этих механизма изучаются в главе 17. Особое внимание уделено реализации с помощью делегатов схемы обратных вызовов.

Глава 18 посвящена обобщениям, механизм которых позволяют существенно повысить уровень абстракции в программировании. Наличие библиотеки обобщенных типов — одна из основных предпосылок снижения трудоемкости разработки программного кода.

Еще несколько слов о языке C#. Это в настоящее время кроссплатформенный язык, который вначале создавался с ориентацией на эффективную работу с платформой .NET Framework. В свою очередь, платформа .NET Framework — это набор библиотек, т. е. модулей с расширением .dll (каждая dll-библиотека — это исполнимый модуль, эквивалентный exe-файлу, но без точки входа, т. е. без метода Main()). Инфраструктура .NET Framework в настоящее время доступна для работы в разных операционных системах, что делает C# весьма универсальным средством для разработки портативных (переносимых) приложений.

В основу книги положена работа автора [11], которая вышла первым изданием в 2011 году и отражала возможности языка C# 3.0. За это время многое изменилось, и в данной книге рассматривается версия C# 7.0. Однако полностью изучать все средства C# 7.0 в начале знакомства с этим языком нет необходимости. В отличие от фундаментальных справочников и монографий по C# (объем которых обычно превышает 1000 страниц) наша книга обеспечит читателя тем оптимальным количеством сведений, которых достаточно, чтобы

знать

- основные парадигмы и методологии создания программных продуктов (процедурный, объектный и объектно-ориентированный подходы, механизм обобщений методов и типов);
- современный язык программирования (синтаксис и семантику языка C#);
- особенности применения средств платформы .NET;

- возможности интегрированных сред разработки;

уметь

- разрабатывать прикладные программы и библиотеки классов с помощью инструментальных интегрированных сред;
- отлаживать и тестировать создаваемые программные продукты, используя диагностические возможности среды разработки;
- применять библиотеку классов платформы .NET и свободно (открыто) распространяемые библиотеки;
- самостоятельно находить новые знания и решения, необходимые для реализации функциональных требований, сформулированных в техническом задании на программный продукт;

владеть навыком

- решения типовых задач программирования с применением языка программирования C# и передовых инструментальных средств;
- проектирования и программирования с использованием процедурного, объектного, объектно-ориентированного и обобщенного подходов;
- применения средств платформы .NET и свободно (открыто) распространяемых библиотек.

Книга доступна начинающему программисту. Для усвоения материала от читателя не требуется предварительных знаний, выходящих за пределы школьной программы по дисциплине «Информатика».

Глава 1

ОБЪЕКТНАЯ ОРИЕНТАЦИЯ ЯЗЫКА C#

1.1. Типы, классы, объекты

Язык C# — объектно-ориентированный язык со строгой типизацией.

Но что такое тип? Тип в программировании — понятие первичное. Тип некоторой сущности декларирует для нее совокупность возможных состояний и набор допустимых действий. Понятие сущности мы пока не уточняем, сущностями могут быть константы, переменные, массивы, структуры и т. д.

Наиболее часто понятие тип в языках программирования используют в связи с понятием «переменная».

Примитивное (но пока достаточное для наших целей) определение: переменная это пара «обозначение переменной + значение переменной».

Для переменной тип вводит совокупность ее возможных значений и набор допустимых операций над этими значениями.

Пример определения переменной в C, C++, C# и некоторых других языках:

```
int spot = 16;
```

`spot` — обозначение (имя) переменной, `16` — ее значение в данный момент (после этого определения), `int` — название типа переменной. Этот тип `int` определяет для переменной `spot` совокупность допустимых значений и набор операций с правилами их выполнения. Например, для `spot` определена операция получения остатка от деления (%) и особым образом определено деление на целую величину (/ в этом случае обозначает операцию целочисленного деления). Результат `spot/5` равен 3, а значением выражения `spot % 3` будет 1.

Типы в языке C# введены с помощью классов (а также структур, перечислений, интерфейсов, делегатов и массивов). Но обо всем по порядку.

Понятие класса в теоретических работах, посвященных объектно-ориентированной методологии, обычно вводится на основе понятия «объект». Объект определяют по-разному.

Приведем одну из форм определений (см., например, [3]).

«Объектом называется совокупность данных (полей), определяющих состояние объекта, и набор функций (методов), обеспечивающих изменение указанных данных и доступ к ним».

После подобного определения в пособиях по объектно-ориентированному программированию перечисляются обязательные признаки объектов:

- 1) различимость;
- 2) возможность одного объекта находиться в разных состояниях (в разное время);
- 3) возможность динамического создания объектов;
- 4) «умение» объектов взаимодействовать друг с другом с помощью обменов сообщениями;
- 5) наличие методов, позволяющих объекту реагировать на сообщения (на внешние для объекта воздействия);
- 6) инкапсуляция данных внутри объектов.

Введя понятие объекта, класс определяют как механизм, задающий структуру (поля данных) всех однотипных объектов и функциональность объектов, т. е. механизм, определяющий все методы, относящиеся к объектам.

В процессе развития объектно-ориентированного подхода и при его реализации в языках программирования стало ясно, что среди данных объекта могут существовать такие, которые принадлежат не единичному объекту, а всем объектам класса. То же было выявлено и для методов — некоторые из них могли определять не функциональность отдельного (каждого) объекта, а быть общими для класса (для всех его объектов). В совокупности поля и методы как класса, так и формируемых с его помощью объектов называются членами класса.

Для иллюстрации этих понятий и особенно отличий полей и методов класса от полей и методов его объектов рассмотрим пример. Класс с названием «студент группы N-го курса»:

- **поля (данные) объекта:** ФИО, оценки за сессию, взятые в библиотеке книги и т. д.;
- **методы объекта:** сдать экзамен, получить книги в библиотеке и т. д.;
- **поля (данные) класса:** номер курса (N), даты экзаменов, количество дисциплин в семестре и т. д.;
- **метод класса:** перевести группу на следующий курс — изменятся все данные класса, но не все объекты останутся в этом измененном классе (не обязательно все студенты будут переведены на следующий курс).

Различие между данными и методами объектов и данными и методами класса существенно используется в языке C#. Чтобы их различать в определении класса, его данные и его методы снабжаются специальным модификатором **static** (статический). Примеры будут приведены позже.

Итак, класс играет две роли:

- класс — это контейнер для методов класса и данных класса;
- класс — это «трафарет», позволяющий создавать конкретные объекты.

Для каждого конкретного объекта класс определяет состояние и поведение. Состояние объекта задается совокупностью значений его полей. Поведение объекта определяется набором методов, применяемых к объектам данного класса.

В соответствии с объектной ориентацией языка C# всякая программа на языке C# представляет собой класс или совокупность классов.

Внутри объявления каждого класса могут быть размещены:

- данные класса (статические поля);
- методы класса (статические методы);
- данные объектов класса (не статические поля);
- методы для работы с объектами класса (не статические методы);
- внутренние классы;
- дополнительные члены, речь о которых еще впереди.

1.2. Программа на C#

Формат простейшего определения (иначе декларации или объявления) класса в C#:

```
class имя_класса  
{поля и методы}
```

Заклученная в обязательные фигурные скобки совокупность полей и методов называется телом класса. Среди полей и методов могут быть статические, относящиеся к классу в целом, и не статические — определяющие состояния конкретных объектов и действия над этими объектами.

Перед телом класса находится заголовок объявления класса. Заголовок в общем случае может иметь более сложную форму, но сейчас более сложную форму не нужно рассматривать. Служебное слово **class** всегда входит в заголовок.

имя_класса — идентификатор, произвольно выбираемый автором класса.

Отметим, что идентификатор в языке C# — это последовательность букв, цифр и символов подчеркивания, которая не может начинаться с цифры. В отличие от многих предшествующих языков, в идентификаторах C# можно использовать буквы разных алфавитов, например, русского или греческого. В языке C# прописная буква отличается от той же самой строчной. Примеры идентификаторов, наверное, излишни.

Среди методов классов исполнимой программы (приложения) на языке C# обязательно присутствует статический метод со спе-

циальным именем Main. Этот метод определяет точку входа в программу — именно с выполнения операторов метода Main() начинается исполнение ее кода. Исполняющая программу система неявно (невидимо для программиста) создает единственный объект класса, представляющего программу, и передает управление коду метода Main().

Прежде чем приводить примеры программ, необходимо отметить, что практически каждая программа на языке C# активно использует классы библиотеки из .NET Framework. Через библиотечные классы программе доступно то окружение, в котором она выполняется. Например, класс Console представляет в программе средства для организации консольного диалога с пользователем.

Применение в программе библиотеки классов предполагает либо создание объектов классов этой библиотеки, либо обращения к статическим полям и методам библиотечных классов.

Чтобы применять методы и поля библиотечных классов и создавать их объекты, необходимо знать состав библиотеки и возможности ее классов. Библиотека .NET настолько обширна, что на первом этапе изучения программирования на C# придется ограничиться только самыми скромными сведениями о ней. Со средствами библиотеки классов будем знакомиться, используя некоторые из этих средств в небольших иллюстративных программах. Вначале будем рассматривать программы, в каждой из которых будут применяться только статические члены некоторых библиотечных классов, и будет только один класс с единственным статическим методом Main(). При таких ограничениях программирование на языке C# превращается в процедурное программирование. Основное отличие от традиционного императивного подхода других процедурных языков — применение особого синтаксиса для обращения к статическим членам библиотечных классов и методам объектов, представляющих в программах данные.

Для иллюстрации приведенных общих сведений о программах на C# рассмотрим программу, которая выводит в консольное окно экрана фразу «Введите Ваше имя:». После нажатия клавиши ENTER программа считывает имя, набираемое пользователем на клавиатуре, а затем приветствует пользователя, используя полученное имя.

```
// 01_01.cs - Первая программа.  
class HelloUser  
{  
    static void Main()  
    {  
        string name;  
        System.Console.WriteLine("Введите Ваше имя: ");  
        name = System.Console.ReadLine();  
        System.Console.WriteLine("Приветствую Вас, " + name + "!");  
    }  
}
```

Для тех, кто не знаком с синтаксисом C, C++ и производных от них языков, отметим, что первая строка — однострочный комментарий.

Вторая строка **class** HelloUser — это заголовок определения класса с именем HelloUser. Напомним, что **class** — служебное слово, а идентификатор HelloUser выбрал автор программы.

Далее в фигурных скобках — тело класса.

В классе HelloUser только один метод с заголовком **static void** Main()

Как уже сказано, служебное слово **static** — это модификатор метода класса (отличающий его от методов объектов). Служебное слово **void** определяет тип, соответствующий особому случаю «отсутствие значения». Его использование в заголовке означает отсутствие возвращаемого методом Main() значения. В заголовке каждого метода после его имени в круглых скобках помещается список параметров (спецификация параметров). В нашем примере параметры у метода не нужны, но круглые скобки обязательны. Отметим, что имя Main не является служебным словом языка C#.

Вслед за заголовком в определении каждого метода помещается его тело — заключенная в фигурные скобки последовательность определений и операторов. Рассмотрим тело метода Main() в нашем примере.

string name; — это определение (декларация) строковой переменной с выбранным программистом именем name. **string** — служебное слово языка C# — обозначение предопределенного типа (System.String) для представления строк. Подробнее о типах речь пойдет позже.

Для вывода информации в консольное окно используется оператор: `System.Console.WriteLine ("Введите Ваше имя:");`

Это обращение к статическому методу WriteLine() библиотечного класса Console, представляющего в программе консоль (консоль — это программные средства текстового интерфейса). System — обозначение пространства имен (**namespace**), к которому отнесен класс Console (о пространствах имен будет сказано чуть позже в параграфе 1.3)

Метод WriteLine() класса Console выводит значение своего аргумента в консольное окно. У этого метода нет возвращаемого значения, но есть параметр. В рассматриваемом обращении к методу WriteLine() аргументом, заменившим параметр, служит строковая константа "Введите Ваше имя:". Ее значение — последовательность символов, размещенная между кавычек. Именно это сообщение в качестве «приглашения» увидит пользователь в консольном окне при выполнении программы.

Текст с приглашением и последующим ответом пользователя для нашей программы могут иметь, например, такой вид:

```
Введите Ваше имя:  
Тимофей<ENTER>
```

Здесь <ENTER> — условное обозначение (добавленное на бумаге, но невидимое на консольном экране) нажатия пользователем клавиши ENTER.

Когда пользователь наберет на клавиатуре некоторое имя (некоторый текст) и нажмет клавишу ENTER, то будут выполнены действия, соответствующие оператору

```
name = System.Console.ReadLine();
```

Здесь выполняется обращение к методу `ReadLine()` класса `Console` из пространства имен `System`. Метод `ReadLine()` не имеет параметров. У него есть возвращаемое значение — строка символов, прочитанная из стандартного входного потока консоли. В нашем примере возвращаемое значение — строка "Тимофей".

Обратите внимание, что метод `ReadLine()` выполнится только после нажатия клавиши `ENTER`. До тех пор пользователь может вносить в набираемый текст любые исправления и изменения.

В рассматриваемом операторе слева от знака операции присваивания «`=`» находится имя `name` той переменной, которой будет присвоено полученное от консоли значение. После приведенного выше диалога значением `name` будет "Тимофей".

Следующий оператор содержит обращение к уже знакомому методу:

```
System.Console.WriteLine ("Приветствую Вас," + name + "!");
```

В качестве аргумента используется конкатенация (сцепление, соединение) трех строк:

- 1) строковой константы `"Приветствую Вас,"`;
- 2) значения строковой переменной с именем `name`;
- 3) строковой константы `"!"`.

В качестве обозначения операции конкатенации строк используется символ «`+`».

Обратите внимание, что в выражениях с арифметическими операндами знак «`+`» означает операцию сложения. Эта способность операций по-разному выполняться для разных типов операндов называется **полиморфизмом**. *Полиморфизм* в переводе с греческого означает «много форм». Полиморфизм и его разновидности мы еще рассмотрим подробнее.

При конкатенации в нашем примере значения строковых констант «присоединяются» к значению переменной с именем `name`.

Таким образом, результат выполнения программы будет таким:

```
Введите Ваше имя:  
Тимофей<ENTER>  
Приветствую Вас, Тимофей!
```

1.3. Пространство имен

В нашей программе использованы обращения к двум статическим методам класса `Console`. В общем виде формат обращения к статическим членам класса:

Название_пространства_имен.имя_класса.имя_члена

Эта конструкция в языке `C#` называется уточненным, квалифицированным или составным именем. Первым элементом полного ква-

лифицированного имени является наименование пространства имен. Поясним это понятие. Вначале приведем опубликованные в литературе определения.

«**Пространство имен** — механизм, посредством которого поддерживается независимость используемых в каждой программе имен и исключается возможность их случайного взаимного влияния.» [6]

«**Пространство имен** определяет декларативную область, которая позволяет отдельно хранить множества имен. По существу, имена, объявленные в одном пространстве имен, не будут конфликтовать с такими же именами, объявленными в другом.» [7]

Каждая программа на C# может либо использовать либо свое собственное уникальное пространство имен, либо размещать свои имена в глобальном пространстве, предоставляемом программе по умолчанию. В ближайшее время нам будет достаточно пространства имен, используемого по умолчанию, и в наших программах объявления пространств имен не понадобятся. Но мы вынуждены использовать пространства имен тех библиотек, средства которых применяются в наших программах.

Понятие пространства имен появилось в программировании в связи с необходимостью различать одноименные понятия из разных библиотек, используемых в одной программе. Пространство имен System объединяет те классы из .NET Framework, которые наиболее часто используются в консольных программах на C#.

Если в программе необходимо многократно обращаться к классам из одного и того же пространства имен, то можно упростить составные имена, используя в начале программы (до определения класса) специальную директиву импорта пространства имен:

```
using имя_пространства_имен;
```

После такой директивы для обращения к статическому члену класса из данного пространства имен можно использовать сокращенное квалифицированное имя, не содержащее названия пространства имен:

```
имя_класса.имя_члена
```

В нашей программе используется пространство имен System. Из этого пространства мы применяем класс Console и два статических метода этого класса — WriteLine() и ReadLine().

Поместив в программу директиву

```
using System;
```

можно обращаться к названным методам с помощью сокращенных квалифицированных имен Console.WriteLine() и Console.ReadLine().

В версии C# 6.0 появилась директива импорта конкретного типа, позволяющая еще больше упростить обращения к статическим членам классов:

```
using static имя_типа;
```

В качестве имени типа допустимо использовать имена классов, структур, перечислений. Директива **using static** позволяет напрямую обращаться ко всем статическим членам (методам, полям и т. д.) того типа, имя которого использовано в директиве. Если включить в программу директиву

```
using static System.Console;
```

то станут допустимы непосредственные обращения к методам:

```
WriteLine ("Введите ваше имя:");  
name = ReadLine();
```

1.4. Создание консольного приложения

В отличие от языков предшествующих поколений, язык C# невозможно применять, изучив только синтаксис и семантику языковых конструкций. На приведенном примере программы мы уже убедились, что даже такие элементарные действия как ввод-вывод текстовой информации требуют применения механизмов, не входящих в язык программирования. Эти механизмы предоставляются программисту в виде средств платформы .NET. Платформа .NET поддерживает не только язык C#, но и десятки других языков, предоставляя им огромную библиотеку классов, упрощающих разработку программ разного назначения. Не пытаясь описать все достоинства и особенности .NET, отметим только следующее: .NET позволяет в одном программном комплексе объединять программы, написанные на разных языках программирования. .NET в настоящее время реализована для разных операционных систем. Для .NET разработаны мощные и наиболее современные интегрированные среды программирования.

Предполагая, что читатель имеет возможность работать с одной из таких сред программирования, поддерживающих язык C#, перейдем к краткому описанию технологии создания простых программ. Будем использовать Visual Studio 2017.

Программируя на C# в .NET Framework, можно в частности разрабатывать (программы других видов в книге просто не рассматриваются):

- консольные приложения;
- Windows-приложения;
- библиотеки классов.

Программу, которую мы привели в предыдущем параграфе, проще всего реализовать как консольное приложение.

Независимо от того, какого вида программа разрабатывается, в Visual Studio необходимо создать решение (Solution) и в этом решении проект (Project). Создание пустого (без проектов) решения не имеет смысла, поэтому решение будет автоматически создано при создании

нового проекта. Прежде чем описать последовательность действий, необходимых для создания и выполнения простой программы, остановимся на соотношении понятий «проект» и «решение». В одно решение могут одновременно входить проекты программ разных видов. Текст (код) программы может быть обработан средой Visual Studio, когда он помещен в проект, а проект — включен в решение. Зачастую в одно решение помещают взаимосвязанные проекты, например, использующие одни и те же библиотеки классов (также помещенные в виде проектов в это решение).

Как только среда разработки (например, Visual Studio) запущена, выполните следующие шаги.

1. Создание нового проекта: File → New → Project.

В окне **New Project** на левой панели (**Project Types**) выберите язык (**Visual C#**) и платформу (**Windows**). На центральной панели выберите вид приложения **Console Application**.

В поле **Name** вместо предлагаемого по умолчанию имени `ConsoleApplication1` напечатайте выбранное вами имя проекта, например `Program_1`. В поле **Location** введите полное имя папки, в которой будет сохранено решение, например, `C:\Программы`. По умолчанию решению приписывается имя его первого проекта (в нашем примере `Program_1`, но это имя можно заменить нужным именем решения). Кнопкой **ОК** запускаем процесс создания проекта (и решения).

Среда Visual Studio создает решение, проект приложения и открывает окно редактора с таким текстом заготовки для кода программы:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
namespace Program_1
{
    class Program
    {
        static void Main(string[] args)
        {
        }
    }
}
```

2. Несмотря на то, что никакого кода в проект мы не добавляли, это приложение вполне работоспособно. Его можно следующим образом запустить на компиляцию и выполнение: Debug → Start Without Debugging (или сочетанием клавиш Ctrl+F5).

Откроется консольное окно с единственной фразой:

"Для продолжения нажмите любую клавишу..."

Это сообщение среды разработки, завершающее исполнение консольного приложения.

3. Дополним созданную средой разработки заготовку кода консольного приложения операторами из нашей первой программы:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
namespace Program_1
{
    class Program
    {
        static void Main(string[] args)
        {
            string name;
            System.Console.WriteLine("Введите Ваше имя: ");
            name = System.Console.ReadLine();
            System.Console.WriteLine("Приветствую Вас, " + name + "!");
        }
    }
}
```

Теперь по нажатию клавиш Ctrl+F5 программа откомпилируется, начнет выполнение, выведет приглашение: «Введите Ваше имя:» и, в ответ на введенное имя, «поздоровается».

В отличие от первой программы 01_01.cs, в тексте заготовки, созданной средой Visual Studio, присутствуют лишние для нашей программы операторы. Во-первых, вместо четырех директив **using** можно обойтись одной **using System**;

Во-вторых, нет необходимости в явном указании параметра в заголовке метода:

```
static void Main(string[] args)
```

Конструкция **string[] args** никак не используется в нашем приложении и может быть удалена. Назначение этой конструкции будет рассмотрено в последующих главах, где мы ее используем в заголовке метода **Main()**.

Третья особенность заготовки — наличие объявления пространства имен:

```
namespace Program_1 {...}
```

Это объявление вводит для программы собственное пространство имен с обозначением **Program_1**. Необходимости в собственном пространстве имен для нашей программы нет. Поэтому проще использовать стандартное (глобальное) пространство имен. Это вполне допустимо для тех небольших программ, которые используются в книге в качестве примеров. И наконец, из пространства имен **System** для нашей программы нужен только класс **Console**. Поэтому импорт пространства **System** можно заменить импортом конкретного класса **Console**. Усовершенствованная версия программы примет вид:


```
// 01_01.cs – Первая программа
using static System.Console;
class HelloUser
{
    static void Main()
    {
        string name;
        WriteLine("Введите Ваше имя: ");
        name = ReadLine();
        WriteLine("Приветствую Вас, " + name + "!");
        WriteLine("Для завершения сеанса нажмите ENTER.");
        ReadLine();
    }
}
```

Результаты выполнения программы:

```
Введите Ваше имя:
Василиса<ENTER>
Приветствую Вас, Василиса!
Для завершения сеанса нажмите ENTER.<ENTER>
```

Контрольные вопросы и задания

1. Объясните, что такое тип.
2. Как можно определить понятие «переменная»?
3. Приведите примеры классов и объектов.
4. Перечислите признаки объектов.
5. В чем отличия членов класса от членов объекта?
6. Дайте определение идентификатора.
7. Объясните назначение отдельных частей простейшей программы на C#.
8. Каково назначение статического метода Main()?
9. Возможно ли написать программу на C#, не применяя классов?
10. Что такое тип **void**?
11. Какие методы класса Console применяются для ввода и вывода данных?
12. В какой момент выполняется чтение вводимых с клавиатуры данных?
13. Что такое полиморфизм?
14. Что такое пространство имен?
15. Какое из слов конструкции System.Console.ReadLine() является названием пространства имен?
16. Для каких целей применяется директива **using**?
17. Чем решение (Solution) в Visual Studio отличается от проекта (Project)?
18. Объясните назначение директивы **using static**.
19. В чем состоит различие статических и нестатических членов класса?

20. Перечислите известные вам виды членов класса.

21. Объясните смысл (назначение) каждого элемента в составном (квалифицированном) имени `System.Console.WriteLine`.

22. Что обозначает операция «+» в бинарном выражении, одним из операндов которого является строка (объект класса **string**)?

Глава 2

ТИПЫ В ЯЗЫКЕ C#

2.1. Типы ссылок и типы значений

В стандарте C# [2] в связи с типами используется выражение «унифицированная система типов». Смысл унификации состоит в том, что все типы происходят от класса **object**, т. е. являются производными от этого класса и наследуют его члены. О механизме наследования речь пойдет позднее, но в примерах программ мы будем применять для объектов и классов некоторые возможности, унаследованные ими от класса **object**.

Типы C# позволяют представлять, во-первых, те «скалярные» данные, которые используются в расчетах (целые и вещественные числа, логические значения) и в обработке текстов (символы, строки). Вторая группа типов соответствует специфическим для программирования на языках высокого уровня «агрегирующим» конструкциям: массивам, структурам, объектам (классов).

Такое деление типов на две названные группы унаследовано языком C# из предшествующих ему языков программирования.

Примечание. В C# имеются типы указателей и тип **void**. Тип **void** означает отсутствие значения. Указатели используются только в небезопасных участках кода программ и рассматриваться не будут.

Однако при разработке C# решили, что в системе типов целесообразно иметь еще одно разделение. Поэтому язык C# поддерживает два вида (две категории) типов: типы значений (*value types*) и типы ссылок (*reference types*).

Принципиальное различие этих двух видов типов заключается в том, что объект ссылочного типа может именоваться одновременно несколькими ссылками, что абсолютно недопустимо для объектов с типами значений.

Для переменных традиционных языков, например C, всегда соблюдается однозначное соответствие:

имя_переменной → значение_переменной

Точно такая же схема отношений справедлива в языке C# для объектов с типами значений:

имя_объекта → значение_объекта

Если рассматривать реализацию такого отношения в программе, то нужно вспомнить, что память компьютера организована в виде последовательности ячеек. Каждая ячейка имеет индивидуальный, обычно числовой, адрес (наименьшая из адресуемых ячеек — байт).

При выполнении программы каждому объекту выделяется блок (участок) памяти в виде одного или нескольких смежных байтов. Адрес первого из них считается адресом объекта. Код, находящийся в выделенном для объекта блоке памяти, представляет значение объекта.

Представить машинную реализацию объекта с типом значений можно так:

адрес_объекта ⇒ код_значения_объекта

Переменные, имеющие типы значений, непосредственно представляют в программе конкретные данные.

Переменные, имеющие типы ссылок, представляют в программе конкретные данные косвенно, хотя косвенность этого представления не показана явно в тексте программы. Доступ к данным по имени переменной с типом ссылки иллюстрирует триада:

имя_переменной ⇒ значение_адреса_данных ⇒ значение_данных

Машинную реализацию такой триады можно изобразить так:

адрес_переменной ⇒ код_адреса_данных ⇒ код_значения_данных

В коде программы на C# доступ к данным с помощью ссылки в ряде случаев можно воспринимать в соответствии со схемой доступа к данным с помощью традиционной переменной (имеющей тип значений):

имя_ссылки ⇒ значение_данных

Но при использовании такой схемы появляется новое и принципиальное отличие — доступность одних и тех же данных (одного участка памяти) с помощью нескольких ссылок:

имя_ссылки_1
имя_ссылки_2
...
имя_ссылки_K } ⇒ значение_данных

Основное и принципиальное для программиста-пользователя отличие типов значений от типов ссылок состоит в следующем. Каждой переменной, которая имеет тип значений, принадлежит её собственная копия данных, и поэтому операции с одной переменной не влияют на значения других переменных.

Несколько переменных с типом ссылок могут быть одновременно соотнесены с одним и тем же объектом. Поэтому операции, выполняемые с одной из этих переменных, могут изменять объект, на который в этот момент ссылаются другие переменные (с типом ссылок).

Различия между типами значений и типами ссылок иллюстрирует еще одна особенность. Объект ссылочного типа никогда не имеет своего собственного имени. В разные моменты выполнения программы для обращения к нему могут использоваться разные переменные ссылочного типа.

Если обратить внимание на принципы организации памяти компьютера, то следует отметить, что на логическом уровне она разделена на две части: стек и управляемую кучу (*managed heap*).

Объекты с типами значений всегда при реализации получают память в стеке. При присваиваниях их значения копируются. Объекты ссылочных типов размещаются в управляемой куче.

Как и объекты, переменные могут быть ссылочных типов (ссылки) и типов значений.

Учитывая этот факт, можно сказать, что типы значений — это те типы, переменные которых непосредственно хранят свои данные, тогда как ссылочные типы — это те типы, переменные которых хранят ссылки, по которым данные могут быть доступны.

Примеры и особенности переменных обоих видов рассмотрим чуть позже, ознакомившись с разнообразием типов языка C#.

2.2. Классификация типов C#

Система типов языка C# — вещь достаточно сложная и требующая аккуратного рассмотрения при первом знакомстве. Общие отношения между типами иллюстрирует иерархическая схема, приведенная на рис. 2.1. Как уже упоминалось и как показано на схеме, все типы языка C# имеют общий базовый тип — класс **object**. О делении типов на типы ссылок и типы значений мы уже рассказывали. А вот с дальнейшей детализацией будем знакомиться постепенно. И знакомство начнем не с классификации, соответствующей иерархии типов, а с другого деления.

Все типы, которые могут использоваться в программах на C#, делятся на три группы:

- *предопределенные* в языке C# (в Стандарте они обозначены термином **Built_In**, который можно перевести как «встроенные»);
- *библиотечные* (обычно из стандартной библиотеки .NET Framework);
- *определенные программистом* (пользовательские).

Предопределенные типы включены в язык C#. К ним относятся:

1) **object** — тип ссылок (класс), который является первоначальным (единственным исходным) базовым для всех других типов языка C#, т. е. все другие типы являются производными от этого типа;

- 2) простые (базовые или фундаментальные) типы — типы значений, для которых в языке C# введены специальные обозначения;
- 3) **string** — тип ссылок (класс) для представления строк — последовательностей символов в кодировке Unicode, и др.

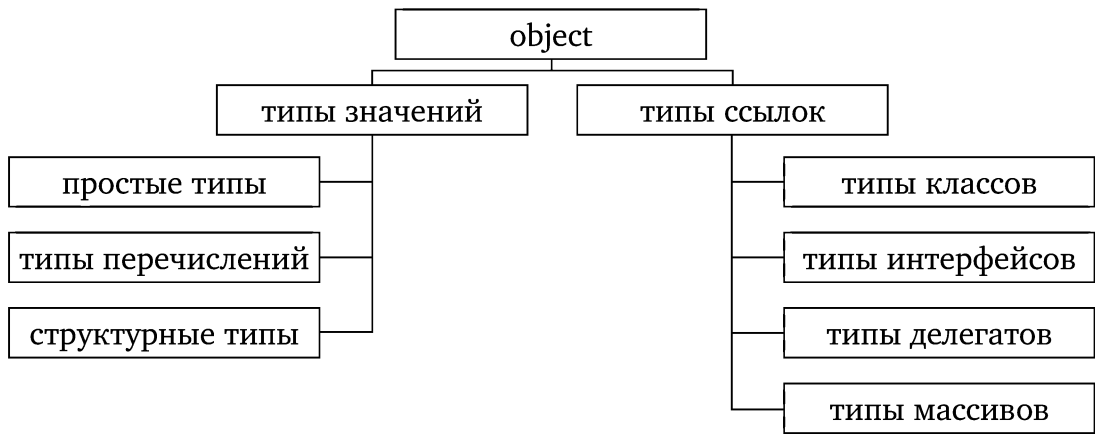


Рис. 2.1. Схема типов языка C#

Библиотечные и пользовательские типы могут быть как типами значений, так и типами ссылок. Чтобы пользоваться библиотечным типом, нужно знать его имя и возможности (поля, методы), а также название того пространства имен, которому он принадлежит.

Примечание. Как мы уже говорили, для сокращения квалифицированного (полного) имени нужного нам класса (типа) в программу включают директивы

```
using название_пространства_имен;
using static min;
```

Например, чтобы написать программу на C# для работы с файлами, в коде используется директива:

```
using System.IO;
```

После этого в программе становятся доступны с помощью сокращенных имен классов типы, необходимые для организации потокового ввода-вывода.

Новые типы значений могут быть введены в программу как перечисления и структуры. Для добавления новых типов ссылок используют классы, массивы, интерфейсы, делегаты. О том, как это делать в своих программах, т. е. как появляются типы, определенные программистом, речь еще впереди, а сейчас рассмотрим базовые типы.

2.3. Простые (базовые) типы. Константы-литералы

Рассмотрение типов начнем с простых (базовых) типов значений, которые в качестве базовых типов используются во многих языках программирования, например, в C и C++.

Язык C# представляет программисту набор предопределенных структурных типов, называемых простыми типами. Так как это типы структурные, то это типы значений. Каждый из этих типов представлен в программе соответствующим зарезервированным идентификатором (служебным словом).

Простые типы значений языка C# можно подразделить на следующие группы:

- числовые (арифметические) типы;
- логический (булевский) тип;
- символьный тип (в стандарте он отнесен к целочисленным типам).

К числовым типам относятся: знаковые и беззнаковые целочисленные типы, вещественные типы и десятичный тип.

Числовые значения представляются в программе с помощью констант (литералов) и переменных трех разных видов:

- целые числа (знаковые типы: **sbyte**, **short**, **int**, **long**, беззнаковые типы: **byte**, **ushort**, **uint**, **ulong**);
- вещественные числа (типы **float**, **double**);
- десятичные числа (тип **decimal**).

Примеры целочисленных литералов:

48 — константа знакового целого типа (**int**);

43L — константа знакового длинного целого типа (**long**);

49U — константа беззнакового целого типа (**uint**);

93UL — константа беззнакового длинного целого типа (**ulong**).

Обратите внимание на необходимость в ряде случаев суффиксов. Суффикс L (или l) используется для отнесения литерала к типу **long**. Суффикс U (или u) превращает литерал в значение беззнакового типа.

Константы (литералы) вещественных типов могут быть записаны в виде с фиксированной точкой:

101.284 — тип **double**;

-0.221F — тип **float**;

12.0f — тип **float**.

Кроме того, широко используется экспоненциальная запись — научная нотация, при которой явно выписываются мантисса и экспонента, а между ними размещается разделитель E или e. Примеры:

-0.24E-13 — тип **double**;

1.44E+11F — тип **float**;

-16.3E+02f — тип **float**.

Обратите внимание на необходимость суффикса F или f в записи вещественной константы с одинарной точностью, относимой к типу **float**. При отсутствии указанного суффикса константа по умолчанию воспринимается и обрабатывается как значение типа **double**.

Тип **decimal** специально введен в язык C#, чтобы обеспечить вычисления, при выполнении которых недопустимы (точнее, должны быть минимизированы) ошибки округления. Например, при финансовых

вычислениях с большими суммами даже минимальные погрешности за счет округления могут приводить к заметным потерям.

Переменные и константы типа **decimal** позволяют представлять числовые значения в диапазоне от 10^{-28} до $7,9 \cdot 10^{28}$. Для каждого числа выделяется 128 двоичных разрядов, причем число хранится в форме с *фиксированной точкой*. С помощью этого типа можно представлять числа, имеющие до 28 десятичных разрядов.

В записи десятичной константы используется суффикс *m* (или *M*).

Примеры десятичных литералов:

308.0008M;

12.6m;

123456789000m.

Для представления логических значений используются константы типа **bool**:

true — истина;

false — ложь.

По сравнению с предшествующими языками (например, C и C++) в C# для представления кодов отдельных символов (для данных типа **char**) используется не 1 байт, а 2 байта и для кодирования используется Unicode. Символьные литералы ограничены обязательными *апострофами* (не путайте с кавычками!):

'A', 'z', '2', '0', 'Я'.

В символьных литералах для представления одного символа могут использоваться эскейп-последовательности, каждая из которых начинается с обратной косой черты «\». В виде эскейп-последовательностей изображаются управляющие символы:

\' — апостроф;

\" — кавычка;

\\ — обратная косая черта;

\a — звуковой сигнал;

\b — возврат на шаг (забой);

\n — новая строка;

\r — возврат каретки (в начало строки);

\t — табуляция (горизонтальная);

\0 — нулевое значение, пусто;

\f — перевод страницы;

\v — вертикальная табуляция.

С помощью явной записи числового значения кода эскейп-последовательностью можно представить любой Unicode-символ. Формат такого представления:

'\uhhhh',

где *h* — шестнадцатеричная цифра, *u* — обязательный префикс. Предельные значения от '\u0000' до '\uFFFF'.

Пример: '\u0066' соответствует символу 'f'.

Разрешены также эскейп-последовательности вида '\xhh', где h — шестнадцатеричная цифра, x — префикс. Например, '\x2B' представляет код символа 't'.

2.4. Объявления переменных и констант базовых типов

По традиции, унаследованной от языков C и C++, новый экземпляр (переменная) простого типа вводится с помощью объявления:

type name = expression;

где *type* — название типа;

name — имя экземпляра (переменной);

expression — инициализирующее выражение (например, константа).
Объявление обязательно завершается точкой с запятой.

Названия базовых типов с примерами объявлений приведены в табл. 2.1. (см. [1]).

В одном объявлении могут быть определены несколько переменных одного типа:

type name1 = expression1, name2 = expression 2;

Переменные одного объявления отделяются друг от друга запятыми.
Пример:

double pi = 3.141593, e = 2.718282, c = 535.491656;

Таблица 2.1

Простые (базовые) типы значений

Тип	Описание	Примеры объявлений
sbyte	8-битовый знаковый целый (1 байт)	sbyte val = 12;
short	16-битовый знаковый целый (2 байта)	short val = 12;
int	32-битовый знаковый целый (4 байта)	int val = 12;
long	64-битовый знаковый целый (8 байтов)	long val1 = 12; long val2 = 34L;
byte	8-битовый беззнаковый целый (1 байт)	byte val1 = 12;
ushort	16-битовый беззнаковый целый (2 байта)	ushort val1 = 12;
uint	32-битовый беззнаковый целый (4 байта)	uint val1 = 12; uint val2 = 34U;
ulong	64-битовый беззнаковый целый (8 байтов)	ulong val1 = 12; ulong val2 = 34U; ulong val3 = 56L; ulong val4 = 78UL;
float	Вещественный с плавающей точкой с одинарной точностью (4 байта)	float val = 1.23F;

Тип	Описание	Примеры объявлений
double	Вещественный с плавающей точкой с двойной точностью (8 байтов)	double val1 = 1.23; double val2 = 4.56D;
bool	Логический тип; значение или false , или true	bool val1 = true ; bool val2 = false ;
char	Символьный тип; значение — один символ Unicode (2 байта)	char val = 'h';
decimal	Точный денежный тип, 28—29 значимых десятичных разрядов (12 байтов)	decimal val = 1.23M;

Введя формат объявления переменных, следует остановиться на вопросе выбора их имен. Имя переменной — выбранный программистом идентификатор. Идентификатор — это последовательность букв, десятичных цифр и символов подчеркивания, которая начинается не с цифры. В языке C# в качестве букв допустимо применять буквы национальных алфавитов. Таким образом, правильными идентификаторами будут, например, такие последовательности:

```
Number_of_Line, объем_в_литрах, x14, mass.
```

В качестве имен, вводимых программистом, запрещено использовать служебные (ключевые) слова языка C#. Чтобы уже сейчас предостеречься от такой ошибки, обратите внимание на их список, приведенный в таблице 2.2.

Таблица 2.2

Служебные слова языка C#

abstract	as	base	bool	break
byte	case	catch	char	checked
class	const	continue	decimal	default
delegate	do	double	else	enum
event	explicit	extern	false	finally
fixed	float	for	foreach	goto
if	implicit	in	int	interface
internal	is	lock	long	namespace
new	null	object	operator	out
override	params	private	protected	public
readonly	ref	return	sbyte	sealed
short	sizeof	stackalloc	static	string
struct	switch	this	throw	true
try	typeof	uint	ulong	unchecked
unsafe	ushort	using	virtual	void
volatile	while	—	—	—

Следует отметить, что в табл. 2.2 не включены идентификаторы, которые играют роль ключевых слов только в конкретном контексте. Примеры: **into**, **set**, **when**, **yield**.

Познакомившись со списком служебных слов, читатель сразу же обнаружит ошибку в следующем объявлении:

```
int try = 15;    // ошибка в имени переменной!
```

Инициализирующее выражение часто константа, однако может быть любым выражением, операнды которого имеют конкретные значения в момент объявления переменной. По типу вычисляемого значения инициализирующее выражение должно соответствовать типу создаваемой переменной.

Примеры объявлений переменных:

```
double pi2 = 6.28;  
int два = 2;
```

В объявлении переменной инициализация может отсутствовать. Однако язык C# очень строг и не допускает существования в программе переменных с неопределенными значениями. Поэтому после объявления переменной без её инициализации необходимо каким-то образом присвоить ей значение. Обычно для этого используют оператор присваивания:

```
имя_переменной = выражение;
```

Пример (удельный заряд электрона в единицах СГСЭ/г):

```
double electron_charge; // объявление  
electron_charge = 5.27e+17; // присваивание значения
```

В отличие от литералов, которые сами по себе представляют собственные значения и не требуют предварительных объявлений, именованные константы вводятся с помощью конструкции:

```
const type name = expression;
```

Отличие от объявления переменной: наличие модификатора **const** и обязательность инициализации.

Наличие инициализатора в объявлении переменной или именованной константы позволяет «поручать» компилятору самостоятельно выводить тип локальной переменной или константы. Для этого вместо названия типа в декларации используется контекстно-зависимое служебное слово **var**. Предыдущие декларации вещественной и целой переменных можно заменить такими:

```
var pi2 = 6.28;  
var два = 2;
```

Хотя такое объявление переменных с неявным указанием их типов не рекомендуется, но в ряде случаев оно может оказаться очень удобным, что будет показано в дальнейшем.

Контрольные вопросы и задания

1. Чем отличаются типы знаковые от беззнаковых.
2. Приведите примеры констант-литералов числовых (арифметических) типов.
3. Укажите назначение десятичного типа и правила записи его констант.
4. Назовите способы записи символьных констант.
5. Приведите примеры эскейп-последовательностей.
6. Назовите размеры (в битах) представления в памяти констант базовых типов.
7. Какие символы допустимы в идентификаторах C#?
8. Приведите примеры служебных слов языка C#.
9. Является ли идентификатор Main служебным словом?
10. Что такое инициализация переменной?
11. Чем именованная константа отличается от константы-литерала?

Глава 3

ОПЕРАЦИИ И ЦЕЛОЧИСЛЕННЫЕ ВЫРАЖЕНИЯ

3.1. Операции языка C#

В предыдущей главе мы ввели понятие типа и рассмотрели классификацию типов, принятую в языке C#. Привели сведения о предельных значениях констант и переменных базовых (простых) типов. Тем самым для базовых типов определена совокупность допустимых значений. Чтобы полностью охарактеризовать базовые типы, требуется рассмотреть допустимые для них операции.

Набор операций языка C# весьма велик, и рассматривать возможности каждой из них мы будем постепенно по мере необходимости. Однако предварительно приведем список операций, разместив их в порядке уменьшения их приоритетов, называемых еще рангами и категориями (табл. 3.1 и 3.2).

Таблица 3.1

Операции, ассоциативные слева — направо

Операция	Значение
Базовые (первичные) операции	
.	выбор члена (класса, структуры или объекта)
()	обращение к методу или к экземпляру делегата
[]	доступ по индексу (индексирование)
++	постфиксный инкремент
--	постфиксный декремент
new	создание объекта (создание экземпляра)
typeof	идентификация типа
sizeof	определение размера операнда (в байтах)
stackloc	выделение памяти в стеке (только в опасном коде)
nameof	строковое представление имени (переменной, типа и т. д.)
checked	контроль за целочисленными переполнениями
unchecked	отмена контроля за целочисленными переполнениями
->	доступ к члену (объекта) по указателю (только в опасном коде)

Операция	Значение
?.	null -условная или элвис-операция обращения к члену (класса, структуры) без генерации исключения, если член равен null
default	получение стандартного (умалчиваемого) значения типа
Унарные операции	
+	унарный плюс (задание знака)
-	унарный минус (задание знака)
++	префиксный инкремент
--	префиксный декремент
~	поразрядное отрицание
!	логическое отрицание
(тип)	приведение к заданному типу
&	получение адреса (только в опасном коде)
*	разыменование указателя (только в опасном коде)
await	ожидание
Арифметические бинарные операции	
*	умножение
/	деление
%	получение остатка при делении
+	сложение
-	вычитание
Операции поразрядных сдвигов	
>>	поразрядный сдвиг вправо
<<	поразрядный сдвиг влево
Операции отношений (сравнений)	
<	меньше
>	больше
<=	меньше или равно
>=	больше или равно
is	сравнение типов операндов (возвращает логическое значение)
as	проверка типов (возвращает значение первого операнда или null)
==	сравнение на равенство
!=	сравнение на неравенство
??	Сравнение с null («поглощение null » — если null , то вернуть стандартное значение)

Операция	Значение
Поразрядные операции	
&	побитовое (поразрядное) И
^	побитовое (поразрядное) исключающее ИЛИ
	побитовое (поразрядное) ИЛИ
Логические бинарные операции	
&	конъюнкция (логическое И)
	дизъюнкция (логическое ИЛИ)
^	исключающая дизъюнкция
&&	условная конъюнкция
	условная дизъюнкция
Тернарная операция	
?:	условная операция

Таблица 3.2

Операции присваивания (ассоциативные справа — налево) и лямбда-операция

Операция	Значение
=	присваивание
+=	сложение с присваиванием
-=	вычитание с присваиванием
*=	умножение с присваиванием
/=	деление с присваиванием
%=	получение остатка от деления с присваиванием
&=	поразрядное И с присваиванием
^=	поразрядное исключающее ИЛИ с присваиванием
=	поразрядное ИЛИ с присваиванием
>>=	поразрядный сдвиг вправо с присваиванием
<<=	поразрядный сдвиг влево с присваиванием
=>	лямбда-операция

В табл. 3.1. операции объединены в группы. В каждую группу помещены операции одного ранга. Операции одного ранга из табл. 3.1 выполняются слева направо. Например, $x*y/z$ будет вычисляться в соответствии с выражением $(x*y)/z$.

Операции группы, размещенной выше, имеют более высокий ранг, нежели операции групп, размещенных ниже. Например, $2 + 6/2$ равно 5, так как операция $/$ имеет более высокий ранг, нежели бинарная операция $+$. Начинаящие программисты, забывая о приоритетах операций, иногда ошибаются, приводя для математического соотношения $\frac{a}{b * c}$ такое выражение: $a/b * c$. Его следует записать, используя скобки: $a/(b * c)$, или без скобок так $a/b/c$.

3.2. Операции присваивания и оператор присваивания

Операции присваивания, помещенные в табл. 3.2, имеют более низкий ранг (меньший приоритет), нежели операции из табл. 3.1. Еще одна особенность операций присваивания состоит в том, что они выполняются *справа налево*. У всех этих операций одинаковый ранг. Например, для выражения $x = y = z = c$ с помощью скобок последовательность вычисления можно обозначить так: $x = (y = (z = c))$. Для сравнения отметим, что выражению $x + y + z + c$ эквивалентно выражение $((x + y) + z) + c$.

Присваивание — фундаментальное понятие программирования. Средства для выполнения присваивания существуют практически в каждом языке программирования.

Начнем с того, что в C# единичный знак « $=$ » обозначает бинарную **операцию** (не оператор!). Формат применения операции присваивания:

имя_переменной = выражение

Конструкция *имя_переменной = выражение* представляет собой бинарное выражение с двумя операндами. Последовательность обработки такого выражения следующая:

- 1) вычислить (получить) значение выражения;
- 2) присвоить полученное значение переменной;
- 3) вернуть в точку размещения выражения значение, присвоенное переменной.

Если конструкция *имя_переменной = выражение* завершается точкой с запятой и размещена в последовательности других операторов программы, то она превращается в оператор присваивания. В этом случае действие ограничивается двумя первыми пунктами: вычислить значение выражения и присвоить полученное значение переменной, поименованной слева от знака присваивания. Третий пункт не нужен — значение всего выражения с операцией присваивания игнорируется. Однако иногда удобно использовать цепочки присваиваний, и в этом случае для всех выражений присваивания, размещенных справа, их значения используются явно.

Пример:

```
int a, b, c, d;  
a = b = c = d = 842;
```


После выполнения такого оператора переменные a, b, c, d принимают одно и то же значение 842. Последовательность вычислений: вычисляется выражение d = 842, тем самым переменная d становится равной 842 и это же значение принимает участие в следующем выражении присваивания, где левым операндом служит переменная c, и т. д.

Кроме обычной операции присваивания в C# есть составные операции присваивания, общую форму которых можно представить так:

бинарная_операция =

Здесь *бинарная_операция* — это одна из следующих арифметических и логических операций:

- + — сложение,
- — вычитание,
- * — умножение,
- / — деление,
- % — получение остатка от деления,
- & — поразрядная конъюнкция,
- | — поразрядная дизъюнкция,
- ^ — поразрядное исключающее ИЛИ,
- >> — поразрядный сдвиг вправо битового представления значения операнда,
- << — поразрядный сдвиг влево битового представления значения операнда.

Назначение составных операций присваивания — упростить запись и ускорить вычисление выражений присваивания, в которых левый операнд (переменная) одновременно используется в качестве левого операнда выражения, расположенного справа от операции присваивания.

Пример оператора присваивания с традиционной операцией:	Эквивалентный по результату оператор:
n = n + 48;	n += 48;
Традиционный оператор, запись которого можно упростить:	Применение составной операции присваивания:
n = n/(n * n);	n /= n * n;

3.3. Операции инкремента (++) и декремента (--)

Сокращенными формами операции присваивания можно считать операции автоувеличения (инкремент) ++ и автоуменьшения (декремент) --.

Операция инкремента ++ увеличивает на 1 значение операнда. Операция декремента -- уменьшает на 1 значение операнда. Для обеих операций операнд может быть целочисленным, символьным, вещественным, десятичным (**decimal**) или элементом перечисления.

Обе операции имеют префиксную и постфиксную формы. Префиксная форма предусматривает изменение значения операнда до использования его значения. Постфиксная — изменяет значение операнда после использования его значения.

Пример:

```
int zone = 240, res; // Объявление двух переменных
res = zone++; // Эквивалент: res = zone; zone = zone + 1;
res = ++zone; // Эквивалент: zone = zone + 1; res = zone;
```

Значение `res` в результате последовательного выполнения всех этих операторов равно 242.

Операндом для операций `++` и `--` может быть только леводопустимое неконстантное выражение (например, переменная). Следовательно, ошибочными будут выражения `84--`, `++0`, `(n - 12)++`, `--(x + y)` и им подобные.

Отметим, что операции `++` и `--` применимы к операндам всех базовых типов значений за исключением логического (`bool`).

3.4. Выражения с арифметическими операциями

Кроме операций инкремента и декремента для целочисленных операндов определены, во-первых, стандартные арифметические операции:

- унарные `-` и `+` (определяют знак выражения);
- бинарные `-`, `+`, `*`, `/` (вычитание, сложение, умножение, деление).

Среди них заслуживает пояснения только операция деления — ее результат при двух целочисленных операндах всегда округляется до наименьшего по абсолютной величине целого значения.

```
// 03_01.cs - целочисленное деление
using static System.Console;
class Program
{
    static void Main()
    {
        WriteLine("-13/4 = " + -13 / 4);
        WriteLine("13/4 = " + 13 / 4);
        WriteLine("15/-4 = " + 15 / -4);
        WriteLine("3/5 = " + 3 / 5);
    }
}
```

Обратите внимание на аргумент метода `WriteLine()`. Это выражение с операцией конкатенации строк `+`. Первая строка — изображение некоторого выражения. Вторая строка — строковое представление значения арифметического выражения, например, выражения `-13/4`.

Результат выполнения программы:

```
-13/4 = -3
13/4 = 3
```

$15 / -4 = -3$

$3 / 5 = 0$

Отдельно следует рассмотреть операцию `%` получения остатка от деления целочисленных операндов. При ненулевом делителе для целочисленных величин выполняется соотношение:

$(x / y * y + x \% y)$ равно x .

Пример:

```
// 03_02.cs – получение остатка от деления
using System;
class Program
{
    static void Main()
    {
        Console.WriteLine("13 % 4 = " + 13 % 4);
        Console.WriteLine("-13 % 4 = " + -13 % 4);
        Console.WriteLine("13 % -4 = " + 13 % -4);
        Console.WriteLine("-13 % -4 = " + -13 % -4);
        Console.WriteLine("-13/-4*-4 + -13 %-4 = "
            + (-13 / -4 * -4 + -13 % -4));
    }
}
```

Результат выполнения программы:

```
13 % 4 = 1
-13 % 4 = -1
13 % -4 = 1
-13 % -4 = -1
-13/-4 * -4 + -13 %-4 = -13
```

3.5. Поразрядные (побитовые) операции

От языков C и C++ язык C# унаследовал операции для работы с битовыми представлениями целых чисел:

- `~` — поразрядное инвертирование (поразрядное НЕ);
- `&` — поразрядная конъюнкция (поразрядное И);
- `|` — поразрядная дизъюнкция (поразрядное ИЛИ);
- `^` — поразрядное исключающее ИЛИ;
- `>>` — поразрядный сдвиг вправо;
- `<<` — поразрядный сдвиг влево.

Для иллюстрации выполнения поразрядных операций удобно использовать операнды беззнакового байтового типа (**byte**). Рассмотрим вначале поразрядную унарную операцию инвертирования (`~`). Операция применяется к каждому разряду (биту) внутреннего представления целочисленного операнда. Предположим, что десятичное значение переменной `bb` беззнакового байтового типа равно 3. Внутреннее представление `bb` имеет вид 00000011. После выполнения опе-

рации $\sim$ битовым представлением результата (т. е. выражения $\sim bb$), станет 11111100, т. е. 252 при записи с использованием десятичного основания. Стоит отметить, что для любого bb значением выражения $\text{byte}(bb + \sim bb)$ всегда будет 255, т. е. $\text{byte}.\text{MAX_VALUE}$.

Таблица 3.3

Правила выполнения поразрядных операций

Значения операндов		Результаты выполнения операции		
Первого	Второго	&		^
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

Операция &. Определим две байтовых переменных: bb со значением 3 и dd со значением 6:

```
byte bb = 3, dd = 6;
```

Поразрядные (битовые) представления: 00000011 и 00000110.

Значением выражения $bb \& dd$ будет десятичное 2, имеющее внутреннее представление 00000010.

Применив к переменным bb и dd бинарную операцию поразрядной дизъюнкции $|$, получим десятичное значение 7 с поразрядным представлением 00000111.

Применив бинарную операцию $\wedge$ исключающего ИЛИ к переменным bb и dd , получим десятичное значение 5 с битовым представлением 00000101.

Следующая программа содержит описанные выше выражения с поразрядными операциями над переменными типа **byte**.

```
// 03_03.cs – поразрядные операции с беззнаковыми переменными
using System;
class Program
{
    static void Main()
    {
        byte bb = 3;
        Console.WriteLine("bb = " + bb + "; ~bb = " + (byte)(~bb));
        byte dd = 6;
        Console.WriteLine("bb & dd = " + (byte)(bb & dd));
        Console.WriteLine("bb | dd = " + (byte)(bb | dd));
        Console.WriteLine("bb ^ dd = " + (byte)(bb ^ dd));
    }
}
```

Поясним еще раз особенности обращений к методу `WriteLine()`. Аргумент метода должен быть строкового типа (типа **string**). Выражение вида

обрабатывается так: вычисляется арифметическое выражение, его значение автоматически преобразуется в строку, которая «присоединяется» к строке, размещенной слева от знака +. Чтобы значение арифметического выражения при преобразованиях «не потеряло» свой беззнаковый тип, явно используется операция приведения типов (**byte**). Более подробно приведение типов рассмотрено в следующей главе.

Результаты выполнения программы:

```
bb = 3; ~bb = 252
bb & dd = 2
bb | dd = 7
bb ^ dd = 5
```

Бинарные поразрядные операции сдвига (>> и <<) по-разному используют значения своих операндов. Левый операнд задает целочисленное значение, к битовому представлению которого применяется сдвиг. Правый операнд указывает количество разрядов (битов), на которое должны сдвигаться все биты внутреннего представления левого операнда. Направление сдвига зависит от операции: << обозначает сдвиг влево, сдвиг вправо обеспечивает операция >>.

При сдвиге влево (<<) все разряды, выходящие за левый край внутреннего представления значения левого операнда, отбрасываются, все «освободившиеся» справа позиции заполняются нулями. Таким образом, сдвинув влево на 3 позиции число 4 с двоичным представлением 00000100, получим 00100000 (десятичное 32). Сдвиг влево числа 00010000 (десятичное 16) на 4 позиции приводит к нулевому значению. Обратим внимание, что сдвиг влево на k позиций эквивалентен умножению левого операнда на 2^k (при условии, что значащие левые ненулевые разряды не выйдут за разрядную сетку). Значением выражения $6 << 3$ будет десятичное число 48, т. е. $6 * 2^3$ или 00110000.

При сдвиге вправо (>>) разряды, выходящие за правый край представления значения левого операнда, отбрасываются. Слева «освободившиеся» позиции заполняются нулями. Таким образом, число 25 с двоичным представлением 00011001 при сдвиге на 2 позиции вправо приводит к получению кода 00000110 со значением 6. Сдвиг вправо на k позиций эквивалентен умножению на 2^{-k} с округлением результата до целого значения. Выражение $6 >> 2$ будет равно 1, т. е. 00000001.

Следующая программа иллюстрирует сказанное относительно поразрядных сдвигов:

```
// 03_04.cs – операции сдвигов для беззнаковых целых
using static System.Console;
class Program
{
    static void Main()
    {
        byte bb = 4;
```

```

WriteLine("bb = " + bb + "; bb << 3 = " +
    (byte)(bb << 3));
bb = 16;
WriteLine("bb = " + bb + "; bb << 4 = "
    + (byte)(bb << 4));
bb = 6;
WriteLine("bb = " + bb + "; bb << 3 = " +
    (byte)(bb << 3));
bb = 25;
WriteLine("bb = " + bb + "; bb >> 2 = " +
    (byte)(bb >> 2));
bb = 6;
WriteLine("bb = " + bb + "; bb >> 2 = " +
    (byte)(bb >> 2));
}
}

```

Результат выполнения программы:

```

bb = 4; bb << 3 = 32
bb = 16; bb << 4 = 0
bb = 6; bb << 3 = 48
bb = 25; bb >> 2 = 6
bb = 6; bb >> 2 = 1

```

3.6. Переполнения при операциях с целыми

Рассматривая поразрядные операции, мы ограничились операндами беззнакового типа **byte**, так как использование знаковых типов требует знакомства с правилами кодирования отрицательных целых чисел. Переменные и константы типа **byte** могут иметь значения от 0 до 255. Соответствующие двоичные коды — от 00000000 (все нули) до 11111111 (все единицы). В то же время для знакового типа **sbyte** установлены пределы от -128_{10} до $+127_{10}$.

Это связано с принятым на аппаратном уровне правилом кодирования знаковых целых чисел. Для их внутреннего представления используется так называемый **дополнительный код**. Если k — количество разрядов, отведенное для представления числа x (для **sbyte** k равно 8), то дополнительный код определяет выражение:

$$\text{доп}(x) = \begin{cases} x, & \text{если } x \geq 0, \\ 2^k - |x|, & \text{если } x < 0. \end{cases}$$

В битовом представлении чисел с использованием дополнительного кода у всех положительных чисел самый левый бит равен 0, а у отрицательных — единице.

Минимальное число типа **sbyte** равно -128_{10} . Его двоичный код 10000000. Число -1 представлено кодом 11111111. Представление нуля 00000000, код единицы 00000001.

Зная правила двоичного кодирования отрицательных целых чисел, легко понять, как меняется значение переменной знакового типа при поразрядных операциях. Например, применяя к положительному числу операцию поразрядного инвертирования $\sim$, мы меняем и знак числа, и на 1 увеличиваем его абсолютное значение. При поразрядном инвертировании отрицательного числа результат равен уменьшенному на 1 его абсолютному значению. Следующая программа иллюстрирует применение операции:

```
// 03_05.cs – поразрядное инвертирование знаковых чисел!!!
using System;
class Program
{
    static void Main()
    {
        sbyte sb = 9;
        sbyte nb = 3;
        Console.WriteLine("~sb = " + ~sb);
        Console.WriteLine("~sb+sb = " + (~sb + sb));
        Console.WriteLine("~nb = " + ~nb);
        Console.WriteLine("~nb+nb = " + (~nb + nb));
    }
}
```

Результат выполнения программы:

```
~sb = -10
~sb+sb = -1
~nb = -4
~nb+nb = -1
```

Поразрядный сдвиг влево $<<$ целочисленного аргумента знакового типа может не только изменить его абсолютное значение, но и, зачастую, изменяет его знак. Приводить примеры здесь нет необходимости. Гораздо важнее рассмотреть особенности выполнения традиционных арифметических операций над беззнаковыми и знаковыми операндами с ограниченным количеством разрядов.

Начнем с беззнаковых целочисленных типов. В результате выполнения следующего фрагмента программы:

```
byte b = 255, c = 1, d;
d = (byte)(b + c);
```

Значением переменной d будет 0. Обоснованность такого результата иллюстрирует следующее двоичное представление

$$\begin{array}{r} 11111111 = 255 \\ + \\ 00000001 = 1 \\ \hline 100000000 = 0 \end{array}$$

(ноль за счет отбрасывания левого разряда).

Теперь обратимся к операндам знаковых типов, например, типа **sbyte**.

Если просуммировать числа -1 (с поразрядным представлением 11111111) и 1 (с кодом 00000001), то получим девятиразрядное число с битовым представлением 100000000. Для внутреннего представления чисел типа **sbyte** отводится 8 разрядов. Девятиразрядное число в эти рамки не помещается, и левая (старшая) единица отбрасывается. Тем самым результатом суммирования становится код нуля 00000000. Все совершенно верно — выражение $(-1 + 1)$ должно быть равно нулю! Однако так правильно завершаются вычисления не при всех значениях целочисленных операндов.

За счет ограниченной разрядности внутреннего представления значений целых типов при вычислении выражений с целочисленными операндами существует опасность аварийного выхода результата за пределы разрядной сетки. Например, после выполнения следующего фрагмента программы:

```
sbyte x = 127, y = 127, z;  
z = (sbyte) (x + y);
```

значением переменной *z* будет -2 . В этом легко убедиться, представив выполнение операции суммирования в двоичном виде:

$$\begin{array}{r} 01111111=127 \\ + \\ 01111111=127 \\ \hline 11111110 = -2 \text{ (в дополнительном коде).} \end{array}$$

Примечание. В операторе

```
z = (sbyte)(x + y);
```

использована операция приведения типов (**sbyte**). При её отсутствии результат суммирования $x + y$ автоматически приводится к типу **int**. Попытка присвоить значение типа **int** переменной *z*, имеющей тип **sbyte**, воспринимается как ошибка, и компиляция завершается аварийно.

Приведенные иллюстрации переполнений разрядной сетки при арифметических операциях с восьмиразрядными целыми (типов **byte**, **sbyte**) могут быть распространены и на целые типы с большим количеством разрядов (типы с указанием разрядностей приведены в табл. 2.1).

Основным типом для представления целочисленных данных в C# является тип **int**. Для представления целочисленных значений типа **int** используются 32-разрядные участки памяти. Тем самым предельные значения для значения типа **int** таковы:

- положительные от 0 до $2^{31} - 1$;
- отрицательные от -1 до -2^{31} .

В следующей программе результаты умножений переменной типа **int** на саму себя выходят за пределы разрядной сетки.


```
// 03_06.cs - переполнение при целочисленных операндах
using System;
class Program
{
    static void Main()
    {
        int m = 1001;
        Console.WriteLine("m = " + m);
        Console.WriteLine("m = " + (m = m * m));
        Console.WriteLine("m = " + (m = m * m));
        Console.WriteLine("m = " + (m = m * m));
    }
}
```

В программе значение целочисленной переменной, вначале равной 1001_{10} , последовательно умножается само на себя.

Результат выполнения программы:

```
m = 1001
m = 1002001
m = -1016343263
m = 554036801
```

После первого умножения $m * m$ значением переменной m становится 1002001_{10} , после второго — результат выходит за разрядную сетку из 32 битов. Левые лишние разряды отбрасываются, однако оставшийся самый левый 32-й бит оказывается равным 1, и код воспринимается как представление отрицательного числа. После следующего умножения 32-й (крайний левый) бит оказывается равным 0, и арифметически неверный результат воспринимается как код положительного числа.

Особо отметим, что в обычном режиме исполняющая система никак не реагирует на выход результата за разрядную сетку, и программисту нужно самостоятельно следить за возможностью появления таких неверных результатов (для этого используют операции **checked**).

В рассмотренных программах с переменными типов **byte** и **sbyte** мы несколько раз применили операцию преобразования (иначе приведения) типов. Например, были использованы конструкции:

```
(byte)(bb&dd)
z = (sbyte)(x + y);
```

В следующей главе приведение типов будет рассмотрено подробно, а сейчас покажем его роль в некоторых выражениях с целочисленными операндами.

Поместим в программу операторы:

```
short dd = 15, nn = 24;
dd = (dd + nn)/dd;
```

При компиляции программы будет выведено сообщение об ошибке:

```
Cannot implicitly convert type 'int' to 'short'.
```

(Невозможно неявное преобразование типа **int** в **short**.)

Несмотря на то, что в операторах использованы переменные только одного типа **short**, в сообщении компилятора указано, что появилось значение типа **int**! Компилятор не ошибся — при вычислении выражений с целочисленными операндами, отличными от типа **long**, они автоматически приводятся к типу **int**. Поэтому результат вычисления $(dd + nn)/dd$ имеет тип **int**. Для значений типа **short** (см. табл. 2.1) выделяется два байта (16 разрядов), значение типа **int** занимает 4 байта. Попытка присвоить переменной **dd** с типом **short** значения типа **int** воспринимается компилятором как потенциальный источник ошибки за счет потери 16 старших разрядов числа. Именно поэтому выдано сообщение об ошибке.

Программист может «успокоить» компилятор, применив следующим образом операцию приведения типов:

```
dd = (short)((dd + nn)/dd);
```

При таком присваивании программист берет на себя ответственность за правильность вычислений.

Обратите внимание на необходимость дополнительных скобок. Если записать $(short)(dd + nn)/dd$, то в соответствии с рангами операций к типу **short** будет приведено значение $(dd + nn)$, а результат его деления на **dd** получит тип **int**.

Контрольные вопросы и задания

1. Перечислите группы (категории) операций языка C#.
2. Перечислите названия групп операций в порядке возрастания их приоритетов (рангов).
3. Знаки каких бинарных операций могут использоваться в составных операциях присваивания?
4. В чем отличия префиксных форм операций декремента и инкремента от постфиксных.
5. К каким операндам применимы операции $++$ и $--$?
6. К каким операндам применима операция $\%$?
7. К каким операндам применима операция $\wedge$?
8. В чем особенность операции деления целочисленных операндов?
9. Назовите правила выполнения операций $\%$.
10. Какому действию эквивалентен сдвиг влево разрядов битового представления целого числа?
11. Приведите дополнительный код отрицательного числа типа **sbyte**, модуль которого не превышает 127.
12. Объясните механизм возникновения переполнения при вычислениях с целочисленными операндами.

Глава 4

ВЫРАЖЕНИЯ С ОПЕРАНДАМИ БАЗОВЫХ ТИПОВ

4.1. Автоматическое и явное приведение арифметических типов

В предыдущей главе мы рассмотрели некоторые особенности кодирования значений целочисленных типов и особенности вычисления выражений с целыми операндами. Теперь уделим внимание вещественным типам.

В соответствии со стандартом ANSI/IEEE Std 754—1985 определены два основных формата представления в ЭВМ вещественных чисел:

- одинарная точность — четыре байта;
- двойная точность — восемь байтов.

В языке C# (точнее, в платформе .NET Framework) для этих двух представлений используются данные типов **float** и **double**. Возможности представления данных с помощью этих типов таковы:

float — мантисса числа 23 бита, т. е. 7 десятичных знаков, показатель степени (экспонента) 8 бит;

double — мантисса числа 52 бита, т. е. 15—16 десятичных знаков, показатель степени (экспонента) 11 бит.

Мантисса хранится в двоично-десятичном представлении, экспонента представлена в двоичном виде. Как и для знаковых целых типов, один бит в представлении вещественных чисел указывает знак.

О внутреннем представлении вещественных данных приходится думать только при нарушениях тех предельных значений, которые существуют для мантисс и экспонент. Поэтому сейчас на этом не будем останавливаться.

Вещественные данные (константы и переменные) могут использоваться в арифметических выражениях совместно и в разных сочетаниях с целочисленными данными. При этом выполняются автоматические преобразования типов. Правила допустимых (и выполняемых автоматически) преобразований иллюстрирует диаграмма, приведенная на рис. 4.1.

При использовании в одном выражении операндов разных, но совместимых типов все они автоматически приводятся к одному типу. При этом имеет место так называемое «расширяющее преобразование», т. е. преобразование выполняется к типу, имеющему больший диапазон значений по сравнению с диапазонами типов других операндов.

Например, для двух операндов с типами `int` и `long` будет выполнено приведение к типу `long`.

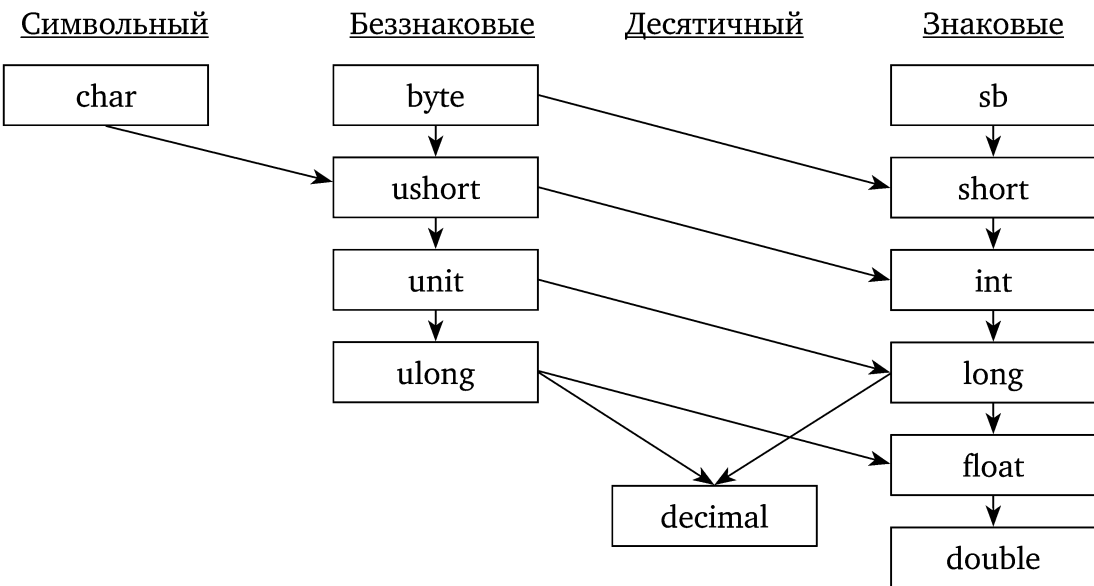


Рис. 4.1. Выполняемые автоматически преобразования арифметических типов

Для операции присваивания правила автоматического приведения типов требуют, чтобы тип операнда слева имел больший диапазон представления чисел, нежели тип правого операнда, или равный ему. В противном случае фиксируется ошибка компиляции.

Обратите внимание, что в тип `double` может быть преобразовано значение любого другого арифметического типа кроме `decimal`. В то же время значение типа `double` не может быть автоматически преобразовано ни к какому другому типу. Распространенная ошибка:

```
float z = 124.3; // недопустимо! - справа значение типа double
```

Такой инициализации компилятор не допускает, так как по умолчанию (без суффикса F) константа 124.3 имеет тип `double`.

Преобразования типов в сложных выражениях выполняются последовательно по мере выполнения операций. Поэтому достаточно разобрать правила преобразования для выражения с бинарной операцией. Алгоритм приведения типов в бинарном выражении можно записать такой последовательностью шагов:

- Если один операнд `decimal`, то второй приводится к типу `decimal`.
- Если один операнд `double` — второй приводится к типу `double`.
- Если один операнд `float`, то второй приводится к типу `float`.
- Если один операнд `ulong` — второй приводится к типу `ulong`.
- Если один операнд `long` — второй приводится к типу `long`.
- Если один операнд `uint`, а второй `sbyte` или `short` или `int`, то оба операнда приводятся к типу `long`.
- Если один операнд `uint`, то второй приводится к типу `uint`.
- Иначе оба операнда приводятся к типу `int`.

Исключение: Если один операнд **ulong**, а второй **sbyte**, или **short**, или **int**, или **long**, то фиксируется ошибка компиляции.

Важным является правило приведения обоих операндов к типу **int**, если в выражении нет операнда типов **decimal**, **double**, **float**, **ulong** или **long**. Об этом мы уже говорили в параграфе 3.6, приведя пример с операндами типа **short**.

В соответствии с этим правилом ошибочной будет последовательность операторов:

```
short dd, nn = 24;  
sbyte bb = -11;  
dd = nn * bb; //ошибка компиляции!
```

В данном примере при вычислении выражения $nn * bb$ оба операнда переводятся к типу **int**, и результат умножения — значение типа **int**. Присваивание недопустимо — автоматическое приведение типа **int** к типу **short** невозможно (см. рис. 4.1).

Для правильного выполнения присваивания необходимо, в этом случае, явное приведение типов.

В нашем примере правильным будет оператор:

```
dd = (short) (nn * bb);
```

Обратите внимание, что операция явного приведения типов имеет высокий приоритет, и бинарное выражение $(nn * bb)$ необходимо поместить в скобки.

4.2. Особые ситуации в арифметических выражениях

Однако и при допустимых преобразованиях существуют особые ситуации, которые необходимо предусматривать. Например, если присваивать переменной вещественного типа целое значение с очень большим количеством значащих цифр, то младшие разряды числа могут быть потеряны.

Пример:

```
float x = 12345678987654321L; // справа long
```

Переменная x получает значение $1.234568E+16$ — *потеряны младшие разряды*.

При делении вещественных чисел существует опасность переполнения, если порядок делимого существенно больше порядка делителя. В этом случае результат, независимо от конкретных величин мантисс делимого и делителя, воспринимается исполняющей системой как **бесконечность** (*Infinity*). Такое же значение получается при делении на нулевое значение.

Переполнение возможно и при других арифметических операциях. Например, в результате выполнения следующего фрагмента программы:

```
float x = 1E+24F;  
Console.WriteLine("x = " + (x *= x));
```

на консольный экран будет выведена строка:

x = бесконечность

Обратная переполнению ситуация — *потеря значимости* возникает, когда получаемое значение слишком мало и не может быть представлено с помощью того количества разрядов (битов), которые выделяются для значения соответствующего типа. В качестве примера рассмотрим следующий фрагмент программы:

```
float x = 1E-24F;  
Console.WriteLine("x = " + (x *= x));
```

Результат в этом случае нулевой:

x = 0

В случае, когда и делитель, и делимое равны нулю, результат неопределенный, и это фиксируется с помощью специального обозначения NaN (Not a Number — не число). При выполнении следующих операторов:

```
float x = 0.0f;  
Console.WriteLine("0/0 = " + x/0);
```

Результат в консольном экране будет таким:

0/0 = NaN

При решении вычислительных задач с помощью итерационных алгоритмов возникает задача оценки необходимого количества итераций. Например, если алгоритм приведен к виду

$$f_{i+1} = f_i + \Delta_i; \quad i = 1, 2, \dots$$

и Δ_i по какому-либо закону убывает с ростом i , то предельная точность достигается в том случае, когда Δ_i становится пренебрежительно малой величиной по сравнению с f_i . В этом случае говорят, что достигнут машинный ноль относительно величины f_i .

Рассмотрим такую последовательность операторов:

```
double x = 1000, y = 1E-08;  
Console.WriteLine("x + y =" + (x + (y *= y)));  
Console.WriteLine(" y= " + y);
```

Результат выполнения:

x + y = 1000 y = 1E-16

В данном примере прибавление значения 1E-16 к 1000 не изменило результата, и 1E-16 является машинным нулем относительно вели-

чины 1000. При необходимости можно ставить и решать задачу оценки максимального по абсолютной величине значения машинного нуля относительно заданной величины.

Если переменной типа **decimal** необходимо присвоить значение, отличное от целого, то его нужно представить в виде константы типа **decimal**. Целое значение можно присваивать десятичной переменной непосредственно. Следующий фрагмент программы (04_01.cs) иллюстрирует некоторые особенности выражений с операндом типа **decimal**. (Напомним, что десятичный литерал имеет суффикс **m** или **M**.)

```
static void Main()
{
    decimal p = 193, d = 0.15431m, e = 1.231E-10M;
    Console.WriteLine("p*d*e= " + (p * d * e));
    float bp = 193F, bd = 0.15431F, be = 1.231E-10F;
    Console.WriteLine("bp*bd*be= " + (bp * bd * be));
    Console.WriteLine("1.0/3.0= " + (1.0 / 3.0));
    Console.WriteLine("1.0m/3.0m= " + (1.0m / 3.0m));
}
```

Переменным p, d, e типа **decimal** и переменным bp, bd, bc типа **float** присвоены одинаковые значения. Результаты одинаковых выражений с операндами разных типов существенно различны по точности:

```
p*d*e= 0,000000003666143273  
bp*bd*be= 3,666143E-09  
1.0/3.0= 0,3333333333333333  
1.0m/3.0m= 0,33333333333333333333333333333333
```

4.3. Логический тип и логические выражения

К предопределенным (еще точнее — к базовым или простым) типам значений относится тип **bool**, используемый для представления логических (булевых) значений. Константами-литералами булева типа являются **true** (истина) и **false** (ложь).

Переменные типа **bool** не могут принимать значений, отличных от логических литералов. Им нельзя присваивать значений других типов, например, целочисленных. Логическое значение (типа **bool**) нельзя преобразовать ни в какое значение другого типа.

Объявление с инициализацией логической переменной:

```
bool really = true;
```

Из логических переменных и констант формируются логические (булевы) выражения. Для этого в языке С# имеются логические операции:

& — конъюнкция (логическое И);
| — дизъюнкция (логическое ИЛИ);
! — логическое отрицание;

$\wedge$ — исключающее ИЛИ.

Семантика этих операций известна из курса математической логики.

Кроме того в C# определены две условные (*conditional*) логические бинарные операции:

$\&\&$ — условная конъюнкция (условное И);

$||$ — условная дизъюнкция (условное ИЛИ).

В выражении $x\&\&y$ значение y не вычисляется, если x имеет значение **false**. В выражении $x||y$ значение y не вычисляется, если x равно **true**.

Кроме данных типа **bool** в логических выражениях часто используются отношения. *Отношение* — это два операнда, соединенные (или разделенные) знаком операции отношений:

$>$ — больше;

$>=$ — больше или равно;

$<$ — меньше;

$<=$ — меньше или равно;

$==$ — сравнение на равенство (равно);

$!=$ — сравнение на неравенство (не равно);

Отметим, что операции сравнения на равенство ($==$ и $!=$) имеют более низкий приоритет, нежели все остальные операции отношений.

Проверку принадлежности числового значения x интервалу (a, b) , где $a < b$, можно выполнить с помощью такого логического выражения:

$x < b \ \& \ a < x$

Последовательность вычислений можно показать с помощью скобок:

$(x < b) \& (a < x)$

Значением выражения будет **true**, если x принадлежит интервалу (a, b) .

Проверку истинности высказывания «значение x находится вне интервала (a, b) » позволяют выполнить логические выражения:

$x > b \ \wedge \ x < a;$

$x > b \ || \ x < a;$

$x > b \ || \ x < a.$

В третьем выражении использована условная дизъюнкция. Остановимся подробнее на особенностях условных логических операций.

Условные версии ($||$ и $\&\&$) бинарных логических операций ($\&$ и $|$) позволяют избежать вычисления значения второго (правого) операнда логического выражения, если значение левого операнда однозначно определяет значение всего выражения. Проверку принадлежности x числовому интервалу (a, b) можно записать так:

$x < b \ \&\& \ a < x$

Если $x < b$ равно **false**, то нет необходимости вычислять значение отношения $a < x$.

Обратите внимание, что знаки бинарных логических операций те же, что и знаки поразрядных операций конъюнкции (&) и дизъюнкции (|). То же самое относится и к знаку ^, который для целочисленных операндов обозначает операцию поразрядного исключающего ИЛИ. Как и в (уже упомянутом) случае применения знака + для обозначения операции конкатенации строк, здесь имеет место перегрузка операций. Это еще один пример полиморфизма.

Не лишним будет, в связи с перегрузкой операций, вспомнить, что символом в информатике называют знак с его смыслом. Встретив в выражениях знак &, компилятор анализирует контекст, и если обнаруживается, что справа и слева операнды типа **bool**, то знак & воспринимается как символ логической операции конъюнкции.

Следующая программа «проверяет» три вещественных переменных *x*, *y*, *z* — могут ли быть их значения длинами сторон треугольника.

```
// 04_02.cs - отношения и логические выражения
using System;
class Program
{
    static void Main()
    {
        double x = 19, y = 31, z = 23.8;
        bool res;
        res = x < y + z & y < x + z & z < x + y;
        Console.WriteLine("res = " + res);
    }
}
```

Результат выполнения программы:

```
res = True
```

В программе логической переменной *res* присваивается значение логического выражения в виде конъюнкции трех отношений. Для заданных значений переменных *x*, *y*, *z* значение результата **true** с помощью метода `Console.WriteLine()` выводится как `True`.

Обратите внимание на порядок вычисления использованного в программе логического выражения. С помощью круглых скобок последовательность выполнения операций можно указать явно таким образом:

```
((x <= (y + z)) & (y <= (x + z))) & (z <= (x + y))
```

Необходимости в таком применении скобок нет — в языке C# определены приоритеты (ранги) всех операций (см. табл. 3.1). В соответствии с этими приоритетами, первыми в нашем логическом выражении вычисляются значения (типа **double**) операндов отношений (т. е. выполняется операция сложения +). Затем последовательно слева направо вычисляются значения (типа **bool**) отношений, и к этим логическим значениям применяется слева направо операция & (конъюнкция).

Результат выполнения программы не изменится, если при вычислении логического выражения использовать условную конъюнкцию:

```
res = x < y + z && y < x + z && z < x + y;
```

Однако при получении значения **false** в любом из отношений, отношения, размещенные правее него, не вычисляются.

Применение условных логических операций особенно удобно в тех случаях, когда проверка истинности одного условия позволяет избежать аварийных ситуаций при вычислении второго условия.

4.4. Выражения с символьными операндами

Если символьное значение (символ или переменная типа **char**) используется в арифметическом выражении, то С# автоматически выполняет его преобразование из типа **char** в числовое значение его кода. То же справедливо и в случае, когда в выражении с арифметическими операциями участвуют несколько символьных переменных или констант.

В качестве иллюстрации сказанного рассмотрим следующую программу:

```
// 04_03.cs - выражения с символьными операндами

using System;
class Program
{
    static void Main()
    {
        char c = 'c', h = '\u0068', a = '\x61', r = '\u0072';
        Console.WriteLine("" + c + h + a + r);
        Console.WriteLine(c + h + a + r);
    }
}
```

Результат выполнения программы:

```
char
414
```

В программе символьные переменные *c* (десятичный код 99), *h* (десятичный код 104), *a* (десятичный код 97), *r* (десятичный код 114) представляют, соответственно, символы 'c', 'h', 'a', 'r'. При первом обращении к методу `WriteLine()` знак `+` играет роль операции конкатенации, аргумент (выражение) `"" + c + h + a + r` преобразуется к строковому виду и имеет значение строки со значением "char". Связано это с правилом, по которому в выражении *строка* + *символ* правый операнд приводится к типу строки, а затем выполняется конкатенация двух строк. Таким образом, вначале `"" + 'c'` превращается в строку "c", затем выражение "c" + 'h' преобразуется в "ch" и т. д. При втором обращении

к методу `WriteLine()` аргумент `s + h + a + r` воспринимается как арифметическое выражение. Знак `+` играет роль операции сложения. Коды символов обрабатываются как целые числа, их сумма равна 414. Это число автоматически преобразуется в строку "414" (так как аргумент метода `WriteLine()` должен иметь строковый тип), и эта строка выводится на консоль.

Как отмечено при объяснении правил вычисления значения аргумента метода `WriteLine()`, результат выражений *строка+символ* и *символ+строка* — это конкатенация строки и строкового представления символа. В следующей программе конкатенация строки и символа выполняется вне метода `WriteLine`.

```
// 04_04.cs - строки и символы
using System;
class Program
{
    static void Main()
    {
        char simb = 'b'; // десятичный код = 98
        string line = "simb = " + simb;
        Console.WriteLine(line);
        line = simb + " = simb";
        Console.WriteLine(line);
        line = simb + simb + " = 2 simb";
        Console.WriteLine(line);
    }
}
```

Результат выполнения программы:

```
simb = b
b = simb
196 = 2 simb
```

Обратите внимание на выражение

```
simb + simb + " = 2 simb"
```

В соответствии с правилами ассоциации для операции `+` оно вычисляется так: $(simb + simb) + " = 2 simb"$. В скобках символы «ведут себя» как целочисленные значения, и их сумма 196 затем преобразуется в строку.

Хотя мы уже применяли строки в виде строк-литералов и в примерах определяли строку как переменную типа **string**, но более подробное рассмотрение строк и их взаимоотношений с другими типами языка C# необходимо перенести дальше в раздел, специально посвященный строкам.

Унарные операции `++` и `--` изменяют на 1 значение кода символьной переменной, однако она не превращается в целочисленную величину, а сохраняет тип **char**. В то же время суммирование символьной переменной с единицей приводит к получению целочисленного значения. Сложение и вычитание символьных переменных также приводят

к соответствующим операциям над их кодами с формированием целочисленных результатов. Следующая программа иллюстрирует приведенные правила.

```
// 04_05.cs - выражения с символами и строками
using System;
class Program
{
    static void Main()
    {
        char cOld = 'a', cNew = cOld;
        Console.Write("В последовательности ");
        Console.Write(++cNew);    // выводит 'b'
        Console.Write(++cNew);    // выводит 'c'
        Console.Write(++cNew + " "); // выводит 'd'
        Console.WriteLine(cNew - cOld + " буквы");
    }
}
```

Результат выполнения программы:

В последовательности bcd 3 буквы

Чтобы получить символ с нужным числовым значением кода, нужно применить операции явного приведения типов.

Например, так:

```
char ch = (char)94; // значение ch - символ '^'
```

Для обратного преобразования (из **char**, например, в **uint**) достаточно автоматического приведения типов:

```
uint cod = ch; // значением cod будет 94
```

4.5. Тернарная (условная) операция

В отличие от классических арифметических и логических операций, унаследованных языками программирования из математики (арифметики и алгебры), условная операция требует трех операндов. В выражении с условной операцией используются два размещенных не подряд символа '?' и ':'. Они разделяют (или соединяют) три операнда:

операнд_1 ? операнд_2 : операнд_3

Операнд_1 — логическое выражение; *операнд_2* и *операнд_3* — выражения одного типа или выражения, которые могут быть неявно (автоматически) приведены к одному типу.

При выполнении выражения с тернарной операцией первым вычисляется значение *операнда_1*. Если оно истинно (**true**), то вычисляется значение *операнда_2*, которое становится результатом. Если *операнд_1* равен **false**, то вычисляется значение *операнда_3*, и оно становится

результатом всего выражения с тернарной операцией. Классический пример:

```
x < 0 ? -x : x.
```

Если x — переменная арифметического типа, то результат выполнения операции — абсолютное значение x .

Ранг операции $?$: очень низок, но она имеет приоритет перед операциями присваивания. Поэтому с операцией присваивания выражения с тернарной операцией можно достаточно часто использовать без скобок. Например, можно так вычислить абсолютное значение разности кодов двух символов, не обращаясь к таблице кодов:

```
int norma = 'f' > 'z' ? 'f' - 'z' : 'z' - 'f';
```

Для наглядности операнды тернарной операции и всё условное выражение удобно заключать в скобки. В качестве примера рассмотрим следующее выражение с переменными арифметического типа:

```
res = (x < y) ? ((y < z) ? z : y) : (x < z) ? z : x;
```

Переменная res получает наибольшее из значений переменных x , y , z .

Приведенное выражение будет правильно вычисляться и при отсутствии скобок:

```
res = x < y ? y < z ? z : y : x < z ? z : x;
```

Однако для наглядности и надежности скобки лучше применить.

Тип результата тернарной операции определяется типом операндов, разделенных двоеточием. Например, если нужно выбрать из двух символов тот, который имеет большее значение кода, то можно записать такое выражение:

```
char ch = 'g' > 'e' ? 'g' : 'e'
```

Приведенные примеры и правила иллюстрирует следующая программа:

```
// 04_06.cs - выражения с тернарной операцией
using System;
class Program
{
    static void Main()
    {
        char c = 'a', h = 'e', ch;
        int norma = c > h ? c - h : h - c;
        Console.WriteLine("|c-h| = " + norma);
        ch = c > h ? c : h;
        Console.WriteLine("ch = " + ch);
        double x = 4, y = 7, z = 5, res;
        res = (x < y) ? ((y < z) ? z : y) : ((x < z) ? z : x);
        Console.WriteLine("res = " + res);
    }
}
```

```
        res = x < y ? y < z ? z : y : x < z ? z : x;  
        Console.WriteLine("res = " + res);  
    }  
}
```

Результат выполнения программы:

```
|c-h| = 4  
ch = e  
res = 7  
res = 7
```

Контрольные вопросы и задания

1. Что такое автоматическое приведение (преобразование) типов?
2. К каким типам может быть автоматически приведено значение типа **int**?
3. Что такое «расширяющее преобразование» типов?
4. При каких сочетаниях типов автоматическое приведение типов невозможно?
5. В каких случаях два операнда разных типов приводятся к типу **int**?
6. Назовите особые ситуации, которые могут возникнуть при вычислении арифметических выражений.
7. Какие значения может принимать переменная типа **bool**?
8. Назовите условные логические бинарные операции языка C#.
9. Что такое отношение?
10. Каковы ранги операций отношений?
11. В выражениях с какими операциями могут использоваться символьные данные?
12. Каков результат применения операции ++ к переменной типа **char**?
13. Какой тип имеет результат суммирования переменной символьного типа с единицей?
14. Сколько операндов должно входить в выражение с операцией «?:»?
15. Какой тип должен иметь первый (левый) операнд операции «?:»?
16. Каков приоритет (ранг) операции «?:» по отношению к операции присваивания?

Глава 5

ТИПЫ C# И ТИПЫ ПЛАТФОРМЫ .NET FRAMEWORK

5.1. Платформа .NET Framework и спецификация CTS

Язык C# и средства его поддержки в настоящее время связаны с платформой .NET Framework, разработанной Microsoft. Названная платформа (см. [4,6,7,15]) включает: общезыковую исполняющую среду (CLR — *Common Language Runtime*) и библиотеку классов (FCL — *Framework Class Library*).

Следует заметить, что язык C# является только одним из многих языков, на которых можно писать программы, работающие на платформе .NET Framework.

При использовании платформы .NET Framework подготовленный программистом код (текст программы, например, на C#) вначале транслируется в код на общем для всех исходных языков промежуточном языке (CIL — *Common Intermediate Language*, иногда сокращенно IL — *Intermediate Language*).

Последовательность процессорных команд появляется позже — во время исполнения средой CLR команд CIL. Этот временной «разрыв» между трансляцией исходного текста и появлением процессорного кода не случаен. Код на промежуточном «универсальном» языке CIL может исполняться на процессорах с разной архитектурой (PowerPC, x86, IA64, Alpha и др.). Единственное, но обязательное требование — на компьютере, где выполняется приложение на языке CIL, должна быть развернута среда .NET Framework, т. е. установлены CLR и FCL, соответствующие стандартам ECMA.

Платформа .NET Framework позволяет разрабатывать приложение, используя одновременно несколько разных языков программирования. Такая возможность обеспечена **общей системой типов** (CTS — *Common Type System*), которую используют все языки, ориентированные на CLR. Так как наша книга посвящена только одному языку программирования, то все ограничения, возникающие при использовании в одном приложении частей, написанных на разных языках, мы рассматривать не будем. Достаточно отметить, что для обеспечения межъязыкового взаимодействия необходимо придерживаться обще-

языковой спецификации (CLS — *Common Language Specification*), предложенной Microsoft. Эта спецификация ограничивает все разнообразие типов того или иного языка программирования тем подмножеством, которое присутствует одновременно во всех языках. Любой из типов, соответствующих спецификации CLS, присутствует в каждом из языков и «понятен» в каждой части многоязыковой программы.

Спецификация CTS описывает правила определения типов и особенности их поведения. При изучении C# мы будем подробно рассматривать правила определения типов именно на языке C#. Сейчас очень кратко остановимся только на основных требованиях CTS.

Во-первых, CTS утверждает, что каждый тип — это класс (или структура), который может включать нуль или более членов. Членами классов (структур) могут быть следующие сущности[6].

Поле — переменная, определяющая состояние класса или объекта. Поле идентифицируется именем и имеет конкретный тип.

Метод — функция, выполняющая действие над классом или объектом. Метод идентифицируется сигнатурой и для него определен тип возвращаемого значения.

Свойство — средство для получения или задания значения некоторой характеристики, зависящей от состояния объекта. Для вызывающей стороны свойство синтаксически неотличимо от поля. В реализации типа свойство представлено одним или двумя методами с фиксированными (внешне невидимыми) именами.

Событие — средство для уведомления адресатов (других объектов, классов, методов) об изменении состояния объекта или о воздействии на него.

Спецификация CTS описывает правила видимости типов и доступа к их членам, правила наследования типов, возможности виртуальных функций и т. д.

Следующее правило CTS состоит в требовании для всех типов иметь единый базовый класс. В соответствии с этим требованием все типы являются производными от класса System.Object. Происхождение всех типов от базового класса System.Object гарантирует для каждого типа присутствие заранее определенной минимальной функциональности. Эта функциональность предусматривает для каждого экземпляра (для объекта) любого типа возможности:

- сравнения с другим экземпляром;
- получения хэш-кода;
- определения (идентификации) типа;
- копирования;
- формирования строкового представления.

Изучая программирование на языке C#, мы знакомимся не с CTS, а с ее «проекцией» на конкретный язык программирования (на C#). В конкретном языке для упрощения вводят «надстройки» над CTS, обеспечивающие более высокий уровень абстракции. В языке C# именно так появляются индексаторы, делегаты, массивы, λ -выражения, кор-

тежи и другие конструкции, которые будут нами подробно рассмотрены.

Кроме того, для упрощения записи программ развернутое обозначения типов, принятых в CTS, в конкретных языках разрешено заменять традиционными для языков программирования более короткими названиями: **int**, **char** и т. п. Именно такие типы языка С# мы рассматривали в предыдущих главах, не отмечая тот факт, что каждый из этих типов просто-напросто представляет в программе на С# один из типов CTS, имеющих в CTS более громоздкие обозначения.

5.2. Простые (базовые) типы С# как типы CTS

Система типов С# построена на основе классов, точнее, типов, имеющих общий базовый класс **object**. Простые (фундаментальные, базовые) типы, такие как **int** и **char**, на самом деле являются не самостоятельными типами, а представляют собой обозначения (условные названия) системных типов, представляемых платформой .NET Framework в виде соответствующих структур (см. табл. 5.1).

Таблица 5.1

Тип языка С# и тип CTS	Диапазон значений	Соответствие CLS
bool System.Boolean	true или false	есть
byte System.Byte	от 0 до 255	есть
sbyte System.SByte	от -128 до 127	нет
char System.Char	от U+0000 до U+ffff	есть
decimal System.Decimal	(от $-7.9 \cdot 10^{28}$ до $7.9 \cdot 10^{28}$)/(10 от 0 до 28)	есть
double System.Double	от $\pm 5.0 \cdot 10^{-324}$ до $\pm 1.7 \cdot 10^{308}$	есть
float System.Single	от $-3.4 \cdot 10^{38}$ до $+3.4 \cdot 10^{38}$	есть
int System.Int32	от -2 147 483 648 до 2 147 483 647	есть
uint System.UInt32	от 0 до 4 294 967 295	нет
long System.Int64	от -9 223 372 036 854 775 808 до 9 223 372 036 854 775 807	есть
ulong System.UInt64	от 0 до 18 446 744 073 709 551 615	нет

Тип языка C# и тип CTS	Диапазон значений	Соответ- ствие CLS
object System.Object	любой тип данных	есть
short System.Int16	от -32 768 до 32 767	есть
ushort System.UInt16	от 0 до 65 535	нет
string System.String	ограничен системными возможностями	есть

Итак, все predefined типы языка C# представляют в программах соответствующие им структуры из пространства имен System библиотеки .NET Framework. Казалось бы, что эквивалентность традиционных названий типов (например, **int**, **char**, **string**...) обозначениям типов в CTS (System.Int32, System.Char, System.String...) делает ненужным знакомство с соответствующими структурами из пространства System. Однако это не так. Имена системных типов используются в названиях средств библиотеки классов .NET Framework. Кроме того, при идентификации типа объекта, например, с помощью метода GetType(), для обозначения типа используется его системное имя, а не имя, принятое в C#.

Отображенное в табл. 5.1 соответствие типов языка C# и системных типов .NET Framework приходится учитывать в тех случаях, когда код на C# нужно использовать в разноязычных приложениях. Например, для целочисленных значений CTS предусматривает применение типов Byte, Int16, Int32, Int64. Тем самым для разноязычных приложений недопустимы беззнаковые целочисленные типы (**uint**, **ulong**, **ushort**) и тип **sbyte**.

Несмотря на то, что в C# введены сокращенные обозначения простых типов, нет ограничения и на применение системных имен. Например, следующие два объявления целочисленной переменной с именем cluster эквивалентны:

```
int cluster = 33;  
System.Int32 cluster = 33;
```

Кроме того, переменную простого типа можно объявить, используя и формат объявления объекта :

```
имя_типа имя_объекта = new имя_типа();
```

В этом случае в качестве имени типа используют или традиционное для C#, или системное обозначение типа. Имя_объекта — выбранный программистом идентификатор, **new** — специальная операция вызова

конструктора того типа, который соответствует типу переменной. С механизмом конструкторов нам еще придется подробно знакомиться. Сейчас достаточно сказать, что назначение конструктора — разместить в памяти и инициализировать новый экземпляр объекта.

Пример двух эквивалентных объявлений с операцией **new**:

```
double radix = new double();  
System.Double radix = new System.Double();
```

В каждом из этих случаев объявлена вещественная переменная с именем **radix** типа **double**, которой с помощью конструктора выделяется участок памяти и присваивается предусмотренное по умолчанию нулевое значение.

Происхождение типов от общего базового класса **System.Object** позволяет применять к каждой переменной и константе следующие методы (перечислены не все):

- **string ToString ()** — возвращает строку, содержащую символьное представление того объекта, к которому этот метод применен;
- **System.Type GetType ()** — возвращает системный тип того объекта, к которому применен метод;
- **bool Equals (object obj)** — проверяет эквивалентность объекта параметра и объекта, к которому этот метод применен.

Прежде чем привести примеры применения перечисленных методов, напомним, что методы могут принадлежать классу или структуре (в этом случае они должны быть статическими), а могут относиться к конкретным объектам (экземплярам) класса или структуры (нестатические). В первом случае для обращения к методу (статическому) используется выражение

```
имя_типа.имя_метода (аргументы)
```

Примером служит обращение **Console.WriteLine()**, где **Console** — имя библиотечного класса, представляющего консольный поток. Во втором случае для обращения к методу обязательно должен быть определен объект (экземпляр типа), и вызов осуществляет выражение

```
имя_ссылки_на_объект.имя_метода (аргументы)
```

Напомним синтаксис обращения к методам, приведем следующую программу, в которой к переменным базового типа значений применяются методы, унаследованные из класса **object**.

```
// 05_01.cs - простые типы как наследники Object  
using System;  
class Program  
{  
    static void Main()  
    {  
        long row = 18L;  
    }  
}
```

```

        System.Int64 col = 423L;
        Console.WriteLine("row.GetType()= " + row.GetType());
        Console.WriteLine("row.Equals(col)= " + row.Equals(col));
        Console.WriteLine("row.ToString() + col.ToString()=" +
            row.ToString() + col.ToString());
        Console.WriteLine("row + col = " + (row + col));
    }
}

```

В программе переменные `row` и `col` определены с помощью разных обозначений одного и того же типа **long**. Значения указанные переменные получают за счет инициализации. Результаты выполнения программы:

```

row.GetType()= System.Int64
row.Equals(col)= False
row.ToString() + col.ToString()=18423
row + col = 441

```

Как уже упомянуто, метод `System.GetType()` возвращает название типа объекта, принятое в .NET Framework, а не в языке C#. Именно поэтому в результатах для `row` выведен тип `System.Int64`, а не **long**. При сравнении переменных с помощью выражения `row.Equals(col)` сравниваются их значения и в примере возвращается логическое значение `False`. Значением выражения `row.ToString() + col.ToString()` является конкатенация строковых представлений значений переменных `row` и `col`. Для сравнения, в следующем обращении к методу `WriteLine()` в скобки заключено выражение `(row + col)`, и в этом случае в скобках выполняется не конкатенация строк, а арифметическое суммирование значений переменных.

5.3. Специфические методы и поля простых типов

Кроме методов, унаследованных от общего базового класса **object**, каждый простой тип имеет набор собственных методов и свойств. Рассмотрим те из них, которые являются достаточно общими и чаще всего применяются в программах.

Метод, без которого трудно обойтись в реальных программах, это метод `Parse()`, назначение которого — преобразовать текстовую строку в числовое или логическое значение. Полезность этого метода связана с тем фактом, что при общении с программой пользователь обычно вводит числовые значения (например, с клавиатуры) в виде последовательности символов, а в программе они должны быть представлены в виде машинных кодов соответствующих чисел.

Задача перекодирования символьной строки в код числового значения, во-первых, трудоемка для программирования, а во-вторых, не каждая последовательность символов может быть преобразована в числовое значение. Метод `Parse()`, определенный в каждом классе

арифметических типов (Int32, Double...), позволяет автоматизировать решение задачи распознавания «правильных» записей числовых значений и задачи их преобразования в числа. Метод Parse() убирает из строки лишние пробелы слева и справа и незначащие левые нули.

Следующая программа иллюстрирует применение метода к строковым литералам, представляющим числовые значения:

```
// 05_02.cs - метод Parse()
using System;
class Program
{
    static void Main()
    {
        long row = long.Parse("18");
        long col = System.Int64.Parse(" 000123 ");
        Console.WriteLine("row + col = " + (row + col));
        double x = double.Parse(" -000,314159e+1");
        Console.WriteLine("x = " + x);
        Console.WriteLine("Double.Parse(\" 0011 \") = " +
            Double.Parse(" 0011 "));
    }
}
```

Обратите внимание, что метод Parse() является методом класса (методом статическим) и поэтому в его вызове нужно указать имя того класса, из которого он взят. Имя класса может быть или системным (в нашей программе System.Int64 и System.Double), или принятым в языке C# (**long**, **double**).

Результат выполнения программы:

```
row + col = 141
x = -3,14159
Double.Parse(" 0011 ") = 11
```

В программе строки (строковые константы–литералы), использованные в качестве аргументов метода Parse(), содержат ведущие и завершающие пробелы, а также незначащие левые нули. Такие отклонения в строковых (символьных) представлениях чисел допустимы. Следует обратить внимание на запятую, отделяющую дробную часть числа от целой в символьном представлении. Это требование локализации, предполагающей, что в России используются не американские, а европейские правила записи вещественных чисел.

Примечание. В текстах программ на C# для отделения целой части вещественного числа от ее дробной части используется точка. При вводе и выводе, т. е. во внешнем символьном представлении вещественных чисел, знак, разделяющий дробную и целую части, определяется локализацией исполняющей системы. Локализация может быть изменена настройкой.

При неверной записи числового или логического значения в аргументе метода `Parse()` генерируется исключение с названием `System.FormatException`. Так как мы еще не рассматривали механизм исключений, то примеры неудачных попыток применять метод `Parse()` с такими ошибками в записи чисел приводить преждевременно.

В `.NET Framework 2.0` появился еще один метод для преобразования строкового представления значения (арифметического, символьного, логического) в значение соответствующего простого типа. Этот метод перегружен и его варианты есть в разных классах. Например, в класс целых чисел типа `int` входит вариант этого метода с таким заголовком:

```
static public bool TryParse (string, out int);
```

Метод анализирует строку, представленную первым параметром, и если ее удастся преобразовать в целочисленное значение, то это значение передается в точку вызова с помощью аргумента, заменяющего второй параметр. Кроме того, метод имеет возвращаемое значение логического типа. При успешном преобразовании возвращается значение **true**. При неверном представлении в строке-аргументе значения соответствующего типа (того, для которого метод вызван) метод возвращает значение **false**.

Принципиальное отличие метода `TryParse()` от метода `Parse()` — отсутствие генерации исключений при неверных преобразованиях. Поэтому его можно использовать, не применяя механизма обработки исключений.

Чтобы привести пример, в котором будут показаны некоторые возможности метода `TryParse()`, нам придется несколько расширить тот арсенал средств, которыми мы до сих пор пользуемся в примерах программ.

Начнем с модификатора **out**, который используется в спецификации второго параметра метода `TryParse()`. Таким модификатором снабжается параметр, с помощью которого метод возвращает в точку вызова некоторое значение. Аргумент, который будет подставлен на место этого параметра, должен быть пригоден для получения возвращаемого значения, и всегда снабжается в обращении к методу модификатором **out**. Таким аргументом может быть имя переменной.

Наиболее часто метод `TryParse()` применяется для анализа и преобразования входных данных, вводимых пользователем с клавиатуры. Как мы уже показывали на примерах, последовательность символов, набранную на клавиатуре, можно «прочитать» с помощью метода

```
static public string Console.ReadLine();
```

Метод принадлежит классу `Console`. У метода `ReadLine()` параметры отсутствуют, он возвращает строковое значение.

Метод «срабатывает» как только пользователь нажимает на клавиатуре клавишу ENTER. В точку вызова метода `ReadLine()` возвращает строку символов, набранную на клавиатуре. Обращение к методу `ReadLine()` можно использовать в качестве первого аргумента метода `TryParse()`. Вторым аргументом должна быть переменная того типа, значение которого мы планируем «прочитать» из входной строки.

Теперь, договорившись о новых средствах, приведем программу, которая читает из входного потока консоли (от клавиатуры) последовательность (строку) символов и анализирует ее следующим образом. Если введено логическое значение (**true** или **false**), то для **true** нужно вывести символ '1', а для **false** — символ '0'. Если введенная последовательность символов не является изображением логического значения, то вывести знак '?'.

```
// 05_03.cs - Метод TryParse().
using System;
class Program
{
    static void Main()
    {
        bool bb;
        char res;
        Console.Write("Введите логическое значение: ");
        res = (bool.TryParse(Console.ReadLine(), out bb) == true) ?
            bb ? '1' : '0' : '?';
        Console.WriteLine("res ==> " + res);
    }
}
```

В программе используется статический метод `TryParse()` класса **bool**. Первый аргумент — обращение к методу `ReadLine()` класса `Console`. Второй аргумент — переменная булевого типа с именем `bb`. Обращение к `TryParse()` — это первый операнд тернарной условной операции `?:`. Если возвращаемое методом значение равно **true**, то переменная `bb` получила одно из двух логических значений. В зависимости от конкретного значения `bb` символьной переменной `res` присваивается '1' или '0'. Если метод `bool.TryParse()` возвращает значение **false**, то переменная `res` получает значение '?'.

Следующие результаты трех выполнений программы дополняют приведенные объяснения:

```
Введите логическое значение: true<ENTER>
res ==> 1
Введите логическое значение: false<ENTER>
res ==> 0
Введите логическое значение: истина<ENTER>
res ==>?
```

В каждый из типов, представляющих в C# простые (базовые) типы, входят несколько открытых статических константных полей, позволя-

ющих оценить предельные значения переменных и констант соответствующего типа.

В целых типах таких полей два:

- `MaxValue` — максимальное допустимое значение (константа);
- `MinValue` — минимальное допустимое значение (константа).

В вещественных типах, кроме названных полей, присутствуют следующие статические константные поля:

- `Epsilon` — наименьшее отличное от нуля числовое значение, которое может принимать переменная заданного типа;
- `NaN` — представление значения, которое не является числом, например, результат выполнения операции при делении нуля на ноль;
- `NegativeInfinity` — представление отрицательной бесконечности, например, при делении отрицательного числа на ноль;
- `PositiveInfinity` — представление положительной бесконечности, например, при делении положительного числа на ноль.

В следующей программе выводятся значения перечисленных констант, определенных для типов **int** и **double**.

```
// 05_04.cs - Константы и предельные значения
using static System.Console;
class Program
{
    static void Main()
    {
        WriteLine("int.MinValue = " + int.MinValue);
        WriteLine("int.MaxValue = " + int.MaxValue);
        WriteLine("double.MinValue = " + double.MinValue);
        WriteLine("double.MaxValue = " + double.MaxValue);
        WriteLine("double.Epsilon = " + double.Epsilon);
        WriteLine("double.NaN = " + double.NaN);
        WriteLine("double.PositiveInfinity = " +
            double.PositiveInfinity);
        WriteLine("double.NegativeInfinity = " +
            double.NegativeInfinity);
        double zero = 0.0, value = 2.7172;
        WriteLine("value/zero = " + value / zero);
        WriteLine("0.0/zero = " + 0.0 / zero);
    }
}
```

Результаты выполнения:

```
int.MinValue = -2147483648
int.MaxValue = 2147483647
double.MinValue = -1,79769313486232E+308
double.MaxValue = 1,79769313486232E+308
double.Epsilon = 4,94065645841247E-324
double.NaN = NaN
double.PositiveInfinity = бесконечность
double.NegativeInfinity = -бесконечность
value/zero = бесконечность
0.0/zero = NaN
```


Контрольные вопросы и задания

1. Назовите две основные части платформы .NET Framework.
2. Что такое CIL (Common Intermediate Language)?
3. Укажите назначение общезыковой спецификации CLS (Common Language Specification)?
4. Какие члены могут присутствовать в классе в соответствии с требованиями общей системы типов CTS?
5. Какую функциональность обеспечивает класс **object** объектам всех типов языка C#?
6. Приведите названия типов CTS, которые представлены в языке C# базовыми типами, например, **double**.
7. Какие базовые типы языка C# не соответствуют CLS?
8. Назовите три метода, унаследованные любым типом языка C# от базового класса **object**?
9. Объясните возможности и ограничения метода Parse().
10. Объясните возможности метода TryParse().
11. Назовите члены базовых типов, позволяющие оценивать их предельные значения.

Глава 6

ОПЕРАТОРЫ

6.1. Общие сведения об операторах

Операторы определяют действия и порядок выполнения действий в программе.

В стандарте языка приведена следующая общая классификация операторов:

- помеченный (*labeled-statement*);
- декларирующий (*declaration-statement*);
- встроенный (*embedded-statement*).

Обратим внимание, что к операторам в С# отнесли объявления (декларации), их частный случай — объявления переменных.

Стандарт относит к встроенным следующие операторы:

- блок (*block*);
- пустой оператор (*empty statement*);
- оператор-выражение (*expression statement*);
- оператор выбора (*selection statement*);
- оператор цикла (*iteration statement*);
- оператор перехода (*jump statement*);
- оператор контроля за исключением (*try statement*);
- оператор организации очистки ресурсов (*using statement*);
- оператор контроля за переполнением (*checked statement*);
- отмена контроля за переполнением (*unchecked statement*);
- оператор блокировки потока (*lock statement*);
- оператор итерации по коллекции (*yield statement*).

В этой главе мы рассмотрим те операторы, для понимания которых не требуется изучения механизмов, отличающих язык С# от его предшественников. Кроме того, не будут рассматриваться операторы, выделенные выше курсивом.

Каждый оператор языка С#, кроме блока, заканчивается разделителем «точка с запятой». Любое выражение, после которого поставлен символ «точка с запятой», воспринимается компилятором как отдельный оператор (исключение составляют выражения, входящие в заголовок цикла **for**).

Часто оператор-выражение служит для вызова функции (метода), не возвращающей в точку вызова никакого значения. Еще чаще оператор-выражение — это не вызов функции, а выражение с операцией при-

сваивания. Обратите внимание, что в языке C# отсутствует отдельный оператор присваивания. Оператор присваивания всего-навсего является частным случаем оператора-выражения (с операцией присваивания).

Специальным случаем оператора служит пустой оператор. Он представляется символом «точка с запятой», перед которым нет никакого выражения или не завершенного разделителем «точка с запятой» оператора. Пустой оператор не предусматривает выполнения никаких действий. Он используется там, где синтаксис языка разрешает присутствие оператора, а по смыслу программы никакие действия не должны выполняться. Пустой оператор иногда используется в качестве тела цикла, когда все циклически выполняемые действия определены в заголовке цикла.

6.2. Метки и оператор безусловного перехода

Перед каждым оператором может быть помещена метка, отделяемая от оператора двоеточием. В качестве метки используется выбранный программистом уникальный идентификатор. Пример помеченного оператора:

```
ABC: x = 4 + x * 3;
```

Как уже сказано, объявления, после которых помещен символ «точка с запятой», считаются операторами. Поэтому перед ними также могут помещаться метки:

```
metka: int z = 0, d = 4; // Метка перед объявлением
```

С помощью пустого оператора (перед которым имеет право стоять метка) метки можно размещать во всех точках программы, где синтаксис разрешает использовать операторы.

Говоря о помеченных операторах и метках, уместно ввести оператор безусловного перехода:

```
goto метка;
```

С помощью этого оператора обеспечивается безусловный переход к тому месту в программе, где размещена метка, использованная после служебного слова **goto**. Например, оператор

```
goto ABC;
```

передаст управление приведенному выше оператору ABC: $x = 4 + x * 3$;

В современном программировании оператор безусловного перехода используется достаточно редко, но в некоторых случаях он может оказаться весьма полезным.

Переход к метке возможен в общем случае не из всех точек программы. Важно, где размещена метка, и где находится оператор пере-

хода. В связи с существующими ограничениями на выполнение переходов нужно познакомиться с понятием блока.

Блок — заключенная в фигурные скобки последовательность операторов. Среди этих операторов могут присутствовать операторы объявлений. В блоке локализованы все объявленные в нем объекты, в том числе и метки. Синтаксически блок является отдельным оператором. Однако блок не требуется завершать точкой с запятой. Для блока ограничителем служит закрывающая фигурная скобка. Внутри блока каждый оператор должен оканчиваться точкой с запятой. Примеры блоков:

```
{int a; char b = '0'; a = (int)b;}  
{func(z + 1.0, 22); e = 4 * x - 1;}
```

Говоря о блоках, нужно понимать правила определения области действия имен и ограничения видимости объектов. Разрешено вложение блоков, причем на глубину вложения синтаксис не накладывает ограничений. О входе в блок стоит сказать уже сейчас — запрещено извне переходить к внутренним операторам блока. Тем самым метки внутри блока недостижимы для оператора перехода, размещенного вне блока. В то же время, для оператора перехода, размещенного внутри блока, разрешен переход к метке вне блока.

6.3. Условный оператор

К операторам выбора, называемым операторами ветвлений, относят: условный оператор (**if...else**) и переключатель (**switch**). Каждый из них служит для выбора «пути» выполнения программы (о переключателе см. параграф 6.6).

Синтаксис условного оператора:

```
if (логическое выражение) оператор_1  
else оператор_2
```

Логическое выражение иногда называют проверяемым условием. Если логическое выражение равно **true**, выполняется *оператор_1*. В противном случае, когда выражение равно **false**, выполняется *оператор_2*. В качестве операторов, входящих в условный оператор, нельзя использовать объявления. Однако здесь могут быть блоки, и в них объявления допустимы. Примеры:

```
if (x > 0) {double x = -4; f(x * 2);}  
else {int i = 2; double x = i * i; f(x);}
```

При использовании блоков нельзя забывать о локализации определяемых в блоке объектов. Например, ошибочна будет такая конструкция:

```
if (j > 0) {int i; i = 2 * j;} else i = -j;
```

Здесь предполагается, что переменная *j* определена до условного оператора и имеет конкретное значение. Ошибка в том, что перемен-

ная *i* локализована в блоке и не существует вне блока, т. е. не может использоваться в операторах после **else**.

Допустима сокращенная форма условного оператора, в которой отсутствует **else** и *оператор_2*. В этом случае при ложности проверяемого условия никакие действия не выполняются. Пример:

```
if (a < 0) a = -a;
```

Оператор_1 и *оператор_2* могут быть условными, что позволяет организовать цепочку проверок условий любой глубины вложенности. В этих цепочках каждый из условных операторов (после проверяемого условия и после **else**) может быть как полным условным, так и иметь сокращенную форму записи. При этом могут быть допущены ошибки неоднозначного сопоставления **if** и **else**. Синтаксис языка предполагает, что при вложениях условных операторов каждое **else** соответствует ближайшему к нему предшествующему **if**.

В качестве примера вложения условных операторов приведем фрагмент программы, в котором переменной *result* необходимо присвоить максимальное из трех значений переменных *x*, *y*, *z* (объявление и инициализация переменных опущены).

```
if (x < y)
    if (y < z) result = z;
    else result = y;
else
    if (x < z) result = z;
    else result = x;
```

В тексте соответствие **if** и **else** показано с помощью отступов.

6.4. Операторы цикла

Операторы цикла организуют многократное исполнение операторов тела цикла. В языке С# определены четыре разных оператора цикла.

Цикл с предусловием:

```
while (выражение-условие)
    тело_цикла
```

Цикл с постусловием:

```
do
    тело_цикла
while (выражение-условие);
```

Параметрический цикл (цикл общего вида):

```
for (инициализатор_цикла;
    выражение-условие;
    завершающее_выражение)
    тело_цикла
```

Цикл перебора элементов массива или коллекции:

foreach (*тип идентификатор in выражение*)
тело цикла

while, **do**, **for**, **foreach**, **in** – служебные слова, применяемые в операторах для циклов. Оператор **foreach** будет рассмотрен при изучении массивов (гл. 7). А теперь рассмотрим остальные циклы и их элементы.

Тело_цикла не может быть объявлением. Это либо отдельный (в том числе пустой) оператор, который всегда завершается точкой с запятой, либо блок. *Выражение-условие*, используемое в первых трех операторах — это логическое выражение, определяющее условие продолжения выполнения итераций (если его значение **true**). Прекращение выполнения цикла возможно в следующих случаях:

- ложное (**false**) значение проверяемого выражения-условия;
- выполнение в теле цикла оператора передачи управления (**break**, **goto**, **return**).

Последнюю из указанных возможностей проиллюстрируем чуть позже, рассматривая особенности операторов передачи управления.

Оператор **while** (оператор «повторять, пока истинно условие») называется оператором *цикла с предусловием*. При входе в цикл вычисляется выражение-условие. Если его значение истинно, то выполняется *тело_цикла*. Затем вычисление *выражения-условия* и выполнение операторов *тела_цикла* повторяются последовательно, пока значение *выражения-условия* остается истинным и нет явной передачи управления за пределы тела цикла.

Оператором **while** удобно пользоваться для просмотра последовательностей, если в конце каждой из них находится заранее известный признак.

В качестве проверяемого выражения-условия в циклах часто используются отношения. Например, следующая последовательность операторов вычисляет сумму квадратов первых *K* натуральных чисел (членов натурального ряда):

```
int i = 0;           // Счетчик
int s = 0;           // Будущая сумма
while (i++ < K)      // Цикл вычисления суммы
    s += i * i;
```

Только при изменении операндов выражения-условия можно избежать заикливания. Поэтому, используя оператор цикла, необходимо следить за тем, чтобы операторы тела цикла воздействовали на выражение-условие, либо оно еще каким-то образом должно изменяться во время вычислений. Например, могут изменяться операнды выражения-условия. Часто для этих целей используют унарные операции ++ и --. В качестве примера использования оператора **while** рассмотрим программу, которая предлагает пользователю ввести текстовую информацию (не пустую строку). Если пользователь нажимает

ENTER, не введя никаких данных, то предложение «Введите имя:» повторяется.

```
// 06_01.cs – цикл с предусловием
using System;
class Program
{
    static void Main()
    {
        string name = "";
        while (name == "")
        {
            Console.Write("Введите имя: ");
            name = Console.ReadLine();
        }
    }
}
```

Результат выполнения программы:

```
Введите имя:<ENTER>
Введите имя:<ENTER>
Введите имя: Rem<ENTER>
```

В данной программе предлагается ввести имя, и введенная строка, представляемая переменной *name*, сравнивается с пустой строкой. Проверка корректности введенных данных в реальных программах может быть гораздо более сложной. Но общая схема применения оператора цикла будет пригодна и в этих более сложных случаях. Цикл не прекращается, пока не будут введены правильные (по смыслу задачи) данные.

Оператор **do** (оператор «повторять») называется оператором цикла с постусловием. Он имеет следующий вид:

```
do
    тело_цикла
while (выражение-условие);
```

При входе в цикл с постусловием *тело_цикла* хотя бы один раз обязательно выполняется. Затем вычисляется *выражение-условие* и, если его значение равно **true**, вновь выполняется *тело_цикла*. При обработке некоторых последовательностей применение цикла с постусловием оказывается удобнее, чем цикла с предусловием. Это бывает в тех случаях, когда обработку нужно заканчивать не перед, а после появления в последовательности конечного признака.

К *выражению-условию* в цикле **do** требования те же, что и для цикла **while** – оно должно изменяться при итерациях либо за счет операторов тела цикла, либо при вычислениях значения выражения-условия.

В предыдущей программе цикл с предусловием можно заменить таким циклом с постусловием:

```
do
{
    Console.Write ("Введите имя:");
```

```

    name = Console.ReadLine();
}
while (name == "");

```

В заголовке оператора параметрического цикла **for** (цикл общего вида) три выражения, разделяемые символом точка с запятой:

```

for(инициализатор_цикла;
    выражение_условие;
    завершающее_выражение).

```

Инициализатор_цикла — выражение или декларация объектов одного типа. Обычно здесь определяются и инициализируются некие параметры цикла. Обратим внимание, что эти параметры должны быть только одного типа. Все выражения, входящие в инициализатор цикла, вычисляются только один раз при входе в цикл. *Инициализатор_цикла* в цикле **for** всегда завершается точкой с запятой, т. е. отделяется этим разделителем от последующего *выражения-условия*, которое также завершается точкой с запятой. Даже при отсутствии в цикле **for** *инициализатора_цикла*, *выражения-условия* и *завершающего_выражения* разделяющие их символы «точка с запятой» всегда присутствуют, т. е. в заголовке цикла **for** всегда имеются два символа ';':

Выражение-условие такое же, как и в циклах **while** и **do**. Если оно равно **false**, то выполнение цикла прекращается. В случае отсутствия *выражения-условия* следующий за ним разделитель «точка с запятой» сохраняется, и предполагается, что значение *выражения_условия* всегда истинно.

Завершающее_выражение (в цикле **for**) достаточно часто представляет собой последовательность скалярных выражений, разделенных запятыми. Эти выражения вычисляются на каждой итерации после выполнения операторов тела цикла, т. е. перед следующей проверкой *выражения-условия*.

Тело_цикла, как уже сказано, может быть блоком, отдельным оператором и пустым оператором. Определенные в инициализаторе цикла объекты существуют только в заголовке и в теле цикла. Если результаты выполнения цикла нужны после его окончания, то их нужно сохранять во внешних относительно цикла объектах.

В следующей программе использованы три формы оператора **for**. В каждом из циклов решается одна и та же задача суммирования квадратов первых *k* членов натурального ряда:

```

// 06_02.cs – параметрический цикл (цикл общего вида).
using System;
class Program
{
    static void Main()
    {
        int k = 3, s = 0, i = 1;
        for (; i <= k; i++)

```



```

        s += i * i;
        Console.WriteLine("Сумма = " + s);
        s = 0; k = 4; // Начальные значения s, k.
        for (int j = 0; j < k;)
            s += ++j * j;
        Console.WriteLine("Сумма = " + s);
        for (i = 0, s = 0, k = 5; i <= k; s += i * i, i++) ;
        Console.WriteLine("Сумма = " + s);
    }
}

```

Результат выполнения программы:

```

Сумма = 14
Сумма = 30
Сумма = 55

```

Все переменные в первом цикле внешние; отсутствует инициализатор цикла; в завершающем выражении заголовка изменяется параметр цикла *i*. После выполнения цикла результат сохраняется в переменной *s* и выводится. Перед вторым циклом *s* обнуляется, а переменной *k* присваивается значение 4. Инициализатор второго цикла определяет параметр цикла — локализованную в цикле переменную *j*. В заголовке отсутствует завершающее выражение, а параметр цикла *j* изменяется в его теле (за пределами заголовка). Инициализатор третьего цикла – выражение, операнды которого разделены запятыми. В этом выражении подготавливается выполнение цикла — обнуляются значения *i* и *s*, и присваивается значение 5 переменной *k*. Остальное должно быть понятно.

Итак, еще раз проследим последовательность выполнения цикла **for**. Определяются и иницируются объекты, т. е. обрабатывается выражение *инициализатора_цикла*. Вычисляется значение *выражения-условия*. Если оно равно **true**, выполняются операторы *тела_цикла*. Затем вычисляется завершающее выражение, вновь вычисляется *выражение-условие*, и проверяется его значение. Далее цепочка действий повторяется. Оператору **for** может быть поставлен в соответствие следующий блок (с оператором цикла **while**):

```

{
    инициализатор_цикла;
    while (выражение-условие)
    {
        операторы_тела_цикла
        завершающее_выражение;
    }
}

```

При выполнении цикла **for** *выражение-условие* может изменяться либо при вычислении его значений, либо под действием операторов тела цикла, либо под действием завершающего выражения. Если *выражение-условие* не изменяется либо отсутствует, то цикл бесконечен.

Следующий оператор обеспечивает бесконечное выполнение пустого оператора:

```
for(;;); // Бесконечный цикл
```

Чтобы проиллюстрировать возможности циклов и показать несколько приемов программирования на C#, рассмотрим задачу об оценке машинного нуля относительно заданного числового значения.

В предыдущей главе, посвященной связи типов языка с типами .NET Framework, мы узнали, что для вещественных чисел введено константное статическое поле `Epsilon`, значение которого — наименьшее отличное от нуля значение, которое может принимать переменная заданного вещественного типа. При решении вычислительных задач иногда требуется использовать не минимально допустимые значения вещественных чисел, а число, которое пренебрежимо мало относительно другого числа, принятого за базовое. Иногда это малое число называют машинным нулем, относительно базовой величины.

Иллюстрируя применение операторов цикла, напомним программу, позволяющую оценить машинный нуль относительно вводимого пользователем значения.

```
// 06_03.cs – циклы и машинный нуль.
using static System.Console;
class Program
{
    static void Main()
    {
        double real, estimate, zero;
        do Write("Введите положительное число: ");
        while (!double.TryParse(ReadLine(), out real) || real <= 0);
        for (estimate = 0, zero = real; estimate != real; zero /= 2.0)
            estimate = real + zero;
        WriteLine("Машинный нуль: " + zero);
    }
}
```

Результат выполнения программы:

```
Введите положительное число: -16<ENTER>
Введите положительное число: двадцать<ENTER>
Введите положительное число: 23<ENTER>
Машинный нуль: 6,38378239159465E-16
```

В программе два цикла. Цикл с постусловием служит для «наблюдения» за правильностью ввода исходного числового значения. От пользователя требуется, чтобы он ввел положительное число. Метод `TryParse()` класса **double** анализирует вводимую пользователем строку символов. Если она корректно представляет запись вещественного числа, то метод `TryParse()` возвращает значение **true** и присваивает значение введенного числа аргументу `real`. Выражение_условие цикла **do** (размещенное в скобках после **while**), истинно, если метод `TryParse()` вернет

значение **false** или значение переменной **real** не положительное. В этом случае цикл повторяется и пользователь повторно видит приглашение «Введите положительное число:».

Как только пользователь введет положительное значение переменной **real**, начинает выполняться второй цикл. В конце каждой итерации вдвое уменьшается значение переменной **zero**. Цикл выполняется до тех пор, пока сумма значения **real** с **zero** не равна значению **real**. Равенство достигается в том случае, когда значение **zero** становится пренебрежимо малой величиной относительно **real**. Таким образом, достигнутое значение **zero** можно считать машинным нулем относительно **real**.

Разрешено и широко используется вложение любых циклов в любые циклы, т. е. цикл может быть телом другого цикла. Примеры таких вложений будут не раз рассмотрены при дальнейшем изложении материала.

6.5. Операторы передачи управления

Кроме оператора безусловного перехода (иначе — оператора безусловной передачи управления **goto**) к операторам передачи управления относят: оператор возврата из метода **return**; оператор выхода из цикла или переключателя **break**; оператор перехода к следующей итерации цикла **continue**; оператор посылки исключения **throw**.

Оператор **return** подробно рассмотрим, перейдя к методам (функциям). Оператор **throw** требует хорошего понимания механизма генерации и обработки исключений, поэтому знакомство с ним нужно отложить. Поэтому сейчас остановимся на операторах **break**, **continue** и **goto**.

Оператор **break** служит для принудительного выхода из цикла или переключателя. Определение «принудительный» подчеркивает безусловность перехода. Например, в цикле не проверяются и не рассматриваются условия дальнейшего продолжения итераций. Оператор **break** прекращает выполнение оператора цикла или переключателя и осуществляет передачу управления (переход) к следующему за циклом или переключателем оператору. При этом, в отличие от перехода с помощью **goto**, оператор, к которому выполняется передача управления, может быть не помечен. Оператор **break** нельзя использовать нигде, кроме циклов и переключателей.

Необходимость в использовании оператора **break** в теле цикла возникает, когда условия продолжения итераций нужно проверять не в начале итерации (циклы **for**, **while**), не в конце итерации (цикл **do**), а в середине тела цикла. В этом случае тело цикла может иметь такую структуру:

```
{  
    операторы  
    if (условие) break;
```

```
}
```

Например, если начальные значения целых переменных i, j таковы, что $i < j$, то следующий цикл определяет наименьшее целое, не меньшее их среднего арифметического:

```
while (i < j)
{
    i++;
    if (i == j) break;
    j--;
}
```

Для $i == 0, j == 3$ результат $(i == 2) \& (j == 2)$ достигается при выходе из цикла с помощью оператора **break**. Для $i == 0$ и $j == 2$ результат $(i == 1) \& (j == 1)$ будет получен при естественном завершении цикла.

Как уже упомянуто, циклы могут быть многократно вложенными. Однако следует помнить, что оператор **break** позволяет выйти только из того цикла, в котором он размещен. При многократном вложении циклов оператор **break** не может вызвать передачу управления из самого внутреннего уровня непосредственно на самый внешний. Например, при решении задачи поиска в матрице хотя бы одного элемента с заданным значением, когда цикл перебора элементов строки вложен в цикл перебора строк, удобнее пользоваться не оператором **break**, а оператором безусловной передачи управления **goto**.

В качестве примера, когда при вложении циклов целесообразно применение оператора **break**, назовем задачу вычисления произведений элементов отдельных строк матрицы. Задача требует вложения циклов. Во внешнем выбирается очередная строка, во внутреннем — вычисляется произведение ее элементов. Вычисление произведения элементов строки можно прервать, если один из сомножителей окажется равным 0. При появлении в строке нулевого элемента оператор **break** прерывает выполнение только внутреннего цикла, однако внешний цикл перебора строк выполнится для всех строк.

Оператор безусловного перехода, который мы рассмотрели в связи с метками и переключателем, имеет вид:

```
goto идентификатор;
```

где идентификатор — имя метки оператора, расположенного в том же блоке, где используется оператор безусловного перехода, или во внешнем блоке.

Принятая в настоящее время дисциплина программирования рекомендует либо вовсе отказаться от оператора **goto**, либо свести его применение к минимуму и строго придерживаться следующих рекомендаций:

- не входить внутрь блока извне (компилятор C# этого не позволит — сообщит об ошибке);

- не входить внутрь условного оператора, т. е. не передавать управление операторам, размещенным после служебных слов **if** или **else**;
- не входить извне внутрь переключателя (компилятор C# сообщит об ошибке);
- не передавать управление внутрь цикла (в тело цикла).

Следование перечисленным рекомендациям позволяет исключить возможные нежелательные последствия бессистемного использования оператора безусловного перехода. Полностью отказываться от оператора **goto** вряд ли стоит. Есть случаи, когда этот оператор обеспечивает наиболее простые и понятные решения. Один из них — это ситуация, когда нужно выйти из нескольких вложенных друг в друга циклов или переключателей. Оператор **break** здесь не поможет, так как он обеспечивает выход только из самого внутреннего вложенного цикла или переключателя.

Оператор **continue** (оператор перехода к следующей итерации) употребляется только в операторах цикла. С его помощью завершается текущая итерация и начинается проверка условия возможности дальнейшего продолжения цикла, т. е. проверяется условие начала следующей итерации. Для объяснений действия оператора **continue** рекомендуется рассматривать операторы цикла в следующем виде:

<pre>while (foo) { . . . contin: ; }</pre>	<pre>do { . . . contin: ; } while (foo);</pre>	<pre>for (;foo;) { . . . contin: ; }</pre>
--	--	--

В каждой из форм многоточием обозначены операторы тела цикла. Вслед за ними размещен пустой оператор с меткой **contin**. Если среди операторов тела цикла есть оператор **continue** и он выполняется, то его действие эквивалентно оператору безусловного перехода на метку **contin**.

Пример использования оператора **continue**. Вводя с клавиатуры последовательность вещественных чисел, подсчитать сумму только положительных. Окончанием ввода последовательности считать ввод нулевого значения.

```
// 06_04.cs – оператор перехода к следующей итерации
using static System.Console;
class Program
{
    static void Main()
    {
        double sum = 0, x;
        do
        {
            do Write("x = ");
            while (!double.TryParse(ReadLine(), out x));
            if (x < 0) continue;
            sum += x;
        }
    }
}
```

```

        while (x != 0);
        WriteLine("Сумма = " + sum);
    }
}

```

Результат выполнения программы:

```

x = 2<ENTER>
x = -4<ENTER>
x = 6<ENTER>
x = 0<ENTER>
Сумма = 8

```

6.6. Переключатель

Переключатель является наиболее удобным средством для организации множественного (мульти-) ветвления. Синтаксис переключателя (до появления версии C# 7.0) таков:

```

switch (переключающее_выражение)
{
    case константное_выражение_1: операторы_1;
        оператор перехода;
    case константное_выражение_2: операторы_2;
        оператор перехода;
    ...
    case константное_выражение_n: операторы_n;
        оператор перехода;
    default: операторы;
        оператор перехода;
}

```

В переключателях используют три служебных слова: **switch**, **case**, **default** и обязательные фигурные скобки, ограничивающие тело переключателя. Конструкция

case константное_выражение:

называется меткой переключателя. Константное выражение может быть целочисленным, может быть символом, строкой или элементом перечисления (о перечислениях речь пойдет в гл. 15).

Заголовок, т. е. управляющая конструкция

switch (переключающее выражение),

передает управление к тому из операторов, помеченных метками переключателя, для которого значение константного выражения после **case** совпадает со значением переключающего выражения. Значение переключающего выражения должно быть целочисленным или иметь тип **char**, тип **string**, тип перечисления, или приводиться к целому. Переключающее выражение не может иметь вещественный тип и не может быть десятичным (**decimal**).

Метка переключателя вводит ветвь или раздел переключателя — последовательность операторов, завершаемая оператором перехода. В качестве оператора перехода может использоваться:

break — выход за пределы переключателя;
goto case i — переход к другой ветви (метке) переключателя;
goto default — переход к ветви, помеченной **default**;
goto Метка — оператор безусловного перехода;
return — оператор выхода из метода;
continue — оператор перехода к следующей итерации цикла;
throw — оператор генерации исключения.

Значения константных выражений, помещаемых за служебными словами **case**, приводятся к типу переключающего выражения. В одном переключателе все константные выражения меток переключателя должны иметь различные значения, но быть одного типа. Любой раздел из операторов, помещенных в фигурных скобках после конструкции **switch(...)**, может быть помечен или одной или несколькими метками переключателя. т. е. отдельный раздел переключателя может начинаться несколькими метками.

Если значение переключающего выражения не совпадает ни с одним из константных выражений его меток, то выполняется переход к оператору, отмеченному меткой **default**. В каждом переключателе может быть не больше одной метки **default**. Если метка **default** отсутствует, то при несовпадении значения переключающего выражения ни с одним из константных выражений, помещаемых вслед за **case**, в переключателе не выполняется ни один из операторов.

В качестве примера приведем программу перевода оценки в баллах при десятибалльной шкале в аттестационную (четырёхбалльную) оценку.

Соответствие:

- 1, 2, 3 балла — не удовлетворительно;
- 4, 5 — удовлетворительно;
- 6, 7 — хорошо;
- 8, 9, 10 — отлично.

```
// 06_05.cs - переключатель
using System;
class Program
{
    static void Main()
    {
        int ball; // оценка в баллах:
        do
            Console.Write("Введите оценку в баллах: ");
        while (!int.TryParse(Console.ReadLine(), out ball)
            || ball <= 0 || ball > 10);
        switch (ball)
        {
            case 1:
            case 2:
```

```

    case 3:
        Console.WriteLine("Неудовлетворительно");
        break;
    case 4:
    case 5:
        Console.WriteLine("Удовлетворительно");
        break;
    case 6:
    case 7:
        Console.WriteLine("Хорошо");
        break;
    case 8:
    case 9:
    case 10:
        Console.WriteLine("Отлично");
        break;
    default:
        Console.WriteLine("Ошибка в данных");
        break;
} // Конец переключателя
}
}

```

Результат выполнения программы:

```

Введите оценку в баллах: гг<ENTER>
Введите оценку в баллах: -9<ENTER>
Введите оценку в баллах: 0<ENTER>
Введите оценку в баллах: 9<ENTER>
Отлично

```

Обратите внимание на обязательность оператора **break** в каждой ветви переключателя. С его помощью управление всегда передается оператору, размещенному за переключателем.

В переключателе могут находиться определения объектов. Тело переключателя, и каждый оператор, входящий в переключатель, может быть блоком. В этом случае нужно избегать ошибок «перескакивания» через определения:

```

switch (n)                // Переключатель с ошибками
{
    char d = 'D';          // Никогда не обрабатывается
    case 1: double f = 3.14; // Обрабатывается только для n == 1
        break;
    case 2:
        ...
        if ((int)d != (int)f) // Ошибка: d и (или) f не определены
            break;
        ...
}

```

Если в переключателе при некотором значении переключающего выражения необходимо выполнить более одного раздела, то можно

заранее выбрать последовательность обхода ветвей, применяя оператор перехода **goto case i** или **goto default**.

Пример программы с переключателем, где выводятся названия нечетных целых цифр, не меньших заданной.

```
// 06_06.cs – переключатель с внутренними переходами
using System;
class Program
{
    static void Main()
    {
        int ic = 5;
        string line = "";
        switch (ic)
        {
            case 0:
            case 1:
                line += "one, ";
                goto case 2;
            case 2:
            case 3:
                line += "three, ";
                goto case 4;
            case 4:
            case 5:
                line += "five, ";
                goto case 6;
            case 6:
            case 7:
                line += "seven, ";
                goto case 8;
            case 8:
            case 9:
                line += "nine.";
                break;
            default:
                line = "Error! It is not digit!";
                break;
        }
        Console.WriteLine("Цифры: " + line);
    }
}
```

Результат выполнения программы:

Цифры: five, seven, nine.

Рассмотренный вариант переключателя существует в языке C# с его первой версии. Основная особенность этого варианта — константа в метке переключателя. т. е. метки переключателя идентифицируются и отличаются друг от друга только константами, которые размещаются после **case** перед символом «двоеточие». Этот вариант декларации метки переключателя называют «шаблоном константы».

При использовании шаблона константы нужная ветвь переключателя выбирается на основе соответствия значения переключающего выражения константе, входящей в метку переключателя.

В версии C# 7.0 для меток переключателей разрешен еще «шаблон типа», при использовании которого появляются следующие дополнительные формы меток переключателя:

```
case тип переменная:  
case null:  
case тип переменная when логическое_выражение:
```

Конструкция «тип переменная» вводит и инициализирует переменную, которая локализована и в соответствующей ветви переключателя и в «логическом_выражении», если оно присутствует.

В шаблоне типа соответствие устанавливается между *типом переключающего выражения* и типом, имя которого размещено вслед за служебным словом **case**.

Следующий фрагмент кода иллюстрирует возможности первых двух форм меток переключателя на основе шаблона типа:

```
object ob = null;  
switch (ob)  
{  
    case int k:  
    case long l:  
        WriteLine("Целое значение!");  
        break;  
    case double x:  
    case float z:  
        WriteLine("Вещественное значение!");  
        break;  
    case null:  
        WriteLine("Нулевая ссылка!");  
        break;  
    default:  
        WriteLine("Тип неизвестен!");  
        break;  
}
```

В коде ссылка *ob* с типом **object** имеет нулевое значение и результатом будет сообщение:

Нулевая ссылка!

Если инициализировать ссылку числовым значением, например, так:

```
object ob = 3.1415;
```

результатом будет сообщение

Вещественное значение!

Так как в одном переключателе разрешено использовать несколько меток **case** с одинаковыми типами, то может возникать неоднозначность выбора ветви. Но метки при переключениях просматриваются по порядку, и для устранения неоднозначности рекомендуется, во-первых, продумывать их корректное расположение. Вторая возможность регулировать выбор ветвей — применение в метке переключения по типу логического выражения со служебным словом **when**. С помощью конструкции «**when** логическое_выражение» можно проверить значение переменной ветви переключателя, *если она имеет соответствующий тип*. Если значение переключающего выражения можно преобразовать к типу, указанному в метке переключателя, то значение переключающего выражения присваивается переменной метки переключателя. Затем эту переменную можно использовать как в логическом выражении конструкции **when**, так и в выбранной ветви переключателя.

Например, в следующем фрагменте кода переключатель включает две ветви, каждая из которых «настроена» на обработку значения типа **int**. Но выбирается ветвь, соответствующая не только типу, но и значению переключающего выражения (оно должно быть либо больше, либо меньше 100).

```
object obj = 430;
switch (obj)
{
    case int k when k < 100:
        Console.WriteLine("Малое целое == " + k);
        break;
    case int m when m > 100:
        Console.WriteLine("Большое значение == " + m);
        break;
}
```

Результат выполнения:

Большое значение == 430

Контрольные вопросы и задания

1. Каково назначение оператора в программах на C#?
2. Перечислите операторы языка C#.
3. Каков обязательный признак отличного от блока оператора в C#?
4. Что такое оператор-выражение?
5. Где часто используется пустой оператор?
6. Что такое метка?
7. Дайте определение блока.
8. Какими правилами определяются вход в блок и выход из него?
9. Назовите операторы выбора (ветвлений).
10. Какие операторы не могут входить в условный оператор?

11. Что такое сокращенная форма условного оператора?
12. Как устанавливается соответствие между **if** и **else** при вложениях условных операторов?
13. Назовите виды операторов циклов в C#.
14. Какой оператор не может быть телом цикла?
15. Какой тип имеет выражение-условие в операторе цикла?
16. Сколько элементов в заголовке цикла общего вида (цикла **for**) и как они разделяются?
17. Что такое инициализатор цикла общего вида (цикла **for**)?
18. Когда вычисляется завершающее выражение цикла **for**?
19. Укажите область существования объектов, объявленных в инициализаторе цикла **for**.
20. Как выполняется вложение циклов?
21. Какие операторы могут прервать выполнение цикла до его завершения, запланированного выражением-условием?
22. Каково минимальное количество итераций в цикле с постусловием?
23. Назовите назначение оператора **break**. Где его можно применять?
24. Укажите возможности оператора **goto** при вложениях циклов.
25. Где и когда употребляется оператор **continue**?
26. Какого типа может быть значение переключающего выражения в переключателе?
27. Что называют меткой переключателя?
28. Каким оператором должна завершиться ветвь переключателя?
29. Какая конструкция вводит ветвь переключателя?
30. В каких случаях выполняется ветвь переключателя, введенная меткой со служебным словом **default**?
31. Приведите примеры шаблона типа в метках переключателя.
32. Почему при переключениях по типу метки переключателя могут оказаться не взаимоисключающими?

Глава 7

МАССИВЫ C#

7.1. Одномерные массивы

Напомним, что система типов языка C# построена на основе классов. Типы делятся на четыре группы (значений, ссылок, указателей и тип **void**). С типами значений мы уже знакомы. К типам ссылок отнесены собственно классы (библиотечные и определяемые программистом), массивы и строки. Рассмотрим в этой главе массивы.

Массив — это совокупность однотипных элементов, воспринимаемая как единое целое. Общеязыковая исполняющая среда (CLR) особым образом воспринимает типы массивов и выделяет каждому экземпляру массива непрерывный участок памяти. Это повышает быстродействие обращений к элементам массивов, но препятствует изменению размеров массивов после их создания. CLR поддерживает одномерные и многомерные массивы. В литературе в качестве еще одного особого вида массивов указывают на массивы, элементы которых содержат ссылки на другие массивы. В английском языке для таких массивов введен специальный термин *jagged* (ступенчатый или зубчатый). Но стандарт языка C# не выделяет такие массивы в отдельный вид. Для краткости такие массивы с элементами, ссылающимися на другие массивы, можно называть «массивами массивов».

С точки зрения синтаксиса языка C# массив — это набор однотипных элементов, обращение к которым выполняется с помощью индексирующих выражений. Каждый массив имеет ранг, определяющий количество индексов, необходимых для идентификации конкретного элемента массива. Ранг массива называют его размерностью. Массив может иметь до 32 измерений, т. е. ранг массива — целое число из интервала [1; 32].

Одномерный массив — набор однотипных элементов, доступ к которым осуществляется с помощью одного индексирующего выражения:

`имя_ссылки_на_массив [индексирующее_выражение]`

Здесь `имя_ссылки_на_массив` — ссылка типа, производного от класса `Array`. `Индексирующее_выражение` должно иметь целочисленный тип (или приводится к целочисленному типу). Значение индексирующего выражения (иначе индекс) должно принадлежать фиксированному

конечному диапазону [indMin, indMax], где indMin — нижняя граница, indMax — верхняя граница индекса. Количество элементов (размер) массива: indMax – indMin + 1. По умолчанию с помощью языковых конструкций C# массивы всегда создаются с нулевой нижней границей индекса, т. е. indMin = 0. Одномерные массивы с нулевым начальным индексом называют векторами. Массивы с ненулевым начальным индексом создаются динамически с помощью метода *CreateInstance* класса *Array*, с которым можно познакомиться в документации.

Имя_ссылки_на_массив — выбираемый пользователем идентификатор. Чтобы идентификатор стал именем ссылки на массив, используется объявление вида

```
тип [ ] идентификатор;
```

Этим оператором объявляется переменная, имеющая тип ссылки на массив, а *тип* определяет тип элементов этого массива. Кроме того, объявление указанного формата вводит в программу новый тип с обозначением *тип*[].

Примеры:

```
int [ ] integers;  
double [ ] points;
```

Здесь *integers* — ссылка на массив типа **int** [] с элементами типа **int**; *points* — ссылка на массив типа **double** [] с элементами типа **double**.

В результате обработки таких объявлений компилятор выделит в стеке участки для размещения переменных (ссылок) *integers* и *points*. Однако массивов, которые могут адресовать эти ссылки, пока не существует, и значениями *integers* и *points* является **null**. Каждый конкретный массив создается как объект класса, производного от класса *Array*, точнее, от соответствующего ему системного класса *System.Array*.

Для создания экземпляра (объекта) конкретного типа массивов используется операция **new**. Выражение:

```
new тип [размер-массива]
```

определяет объект-массив с заданным в квадратных скобках количеством элементов указанного типа. Этот объект компилятор размещает в области памяти, называемой управляемой кучей (managed heap). Результат выполнения операции **new** — ссылка на выделенный для объекта (для экземпляра массива) участок памяти.

Чтобы связать ссылку с этим объектом, используют операцию присваивания:

```
имя_ссылки_на_массив = new тип[размер-массива];
```

Тип ссылки на массив должен соответствовать типу, указанному после операции **new**.

Примеры создания экземпляров массивов:

```
integers = new int [4];  
points = new double [3];
```

После этих и предыдущих объявлений ссылка `integers` (размещенная в стеке) будет адресовать (ссылаться на) участок памяти в куче, отведенный для конкретного массива из четырех элементов типа `int`. Ссылка `points` будет связана с массивом из трех элементов типа `double`.

Приведенному объявлению двух ссылок (`integers` и `points`) и «адресуемых» ими одномерных массивов соответствует схема распределения памяти, показанная на рис. 7.1.

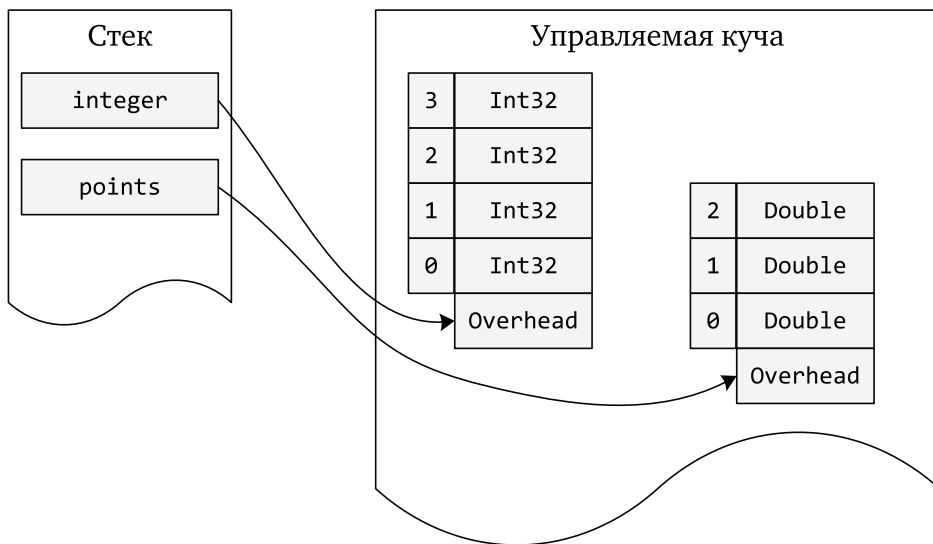


Рис. 7.1. Размещение в памяти двух ссылок и двух одномерных массивов

На рис. 7.1 для каждого из массивов условно показаны номера (индексы) их элементов. Кроме того, показано, что при выделении блока памяти для массива в этом блоке создается участок памяти (Overhead) для служебной информации. К служебной информации, сохраняемой вместе с элементами массива, относятся: размерность (иначе ранг или число измерений) массива; нижняя граница индекса для каждого измерения; тип элементов массива и количество элементов в каждом измерении. Для каждого из массивов перечисленные сведения из области служебной информации можно получить с помощью специальных свойств и методов класса `Array`.

Допустимо объединение объявления ссылки на массив и определения массива-объекта:

```
тип[ ] имя_ссылки_на_массив = new тип [размер_массива];
```

Пример:

```
char[ ] line = new char[12];
```

Такое объявление вводит тип массива `char[ ]`, определяет одномерный объект-массив `char[12]`, объявляет ссылку `line` на одномерный массив с элементами типа `char` и связывает эту ссылку с объектом-массивом.

После определения массива его элементы получают значения по умолчанию. Для элементов арифметических типов по умолчанию устанавливаются нулевые значения, для элементов типа **char** — значения `'\0'`, для элементов логического типа — значения **false**, для элементов типа **string** — пустые строки, для элементов с типами ссылок и для элементов типа **object** — значения **null**.

Определив ссылку на массив и связав ее с конкретным объектом-массивом, можно обращаться к элементам массива, используя выражения с операцией индексирования. В следующей программе определены массивы разных типов и выведены значения по умолчанию их отдельных элементов.

```
// 07_01.cs – массивы – инициализация по умолчанию
using static System.Console;
class Program
{
    static void Main()
    {
        bool[] bar = new bool[5];
        WriteLine("bar[2] = " + bar[2]);
        int[] integers = new int[4];
        WriteLine("integers[0] = " + integers[0]);
        double[] points = new double[3];
        WriteLine("points[1] = " + points[1]);
    }
}
```

Результат выполнения программы:

```
bar[2] = False
integers[0] = 0
points[1] = 0
```

Можно изменить значение элемента массива, используя в левой части выражения с операцией присваивания ссылку на массив с соответствующим индексом.

При создании экземпляра (объекта) типа массивов разрешена его явная инициализация, т. е. общий вид выражения с операцией **new** таков:

```
new тип [размер-массива] инициализатор;
```

Инициализатор не обязателен, поэтому ранее мы применяли операцию **new** без него. Инициализатор — заключенный в фигурные скобки список инициализирующих выражений. Инициализатор имеет вид:

```
{список_выражений}
```

Каждое выражение в списке инициализатора (они разделены в списке запятыми) определяет значение соответствующего элемента создаваемого массива. Количество инициализирующих выражений в списке

инициализатора должно соответствовать размеру массива, т. е. числу его элементов.

Пример:

```
double a = 1.1;
double [4] x = new double [4] {2.0/4, 3.0, 4.0, a};
```

Здесь *x* — ссылка на массив с четырьмя элементами типа **double**, элементы имеют значения:

x[0]==>0.5, *x*[1]==>3.0, *x*[2]==>4.0, *x*[3]==>1.1.

Тот же результат получится, если удалить из обеих квадратных скобок размер массива 4. Дело в том, что если размер массива не указан, то он определяется количеством инициализирующих выражений.

Для объявления экземпляра массива с применением инициализатора допустима и зачастую используется сокращенная форма без явного использования операции **new**:

```
тип [ ] имя_ссылки_на_массив = инициализатор;
```

Как и ранее, инициализатор — заключенная в фигурные скобки последовательность выражений, разделенных запятыми. Размер массива в этом случае может быть не указан явно и определяется количеством инициализирующих выражений. Пример:

```
long[ ] g = {8/4, 3L, 4, (int)2.4};
```

Тот же результат можно получить, явно применив операцию **new**:

```
long [ ] s = new long [ ]{8/4, 3L, 4, (int)2.4};
```

В одном объявлении могут быть определены несколько однотипных ссылок на массивы:

```
byte [ ] x, y, z;
```

Инициализация массивов допустима и при таком определении нескольких однотипных ссылок в одном объявлении. Например, так:

```
int [ ] b = {1, 2, 3, 4, 5},
z = new int [ ] {6, 7, 8, 9},
d = z;
```

Определяя ссылки на массивы и массивы как объекты, необходимо помнить их различия. Компилятор в стеке выделяет память для каждой ссылки при ее объявлении. А память в куче для объекта-массива выделяется только при явном (или неявном) применении операции **new**.

В приведенном примере ссылки *d* и *z* адресуют один и тот же массив (объект класса **int** []) из четырех элементов типа **int**. Таким образом, к одному и тому же элементу массива в нашем примере возможен доступ с помощью двух ссылок. В результате выполнения операторов

```
d[3] = 43;  
Console.WriteLine("z [3] =" + z [3]);
```

будет выведено

```
z [3] = 43.
```

Обратите внимание на тот факт, что у объекта-массива (т. е. у той конструкции, которую называют массивом) нет имени. Ссылка на массив не является уникальным именем экземпляра массива, ведь с массивом могут быть ассоциированы одновременно несколько ссылок (переменные *d* и *z* в нашем примере).

К массивам (которые являются на самом деле частным случаем контейнеров или коллекций) применим еще не рассмотренный нами оператор цикла, вводимый служебным словом **foreach**. Формат этого оператора:

```
foreach (тип имя_переменной in ссылка_на_массив)  
    тело_цикла
```

В заголовке цикла **foreach** используется ссылка на уже существующий одномерный массив, тип элементов которого должен соответствовать типу, указанному перед именем переменной. Тело цикла — это либо отдельный оператор (завершаемый обязательной точкой с запятой) либо блок — заключенная в фигурные скобки последовательность операторов. В заголовке оператора цикла конструкция *тип имя_переменной* определяет переменную цикла с именем, выбранным программистом. Эта переменная автоматически последовательно принимает значения всех элементов массива, начиная с элемента, имеющего начальное (обычно нулевое) значение индекса. Для каждого из значений переменной цикла **foreach** выполняется тело цикла. Количество итераций цикла определяется количеством элементов массива.

Определенная в заголовке цикла переменная доступна только в теле цикла, и там ее значение может быть использовано. Однако попытки изменить с помощью обращений к этой переменной элементы массива будут безуспешными. Переменная цикла **foreach** получает значения элементов массива, но не может их изменять.

В качестве примера приведем следующую программу, где цикл **foreach** используется для вывода значений элементов символьного массива.

```
// 07_02.cs – массивы одномерные и цикл foreach  
using static System.Console;  
class Program  
{  
    static void Main()  
    {  
        char[] hi = { 'H', 'e', 'l', 'l', 'o', ',', ' ', 'C', '#', '!' };  
        foreach (char ch in hi)
```

```
        Write(ch);  
        WriteLine();  
    }  
}
```

Результат выполнения программы:

Hello, C#!

Обратите внимание, что при объявлении объекта-массива и адресуемой его ссылки `hi` выполнена инициализация массива, и не потребовалось явно применять операцию **new**.

До сих пор мы рассматривали массивы, размеры которых определялись при их создании — инициализацией либо явным указанием числа элементов. При выполнении операции **new** число элементов массива может быть задано не только константой, но и каким-либо выражением. Наиболее типичный случай — размер массива определяет пользователь, вводя нужное для решения задачи числовое значение. Не менее редко возникает ситуация, когда размер (число элементов) массива определяется как результат предыдущих вычислений. В каждом из названных случаев размер массива определяется до его декларации. Однако как только массив (объект класса массивов) определен, его размер изменить невозможно. Если размер нужно увеличить или уменьшить, приходится сначала создать новый массив нужных размеров, потом копировать в него значения элементов исходного массива и уничтожить старый массив. К счастью, существуют методы класса `Array`, которые позволяют существенно упростить названную последовательность действий. Покажем, как это можно сделать, вначале кратко перечислив основные библиотечные методы обработки массивов.

7.2. Массивы как наследники класса `Array`

Массивы наследуют из класса `Array` несколько свойств и много методов, упрощающих работу с массивами.

Важных свойств два:

- свойство **int** `Rank` — размерность (число измерений) массива;
- свойство **int** `Length` — целочисленное (32 бита) значение, определяющее общее количество элементов экземпляра массива (изменять его нельзя). Это свойство незаменимо в методах, параметрами которых служат ссылки на массивы. Обратите внимание, что для многомерных массивов свойство `Length` возвращает общее количество элементов массива с учетом всех измерений.

Методы класса `Array` позволяют решать многие типовые задачи обработки массивов. В табл. 7.1 и 7.2 приведены наиболее употребительные из статических методов и методов экземпляров для работы с массивами.

Статические методы класса Array (выборочно)

Метод	Описание
BinarySearch	Бинарный поиск в упорядоченном массиве; результат выполнения — индекс соответствующего условию поиска элемента или отрицательное число, если элемента с искомым значением в массиве нет
Clear	Очистка (обнуление, присваивание пробелов или значения null) всех элементов массива или элементов из указанного параметрами интервала
Copy	Копирование части массива или всего массива в другой массив
Exist	Проверка содержания в массиве элементов, удовлетворяющих условиям предиката-параметра
Find	Выбор первого элемента, удовлетворяющего условиям предиката-параметра
FindAll	Выбор всех элементов, удовлетворяющих условиям предиката-параметра
FindIndex	Поиск индекса первого элемента, удовлетворяющего условиям предиката-параметра
FindLast	Выбор последнего элемента, удовлетворяющего условиям предиката-параметра
FindLastIndex	Поиск индекса последнего элемента, удовлетворяющего условиям предиката-параметра
ForEach	Выполняет указанное параметром действие с каждым элементом массива
IndexOf	Возвращает наименьший индекс (минимальный номер) элемента, значение которого совпадает с образцом
LastIndexOf	Возвращает последний индекс (максимальный номер) элемента, значение которого совпадает с образцом
Resize	Изменяет (уменьшает или увеличивает) количество элементов в одномерном массиве до указанной параметром величины
Reverse	Меняет на обратный порядок элементов в массиве
Sort	Выполняет сортировку (упорядочивание) элементов массива.

Практически все методы класса Array перегружены, т. е. для каждого имеется несколько вариантов задания аргументов. В качестве очень характерного примера кратко рассмотрим особенности разных версий метода Sort().

Простейший вариант метода Sort() имеет единственный параметр — ссылку на одномерный сортируемый массив. Заголовок этой версии метода:

```
public static void Sort (Array array);
```

В этой версии метода сортировки нет параметра, определяющего правило упорядочивания элементов массива. В отсутствие этого параметра по умолчанию арифметические массивы сортируются в порядке возрастания значений элементов. Строки упорядочиваются лексикографически.

Следующий достойный упоминания метод сортировки имеет два параметра-массива, второй из которых — сортируемый массив, а первый — массив ключей, по которым выполняется упорядочивание элементов второго массива:

```
public static void Sort (Array keys, Array items);
```

Для каждого из названных двух вариантов метода сортировки (и в других версиях метода Sort()) может быть задана функция (метод) сравнения элементов или ключей, а также диапазоны изменения индексов для обрабатываемых элементов массивов. Кроме того, существуют обобщенные версии названных методов. Об обобщенных методах и об особенностях декларации функций (методов) сравнения речь пойдет в следующих разделах, посвященных обобщениям и делегатам. А сейчас приведем пример применения одного из перечисленных в таблице статических методов класса Array.

Предположим, что в программе необходимо нормировать положительные вещественные числа, вводимые пользователем с клавиатуры, разделив каждое из чисел на максимальное из них. Ввод чисел продолжается до тех пор, пока пользователь не введет нулевое значение очередного числа.

Общее количество чисел и значение максимального из них заранее неизвестны, поэтому числа придется сохранять до тех пор, пока во входной последовательности не появится нулевое значение. Только после этого можно быть уверенным, что уже введено максимальное из чисел и можно выполнять нормирование всех значений. Так как до начала выполнения программы общее число чисел в нормируемой последовательности неизвестно, то хранить все введенные числа будем в массиве, размер которого возрастает и в каждый момент равен количеству уже введенных положительных чисел.

Программа, решающая сформулированную задачу, может быть такой:

```
// 07_03.cs – массив с изменяемым (растущим) размером
using System;
using static System.Console;
class Program
{
    static void Main()
    {
        double[] numbers = new double[0];
        int Len = 0;
        double numb, maxNumb = double.MinValue;
        while (true)
```

```

{
    do Write("Введите не отрицательное число: ");
    while (!double.TryParse(ReadLine(), out numb) || numb < 0);
    Len = numbers.Length;
    if (numb == 0) break;
    Array.Resize(ref numbers, Len + 1);
    numbers[Len] = numb;
    maxNumb = maxNumb < numb ? numb : maxNumb;
}
WriteLine("Количество элементов в массиве: {0}.", Len);
if (Len == 0)
{
    WriteLine("Обработка завершена!");
    return;
}
WriteLine("Максимальный элемент: {0}.", maxNumb);
WriteLine("Нормированные числа:");
for (int i = 0; i < Len; i++)
    Write("{0,5:F3} ", numbers[i] / maxNumb);
}
}

```

Результат выполнения программы:

```

Введите не отрицательное число: 4<ENTER>
Введите не отрицательное число: 8<ENTER>
Введите не отрицательное число: e4<ENTER>
Введите не отрицательное число: 15<ENTER>
Введите не отрицательное число: 0<ENTER>
Количество элементов в массиве: 3.
Максимальный элемент: 15.
Нормированные числа:
0,267 0,533 1,000

```

В программе объявлена ссылка *numbers*, ассоциированная с массивом нулевого размера (с объектом типа **double**[]). Переменная **int** *Len* — текущее значение размера массива до его увеличения и одновременно индекс добавленного в массив элемента. Переменная **double** *numb* — очередное число, вводимое пользователем, которое записывается в добавляемый элемент массива. Переменная **double** *maxNumb* — максимальное из вводимых пользователем чисел. Ввод чисел и определение максимального из них выполняется в «бесконечном» цикле с заголовком **while(true)**. Цикл прерывается (за счет выполнения оператора **break**), когда пользователь вводит нулевое значение переменной *numb*. Если ее значение отлично от нуля, то выполняется обращение к методу

```
Array.Resize(ref numbers, Len + 1);
```

Первый аргумент метода — ссылка на изменяемый этим методом массив. Второй аргумент — тот размер, который получит новый экземпляр массива, адресуемый ссылкой *numbers*. В нашей программе к существу-

ющим элементам массива (сразу после них) добавляется один элемент. Его индекс равен значению длины предыдущего экземпляра массива, сохраняемому в переменной *Len*. Очень важно, что метод *Resize()* не только создает массив с новыми размерами, но и копирует в него элементы изменяемого массива. В добавленный элемент с индексом *Len* записывается очередное (не нулевое) значение переменной *numb* и оно сравнивается со значением текущего максимального значения (переменная *maxNumb*).

Остальные операторы кода программы, не требуют комментариев. Приводимые результаты одного из сеансов работы с программой поясняют ее особенности. Обратим только внимание, что при пустом массиве (когда *Len* равно нулю) программу после информационного сообщения завершает оператор **return**;

Проиллюстрировав возможности статического метода *Array.Resize*, приведем список нестатических методов класса *Array*.

Таблица 7.2

Нестатические методы (методы объектов) класса *Array* (выборочно)

Метод	Описание
Clone	Создает копию массива-объекта, к которому применен метод. Возвращает ссылку на новый массив (на копию), всегда имеющую тип object
CopyTo	Копируются элементы вызывающего массива в массив-параметр, начиная с его индекса, указанного параметром
GetLength	Возвращает размер массива (количество элементов) по указанному аргументом измерению
GetType	Возвращает объект класса <i>Type</i> , идентифицирующий тип вызывающего массива
GetUpperBound	Возвращает верхнюю границу индекса массива по указанному аргументом измерению
GetValue	Возвращает значение элемента массива, указанного параметром-индексом
SetValue	Присваивает значение, заданное параметром, указанному параметром-индексом элементу.

Краткие сведения о методах, приведенные в таблицах, должны служить только «путеводителем» для изучения возможностей методов, упрощающих обработку массивов языка C#. Наиболее интересные возможности методов для обработки массивов нужно демонстрировать, используя делегаты и лямбда-выражения, что будет сделано в следующих главах. А сейчас на примере покажем применение методов *Clone()*, *Sort()*, *Reverse()*.

```
// 07_04.cs – методы класса Array
using System;
class Program
```

```

{
    static void Main()
    {
        char[] hi = {'1', 'A', '2', 'B', '3', 'C', '4', 'D', '5', 'E'};
        char[] hiNew = (char[])hi.Clone(); // копирование
        Array.Sort(hiNew); // сортировка
        Console.Write("Сортировка: ");
        foreach (char ch in hiNew)
            Console.Write(ch);
        Console.WriteLine();
        Array.Reverse(hiNew); // реверсирование
        Console.Write("Реверсирование: ");
        foreach (char ch in hiNew)
            Console.Write(ch);
        Console.WriteLine();
        Console.Write("Исходный массив: ");
        foreach (char ch in hi)
            Console.Write(" " + ch);
        Console.WriteLine();
    }
}

```

Результат выполнения программы:

```

Сортировка: 12345ABCDE
Реверсирование: EDCBA54321
Исходный массив: 1A2B3C4D5E

```

В коде используются члены двух разных классов (Console и Array) из одного пространства имен System и вместо двух директив «**using static**» применена директива «**using System;**». В программе определен и инициализирован символьный массив и адресующая его ссылка hi. Затем определена ссылка hiNew, и создана копия массива с помощью метода Clone(), который применен к объекту, адресованному ссылкой hi. Так как метод Clone() возвращает ссылку на базовый класс **object**, то выполнено явное приведение типов (**char []**). Только так можно присвоить результат ссылке hiNew. Методом Array.Sort() выполнена сортировка массива, связанного со ссылкой hiNew. Элементы по умолчанию упорядочены по возрастанию целочисленных значений кодов символов. Далее выводится отсортированный массив и выполняется его обработка методом Reverse(). Обратите внимание, что исходный массив остается без изменений.

7.3. Виды массивов и массивы многомерные

Размерностью массива (его рангом) называют количество индексов, которое необходимо для получения доступа к отдельному элементу массива.

Рассмотренные нами одномерные массивы являются частным случаем всего разнообразия массивов, которые можно использовать в C#.

Для этих массивов размерность равна 1, а размер одномерного массива определяется числом возможных значений индекса.

В общем случае тип массива объявляется с помощью конструкции:

тип_не_массива спецификаторы_размерностей

Здесь *тип_не_массива* это один из следующих типов:

- *тип_значений*;
- *тип_класса*;
- *тип_интерфейса*;
- *тип_делегата*.

Спецификатор размерности — это либо пустые квадратные скобки [], либо конструкция: [*разделители_размеров*];

Разделитель_размеров — это запятая. Количество запятых на единицу меньше размерности массива.

Спецификаторы размерностей размещаются подряд в нужном количестве после *типа_не_массива*. Рассмотрим случай, когда спецификатор размерности один. Например, *type[R]* — тип одномерного массива с R элементами типа *type*.

Важными частными случаями типов массивов с одним спецификатором размерности кроме типов одномерных массивов являются типы «прямоугольных» массивов, представляющие массивы двумерные (матрицы), трехмерные («параллелепипеды») и т. д. Именно такие массивы традиционно принято называть многомерными массивами. Примеры:

```
int [,] dots; // ссылка на двумерный массив
byte [,,] bits; // ссылка на трехмерный массив
```

Для определения объекта — конкретного экземпляра типа многомерных массивов, используется выражение с операцией **new**:

new *тип_не_массива* [*d1*, *d2*, *d3*, ...] *инициализатор*

Здесь *di* — выражение, определяющее размер, т. е. количество элементов по соответствующему измерению. Инициализатор представляет собой вложение конструкций {*список_инициализаторов*}.

Элементами такого списка в свою очередь служат заключенные в фигурные скобки списки инициализаторов. Глубина вложения соответствует размерности массива. Размерность массива (его ранг) можно получить с помощью нестатического свойства *Rank*.

Пример определения с использованием инициализации матрицы (двумерного массива) с размерами 4 на 2:

```
int [,] matr = new int[4,2] {{1, 2}, {3, 4}, {5, 6}, {7, 8}};
```

Как и для одномерных массивов, при наличии инициализатора конструкцию **new int[4,2]** можно опустить.

Так как размеры объектов-массивов задаются выражениями, в которые могут входить переменные, то допустимы многомерные массивы

с динамически определяемыми размерами. В качестве примера рассмотрим программу, формирующую единичную матрицу, размеры которой вводит пользователь с клавиатуры:

```
// 07_05.cs – двумерный массив – единичная матрица
using static System.Console;
class Program
{
    static void Main()
    {
        int size;
        do Write("size = ");
        while (!int.TryParse(ReadLine(), out size) || size < 1);
        int[,] one = new int[size, size];
        for (int i = 0; i < size; i++, WriteLine())
            for (int j = 0; j < size; j++)
            {
                if (i == j) one[i, j] = 1;
                Write(one[i, j] + "\t");
            }
        foreach (int mem in one)
            Write(mem + " ");
        WriteLine();
        WriteLine("one.Length = " + one.Length);
        WriteLine("one.Rank = " + one.Rank);
    }
}
```

Результат выполнения программы:

```
size = 4<ENTER>
1 0 0 0
0 1 0 0
0 0 1 0
0 0 0 1
1 0 0 0 0 1 0 0 0 0 1 0 0 0 0 1
one.Length = 16
one.Rank = 2
```

В программе формируется двумерный массив (экземпляр массивов типа `int[,]`), размеры которого определены переменной `size`. Значение `size` вводит пользователь. По умолчанию все элементы массива получают нулевое значение. Во вложенных циклах диагональным элементам присвоены единичные значения, и выводятся значения всех элементов. Обратите внимание, как в заголовке внешнего цикла `for` используется обращение к методу `Console.WriteLine()` для перехода на новую строку при выводе (заметьте, что в коде имя класса `Console` опущено). Далее иллюстрируется применение цикла `foreach` к многомерному массиву. Перебор значений массива, представляющего матрицу, выполняется по строкам (быстрее изменяется правый индекс).

В конце программы выведены значение свойства `one.Length` — это общее количество элементов в массиве и `one.Rank` — значение ранга массива — ранг равен двум.

7.4. Массивы массивов и «непрямоугольные» массивы

Как следует из синтаксического определения, при объявлении типа массива можно задавать несколько спецификаторов размерностей. В стандарте приведен пример такого типа массивов (с тремя спецификаторами размерностей):

`int [] [,][,]` — одномерный массив, элементы которого — трехмерные массивы, каждый с элементами типа «двумерный массив с элементами типа `int`». Такой массив можно рассматривать как массив массивов.

Так как размеры массивов, входящих как элементы в другой массив, могут быть разными, то массив массивов в общем случае не является «прямоугольным». Такие непрямоугольные массивы в литературе по C# называют **jagged array** (зубчатые массивы). Пример из стандарта:

```
int[][] j2 = new int[3][];  
j2[0] = new int[] {1, 2, 3};  
j2[1] = new int[] {1, 2, 3, 4, 5, 6};  
j2[2] = new int[] {1, 2, 3, 4, 5, 6, 7, 8, 9};
```

Обратите внимание, что с помощью одной операции **new** невозможно создать массив массивов. В примере `j2` — ссылка на объект типа массив массивов с элементами типа `int`. Выражение `new int[3][]` создает объект-массив с тремя элементами типа «ссылка на одномерный массив с элементами типа `int`». Каждая из этих ссылок доступна с помощью соответствующего выражения `j2[0]`, `j2[1]`, `j2[2]`. Однако вначале значения этих ссылок не определены. Только присвоив каждой из них результат выражения «`new int[]` инициализатор», мы связываем ссылки с конкретными одномерными массивами, память для которых за счет выполнения операции **new** будет выделена в куче.

Приведенные четыре объявления можно заменить одним, используя правила инициализации (количество операций **new** при этом не изменится):

```
int[][] j3 = new int[3][]  
{  
    new int[] {1, 2, 3},  
    new int[] {1, 2, 3, 4, 5, 6},  
    new int[] {1, 2, 3, 4, 5, 6, 7, 8, 9}  
};
```

При объявлении массива массивов, как и для других видов массивов, можно не применять инициализаторы, но тогда придется явно указать размеры массивов, и их элементы получат умалчиваемые значения.

Применяя инициализатор, разрешено опускать для массива массивов выражение с операцией **new**, например, декларировать массив таким образом:

```
int[][] j4 =  
{
```

```

    new int[3],
    new int[6],
    new int[9]
};

```

В этом объявлении размер массива верхнего уровня задан списком инициализаторов. Размеры массивов нижнего уровня с элементами типа **int** здесь заданы явно, и значения элементов определяются по умолчанию как нулевые.

В качестве примера с непрямоугольным (зубчатым) массивом рассмотрим следующую программу, формирующую нижнюю нулевую треугольную матрицу. Элементам ее диагонали присвоим номера строк (нумерацию строк будем выполнять, начиная от 1). Для элементов над диагональю память выделяться не будет. Как в примере с единичной матрицей размер матрицы (число ее строк) будет вводить пользователь, как значение переменной **int size**. Текст программы:

```

// 07_06.cs – нижняя треугольная матрица
using static System.Console;
class Program
{
    static void Main()
    {
        int size;
        do Write("size = ");
        while (!int.TryParse(ReadLine(), out size) || size < 1);
        int[][] tre = new int[size][];
        for (int j = 0; j < size; j++)
        {
            tre[j] = new int[j + 1];
            tre[j][j] = j + 1;
        }
        for (int i = 0; i < tre.Length; i++, WriteLine())
            for (int j = 0; j < tre[i].Length; j++)
                Write(tre[i][j] + "\t");
        WriteLine("tre.Length = " + tre.Length);
        WriteLine("tre.Rank = " + tre.Rank);
    }
}

```

Результат выполнения программы:

```

size = 4<ENTER>
1
0 2
0 0 3
0 0 0 4
tre.Length = 4
tre.Rank = 1

```

В программе объявлена ссылка **tre** на массив массивов. Операцией **new** определен одномерный массив из **size** элементов — ссылок на массивы. Каждый элемент **tre[j]** — ссылка на еще не существующий одно-

мерный массив с элементами типа `int`. Эти массивы — реальные строки треугольной матрицы — формируются в цикле. Длина j -го массива равна $j + 1$.

В цикле печати массива для определения числа элементов используется свойство `Length`. Выражение `tre.Length` возвращает число строк матрицы. Обратите внимание, что в отличие от многомерного массива свойство `Length` равно числу элементов только «верхнего» уровня массива массивов. Выражение `tre[j].Length` позволяет определить длину j -й строки. Свойство `Rank`, относящееся к объекту типа `int[ ][ ]`, равно 1, так как это одномерный массив ссылок на массивы. Остальное очевидно из результатов выполнения программы.

Декларируя ссылку на массив и объявляя конкретный объект — экземпляр массива, программист каждый раз определяет некоторый тип именно таких массивов, которые ему нужны. Синтаксис объявления этих типов мы уже разобрали и объяснили с помощью примеров. Следует обратить внимание, что имена этих типов массивов и синтаксис определения типов массивов не похожи на те конструкции, которые применяются для определения пользовательских классов как таковых (вводимых с помощью служебного слова `class`). Однако каждый декларируемый в программе тип массивов является настоящим классом и создается как производный (как наследник) системного класса `Array`. Будучи наследником, каждый тип массивов получает или по-своему реализует методы и свойства (см. табл. 7.1—7.3) класса `Array`. Следующая программа иллюстрирует возможности некоторых методов, о которых мы еще не говорили.

```
// 07_07.cs - методы и свойства класса Array
using static System.Console;
class Program
{
    static void Main()
    {
        double[,] ar = {
            {10, -7, 0, 7},
            {-3, 2.099, 6, 3.901},
            {5, -1, 5, 6},
        };
        WriteLine("ar.Rank = " + ar.Rank);
        WriteLine("ar.GetUpperBound(1) = " + ar.GetUpperBound(1));
        WriteLine("ar.GetLength(1) = " + ar.GetLength(1));
        for (int i = 0; i < ar.GetLength(0); i++, WriteLine())
            for (int j = 0; j <= ar.GetUpperBound(1); j++)
                Write("\t" + ar[i, j]);
    }
}
```

Результат выполнения программы:

```
ar.Rank = 2
ar.GetUpperBound(1) = 3
```

```

ar.GetLength(1) = 4
    10      -7      0      7
    -3      2,099    6      3,901
    5       -1      5      6

```

В программе определен и инициализирован двумерный массив с элементами типа **double**. Результаты выполнения программы поясняют особенности свойств и методов типа массивов **double[,]**, производного от класса **Array**. Обратите внимание, что **GetUpperBound(1)** — верхняя граница второго индекса, а не количество значений этого индекса.

7.5. Массивы массивов и поверхностное копирование

Итак, массив массивов представляет собой одномерный массив, элементами которого являются ссылки на массивы следующего (подчиненного) уровня. Этот факт требует особого внимания, так как затрагивает фундаментальные вопросы копирования ссылок и тех объектов, которые ими адресуются.

Независимо от того, какого вида массив мы рассматриваем, присваивание ссылке на массив значения другой ссылки на уже существующий массив (на объект с типом массива) приводит к появлению двух ссылок на один массив. Это мы уже иллюстрировали и разобрали.

Метод **Clone()** позволяет создать новый экземпляр массива. В программе **07_04.cs** показано, что изменяя один из одномерных массивов-копий, мы не изменяем второй. Следующая программа иллюстрирует применение копирования к многомерному массиву:

```

// 07_08.cs – двумерный массив – полное клонирование
using static System.Console;
class Program
{
    static void Main()
    {
        int size;
        do Write("size = ");
        while (!int.TryParse(ReadLine(), out size) || size < 1);
        int[,] one = new int[size, size];
        WriteLine("Массив one:");
        for (int i = 0; i < size; i++, WriteLine())
            for (int j = 0; j < size; j++)
            {
                if (i == j) one[i, j] = 1;
                Write(one[i, j] + "\t");
            }
        WriteLine("one.Length = " + one.Length);
        int[,] two = (int[,])one.Clone(); // клонирование
        two[0, 0] = - size;
        WriteLine("Массив two:");
        for (int i = 0; i < size; i++, WriteLine())
            for (int j = 0; j < size; j++)
                Write(two[i, j] + "\t");
    }
}

```

```

        WriteLine("Массив one:");
        for (int i = 0; i < size; i++, WriteLine())
            for (int j = 0; j < size; j++)
                Write(one[i, j] + "\t");
    }
}

```

Результат выполнения программы:

```

Массив one:
1 0 0 0
0 1 0 0
0 0 1 0
0 0 0 1
one.Length = 16
Массив two:
-4 0 0 0
0 1 0 0
0 0 1 0
0 0 0 1
Массив one:
1 0 0 0
0 1 0 0
0 0 1 0
0 0 0 1

```

В программе определена ссылка `two` типа `int[,]`, и ей присвоен результат копирования массива, связанного со ссылкой `one`, имеющей тот же тип `int[,]`. Выведена единичная матрица, адресованная ссылкой `one`, затем изменен обычным присваиванием один элемент массива-копии:

```
two[0, 0] = - size;
```

Выведенные на консоль значения элементов массивов иллюстрируют независимость массива-оригинала от массива-копии.

Программы `07_04.cs` и `07_08.cs` выполняют с помощью метода `Clone()` копирование массивов, у каждого из которых один спецификатор размерности. В первом случае массив одномерный, во второй программе копируется двумерный массив. Результат копирования в обоих случаях вполне ожидаемый.

Применяя метод `Clone()` к массиву массивов, мы сталкиваемся с очень важной особенностью. Строго говоря, действия метода остаются прежними — он создает массив-копию и присваивает его элементам значения элементов массива-оригинала. Однако в этом случае копирования тех подчиненных массивов, на которые «смотрят» ссылки (элементы массива-оригинала), не происходит. Выполняется так называемое поверхностное или поразрядное копирование. Иначе и быть не должно — «не знает» метод `Clone()`, что код, который является значением элемента копируемого массива, представляет собой ссылку, и по этой ссылке нужно еще что-то «доставать».

Таким образом, копируя с помощью метода Clone() массив массивов, мы получаем два экземпляра массива верхнего уровня, элементы которых адресуют одни и те же участки памяти, выделенные для подчиненных массивов объекта-оригинала.

В качестве иллюстрации указанной ситуации приведем следующую программу, построенную на основе 07_06.cs:

```
// 07_09.cs - массив массивов - клонирование поверхностное!
using static System.Console;
class Program
{
    static void Main()
    {
        int size;
        do Write("size = ");
        while (!int.TryParse(ReadLine(), out size) || size < 1);
        int[][] tre = new int[size][];
        for (int j = 0; j < size; j++)
        {
            tre[j] = new int[j + 1];
            tre[j][j] = j + 1;
        }
        WriteLine("Массив tre:");
        for (int i = 0; i < tre.Length; i++, WriteLine())
            for (int j = 0; j < tre[i].Length; j++)
                Write(tre[i][j] + "\t");
        WriteLine("tre.Length = " + tre.Length);
        int[][] two = (int[][])tre.Clone();
        two[0][0] = -size;
        WriteLine("Массив two:");
        for (int i = 0; i < two.Length; i++, WriteLine())
            for (int j = 0; j < two[i].Length; j++)
                Write(two[i][j] + "\t");
        WriteLine("Массив tre:");
        for (int i = 0; i < tre.Length; i++, WriteLine())
            for (int j = 0; j < tre[i].Length; j++)
                Write(tre[i][j] + "\t");
    }
}
```

Результат выполнения программы:

```
size = 4<ENTER>
Массив tre:
1
0 2
0 0 3
0 0 0 4
tre.Length = 4
Массив two:
-4
0 2
0 0 3
0 0 0 4
```


Массив `tre`:

```
-4  
0 2  
0 0 3  
0 0 0 4
```

В программе определена ссылка `two` типа `int[ ][ ]` и ей присвоен результат копирования (клонирования) «треугольного» массива, адресованного ссылкой `tre`, имеющей тип `int[ ][ ]`. С помощью оператора

```
two[0][0] = -size;
```

изменен один целочисленный элемент «нижнего уровня» массива массивов `two`. После присваивания изменилось значение, соответствующее выражению `tre[0][0]`.

Контрольные вопросы и задания

1. Являются ли типы массивов типами значений?
2. Какой тип может иметь индексирующее выражение?
3. Где размещается (в стеке или в куче) объявленная в статическом методе ссылка на массив?
4. При выполнении какой операции создается объект класса массивов?
5. Что такое класс массивов?
6. Какие значения принимают элементы массива при отсутствии в его определении инициализатора?
7. Какова структура инициализатора массива?
8. Чем определяется количество инициализирующих выражений в инициализаторе массива?
9. Объясните назначение всех элементов заголовка цикла **foreach**.
10. Каково назначение и возможности переменной цикла **foreach**?
11. Какими средствами можно изменить размер массива-объекта в процессе выполнения программы?
12. Назовите свойства массивов, унаследованные ими от класса `Array`.
13. Приведите примеры нестатических методов класса `Array`.
14. Приведите примеры статических методов класса `Array`.
15. Выясните, в чем различия методов `Copy()` и `Clone()`.
16. Что такое размерность массива?
17. Что такое спецификатор размерности массива?
18. Допустимо ли динамическое определение размеров многомерных массивов?
19. Чему равно свойство `Length` для многомерного массива?
20. С помощью каких средств можно получить размер многомерного массива по нужному измерению?
21. Сколько спецификаторов размерности в объявлении типа четырехмерного массива?

22. Перечислите синтаксические отличия массива массивов от двумерного массива.
23. Сколько операций **new** в определении объекта (экземпляра) трехмерного массива?
24. Чему равно свойство Rank массива массивов?
25. В каком случае при клонировании массива проявляется эффект поверхностного копирования?

Глава 8

СТРОКИ — ОБЪЕКТЫ КЛАССА `STRING`

8.1. Строковые литералы

Для представления текстовой информации в C# используются объекты класса **string**. Класс **string** представляет собой один из predefined типов языка C#. В .Net Framework этому типу соответствует класс `System.String`. Один из видов объектов класса **string** мы уже многократно применяли — это строковые константы или строковые литералы.

Строковая константа или строковый литерал имеет (в соответствии со стандартом C# [1]) две формы — обычный (*регулярный*) строковый литерал (*regular-string-literal*) и буквальный строковый литерал (*verbatim-string-literal*). До сих пор мы использовали только обычную (*регулярную*) форму, традиционную для языков программирования. Регулярный строковый литерал — это последовательность символов и эскейп-последовательностей, заключенная в кавычки (не в апострофы). В регулярных строковых константах для представления специальных символов используются те же эскейп-последовательности, которые применяются в константах типа **char**. Обработывая регулярный строковый литерал, компилятор из его символов формирует строковый объект и при этом заменяет эскейп-последовательности соответствующими кодами (символов или управляющих кодов). Таким образом, литералу

```
"\u004F\x4E\u0045\ttwo"
```

будет соответствовать строка, при выводе которой на экране текст появится в таком виде:

```
ONE  two
```

Здесь `\u004F` — юникод символа 'O', `\x4E` — шестнадцатеричный код символа 'N', `\u0045` — юникод символа 'E', `\t` — эскейп-последовательность, представляющая код табуляции.

Буквальный строковый литерал начинается с префикса `@`, за которым в кавычках размещается возможно пустая последовательность символов. Символы такого литерала воспринимаются буквально, т. е. в такой строке не обрабатываются эскейп-последовательности, а каждый символ воспринимается как таковой.

В результате выполнения оператора:

```
Console.WriteLine(@"\u004F\x4E\u0045\ttwo");
```

на экране появится

```
\u004F\x4E\u0045\ttwo
```

Если в буквальном литерале необходимо поместить кавычку, то она изображается двумя рядом стоящими кавычками.

Важно отметить, что буквальный литерал может быть размещен в коде программы на нескольких строках и это размещение сохраняется при его выводе. В тоже время попытка перехода при выводе на новую строку с помощью размещенной в буквальном литерале эскейп-последовательности `\n` будет безуспешной.

В язык C# 7.0 введен синтаксис интерполируемых строк, которые мы рассмотрим в параграфе 8.6. Литерал интерполируемой строки снабжается префиксом `$`.

8.2. Строковые объекты и ссылки типа `string`

Каждый строковый литерал — это объект класса **string**. Класс **string** является типом ссылок. Кроме литералов можно определить объекты класса **string** с использованием конструкторов (конструктор — специальный метод класса, предназначенный для инициализации объекта класса в процессе его создания). Конструкторы класса **string** позволяют инициализировать объекты-строки несколькими способами.

Наиболее простая форма создания строки-объекта — применение строкового литерала в качестве инициализирующего выражения. Пример:

```
string line1 = "Это строка 1";
```

После выполнения этого объявления создается ссылка `line1` типа **string**, и она ассоциируется со строковым литералом, который является объектом класса **string**.

Строковый объект можно создавать, используя массив символов:

```
char [] car = {'М', 'а', 'с', 'с', 'у', 'в'};  
string line2 = new string (car);
```

В данном примере определен символьный массив. Представляющая его ссылка `car` используется в качестве аргумента при обращении к конструктору класса **string**. Значением создаваемого объекта будет строка "Массив". Чтобы получить строку, содержащую один символ, используется конструктор с первым параметром типа **char** и вторым параметром, равным 1.

Пример:

```
string line3 = new string ('W', 1);
```

`line3` представляет строку "W".

Если нужна строка, содержащая последовательность одинаковых символов, то применяется конструктор с двумя параметрами. Первый из них определяет нужный символ, а второй — число его повторений.

Пример:

```
string line4 = new string ('7', 3);
```

В данном случае line4 это ссылка на строку "777".

Второй параметр может быть любым целочисленным выражением, поэтому этот конструктор удобно применять в тех случаях, когда количество повторений символа заранее неизвестно, т. е. зависит от каких-то изменяемых при выполнении программы данных.

Обратите внимание, что среди конструкторов класса **string** нет конструктора с параметром типа **string**.

Строковые объекты, как создаваемые с применением конструкторов, так и формируемые для представления строковых литералов, компилятор размещает в куче. Ссылки на строки размещаются в стеке. Размер строки при определении строкового объекта явно не указывается, он определяется автоматически при инициализации. Ни размер строки, ни ее содержимое не могут изменяться после создания строки!!!

Если инициализация при объявлении строковой ссылки отсутствует, то ей присваивается значение **null**, и ее нельзя использовать в выражениях до присваивания ей конкретного значения. Пример ошибки:

```
string line;  
line += "asdfg"; // ошибка компиляции
```

Кроме явного задания строковых литералов и применения конструкторов для создания строковых объектов используют метод ToString(). Этот метод определен для всех встроенных типов. Например, значением выражения

```
242.ToString()
```

будет строковый объект, содержащий изображение символов целого числа "242".

После выполнения операторов

```
bool b = 5 > 4;  
string sb = b.ToString();
```

значением sb будет строка "True".

8.3. Операции над строками

Строки языка C# предназначены для хранения последовательностей символов, для каждого из которых отводится 2 байта, и они хранятся в кодировке Unicode (как данные типа **char**). В некотором смысле

строка подобна одномерному массиву с элементами типа **char**. Элементы (символы строки) последовательно нумеруются, начиная с 0. Последний символ имеет номер на 1 меньший длины строки.

Для строковых объектов определена операция индексирования:

строка[индексирующее_выражение]

строка — это ссылка на объект класса **string** или строковая константа; *индексирующее_выражение* должно быть целочисленным, не может быть отрицательным и всегда меньше длины строки.

Результат выражения с операцией индексирования — символ (значение типа **char**), размещенный в той позиции строки, номер которой соответствует индексному выражению. Если значение индекса меньше нуля, а также больше или равно длине строки, возникает исключительная ситуация (генерируется исключение).

Выражение с операцией индексирования, примененное к строке, только *правдоподобное*. С его помощью запрещено (невозможно) изменять соответствующий элемент строки.

В качестве примера рассмотрим выражение, формирующее символьное представление шестнадцатеричной цифры по ее десятичному значению в диапазоне от 0 до 15:

```
"0123456789ABCDEF"[ind]
```

Если `ind == 13`, то значением выражения будет 'D'.

Если нужно применить цикл **foreach** для перебора элементов объекта типа **string**, то в цикле **foreach** используется итерационная переменная типа **char**. В этом случае строковый литерал либо объект класса **string** (представляемый ссылкой на такой объект) выступает в качестве контейнера с элементами типа **char**. Пример со строковым литералом:

```
foreach (char numb in "0123")  
    Console.Write("\t" + numb + "->" + (int)numb);
```

Результат:

```
0->48    1->49    2->50    3->51
```

Операция присваивания (=) для строк выполняется не так, как, например, для массивов. Когда ссылке с типом массива присваивается значение ссылки на другой, уже существующий, массив, изменяется только значение ссылки. Массив как объект становится доступен для нескольких ссылок.

Операция присваивания для строк приводит к созданию нового экземпляра той строки, на которую ссылается выражение справа от знака операции =. Ранее существовавшая строка никак не ассоциируется с новой ссылкой.

Сказанное иллюстрирует рис. 8.1, на котором приведена схема, соответствующая следующим объявлениям:

```
char [] aCh = {'Б','и','т'};
char[] bCh = aCh;

string aSt = "Байт";
string bSt = aSt;
```

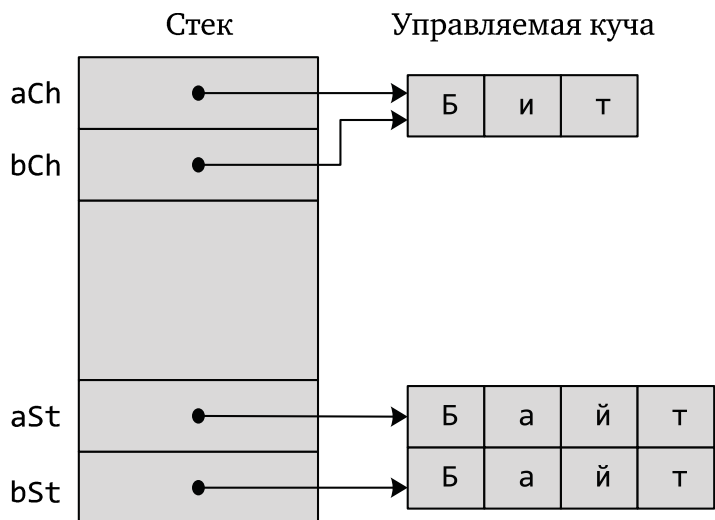


Рис. 8.1. Присваивания для ссылок на массивы и на строки

Операции сравнения на равенство `==` и неравенство `!=`, применяемые к ссылкам на строки, сравнивают последовательности символов в строках (отметим, что для массивов сравниваются значения ссылок). Следующий фрагмент кода демонстрирует различие между массивами и строками при выполнении сравнений:

```
char [] ar1 = {'A', 'r', 'r', 'a', 'y'};
char [] ar2 = {'A', 'r', 'r', 'a', 'y'};
Console.WriteLine(ar1 == ar2); // False
string st1 = "String";
string st2 = "String";
Console.WriteLine(st1 == st2); // True
```

Конкатенацию строк выполняет операция `+`. Например, после приведенного выше определения ссылки `aSt` выражение

```
aSt + " состоит из 8 бит"
```

приведет к формированию строки "Байт состоит из 8 бит".

Операция `+` перегружена и обеспечивает конкатенацию не только двух строк, а также строки и операнда другого типа, значение которого автоматически (за счет неявного обращения к методу `ToString()`) приводится к типу **string**. Например, выражение

```
aSt + " содержит " + 8 + " бит"
```

формирует строку "Байт содержит 8 бит".

Однако значением выражения

```
aSt+" содержит " + 8 + aCh
```

будет строка "Байт содержит 8 System.Char[]". В этом случае ссылка на массив aCh приводится к обозначению типа символьного массива (значением выражения aCh.ToString() является "System.Char[]").

Операция + как конкатенация имеет тот же ранг, что и операция +, обозначающая сложение арифметических данных. Поэтому значением выражения

```
aSt + " содержит "+ 4 + 4 + " бит"
```

будет строка "Байт содержит 44 бит".

По-видимому, это не то, что ожидали. Но ведь операции одного ранга выполняются слева направо.

8.4. Некоторые методы и свойства класса string

Хотя мы еще не рассматривали механизм и синтаксис наследования и не умеем конструировать производные классы, но уже сейчас полезно отметить, что класс **string** есть *sealed*-класс, т. е. он не может служить базовым для других (производных) классов.

Наиболее важным свойством класса **string** является свойство Length, позволяющее получить длину (количество символов) конкретной строки (объекта класса **string**). Значением выражения "\tБином\u0068".Length будет 7, так как каждая эскейп-последовательность представляет только один символ.

Среди многочисленных методов класса **string** рассмотрим наиболее важные, не останавливаясь на существовании их многочисленных версий.

Таблица 8.1

Статические методы класса String (выборочно)

Метод	Описание
Compare	Сравнивает два строки (или подстроки) и возвращает целое число, которое показывает их относительное положение в порядке сортировки
CompareOrdinal	Сравнивает два строки (или подстроки) при помощи сравнения кодов их символов
Concat	Выполняет конкатенацию строк-параметров, строковых представлений параметров, элементов коллекций или массивов
Copy	Создает копию строки-параметра
Format	Заменяет в первом параметре (в строке форматирования) поля подстановок строковыми представлениями последующих аргументов (метод Format подробно рассматривается в параграфе 8.5)
Join	Объединяет в одну строку строки массива-параметра. Первый параметр типа string задает разделитель, которым будут отделены друг от друга в результирующей строке элементы массива

Нестатические методы класса String (выборочно)

Метод	Описание
CompareTo	Сравнивает две строки и возвращает целочисленное значение. Для двух строк S1, S2 значение S1.CompareTo(S2) равно +1, если S1 > S2, и равно -1, если S1 < S2 и равно нулю, если S1 == S2. Сравнение строк выполняется лексикографически
EndsWith	Проверяет, совпадает ли конец вызывающей строки со строкой, определенной параметрами
Equals	Сравнивает значение вызывающей строки со строкой, определенной параметрами
IndexOf	Выполняет поиск в вызывающей строке подстроки, определенной параметрами. Возвращает индекс или -1, если поиск неудачен
Insert	Вставляет строку-параметр в копию вызывающей строки с позиции, заданной дополнительным параметром
LastIndexOf	Поиск с конца вызывающей строки подстроки, заданной параметром. Возвращает индекс или -1, если поиск неудачен
Remove	Возвращает копию вызывающей строки, в которой удалено указанное число символов, начиная с указанной позиции
Replace	Заменяет символы или подстроки в копии вызывающей строки
Split	Формирует массив строк из фрагментов вызывающей строки. Параметр задает разделители, которыми в вызывающей строке разделены выделяемые фрагменты
StartsWith	Проверяет, совпадает ли начало вызывающей строки со строкой, определенной параметрами
Substring	Выделяет из вызывающей строки подстроку. Параметры задают начало и длину выделяемой части строки
ToCharArray	Копирует символы вызывающей строки в массив типа char[]
ToLower	Возвращает копию вызывающей строки, переведенную в нижний регистр
ToUpper	Возвращает копию вызывающей строки, переведенную в верхний регистр
Trim	Удаляет из копии вызывающей строки вхождение заданных символов (например, пробелов) в начале и в конце строки
TrimEnd	Удаляет из копии вызывающей строки вхождение заданных символов (например, пробелов) в конце строки
TrimStart	Удаляет из копии вызывающей строки вхождение заданных символов (например, пробелов) в начале строки.

8.5. Форматирование

Под форматированием понимают преобразование значения конкретного типа в соответствующее ему строковое представление.

При выводе, например, с помощью `Console.Write()` значений базовых типов (например, **int** или **double**) они автоматически преобразуются в символьные строки. Если программиста не устраивает автоматически выбранный формат их внешнего представления, он может его изменить. Для этого можно воспользоваться статическим методом `Format` класса **string** или использовать так называемую строку форматирования в качестве первого параметра методов, поддерживающих форматирование, например, `Console.Write()` и `Console.WriteLine()`. В обоих случаях правила подготовки исходных данных для получения желаемого результата (новой строки) одинаковы. Рассмотрим эти правила применительно к методу `Format` класса **string**:

```
public static string Format(string form, params object[ ] ar)
```

Так как синтаксис заголовков методов (функций) мы формально еще не рассматривали, поясним элементы приведенного заголовка.

public — модификатор доступа. Указывает что метод открытый, т. е. доступен вне класса **string**.

static — модификатор, относящий метод к классу в целом, а не к его объектам.

string — тип возвращаемого методом значения

`Format` — имя метода.

`form` — произвольно выбранное имя параметра (типа **string**), представляющего строку форматирования.

params — модификатор, указывающий, что следующий за ним массив-параметр `ar` может быть заменен при обращении к методу любым количеством аргументов (включая нуль, т. е. аргументы могут отсутствовать).

object — тип элементов массива-параметра `ar`. Так как **object** — это базовый класс для всех типов языка C#, то аргументы, соответствующие элементам массива `object[ ] ar`, могут иметь любой тип.

Итак, метод `string.Format()` возвращает строку, построенную на основе строки форматирования и значений аргументов, следующих за этой строкой в обращении к методу `Format()`.

Строка форматирования включает неизменяемые символы и конструкции, называемые *полями подстановок*. Структура поля подстановки:

```
{N[,W][:S[R]]},
```

где `N` — номер аргумента; `W` — ширина поля; `S` — спецификатор формата; `R` — спецификатор точности.

Квадратные скобки в поля подстановки не входят и обозначают необязательность ограниченного ими фрагмента (элемента) поля. В поле

подстановки разрешено опускать все элементы кроме фигурных скобок и номера аргумента. Именно поэтому все остальные элементы поля подстановки мы ограничили квадратными скобками. Воспользуемся этим правилом, чтобы привести такой пример.

```
int num = 23, den = 6;
string result, // ссылка на строку, с результатом
    form = "Числитель: {0}, знаменатель: {1}"
        + ", дробь: {0}/{1}=={2}";
result = string.Format(form, num, den, num/den);
Console.WriteLine(result);
```

В коде определены две ссылки `result` и `form` на объекты типа **string**. Значение строки, связанной со ссылкой `form`, определено инициализирующим выражением в виде конкатенации двух строковых литералов. В них пять полей подстановок. Номера в полях указывают, символьные представления значений каких аргументов (по счету, начиная с 0) будут подставлены вместо соответствующих полей. В обращении к методу `Format()` первый аргумент — строка `form`, затем `num` — аргумент, с номером 0, `den` — аргумент, которому соответствует номер 1, `num/den` — выражение, значение которого воспринимается как аргумент с номером 2. В результате выполнения подстановок метод `Format()` возвращает строку, ссылка на которую присвоена переменной `result`. Выводя эту строку на консоль, получим:

```
Числитель: 23, знаменатель: 6, дробь: 23/6==3
```

Обратите внимание на округление результата целочисленного деления.

В приведенном примере поля подстановок содержали только номера аргументов. Разберем назначение других элементов, которые в поле подстановки необязательны. Запятая и следующее за ним число (обозначенное в структуре поля подстановки как `W`) в поле подстановки определяет количество позиций, выделяемых для изображения подставляемого значения. Если эти элементы (запятая и число) опущены, то число позиций определяется изображением выводимого значения, т. е. ширина поля выбирается минимально достаточной для изображения значения. Если ширина поля `W` указана, и она превышает длину помещаемого в поле значения, то при положительной длине поля `W` изображаемое значение выравнивается по правой границе. Если перед шириной поля `W` стоит минус, то выравнивание выполняется по левой границе поля.

Спецификатор формата `S` задает вид изображаемого значения. Для разных типов данных этот спецификатор выбирается по-разному. Следующая за ним необязательная цифра — спецификатор точности `R` — влияет на формируемое значение, и это влияние зависит от спецификатора формата.

Спецификаторы формата S и точности R

Специфи- катор S	Название формата	Роль спецификатора точности R
C или c	валютный	Количество десятичных разрядов
D или d	целочисленный	Минимальное число цифр
E или e	экспоненциальный	Число разрядов после точки
F или f	с фиксированной точкой	Число разрядов после точки
N или n	число с разделителем триад	Число знаков после точки
P или p	значение в процентах (%)	Число знаков после точки
G или g	короткий из E или F	Подобен E или F
X или x	шестнадцатеричный	Минимальное число цифр

Пример:

```
double dou = 1234.567;
string form = "Спецификация E4: {0:E4}" +
    ", спецификация F4: {0:F4}";
string result = string.Format(form, dou);
Console.WriteLine(result);
```

В обоих полях подстановки строки форматирования опущена ширина поля, и предполагается дважды использовать только один аргу-
мент (с номером 0).

Выводимая на консоль строка:

```
Спецификация E4: 1,2346E+003, спецификация F4: 1234,5670
```

Обратите внимание на округление при выводе по формату E4 и на дополнительный нуль в изображении числа с фиксированной точкой при выводе по формату F4. Это определяется спецификатором точности, равным 4.

Пример с целочисленным значением, выводимым с разными осно-
ваниями в поля из четырех позиций. Обратите внимание, что ширина
первого поля отрицательна:

```
int num = 23;
string form = "Основание 10: {0, -4:D}\n основание 16: {0,4:X}";
Console.WriteLine(form, num);
```

Результат в консольном окне:

```
Основание 10: 23
основание 16: 17
```

В этом примере форматирование выполняется непосредственно при
обращении к методу WriteLine(). Первый аргумент выступает в роли

форматной строки, а вызов метода `string.Format()` осуществляется неявно.

Если в этом примере в полях подставки указать спецификаторы точности, то в изображениях целых чисел могут появиться незначащие нули. Пример:

```
int num = 23;  
string form = "Основание 10: {0, -4:D3}\n основание 16: {0,4:X3}";  
Console.WriteLine(form, num);
```

Результат:

```
Основание 10: 023  
основание 16: 017
```

Примечание. Если в поле подстановки указана ширина, недостаточная для представления выводимого значения, то ширина поля будет автоматически увеличена до необходимого количества позиций.

8.6. Форматирование в `ToString()` и интерполяция строк

До сего времени мы использовали метод `ToString()` без параметров. Именно в таком виде метод наследуется всеми классами от общего базового класса **object**. Реализация этого метода в **object** предполагает, что метод возвращает строку, представляющую полное имя типа того объекта для которого метод без параметров вызван. Чтобы метод представлял в виде строки «значение объекта», этот метод в каждом из базовых классов платформы .NET перегружен. Именно поэтому, применяя метод `ToString()`, например, к целочисленному литералу, мы получаем строковое представление значения соответствующего целого числа, а не строку "System.Int32".

Однако, применяя метод без параметров, программист не может по своему усмотрению форматировать строку с изображением обрабатываемого значения. Кроме того, строку нельзя форматировать с учетом региональных стандартов. Например, в Европейских государствах при записи вещественного числа дробную часть отделяют от целой запятой, а в США для этой цели используют символ точки.

Для устранения указанных затруднений в каждом из классов, представляющие базовые типы платформы .NET, кроме метода `ToString()` без параметров, включены варианты того же метода с одним и с двумя параметрами. Первый из этих параметров обеспечивает управление форматированием создаваемой строки. Второй параметр служит для указания региональных стандартов.

Остановимся на возможностях первого из названных параметров метода `ToString()`. Для задания соответствующего ему аргумента используется форматная строка со спецификатором формата (S) и при

необходимости со спецификатором точности (R, см. табл. 8.3.) В качестве примера приведем следующий фрагмент кода:

```
double pi = Math.PI;  
Console.WriteLine(pi.ToString("F"));  
Console.WriteLine(pi.ToString("F4"));  
Console.WriteLine((pi*100).ToString("E4"));
```

На консольный экран будет выведено:

```
3,14  
3,1416  
3,1416E+002
```

Более сложное форматирование числовых данных в методе ToString() можно организовать, используя в форматной строке так называемые «пользовательские спецификаторы формата». С ними можно познакомиться, обратившись к документации или к справочной литературе ([15]). А сейчас рассмотрим еще одну возможность модифицировать формируемые строки.

В C# 7.0 введены так называемые *интерполируемые строки*. Интерполируемая строка — это строковый литерал, перед которым помещен символ \$. Внутри этого литерала могут находиться *интерполируемые выражения*. Каждое из них представляет собой пару фигурных скобок, между которыми помещено любое выражение языка C#. Чтобы сразу кое-что прояснить, приведем пример кода:

```
int числитель = 5,  
    знаменатель = 3;  
string report = $"Дробь: {числитель}/{знаменатель} равна "  
    + $"{{(double)числитель/знаменатель:F4}}";  
Console.WriteLine(report);
```

На консольный экран будет выведено:

```
Дробь: 5/3 равна 1,6667
```

В коде значение переменной **string report** определяют две интерполируемых строки, соединенные знаком конкатенации. В первой из названных строк два интерполируемых выражения, каждое из которых при обработке заменяется значением соответствующей целочисленной переменной *числитель* и *знаменатель*. Во второй строке интерполируемое выражение более сложное. С его помощью вычисляется *вещественное* значение дроби, к которому затем применяется форматная строка "F4". Интерполируемое выражение отделяется от форматной строки двоеточием.

Отметим некоторые особенности механизма интерполируемых строк. Каждая интерполируемая строка должна размещаться в одной строке кода (без переносов). В интерполируемую строку можно включать эскейп-последовательности.

Чтобы включить в интерполируемую строку символы '{' или '}', не входящие в интерполируемые выражения, каждый из таких символов фигурной скобки необходимо дублировать.

Строковый литерал, входящий в интерполируемую строку, может быть буквальным. В этом случае символ \$ должен размещаться перед символом @.

Подчеркнем, что в интерполируемые выражения можно помещать любые выражения, допустимые в языке C#. Кроме того, значения выражений можно очень гибко форматировать. Например, можно задавать ширину поля и выравнивание изображения интерполируемого выражения.

8.7. Применение строк в переключателях

Мы уже упоминали, что объекты класса **string** (и ссылки на них) могут использоваться в метках переключателя и служить значением переключающего выражения. Для иллюстрации этих возможностей рассмотрим схему решения такой задачи: «Ввести фамилию человека (например, служащего компании) и вывести имеющиеся сведения о нем». Характер этих сведений и конкретные сведения нам не важны — покажем общую схему решения подобных задач с использованием переключателя и строк.

```
Console.Write("Введите фамилию:");
string name = Console.ReadLine();
switch(name)
{
    case "Сергеев": Console.WriteLine("Фрол Петрович");
        ...Вывод данных о Сергееве...
        break;
    case "Туров":
        ...Вывод данных о Турове...
        break;
    ...
    default: Console.Write("Нет сведений");
}
```

8.8. Массивы строк

Как любые объекты, строки можно объединять в массивы. Хотя внимательный читатель заметит, что в массив помещаются не строки, а только ссылки на них, но при использовании массивов ссылок на строки не требуются никакие специальные операции для организации обращения к собственно строкам через ссылки на них. Поэтому в литературе, посвященной языку C#, зачастую говорят просто о массивах строк. В следующей программе создается массив ссылок на строки, по умолчанию инициализированный пустыми строками. Затем в цикле

элементам массива (ссылкам) присваиваются значения ссылок на объекты-строки разной длины. Далее к массиву применяется оператор перебора элементов контейнеров **foreach**, и строки выводятся на консоль.

```
// 08_01.cs – массивы строк...
using System;
class Program
{
    static void Main()
    {
        string[] stAr = new string[4];
        for (int i = 0; i < stAr.Length; i++)
            stAr[i] = new string('*', i + 1);
        foreach (string rs in stAr)
            Console.Write("\t" + rs);
    }
}
```

Результат выполнения программы:

```
* ** *** ****
```

В программе создан массив из четырех пустых строк. Он представлен ссылкой `stAr`. Обратите внимание, что для создания объектов, адресуемых элементами массива, применяется конструктор с прототипом **string (char, int)**; Этот конструктор создает строку в виде последовательности одинаковых символов, количество которых определяет второй параметр (первый параметр позволяет задать повторяемый символ). Итерационная переменная цикла **foreach** имеет тип **string**, так как просматриваемым контейнером служит массив типа **string[]**.

Для иллюстрации возможностей методов `Split()`, `Join()` рассмотрим следующую задачу. Пусть значением строки является предложение, слова которого разделены пробелами. Требуется заменить каждый пробел последовательностью символов `"-:-"`. Следующая программа решает сформулированную задачу.

```
// 08_02.cs - декомпозиция и объединение строк
using System;
class Program
{
    static void Main()
    {
        string sent = "De gustibus non est disputandum";
        // О вкусах не спорят
        string[] words; // ссылка на массив строк
        words = sent.Split(' ');
        Console.WriteLine("words.Length = " + words.Length);
        foreach (string st in words)
            Console.Write("{0}\t", st);
        sent = string.Join("-:-", words);
        Console.WriteLine("\n" + sent);
    }
}
```


Результат выполнения программы:

```
words.Length = 5  
De gustibus non est disputandum  
De-:-gustibus-:-non-:-est-:-disputandum
```

В строке, связанной со ссылкой `sent`, помещены слова, разделенные пробелами. Определена ссылка `words` с типом массива строк (объектов класса **string**). Обращение `sent.Split(' ')` «разбивает» строку, адресованную ссылкой `sent`, на фрагменты. Признак разбиения — символ пробела ' ', использованный в качестве аргумента. Из выделенных фрагментов формируется массив (объект класса **string[]**), и ссылка на него присваивается переменной `words`.

Выражение `words.Length` равно длине (количеству элементов) сформированного массива. Напомним, что `Length` — свойство класса массивов **string[]**, унаследованное от базового класса `Array`.

Оператор **foreach** перебирает элементы массива `words`, и итерационная переменная `st` последовательно принимает значения каждого из них (напомним, что особенность итерационной переменной состоит в том, что она позволяет только получать, но не позволяет изменять значения перебираемых элементов). В теле цикла **foreach** один оператор — обращение к статическому методу `Write()` класса `Console`. Его выполнение обеспечивает вывод значений элементов массива (строк). Эскейп-последовательность `'\t'` в форматной строке разделяет табуляцией выводимые слова.

Статический метод `Join()` предназначен для выполнения действий в некотором смысле обратных действиям метода `Split()`. Обращение `string.Join("-:-", words)` объединяет (конкатенирует) строки массива, представленного ссылкой `words`, т. е. соединяет в одну строку слова исходного предложения. Между словами вставляется как разделитель строка, использованная в качестве первого аргумента метода `Join()`. Тем самым каждый пробел между словами исходной строки заменяется строкой `"-:-"`.

8.9. Сравнение строк

Для объектов класса **string** определены только две операции сравнения `==` и `!=`. Если необходимо сравнивать строки по их упорядоченности, например, в лексикографическом порядке, то этих двух операций недостаточно. Поэтому часто используют нестатический метод `CompareTo()` и статический метод класса **string** (точнее набор перегруженных методов) с именем `Compare`, позволяющие сравнивать строки или их фрагменты. Так как сравнение и сортировка строк очень важны в задачах обработки текстовой информации, то рассмотрим два варианта метода `Compare()`.

Наиболее простой метод имеет следующий заголовок:

```
int static Compare (string, string)
```

Сравнивая две строки, использованные в качестве аргументов, метод возвращает значение 0, если строки равны. Если первая строка лексикографически меньше второй — возвращается отрицательное значение. В противном случае возвращаемое значение положительно.

В следующем фрагменте кода определен массив ссылок на строки, из него выбирается и присваивается переменной `res` ссылка на лексикографически наибольшую строку:

```
string[] eng = {"one", "two", "three", "four"};  
string res = eng[0];  
    foreach (string num in eng)  
        if (string.Compare(res, num) < 0)  
            res = num;
```

Значением строки, связанной со ссылкой-переменной `res`, будет "two".

Проиллюстрированная форма метода `Compare()` оценивает лексикографическую упорядоченность, соответствующую английскому алфавиту. Следующий вариант этого метода позволяет использовать разные алфавиты.

```
int static Compare (string, string, Boolean, CultureInfo)
```

Первые два параметра — сравниваемые строки. Третий параметр указывает на необходимость учитывать регистр символов строк. Если он равен **true**, то регистры не учитываются и строки "New" и "nEw" будут считаться равными. Четвертый параметр — объект класса `System.Globalization.CultureInfo` — позволяет указать алфавит, который необходимо использовать при лексикографическом сравнении строк. Для построения объекта, который может быть использован в качестве аргумента, заменяющего четвертый параметр, используют конструктор класса `CultureInfo(string)`. Аргумент конструктора — строка, содержащая условное обозначение нужного алфавита. Точнее сказать, обозначается не алфавит, а культура (Culture), которой принадлежит соответствующий язык. Для обозначения национальных культур приняты имена и коды, таблицу которых можно найти в литературе и документации. Мы в наших примерах будем использовать два алфавита: английский (имя культуры "en", код культуры 0x0009) и русский ("ru" и 0x0019).

В следующем фрагменте кода, который построен по той же схеме, что и предыдущий, определен массив ссылок на строки с русскими названиями представителей семейства тетеревиных из отряда куриных. Из массива выбирается ссылки на лексикографически наименьшую строку, т. е. после упорядочения по алфавиту расположенную в начале списка.

```
string[] hens = {"Куropатка белая", "Куropатка тундровая",  
    "Тетерев", "Глухарь", "Рябчик"};
```

```
string res = hens[0];
foreach (string hen in hens)
    if (string.Compare(res, hen, true,
        new System.Globalization.CultureInfo("ru")) > 0)
        res = hen;
Console.WriteLine(res);
```

Результат, выводимый на консоль:

Глухарь

Обратите внимание, что четвертый параметр метода Compare() заменен в вызове безымянным объектом класса CultureInfo, сформированным конструктором класса в выражении с операцией **new**. Аргумент конструктора — литерная строка "ru" — обозначение нужного алфавита (в данном примере — русского языка). Если возвращаемый результат больше нуля, т. е. первая строка, именуемая ссылкой res, лексикографически больше второй, то вторая строка, связанная со ссылкой hen, принимается в качестве претендента на наименьшее значение.

8.10. Преобразования с участием строкового типа

Рассматривая арифметические типы, мы привели правила неявных преобразований и операцию явного приведения типов:

(тип) первичное_выражение

К строковому типу неявные преобразования неприменимы, и невозможно использование операции явного приведения типов.

Как уже отмечалось, для всех типов существует метод ToString(), унаследованный всеми классами от единого базового класса **object**. Таким образом, значение любого типа можно представить в виде строки, например, так:

```
int n = 8, m = 3;
Console.WriteLine(m.ToString()+n.ToString() + " попугаев");
```

Результат вывода на консоль:

38 попугаев

Для обратного преобразования из строки в значение нужного типа можно воспользоваться статическими методами библиотечного класса Convert, принадлежащего пространству имен System. Еще один путь — применение статического метода Parse() или метода TryParse(). Указанные методы (их применение кратко рассмотрено в предыдущих главах) определены в каждом классе предопределенного типа, за исключением класса **object** (в котором они не нужны). Эти методы часто применяются при чтении данных из входного консольного потока. Метод Console.ReadLine() возвращает в виде строки набранную на клавиатуре после-

довательность символов. «Расшифровку» этой последовательности, т. е. превращение символьного представления во внутреннее представление (в код) соответствующего значения, удобно выполнить с помощью метода `Parse()` или `TryParse()`. Наиболее просто, но небезопасно, применить метод `Parse()`, например таким образом:

```
int res = int.Parse(Console.ReadLine());
```

В данном случае изображение целого числа в виде набранной на клавиатуре последовательности цифр (возможно со знаком) передается в виде строки как аргумент методу `Parse()` класса `int` (иначе `System.Int32`). Задача метода `int.Parse()` — сформировать код целого числа, которое станет значением переменной `int res`. Особенность (и опасность) — в прочитанной строке не должно быть символов, отличных от десятичных цифр и знака числа (+ или -). Перед изображением числа и после него могут находиться пробелы, которые будут отброшены (проигнорированы). Например, набранная на клавиатуре строка может быть такой:

```
"      -240      "
```

Значением переменной `res` будет `-240`.

Как уже говорилось, при неверной строковой записи значения анализируемого типа метод `Parse()` генерирует исключение. При отсутствии в программе операторов обработки этих исключений (а мы их еще не рассматривали) программа завершается аварийно.

Для решения той же задачи чтения из входной строки целочисленного значения метод `int.TryParse()` можно применить так:

```
int res;  
do Console.Write("Введите целое число: ");  
while(int.TryParse(Console.ReadLine(),out res) == false);
```

В цикле с постусловием на консоль выводится приглашение "Введите целое число: ". Набранную на клавиатуре последовательность символов считывает метод `Console.ReadLine()`. Возвращаемая методом строка служит первым аргументом метода `int.TryParse()`. Если строка является корректным изображением целого числа, то его значение присваивается аргументу `res`, а метод `TryParse()` возвращает значение `true`. Тем самым цикл завершается. В противном случае параметр `res` остается без изменений, метод `TryParse()` возвращает значение `false`, что приводит к следующей итерации цикла. Цикл будет повторяться, пока пользователь не введет правильное изображение целого числа.

Методов преобразований для predefined типов в классе `System.Convert` много и у них разные имена. Например, для преобразования строки в код целого числа типа `int` предназначен метод:

```
Convert.ToInt32(строка);
```

При использовании преобразований с помощью методов класса `Convert` в строке-аргументе должны быть только символы, допустимые для представления того значения, к типу которого выполняется преобразование. В противном случае возникает ошибочная ситуация, генерируется исключение и, если в программе не предусмотрена обработка этого исключения, программа завершается аварийно. Приведем пример с одним из методов класса `Convert`:

```
string sPi = "3,14159", radius = "10,0";  
double circle = 2 * Convert.ToDouble(sPi) *  
    Convert.ToDouble(radius);  
Console.WriteLine("Длина окружности=" + circle.ToString());
```

В примере определены две строки, содержащие изображения вещественных чисел (типа **double**). Обратите внимание, что дробная часть строкового представления каждого числа отделена от целой части запятой, а не точкой. Это связано (как мы уже упоминали) с правилами локализации системы, в которой исполняется программа. В Европе и России целая и дробная части числа традиционно разделяются запятой. В инициализаторе переменной **double** `circle` использованы два обращения к одному методу класса `Convert`. Возвращаемые этими методами значения использованы для вычисления инициализирующего выражения. Как догадался внимательный читатель, будет получено приближенное значение длины окружности с радиусом 10. В аргументе метода `Console.WriteLine()` явно выполнено (хотя это и не обязательно) преобразование значения переменной `circle` к значению типа **string**. На консоль будет выведено:

```
Длина окружности=62,8318
```

(Опять запятая в изображении числа!)

8.11. Аргументы метода `Main()`

До сего времени мы использовали вариант метода `Main()` без параметров. Имеется возможность определять метод `Main()` с таким заголовком:

```
public static void Main (string [] arguments)
```

где `arguments` — произвольно выбираемое программистом имя ссылки на массив с элементами типа **string**.

Эти элементы массива представляют в теле метода `Main()` аргументы командной строки. Конкретные аргументы командной строки — это разделенные пробелами последовательности символов, размещенные после имени программы при ее запуске из командной строки.

Если программа запускается не из командной строки, а из среды `Visual Studio`, то для задания аргументов командной строки нужно

использовать следующую схему. В основном меню выбираете пункт «Project» («Проект»), затем в выпадающем меню выбираете команду «Properties: имя_проекта» («Свойства: имя_проекта»). В открывшемся окне на панели слева («Application» — «Приложение») выбираете закладку «Debug» («Отладка»). Справа открывается панель, одно из текстовых полей которой названо «Command line arguments» («Аргументы командной строки»). Текст, который вводится в это поле, воспринимается как последовательность (разделенных пробелами) значений аргументов метода Main(). Как воспользоваться этими значениями (этим массивом строк) — дело автора программы. Продемонстрируем на следующем примере основные особенности обработки аргументов командной строки. Пусть требуется подсчитать сумму целых чисел, записанных через пробелы при запуске программы в командной строке (или введенные в текстовое поле «**Command line arguments**»).

Числа вводятся в виде наборов символов, которые отделены друг от друга (и от имени запускаемой программы) пробелами. В программе предусмотрим печать сообщения об отсутствии аргументов в командной строке. Текст программы:

```
// 08_04.cs – Аргументы метода Main()
using System; // Для класса Convert
using static System.Console;
class Program
{
    static void Main(string[] numbs)
    {
        int sum = 0;
        if (numbs.Length == 0)
        {
            WriteLine("Нет аргументов в командной строке!");
            return;
        }
        for (int i = 0; i < numbs.Length; i++)
            sum += Convert.ToInt32(numbs[i]);
        WriteLine("Сумма чисел = " + sum);
    }
}
```

В теле метода Main() определена целочисленная переменная sum для подсчета суммы. Параметр numbs — ссылка на массив ссылок на объекты типа **string**. Если при запуске программы в командной строке нет аргументов — массив numbs пуст, значение свойства numbs.Length равно нулю. Выводится сообщение:

Нет аргументов в командной строке

и оператор **return**; завершает выполнение программы. При наличии аргументов, выполняется цикл **for** с параметром **int i** (можно применить и цикл **foreach**). Строка — очередной элемент массива numbs[i] — служит аргументом метода Convert.ToInt32(). Возвращаемое целочислен-

ное значение увеличивает текущее значение переменной `sum`. Если при запуске программы в командной строке следующая информация

```
Имя_проекта.exe 24 16 -15<ENTER>
```

Результат, выводимый в консольное окно, будет таким:

```
Сумма чисел = 25
```

8.12. Неизменяемость объектов класса `string`

К символам объекта класса `string`, будь то объект, созданный компилятором для представления строки-литерала, или объект, созданный с помощью обращения к конструктору класса `string`, можно обращаться только для получения их значений. Например, для получения значения одного символа строки используется выражение с операцией индексирования `[]`.

Чтобы «изменить» строку, приходится прибегать к «обходным маневрам». Например, можно переписать символы строки во вспомогательный массив с элементами типа `char`. Элементы такого массива доступны изменению. Выполнив нужные преобразования, создадим на основе измененного массива новую строку, используя конструктор `string(char[])`. Если исходная строка не нужна — можем присвоить ее ссылке значение ссылки на полученный объект. Схема преобразования показана на рис. 8.2.

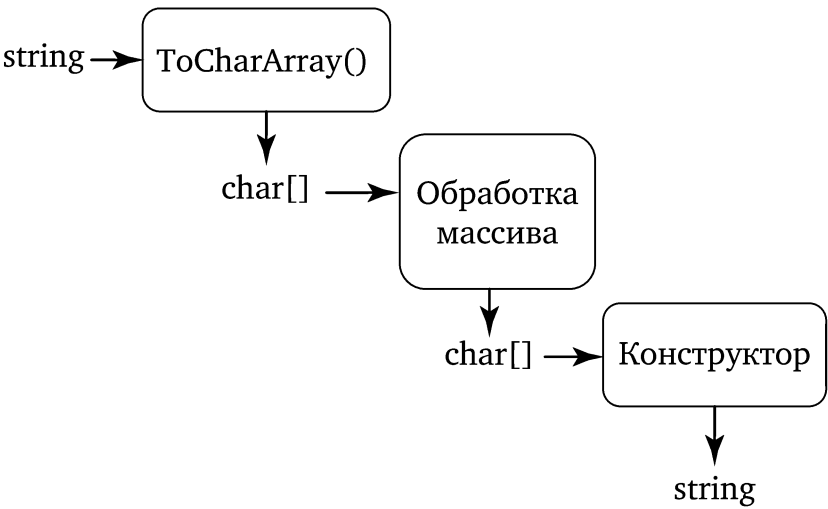


Рис. 8.2. Как изменить объект типа `string`

Последовательность операторов, соответствующая описанной схеме:

```
string row = "0123456789";
char[] rev;
rev = row.ToCharArray();
Array.Reverse(rev);
row = new string(rev);
Console.WriteLine(row);
```

Результат выполнения этого фрагмента программы:

9876543210

В примере определена ссылка `row` на объект класса **string**, инициализированный строковым литералом `"0123456789"`. Определена без инициализации ссылка `rev` на символьный массив. Затем к объекту, связанному со ссылкой `row`, применен метод `ToCharArray()`, и результат присвоен ссылке `rev`. Метод `Reverse()` класса `Array` меняет на обратный порядок размещения значений элементов массива, связанного со ссылкой `rev`, использованной в качестве аргумента. Из измененного массива, адресованного ссылкой `rev`, конструктор **string()** создает новую строку, ссылка на которую присваивается переменной `row`. Тем самым разрывается связь ссылки `row` со строкой `"0123456789"`, и `row` связывается с объектом, содержащим последовательность `"9876543210"`.

Еще одна возможность динамически изменять строки — это использовать тип `System.Text.StringBuilder`. Объекты этого класса подобно объектам класса **string** представляют последовательности символов (типа **char**), однако строка, представляемая объектом типа `StringBuilder`, является изменяемой. К сожалению, язык C# не считает `StringBuilder` элементарным типом. Поэтому для создания строки типа `StringBuilder` необходимо явно обратиться к одному из его конструкторов. Для ознакомления с особенностями и возможностями `StringBuilder` следует обратиться к документации или к литературе [6, стр. 251—254].

Контрольные вопросы и задания

1. Объясните различия между регулярным и буквальным строковыми литералами.
2. Каким образом в буквальный строковый литерал поместить символ кавычки?
3. Перечислите способы создания объектов типа **string**.
4. Перечислите операции над строками.
5. В чем особенность операции индексирования для строк?
6. В чем отличия и в чем сходство строк и массивов типа **char[]**?
7. Как выполняется операция присваивания для строк?
8. Какие операции сравнения применимы к строкам?
9. Перечислите особенности конкатенации строк со значениями других типов.
10. В каких случаях метод `ToString()` вызывается неявно?
11. Каково значение свойства `Length` для регулярного строкового литерала, содержащего эскейп-последовательности?
12. Как выполняется сравнение строк?
13. Как выполняется метод `Join()`?
14. Как выполняется метод `Split()`?
15. Объясните правила применения метода `Format()`.

16. Назовите назначения элементов поля подстановки строки форматирования.
17. Перечислите спецификаторы формата поля подстановки.
18. Назовите назначение параметров метода ToString().
19. Приведите примеры спецификаторов формата в аргументах метода ToString().
20. Что такое интерполируемая строка?
21. Что такое интерполируемое выражение?
22. Какой тип должна иметь переменная цикла **foreach**, применяемого к строке?
23. Как инициализировать массив строк?
24. Как получить строку, символы которой представляют значение типа **long**?
25. Какими средствами можно получить код значения базового типа, символьная запись которого находится в строке?
26. Как при запуске программы задать аргументы метода Main()?
27. Как в теле программы получить аргументы из командной строки?

Глава 9

МЕТОДЫ C#

9.1. Методы-процедуры и методы-функции

Как писал Никлаус Вирт, программы — это алгоритмы + структуры данных. Можно считать, что данные в C# — это поля объектов и статические поля классов. Алгоритмы (т. е. функциональность программ) представляются с помощью методов класса и методов его объектов. В языке C# методы не существуют вне классов или структур. Нестатические методы реализуют функциональность объектов. Статические методы обеспечивают функциональность классов и, тем самым, программы в целом. Каждый метод предназначен для выполнения действий, определяемых совокупностью входящих в метод (в его тело) операторов. Начиная с версии C# 7.0, в каждом методе может быть определен и использован другой (локальный) метод (локальные методы рассмотрены в параграфе 9.2).

Для определения методов и при их вызове в C# не используются никакие служебные слова. В «языках древности», таких как ФОРТРАН или КОБОЛ, и в некоторых более современных языках, не входящих в семейство С-образных, для обозначения подпрограмм, функций, процедур используются специальные термины, включаемые в список служебных слов соответствующего языка (FUNCTION, SUBROUTINE — в Фортране, procedure в Паскале и т. д.). Для вызова подпрограмм в Фортране используется конструкция со служебным словом CALL и т. п.

В языках, ведущих свое происхождение от языка С (C++, Java, C# и др.) при определении функций и для обращения к ним специальные термины не применяются. Это в полной мере относится и к методам языка C#. Однако синтаксически и семантически методы C# можно разделить на процедуры и функции. Это деление не особенно жесткое и в ряде случаев метод может играть две роли — и процедуры, и функции.

В упрощенном варианте формат декларации метода, который может играть роль как процедуры, так и функции, можно представить так:

```
модификаторы_методаopt  
тип_возвращаемого_значения имя_метода (спецификация_параметров)  
{операторы_тела_метода}
```

Индекс _{opt} указывает на необязательность модификаторов метода.

Фигурные скобки ограничивают тело метода, часть объявления перед телом метода называют заголовком метода.

Минимальная конструкция, применяемая для обращения к методу:

```
имя_метода(список_аргументов)
```

В общем случае имя метода при обращении дополняется префиксами, указывающими на принадлежность метода пространству имен, конкретному классу или реально существующему объекту.

Метод-процедура отличается от метода-функции следующими свойствами:

- в качестве типа возвращаемого значения используется **void**, т. е. процедура не возвращает в точку вызова никакого результата;
- в теле процедуры может отсутствовать оператор возврата **return**, а когда он присутствует, то в нем нет выражения для вычисления возвращаемого значения. Если оператор **return** отсутствует, то точка выхода из процедуры (из метода) расположена за последним оператором тела метода;
- для обращения к методу-процедуре используется вызов метода в виде отдельного оператора:

```
имя_метода (список_аргументов);
```

Метод-функция всегда возвращает в точку вызова результат (некоторое значение). Типом возвращаемого значения служит тип ссылки или тип значения.

В теле функции всегда присутствует, по крайней мере, один оператор возврата:

```
return выражение;
```

где выражение определяет возвращаемый функцией результат.

Обращение к методу-функции может использоваться в качестве операнда подходящего выражения. Термин «подходящего» относится к необходимости согласования операндов в выражении по типам.

Если результат, возвращаемый методом-функцией, по каким-то причинам не нужен в программе, а требуется только выполнение операторов тела функции, то обращение к ней может оформляться как отдельный оператор и в этом случае функция выступает в роли процедуры.

Когда вы знакомитесь с некоторым методом (или пишете свой метод), очень важно понимать, откуда метод берет (или будет брать) исходные данные и куда (и как) метод отправляет результаты своей работы.

Исходные данные могут быть получены:

- через аппарат параметров метода;
- как глобальные по отношению к методу;
- от внешних устройств (потoki ввода, файловые потоки).

Результаты метод может передавать:

- в точку вызова как возвращаемое значение;
- в глобальные по отношению к методу объекты;
- внешним устройствам (потoki вывода, файловые потоки);
- через аппарат параметров метода.

Глобальными по отношению к методу объектами в С# являются статические поля класса, в котором, метод определен, и поля (статические) других классов, непосредственный доступ к которым имеет метод. Для нестатического метода в качестве глобальных данных доступны нестатические члены класса, т. е. члены объектов этого класса. Для локальных методов глобальными объектами являются параметры и локальные переменные того метода, в котором объявлен локальный метод. Обмены через глобальные объекты являются нарушением принципов инкапсуляции и обычно в реальных разработках не рекомендуются.

Обмены со стандартными потоками ввода-вывода, поддерживаемые средствами класса `Console`, нам уже знакомы. Для организации обменов с файловыми потоками используются те средства библиотеки классов, которые мы еще не рассматривали. Сосредоточимся на особенностях обменов через аппарат параметров.

При определении метода в его заголовке размещается спецификация параметров (возможно пустая) — разделенная запятыми последовательность спецификаторов параметров. Каждый спецификатор параметра имеет вид:

модификатор тип_параметра имя_параметра

Модификатор параметра может отсутствовать или имеет одну из следующих форм: **ref**, **out**, **params**, **this**.

Если модификатор параметра отсутствует, то спецификатор параметра может включать умалчиваемое значение аргумента и принимает вид:

тип_параметра имя_параметра = выражение

Такие параметры в литературе называют необязательными (*optional parameter*). Выражение после знака присваивания определяет умалчиваемое значение аргумента и должно быть константным.

Ограничений на тип параметра не накладывается. Параметр может быть: предопределенного типа (базовые типы, строки, **object**); перечислением; структурой; классом; массивом; интерфейсом; делегатом.

Имя параметра — это идентификатор, который выбирает автор метода. Область видимости и время существования параметра ограничиваются заголовком и телом метода. Таким образом, параметры не видны и недоступны для кода, который не размещен в теле метода.

Стандарт С# отмечает существование четырех видов параметров:

- параметры, передаваемые по значениям;
- параметры, передаваемые по ссылкам (**ref**);
- выходные параметры (**out**);

- массив-параметр (**params**).

Параметры первых трех видов в Стандарте C# называют фиксированными параметрами. Спецификация фиксированного параметра включает необязательный модификатор **ref** или **out**, обозначение типа и идентификатор (имя параметра).

Список параметров представляет собой последовательность разделенных запятыми спецификаторов параметров (возможно, пустую), из которых *только последний* может быть массивом-параметром, имеющим модификатор **params**.

Модификаторы метода необязательны, но их достаточно много. Вот их список, пока без комментариев:

```
new, public, protected, internal, private, static, virtual,  
sealed, override, abstract, extern, async.
```

В данной главе мы будем рассматривать только статические методы (методы классов, не объектов) и локальные методы. В декларацию каждого метода класса (или структуры) входит модификатор **static** и такой метод называют статическим методом.

Чтобы продемонстрировать некоторые возможности и отличия метода-процедуры от метода-функции, рассмотрим следующую программу

```
// 09_01.cs - Статические методы - процедура и функция  
using System;  
using static System.Console;  
class Program  
{  
    static void print(string line)  
    {  
        WriteLine("Длина строки: " + line.Length);  
        WriteLine("Значение строки: " + line);  
    }  
    static string change(string str)  
    {  
        char[] rev = str.ToCharArray();  
        Array.Reverse(rev);  
        return new string(rev);  
    }  
    static void Main()  
    {  
        string numbers = "123456789";  
        print(numbers);  
        numbers = change(numbers);  
        print(numbers);  
    }  
}
```

Результат выполнения программы:

```
Длина строки: 9  
Значение строки: 123456789
```

Длина строки: 9

Значение строки: 987654321

В классе Program три статических метода. Метод print() получает исходные данные в виде строки-параметра и выводит длину и значение этой строки. В точку вызова метод print() ничего не возвращает — это процедура.

Метод change() — это функция. Он получает в качестве параметра строку, формирует ее «перевернутое» значение и возвращает в точку вызова этот результат.

В статическом методе Main() определена ссылка numbers на строку "123456789". В отдельном операторе вызван метод-процедура print(numbers), который выводит сведения о строке, именованной ссылкой numbers. Затем та же ссылка использована в качестве аргумента метода-функции change(). Обращение к этому методу размещено в правой части оператора присваивания, поэтому возвращаемый методом change() результат становится новым значением ссылки numbers. Повторное обращение к методу print() иллюстрирует изменения.

9.2. Методы локальные и сжатые до выражений

В версии C# 7.0 введены еще два вида методов, которые в ряде случаев существенно упрощают программирование. Это *локальные методы* и *методы, сжатые до выражений*.

Локальным называют метод, определенный внутри другого метода. Этот другой (охватывающий) метод может быть статическим методом, которые рассматриваются в этом разделе. Кроме того, локальный метод можно определить в другом локальном методе и в тех методах, которые пока нами не рассмотрены (конструктор, метод объекта, аксессор свойства, аксессор индексатора).

При определении локального метода запрещено снабжать его модификатором **static**. Но если охватывающий метод является статическим, то и все его локальные методы также статические. Локальный метод имеет доступ ко всем локальным переменным и к параметрам охватывающего метода. В дополнение отметим, что локальный метод может быть рекурсивным и может быть определен как асинхронный.

Пример иллюстративной программы с локальными методами.

```
// 09_02.cs – Локальные методы
using System;
class Program
{
    static void Main()
    {
        double Sring(double R1, double R2)
        {
            double Rmax = Math.Max(R1, R2);
            double Rmin = Math.Min(R1, R2);
        }
    }
}
```

```

        return Scircle(Rmax) - Scircle(Rmin);
    }
    double Scircle(double R)
    {
        return Math.PI * R * R;
    }
    double A = 10, a = 4;
    Console.WriteLine("Площадь кольца = {0:F4}", Sring(A, a));
}
}

```

Результат выполнения программы:

Площадь кольца = 263,8938

В приведенном коде программы охватывающим является метод `Main()`, в котором определены два локальных метода: `Sring()` и `Scircle()`. Первый из названных локальных методов вычисляет площадь кругового кольца, а второй — площадь круга. Большее из значений параметров `R1` и `R2` метода `Sring()` служит радиусом внешнего круга кольца, меньшее определяет радиус внутренней границы кольца. В теле метода `Sring()` дважды выполняется обращение к локальному методу `Scircle()`. Остальное тривиально. В метод `Main()` декларированы две вещественные переменные (**`double A=10, a=4`**), которые используются в качестве аргументов метода `Sring()`.

Упростить декларацию метода в тех случаях, когда в его теле всего один оператор, позволяет синтаксис методов, сжатых до выражений. Объявление метода, сжатого до выражения, имеет следующий вид:

```

тип_возвращаемого_значения имя_метода (спецификация_параметров) =>
выражение;

```

Если тип возвращаемого значения отличен от **`void`**, то «выражение» в приведенной конструкции интерпретируется как оператор «**`return`** выражение;». Если тип «возвращаемого значения» определен как **`void`**, то значение выражения никак не учитывается (игнорируется), а интерес представляет только побочный эффект его вычисления.

В качестве примера декларации методов, сжатых до выражений, можно было бы изменить объявления локальных методов `Sring()` и `Scircle()` из предыдущей программы (в этом случае тело метода `Sring()` пришлось бы преобразовать, чтобы в нем остался только один оператор **`return`**).

Приведем другую программу с методами, сжатыми до выражений, где вычисляется длина гипотенузы по длинам катетов прямоугольного треугольника.

```

// 09_03.cs – методы, сжатые до выражений
using System;
class Program
{
    static void Print(string st, double z) =>

```

```

        Console.WriteLine(st + " = {0}", z);
static void Main()
{
    double Hypotenuse(double x, double y) =>
        Math.Sqrt(x * x + y * y);
    double a = 3, b = 4;
    Print("Катет A", a);
    Print("Катет B", b);
    Print("Гипотенуза", Hypotenuse(a, b));
}
}

```

Результат выполнения программы:

```

Катет A = 3
Катет B = 4
Гипотенуза = 5

```

В программе определены два метода, сжатые до выражений: `Print()` и `Hypotenuse()`. Метод `Print()` декларирован как статический, принадлежащий классу `Program`. Первый параметр метода `Print()` типа **string** определяет название результата, выводимого с помощью обращения к методу `Console.WriteLine()` из метода `Print()`. Вторым параметром метода `Print()` имеет тип **double**. Он определяет выводимое значение (числовой результат).

Метод `Hypotenuse()` локализован в статическом методе `Main()`. Его назначение — вычислить по длинам катетов, значения которых заданы параметрами (**double** `x`, **double** `y`), (и вернуть) длину гипотенузы.

В методе `Main()` декларированы две локальные вещественные переменные (**double** `a = 3`, `b = 4`), играющие роль катетов прямоугольного треугольника. С помощью обращений к методу `Print()` выводятся сведения о катетах и длина гипотенузы (см. результаты выполнения программы). Обратите внимание, что в третьем обращении вещественный параметр метода `Print()` заменен вызовом метода `Hypotenuse()`, аргументы которого — локальные вещественные переменные охватывающего метода.

Отметим, что методы, сжатые до выражений, не обязательно должны быть локальными или статическими методами. Особых ограничений на них не наложено. Например, метод, сжатый до выражения, может быть конструктором в декларации класса. Единственное требование — тело метода должно состоять из единственного оператора.

9.3. Соотношение параметров и аргументов

Чтобы объяснить возможности и особенности разных видов параметров, начнем с того, что при обращении к методу каждый фиксированный параметр замещается некоторым аргументом. Соответствие

между параметрами и замещающими их аргументами устанавливается по их взаимному расположению, т. е. параметры определены позиционно.

Аргумент, заменяющий фиксированный параметр, представляет собой выражение, тип которого совпадает или может быть приведен к типу, указанному в спецификации соответствующего параметра.

Для параметра, передаваемого по значению, при обращении к методу создается внутри метода временный объект, которому присваивается значение аргумента. Имя параметра в теле метода соотносено с этим временным объектом и никак не связано с тем конкретным аргументом, который использован в обращении вместо параметра. Операции, выполняемые в теле метода с участием такого параметра, действуют только на временный объект, которому присвоено значение аргумента.

В Стандарте C# для иллюстрации независимости аргумента от изменений параметра, передаваемого по значению, приведена следующая программа:

```
// 09_04.cs - параметр, передаваемый по значению
using System;
class Program
{
    static void F(int p)
    {
        p++;
        Console.WriteLine("p = {0}", p);
    }
    static void Main()
    {
        int a = 1;
        Console.WriteLine("pre: a = {0}", a);
        F(a);
        Console.WriteLine("post: a = {0}", a);
    }
}
```

Результат выполнения программы:

```
pre: a = 1
p = 2
post: a = 1
```

Изменение параметра `p` в теле метода `F()` не повлияло на значение аргумента `a`.

Таким образом, как показывает приведенный пример, параметры, передаваемые по значениям, служат для передачи данных в метод, но не позволяют вернуть какую-либо информацию из метода.

В классе может быть любое количество статических методов, и они могут беспрепятственно обращаться друг к другу. Покажем на примере какие при этом появляются возможности.

Задача: написать функцию (метод), возвращающую значение минимального из четырех аргументов. Прямой путь — использовать в теле метода вложенные условные операторы или условные выражения. Поступим по-другому — определим вспомогательную функцию, которая возвращает минимальное значение одного из двух параметров. Программа может быть такой:

```
// 09_05.cs - вложенные вызовы функций
using System;
class Program
{
    // Вспомогательная функция:
    static double min2(double z1, double z2)
    {
        return z1 < z2 ? z1 : z2;
    }
    // функция возвращает минимальное из значений параметров:
    static double min4(double x1, double x2, double x3, double x4)
    {
        return min2(min2(min2(x1, x2), x3), x4);
    }
    static void Main()
    {
        Console.WriteLine(min4(24, 8, 4, 0.3));
    }
}
```

Результат выполнения программы:

0.3

В данном примере оба статических метода `min2()` и `min4()` выступают в роли функций. Обратите внимание на вложение обращений к функции `min2()` в выражении оператора **return** из функции `min4()`.

Чтобы метод мог с помощью параметров изменять внешние по отношению к методу объекты, соответствующие им параметры должны иметь модификатор **ref**, т. е. передаваться по ссылке.

Для иллюстрации этой возможности модифицируем программу `09_04.cs` — снабдим параметр новой функции `FR()` модификатором **ref**.

```
// 09_06.cs – параметр типа int, передаваемый по ссылке
using System;
class Program
{
    static void FR(ref int p)
    {
        p++;
        Console.WriteLine("p = {0}", p);
    }
    static void Main()
    {

```

```

    int a = 1;
    Console.WriteLine("pre: a = {0}", a);
    FR(ref a);
    Console.WriteLine("post: a = {0}", a);
}
}

```

Результат выполнения программы:

```

pre: a = 1
p = 2
post: a = 2

```

После обращения к методу FR() значение внешней по отношению FR() переменной изменилось.

Обратите внимание, что аргумент, подставляемый на место передаваемого по ссылке параметра, должен быть снабжен модификатором **ref**.

В качестве примера метода с параметрами, передаваемыми по ссылкам, часто рассматривают метод `swap()`, меняющий местами значения двух объектов, адресованных аргументами. Рассмотрим чуть более сложную задачу — метод упорядочения (в порядке возрастания) значений трех целочисленных переменных. Для демонстрации возможностей вспомогательных методов определим в том же классе метод упорядочения двух переменных:

```

// Вспомогательный метод
static void rank2(ref int x, ref int y)
{
    int m;
    if (x > y)
    {
        m = x; x = y; y = m;
    }
}

```

Метод `rank2()` играет роль процедуры — он в точку вызова ничего не возвращает, так как тип возвращаемого значения **void**. Параметры — целочисленные переменные, передаваемые по ссылкам. В теле метода объявлена вспомогательная локальная переменная **int m**. Она используется как промежуточный буфер при обмене значений параметров `x` и `y`.

Имея приведенный метод, можно написать следующую процедуру, решающую нашу задачу сортировки трех переменных:

```

static void rank3(ref int x1, ref int x2, ref int x3)
{
    rank2(ref x1, ref x2);
    rank2(ref x2, ref x3);
    rank2(ref x1, ref x2);
}

```

В теле метода `rank3()` его параметры используются в качестве аргументов при обращениях к методу `rank2()`. Процедура `rank2()` вызывает

ется трижды, последовательно сортируя значения, на которые указывают сначала параметры `x1`, `x2`, затем `x2`, `x3` и затем `x1`, `x2`. Функция `Main()`, иллюстрирующая применение метода `rank3()`:

```
static void Main()
{
    int i = 85, j = 23, k = 56;
    rank3(ref i, ref j, ref k);
    Console.WriteLine("i={0}, j={1}, k={2}", i, j, k);
}
```

Результат выполнения программы:

```
i=23, j=56, k=85
```

Итак, параметры, передаваемые по ссылке, используются для изменения уже существующих значений внешних по отношению к методу объектов.

Еще раз обратите внимание, что модификатор **ref** используется не только перед параметром, но и перед замещающим его аргументом. Особо отметим, что аргументом может быть только имеющая значение переменная (не константа и не выражение) того же типа, что и параметр.

```
Console.WriteLine(rank3(ref 24, ref 8, ref 4)); // ошибка!
// ERROR – аргументы должны быть переменными!!!
```

Выходные параметры снабжаются модификатором **out** и позволяют присвоить значения объектам вызывающего метода даже в тех случаях, когда эти объекты значений еще не имели.

Возникает вопрос — зачем нужен модификатор **out**, если те же действия обеспечивает применение модификатора **ref**? Связано появление модификатора **out** с правилом обязательной инициализации переменных. Модификатор **out** «говорит» компилятору — «не обращай внимание на отсутствие значения у переменной, которая использована в качестве аргумента», и компилятор не выдает сообщения об ошибке. Однако эта переменная обязательно должна получить конкретное значение в теле метода. В противном случае компилятор зафиксирует ошибку. Ошибкой будет и попытка использования в теле метода значения выходного параметра без предварительного присваивания ему значения.

Метод с выходными параметрами (имеющими модификаторы **ref** или **out**) обычно выступает в роли процедуры. Хотя не запрещено такому методу возвращать значение с помощью оператора **return**. Примером такого метода служит метод `TryParse()`, который мы уже использовали в примерах программ.

В качестве иллюстрации применения выходных параметров приведем метод, возвращающий целую (**int integer**) и дробную (**double fra**) части вещественного числа (**double x**). В функции `Main()` определим

три переменных: **double** *real* — исходное число, **double** *dPart* — дробная часть, **int** *iPart* — целая часть. Переменная *real* инициализирована, переменные *iPart*, *dPart* не имеют значений до обращения к методу.

```
// 09_08.cs - параметры-результаты
using System;
class Program
{
    // Метод возвращает значения целой и дробной частей
    // вещественного параметра
    static void fun(double x, out int integer, out double fra)
    {
        integer = (int)x;
        fra = x - integer;
    }
    static void Main()
    {
        double real = 53.93;
        double dPart;
        int iPart;
        fun(real, out iPart, out dPart);
        Console.WriteLine("iPart={0}, dPart={1}", iPart, dPart);
    }
}
```

Результат выполнения программы:

iPart=53, dPart=0,93

Необходимо отметить, что при больших значениях аргумента, заменяющего параметр **double** *x*, величина **(int)***x* может превысить предельное значение типа **int**. Это приведет к неверным результатам за счет переполнения, о котором мы говорили в гл. 3.

Для параметров с модификаторами **out** в версии C# 7.0 добавлен ряд синтаксических усовершенствований, призванных упростить (а возможно, просто украсить) код программы. Одним из названных изменений является возможность объявления «на лету» переменных, используемых в качестве аргументов при обращении к методу.

Результаты выполнения предыдущей программы не изменятся, если в методе *Main()* объявления переменных, возвращающих результаты выполнения метода *fun()*, разместить непосредственно в обращении к методу:

```
static void Main()
{
    double real = 53.93;
    fun(real, out int iPart, out double dPart);
    Console.WriteLine("iPart={0}, dPart={1}", iPart, dPart);
}
```

Вторая особенность параметра с модификатором **out** — возможность исключить соответствующий ему аргумент из обращения к методу,

если результат, возвращаемый параметром, не нужен в конкретном использовании метода. Для названного «отбрасывания» аргументов каждый ненужный параметр заменяется аргументом (подстановкой) такого вида: «**out _**».

Продолжая предыдущий пример, покажем, как можно получить с помощью обращения к методу `fun()` только значение дробной части вещественного числа, заданного первым аргументом:

```
fun(real, out _, out double fraction);  
Console.WriteLine("fraction={0}", fraction);
```

У метода два параметра с модификаторами **out**. Так как параметр **out int integer** не нужен, то в обращении к методу он заменен аргументом «**out _**». Для получения результата использован объявляемый на лету аргумент **out double fraction**.

Если в спецификации параметра задано значение по умолчанию, и именно такое значение требуется при конкретном обращении к методу, то соответствующий аргумент можно опустить. Если соответствие между параметрами и аргументами устанавливается по их взаимному расположению (т. е. по занимаемым ими позициям), то параметры со значениями по умолчанию должны размещаться в конце списка параметров. Рассмотрим следующий пример:

```
static void Main()  
{  
    void PrintReal(double x, string format = "Значение = {0:F4}")  
        => Console.WriteLine(format, x);  
    PrintReal(5.0/3);  
}
```

В методе `Main()` локализован метод `PrintReal()`, сжатый до выражения. Для второго параметра **string format** задано значение по умолчанию в виде строки "Значение = {0:F4}". В теле метода `PrintReal()` параметр `format` используется в качестве форматной строки метода `WriteLine()`.

В обращении `PrintReal(5.0/3)` второй аргумент опущен, и вместо него используется умалчиваемое значение второго параметра.

Результат выполнения:

Значение = 1,6667

При позиционном сопоставлении аргументов с заменяемыми ими параметрами размещать спецификации параметров с умалчиваемыми значениями нужно только после обязательных параметров, не содержащих умалчиваемых значений. Это не всегда удобно. Чтобы устранить указанное неудобство, в обращении к методам можно использовать именованные аргументы. В этом случае каждый аргумент идентифицируется именем того параметра, который он должен заменить. Аргумент выглядит так:

имя_параметра: выражение

Обращение:

```
PrintReal(format:"Научная нотация: {0:E4}", x: 479.77);
```

Результат выполнения:

Научная нотация: 4,7977E+002

9.4. Параметры с типами ссылок

Мы уже достаточно подробно на примерах рассмотрели возможности параметров с типами значений. Отметим существенное различие их передачи по значению и по ссылке. Разобрали два варианта передачи по ссылке — с применением модификатора **ref** и с использованием модификатора **out**. Теперь остановимся на особенностях параметров с типами ссылок. Для них также возможна как передача по значению, так и передача по ссылке.

Если параметр с типом ссылки передается методу по значению, то в теле метода создается копия использованного аргумента (копия ссылки). Эта копия аргумента ссылается на какой-то внешний для метода объект и операторы тела метода через ссылку могут изменить этот внешний объект. Пример:

```
// 09_09.cs - параметр с типом ссылки
using System;
class Program
{
    static void sorting(int[] vector) // упорядочить массив
    {
        int t;
        for (int i = 0; i < vector.Length - 1; i++)
            for (int j = i + 1; j < vector.Length; j++)
                if (vector[i] > vector[j])
                {
                    t = vector[i];
                    vector[i] = vector[j];
                    vector[j] = t;
                }
    }
    // Вывести вектор
    static void arrayPrint(int[] a, string формат)
    {
        int i;
        for (i = 0; i < a.Length; i++)
            Console.Write(формат, a[i]);
    }
    static void Main()
    {
        int[] array = { 1, 4, 8, 2, 4, 9, 3 };
        arrayPrint(array, "{0,6:d}");
    }
}
```

```

        Console.WriteLine();
        sorting(array);
        Console.WriteLine("Измененный массив:");
        arrayPrint(array, "{0,6:d}");
        Console.WriteLine();
    }
}

```

Результат выполнения программы:

```

    1    4    8    2    4    9    3
Измененный массив:
    1    2    3    4    4    8    9

```

Метод **sorting()** в качестве аргумента, передаваемого по значению, должен принимать ссылку на некоторый массив типа **int[]**. В теле метода выполняется перестановка значений элементов того массива, ссылка на который использована в качестве аргумента. Таким образом, метод изменяет внешний для него объект — одномерный целочисленный массив.

Однако не следует считать, что метод **sorting()** принимает параметр **vector** по ссылке. Этот параметр с типом ссылки на одномерный целочисленный массив *передается по значению*.

Чтобы хорошо понять различие передач по значениям от передач по ссылкам для параметров с типами ссылок, рассмотрим следующий пример. Пусть поставлена задача обменивать значения ссылок на два объекта-массива. Сделаем это двумя способами. Определим очень похожие методы:

```

static void change1(int[] vec1, int[] vec2)
{
    int[] temp;
    temp = vec1;
    vec1 = vec2;
    vec2 = temp;
}
static void change2(ref int[] vec1, ref int[] vec2)
{
    int[] temp;
    temp = vec1;
    vec1 = vec2;
    vec2 = temp;
}

```

Методы отличаются только модификаторами параметров. Для **change1()** параметры передаются по значениям, для **change2()** — по ссылкам.

```

static void Main()
{
    int[] ar1 = {1, 4, 8, 2, 4, 9, 3};
    int[] ar2 = {1, 2, 3};
}

```



```

change1(ar1, ar2); // передача по значениям
Console.WriteLine("ar1.Length=" + ar1.Length);
Console.WriteLine("ar2.Length=" + ar2.Length);
change2(ref ar1, ref ar2); // передача по ссылкам
Console.WriteLine("ar1.Length=" + ar1.Length);
Console.WriteLine("ar2.Length=" + ar2.Length);
}

```

Результаты выполнения программы:

```

ar1.Length=7
ar2.Length=3
ar1.Length=3
ar2.Length=7

```

В методе Main() определены два конкретных массива:

```

int[] ar1 = { 1, 4, 8, 2, 4, 9, 3 };
int[] ar2 = { 1, 2, 3 };

```

Обращение `change1(ar1, ar2);` не изменяет ссылок, `ar1` останется связанной с массивом из семи элементов, `ar2` будет ссылаться, как и ранее, на массив из трех элементов.

При вызове `change2(ref ar1, ref ar2);` ссылки обменяются значениями — `ar1` будет ссылаться на массив из трех элементов, `ar2` будет адресовать массив из семи элементов.

На этом примере мы убедились, что аргумент с типом ссылки (так же как аргумент с типом значения) может изменить свое значение при выполнении тела метода только в том случае, когда он передан методу по ссылке, т. е. имеет модификатор **ref** или **out**.

Итак, параметр ссылочного типа может быть снабжен модификатором **ref**. Без него аргумент всегда «смотрит» на свой объект (внешний), может его менять, но не может изменить своего значения. С **ref** аргумент с типом ссылки может сменить свое значение и «отцепиться» от своего объекта.

В ряде случаев интересно иметь метод, который пригоден не для одного фиксированного типа параметра, а до некоторой степени универсален и допускает подстановку вместо параметра аргументов разных типов. Так как все типы языка C# являются производными от одного базового класса **object**, то метод для обмена значений двух ссылок можно написать так:

```

static void change(ref object ob1, ref object ob2)
{
    object temp;
    temp = ob1;
    ob1 = ob2;
    ob2 = temp;
}

```

Чтобы обратиться к этому методу, необходимо привести типы нужных нам аргументов к типу **object**. После выполнения метода нужно

«вернуть» полученным результатам тип исходных ссылок. В следующем методе проделаны указанные действия:

```
static void Main()
{
    int[] ar1 = {1, 4, 8, 2, 4, 9, 3};
    int[] ar2 = {1, 2, 3};
    object obj1 = ar1, obj2 = ar2;
    change(ref obj1, ref obj2); // передача по ссылкам
    ar1 = (int []) obj1;
    ar2 = (int []) obj2;
    Console.WriteLine("ar1.Length=" + ar1.Length);
    Console.WriteLine("ar2.Length=" + ar2.Length);
}
```

Результаты выполнения программы:

```
ar1.Length=3
ar2.Length=7
```

Продемонстрированное в этой программе приведение типов аргументов к типам параметров при *передаче по ссылкам* обусловлено синтаксисом языка и преследует обеспечение безопасности кода программы.

В случае передачи по значению параметра с типом **object** необходимости в явном приведении типа аргумента к типу параметра нет. Вместо передаваемого по значению параметра с типом **object** разрешено подставить аргументы любых типов.

Следующий пример иллюстрирует эту возможность, печатая сведения о типе переданного ему аргумента.

```
static void printType(object param)
{Console.WriteLine(param.GetType());}
```

Аргументы при обращениях к методу printType() могут иметь любой тип, производный от типа **object**. Иллюстрация:

```
static void Main()
{
    int[] ar1 = {1, 4, 8, 2, 4, 9, 3};
    printType(ar1);
    printType("строка");
    printType(440);
}
```

Результаты выполнения программы:

```
System.Int32[]
System.String
System.Int32
```

Предположим, что метод должен определить внутри своего тела некоторый объект и сделать этот объект доступным в точке вызова. Это можно сделать двумя способами.

Во-первых, метод может вернуть ссылку на этот объект как значение, возвращаемое функцией (методом) в точку вызова. Во-вторых, в методе можно использовать параметр с типом ссылки, снабженный модификатором **ref** либо **out**. Первый способ иллюстрирует следующая программа.

```
// 09_13.cs - ссылка как возвращаемое значение
using System;
class Program
{
    static int[] newAr(uint numb)
    {
        int[] temp = new int[numb];
        for (int i = 0; i < numb; i++)
            temp[i] = (i + 1) * (i + 1);
        return temp;
    }
    static void Main()
    {
        int[] vector = newAr(6);
        foreach (int el in vector)
            Console.Write(el + " ");
        Console.WriteLine();
    }
}
```

Результаты выполнения программы:

1 4 9 16 25 36

В программе метод (функция) `newAr()`, получив в качестве аргумента целое неотрицательное значение, создает одномерный целочисленный массив, присваивает его элементам значения квадратов натуральных чисел и возвращает в точку вызова ссылку на сформированный массив. В методе `Main()` определена переменная `vector` — ссылка на целочисленный массив, и ей присвоен результат вызова метода `newAr()`. Результаты выполнения программы иллюстрируют сказанное.

9.5. Методы с переменным числом аргументов

У параметра с модификатором **params** назначение особое — он представляет в методе список аргументов неопределенной (заранее не фиксированной) длины. Его тип — одномерный массив с элементами типа, указанного в спецификации параметра.

С применением метода, имеющего параметр со спецификацией **params string []**, мы уже познакомились в гл. 8, где рассмотрели (параграф 8.10) возможности передачи в метод `Main()` аргументов из командной строки.

Как уже сказано, этот параметр может быть только последним (или единственным) в списке параметров. В обращении к методу этот пара-

метр заменяется списком аргументов, каждый из которых должен иметь тот же тип, что и элементы массива параметров. В теле метода отдельные реально использованные аргументы представлены элементами массива-параметра. Количество аргументов соответствует длине массива.

В качестве примера объявления и использования метода с таким параметром приведем программу с методом для вычисления значений полинома

$$P_n(x) = a_0 * x^n + a_1 * x^{n-1} + \dots + a_{n-1} * x + a_n.$$

```
// 09_14.cs - массив-параметр
using System;
class Program
{
    // Вычисляет значение полинома с целыми коэффициентами:
    static double polynom(double x, params int[] coef)
    {
        double result = 0.0;
        for (int i = 0; i < coef.Length; i++)
            result = result * x + coef[i];
        return result;
    }
    static void Main()
    {
        Console.WriteLine(polynom(3.0, 3, 1, 2));
    }
}
```

Результат выполнения программы:

32

Метод `polynom()` возвращает значение типа **double**, т. е. играет роль функции. Первый параметр **double** `x` представляет значение аргумента полинома. Вторым параметром **params int []** `coef` дает возможность передать в метод список коэффициентов полинома $a_0, a_1, \dots, a_{n-1}, a_n$. Все коэффициенты должны иметь целочисленные значения. Их реальное количество, т. е. степень полинома, при обращении определяется числом использованных аргументов, а в теле метода — значением свойства `coef.Length`. В теле метода:

`coef[0]= $a_0$ , coef[1]= $a_1$ , ..., coef[n]= $a_n$ .`

Для вычисления полинома использована схема Горнера, позволяющая заменить явные возведения в степень последовательными умножениями:

$$P_n(x) = (\dots(a_0 * x + a_1) * x + a_2) * x + \dots + a_{n-1}) * x + a_n.$$

В том случае, когда требуется метод, принимающий произвольное число параметров любых разных типов, Дж. Рихтер [6] предлагает

использовать параметр вида **params object[]**. Он приводит в качестве примера метод, который выводит обозначения (наименования) типов всех переданных методу аргументов. Вот программа с этим методом:

```
// 09_15.cs – массив-параметр "универсального" типа
using System;
class Program
{
    // Метод выводит названия типов аргументов:
    static void DisplayTypes(params object[] objects)
    {
        foreach (object o in objects)
            Console.WriteLine(o.GetType());
    }
    static void Main()
    {
        DisplayTypes("yes", 432, new object());
    }
}
```

Результат выполнения программы:

```
System.String
System.Int32
System.object
```

Обратим внимание, что параметр, снабженный модификатором **params**, обеспечивает передачу аргументов по значению, т. е. значения аргументов после выполнения метода не изменяются. Следующая программа иллюстрирует это правило.

```
// 09_16.cs - переменное число аргументов
using System;
class Program
{
    static void varParams(params int[] ar)
    {
        for (int i = 0; i < ar.Length; i++)
        {
            ar[i] *= 2;
            Console.Write("ar[{0}]={1} ", i, ar[i]);
        }
    }
    static void Main()
    {
        int a = 2, b = 3, c = 5;
        varParams(a, b, c);
        Console.WriteLine("\na={0}, b={1}, c={2}", a, b, c);
    }
}
```

Результат выполнения программы:

```
ar[0]=4  ar[1]=6  ar[2]=10
a=2, b=3, c=5
```

Значения переменных *a*, *b*, *c* после их использования в качестве аргументов метода `varParams()` не изменились, хотя в теле метода элементам массива-параметра присвоены новые значения.

9.6. Перегрузка методов

Перегрузка методов представляет собой один из случаев *полиморфизма*.

В отношении к методам полиморфизм позволяет с помощью одного имени представлять различный код, т. е. различное поведение. Важно здесь, что выбор подходящего кода выполняется автоматически на этапе трансляции или исполнения программы.

Полиморфизм не обязательно связан с перегрузкой методов и присущ не только *полиморфным языкам*, к которым относится С#. Практически во всех языках программирования знаки арифметических операций применимы к операндам разных типов. Например, умножать и суммировать можно и целые, и вещественные операнды. Для этого используются операции $*$ и $+$. Однако на уровне машинных команд операции с целочисленными операндами могут быть реализованы не так, как операции над вещественными числами. Компилятор автоматически по типам операндов определяет, как должны выполняться соответствующие действия. В языках С++ и С# эта возможность применима и к типам, которые вводит программист с помощью определений классов. В этом случае говорят о *перегрузке* или *распространении действия операций на объекты* новых классов. Этому виду полиморфизма нужно уделить особое внимание при изучении пользовательских классов. А сейчас вернемся к полиморфизму перегрузки методов.

В программе на языке С# одно имя (идентификатор) может обозначать разные методы, реализующие в общем случае совершенно разные алгоритмы. Выбор нужного метода при вызове определяется конкретным набором аргументов, размещенных в скобках после имени метода, и тем пространством имен, к которому отнесено имя метода. О таком различии говорят, что методы различаются своими *сигнатурами*.

Продemonстрируем особенности и удобство перегрузки на следующем примере. Пусть требуется вычислять площадь *S* треугольника на основе разных исходных данных.

Вариант 1 — известны длины основания (*d*) и высоты (*h*):

$$S = (d * h) / 2.$$

Вариант 2 — известны длины сторон (*a*, *b*, *c*):

$$S = (p(p - a)(p - b)(p - c))^{0.5}, \text{ где } p = (a + b + c) / 2.$$

Вариант 3 — известны длины сторон (*a*, *b*, *c*) и радиус (*R*) описанной около треугольника окружности:

$$S = (a * b * c) / (4 * R).$$

Три метода, реализующие приведенные варианты вычисления площади треугольника, можно обозначить одним именем. Методы будут отличаться спецификациями параметров и алгоритмами вычислений. Объявить методы можно так:

```
static double area(double d, double h)
{return d * h/2;}
static double area(double a, double b, double c)
{double p = (a + b + c)/2;
return Math.Sqrt(p * (p - a) * (p - b) * (p - c));}
static double area(double a, double b, double c, double R)
{return(a * b * c)/(4 * R);}
```

В данном примере предполагается, что все методы являются методами одного класса. В функции Main() того же класса можно так обратиться к каждому из них:

```
Console.WriteLine("area1="+ area(4.0, 3.0));
Console.WriteLine("area2="+ area(3.0, 4.0, 5.0));
Console.WriteLine("area3="+ area(3.0, 4.0, 5.0, 2.5));
```

Подводя итоги обсуждения перегрузки методов, еще раз повторим, что **сигнатура метода** — это комбинация его имени, спецификации параметров, и пространства имен, к которому относится метод. Кроме того, имя класса, в котором объявлен метод, и модификаторы параметров **out** и **ref** входят в сигнатуру, а модификаторы метода (например, **static public**) в сигнатуру не входят.

Обратите внимание, что тип возвращаемого методом значения не входит в сигнатуру, влияющую на перегрузку методов.

9.7. Рекурсивные методы

Рекурсивным называют метод, который прямо (непосредственно) или косвенно вызывает самого себя. Метод называют косвенно рекурсивным, если он содержит обращение к другому методу, содержащему прямой или косвенный вызов определяемого (первого) метода. В случае косвенной рекурсивности по тексту определения метода его рекурсивность может быть не видна. Если в теле метода явно используется обращение к этому методу, то имеет место прямая рекурсия. В этом случае говорят, что метод самовывзывающий (*self-calling*). Именно самовывывывающие методы будем называть рекурсивными, а для методов с косвенной рекурсией будем использовать термин «косвенно рекурсивные методы».

Классический пример рекурсивного метода — функция для вычисления факториала неотрицательного целого числа. На языке C# ее можно записать таким образом:

```
static long fact(int k)
{
    if (k < 0) return 0;
```

```

    if (k == 0 || k == 1) return 1;
    return k * fact(k - 1);
}

```

Для отрицательного аргумента результат (по определению факториала) не существует. В этом случае функция возвращает нулевое значение (можно было бы возвращать, например, отрицательное значение). Для нулевого и единичного аргумента, по определению факториала, возвращаемое значение равно 1. Если $k > 1$, то вызывается та же функция с уменьшенным на 1 значением аргумента и возвращаемое ею значение умножается на текущее значение аргумента k . Тем самым организуется вычисление произведения $1 * 2 * 3 * \dots * (k - 2) * (k - 1) * k$.

При проектировании рекурсивного метода нужно убедиться:

- что он может завершить работу, т. е. невозможно возникновение зацикливания;

- что метод приводит к получению правильных результатов.

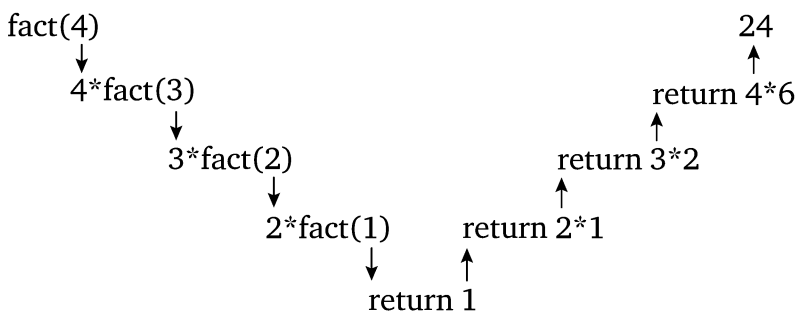
Для удовлетворения первого требования должны соблюдаться два правила:

- 1) в последовательности рекурсивных вызовов должен быть явный разрыв, т. е. самовыводы должны выполняться до тех пор, пока истинно значение некоторого выражения, операнды которого изменяются от вызова к вызову;

- 2) при самовыводах должны происходить изменения параметров и эти изменения после конечного числа вызовов должны привести к нарушению проверяемого условия из пункта 1.

В нашем примере условием выхода из цепочки рекурсивных вызовов является истинность выражения $(k == 0 || k == 1)$. Так как значение k конечно и параметр при каждом следующем вызове уменьшается на 1, то цепочка самовыводов конечна. Идя от конца цепочки, т. е. от $1 \neq 1$, к началу, можно убедиться, что все вызовы работают верно и метод работает правильно.

Иллюстрация цепочки самовыводов для метода вычисления факториала:



Эффективность рекурсивного метода зачастую определяется глубиной рекурсии, т. е. количеством самовыводов в одной цепочке при обращении к методу.

В качестве еще одного примера рекурсии приведем метод, преобразующий все цифры натурального числа в соответствующие им символы. Параметр метода — целое число типа `int`, результат — массив `char[]`. Код метода:


```
// Цифры натурального числа:
static char[] arNumbs(int n)
{
    if (n / 10 == 0)
    { return new char[] { (char)(n + (int)'0') }; }
    else
    {
        char[] res = arNumbs(n / 10);
        int L = res.Length;
        Array.Resize(ref res, L + 1);
        res[L] = (char)((n % 10) + (int)'0');
        return res;
    }
}
```

В коде метода `arNumbs(int n)` первый шаг — проверка результата целочисленного деления значения параметра на 10. Если результат нулевой, то значение параметра — число меньше 10 и представлено одной цифрой. В этом случае в операторе возврата из метода создается символьный массив из одного элемента со значением, равным символьному представлению значения параметра (одной цифры). Ссылка на массив из одного элемента — это результат, возвращаемый методом при прерывании цепочки рекурсивных самовывозов.

В более общем случае параметр больше 10 и выполняется блок операторов, размещенных в ветви **else**. Объявляется ссылка `char [] res`, которой присваивается результат рекурсивного обращения к методу `arNumbs(n/10)`. Для полученного из этого метода массива вычисляется размер (свойство `res.Length`). Затем метод `Array.Resize()` увеличивает на 1 размер массива, связанного со ссылкой `res`. В последний элемент нового массива заносится символьное представление последней цифры значения параметра. Ссылка на полученный массив возвращается как результат рекурсивного вызова.

Для иллюстрации выполнения метода используем следующий код:

```
static void Main()
{
    foreach (var c in arNumbs(0090580))
        Console.Write(c + " ");
}
```

Результат выполнения программы:

9 0 5 8 0

Цепочка рекурсивных вызовов для приведенного кода:

`arNumbs(0090580) -> arNumbs(009058) -> arNumbs(00905) -> arNumbs(0090) -> arNumbs(009) -> arNumbs(00) -> arNumbs(0) -> разрыв цепочки.`

Цепочка возвратов из самовывозов:

`res[0] = '9' -> res[1] = '0' -> res[2] = '5' -> res[3] = '8' -> res[4] = '0'`

Многие задачи, требующие циклического повторения операций, могут быть представлены как рекурсивными, так и итерационными алгоритмами. Основной для рекурсивного алгоритма является его самовывоз, а для итерационного алгоритма — цикл.

В качестве иллюстрации замены цикла на рекурсию приведем рекурсивный метод (процедуру) для вывода на консольный экран значений значимых элементов символьного массива. Пусть в массиве только начальные элементы имеют значимые значения, а элементам, расположенным за ними (до конца массива), присвоены умалчиваемые значения. Напомним, что при создании символьного массива без явной инициализации его элементам по умолчанию присваиваются значения '\0'.

Параметры метода — ссылка на одномерный символьный массив и индекс первого выводимого элемента. Окончание вывода — конец массива либо терминальный символ '\0' в качестве значения очередного элемента

```
// Рекурсивный метод печати элементов символьного массива
static void chPrint(char[] ch, int beg)
{
    if (beg >= ch.Length-1 || ch[beg] == '\0')
    {
        Console.WriteLine();
        return;
    }
    Console.Write(ch[beg] + " ");
    chPrint(ch, beg + 1);
}
```

Первый параметр метода **char[] ch** — ссылка на обрабатываемый символьный массив. Второй параметр **int beg** — индекс элемента, начиная с которого выводятся значения. Условие выхода из метода — достижение конца массива **beg >= ch.Length-1** или значение '\0' элемента с индексом **beg**. Если условие выхода не достигнуто, выполняются операторы:

```
Console.Write(ch[beg] + " ");
chPrint(ch, beg + 1);
```

Тем самым первым выводится значение начального элемента массива и происходит самовывоз метода для следующего значения второго параметра, и т. д....

Вызов последнего отличного от '\0' элемента или достижение конца массива завершает цепочку рекурсивных обращений.

Пример использования приведенного метода (09_17.cs):

```
static void Main()
{
    int size = 9;
    char[] symbols = new char[size];
    string line = "AaBbCc";
```

```

    for (int k = 0; k < line.Length; k++)
        symbols[k] = line[k];
    chPrint(symbols, 0);
}

```

Результат выполнения программы:

A a B b C c

Обратите внимание, что при обращении к методу `chPrint()` второй аргумент равен 0, т. е. вывод идет с начала массива.

9.8. Применение метода `Array.Sort()`

Несмотря на то, что обоснования возможности использования имени метода в качестве аргумента другого метода будут рассмотрены только в главе, посвященной делегатам, уже сейчас полезно научиться применять собственные методы для «настройки» некоторых методов библиотечных классов. Сделаем это на примере очень полезного метода `Array.Sort()`, который мы упомянули в гл. 7. Это статический метод, упорядочивающий (сортирующий) элементы массива. Этот метод перегружен, и в классе `Array` определены 17 методов `Sort()` с разными сигнатурами. Тривиальный вариант метода `Sort()` имеет только один параметр типа `Array`. Так как все типы массивов являются наследниками класса `Array`, то в качестве аргумента при обращении к методу сортировки допустимо использовать ссылку на любой одномерный массив (имеется в виду одномерный массив с элементами любых типов). Метод `Sort()` упорядочивает значения элементов массива в соответствии с их типом. Массивы с элементами арифметических типов упорядочиваются по возрастанию значений, строки сортируются лексикографически и т. д.

Такое «поведение» метода `Sort()` с одним параметром очень полезно, но этого зачастую недостаточно для конкретных задач. Рассмотрим еще один вариант метода сортировки, в котором после ссылки на сортируемый массив в качестве второго аргумента применяется имя функции, задающей правило сравнения элементов. Напомним, что функция в C# — это метод, возвращающий значение, отличное от **void**. Такую функцию для правила сравнения программист-пользователь должен написать самостоятельно по следующей схеме.

```

int имя_функции(тип параметр_1, тип параметр_2)
{
    if(условие 1) return +1; // нарушен порядок
    if(условие 2) return -1; // порядок соблюден
    return 0;                // безразличие
}

```

Имя_функции выбирается произвольно. Тип возвращаемого значения `int`, тип параметров должен быть тем же, что и у элементов сорти-

руемого массива. Условия 1 и 2 — логические выражения, в которые входят параметры 1 и 2. Обычно это отношения между параметрами.

Получив в качестве аргумента имя такой функции, метод Sort() применяет ее для принятия решения об относительной упорядоченности пар элементов сортируемого массива. Делается это приблизительно (мы ведь не знаем алгоритма работы метода Sort()) так. Если результат равен 0, то сравниваемые элементы равнозначны, их не нужно менять местами. Если результат равен +1, то первый параметр должен быть помещен после второго. При результате -1 первый параметр должен размещаться до второго.

В следующей программе две функции сравнения реализованы в виде статических методов класса, где размещен метод Main(). Первая функция even_odd() проверяет четность своих параметров и возвращает +1, если первый параметр нечетный, а второй — четный. При равенстве параметров возвращается 0, а при ложности обоих условий возвращается -1. Результат -1 означает, что первый параметр четный — его никуда не нужно перемещать. Таким образом, функция «приказывает» методу Array.Sort() так поместить элементы массива, чтобы вначале находились четные, а за ним — нечетные значения.

Вторая функция drop() «настраивает» метод сортировки на упорядочение массива по убыванию значений его элементов. Значение +1 возвращается в том случае, если первый параметр меньше второго.

В методе Main() определен и инициализирован целочисленный массив, который трижды сортируется методами Sort(). Первый раз по четности, второй раз по возрастанию, третий раз по убыванию значений элементов. После каждой сортировки выводятся значения элементов на консольный экран. Текст программы и полученные результаты поясняют сказанное.

```
// 09_18.cs - применение метода Array.Sort()
using System;
class Program
{
    static int even_odd(int x, int y) // по четности
    {
        if (x % 2 != 0 & y % 2 == 0) return 1;
        if (x == y) return 0;
        return -1;
    }
    static int drop(int x, int y) // по убыванию
    {
        if (x < y) return 1;
        if (x == y) return 0;
        return -1;
    }
    static void Main()
    {
        int[] ar = { 4, 5, 2, 7, 8, 1, 9, 3 };
        Array.Sort(ar, even_odd); // по четности
```

```

foreach (int memb in ar)
    Console.Write("{0} ", memb);
Console.WriteLine();
Array.Sort(ar);    // по умолчанию - по возрастанию
foreach (int memb in ar)
    Console.Write("{0} ", memb);
Console.WriteLine();
Array.Sort(ar, drop); // по убыванию
foreach (int memb in ar)
    Console.Write("{0} ", memb);
Console.WriteLine();
    }
}

```

Результат выполнения программы:

```

8 2 4 3 9 1 7 5
1 2 3 4 5 7 8 9
9 8 7 5 4 3 2 1

```

9.9. Кортежи и ссылки в методах

Предупреждение. Кортежи появились в .NET Framework 4.7. Чтобы использовать кортеж и в более ранней версии .NET Framework, необходимо в проекте установить (и включить в число ссылок) NuGet пакет System.ValueTuple.

Порядок действий. В контекстном меню ветви «Ссылки» Обзорера решений MS VS 2017 выбрать пункт «Управление пакетами NuGet»; в появившемся диалоге выбрать «nuget.org» из выпадающего списка «Источник пакета»; в строке поиска ввести «System.ValueTuple»; перейти на вкладку «Обзор»; выбрать «System.ValueTuple»; «Установить»; закрыть вкладку; проверить для проекта наличие ссылки System.ValueTuple.

Подобно тому, как класс Array поддерживает работу с массивами, статический класс Tuple дает возможность создавать и применять в проекте кортежи. В отличие от массива, элементы которого имеют один тип, кортеж может представлять набор значений разных типов. Кортеж, по существу, это экземпляр класса или структуры с членами данных, но без поведения. Таким образом, кортеж — контейнер разнотипных данных.

Для объявления переменной с типом кортежа используется конструкция:

```
(список_типов_элементов) имя_кортежа;
```

В отличие от ссылочных типов, создавая переменную с типом кортежа, не требуется использовать операцию **new** — кортеж сразу размещается в памяти, и элементы кортежа доступны с помощью свойств с фиксированными именами Item1, Item2, ...

Пример:

```
(string, double, string) tup ;  
tup.Item1 = "Расстояние: ";  
tup.Item2 = 234.0;  
tup.Item3 = " километра";  
Console.WriteLine($"{tup.Item1}{tup.Item2}{tup.Item3}");
```

Результат на экране:

Расстояние: 234 километра

Создавать кортеж можно с инициализацией его элементов, что позволяет применять неявную типизацию кортежа. При этом возможны два подхода — с явным именованием элементов кортежа и с их неявным именованием.

Создание кортежа с неявной типизацией при инициализации неименованных элементов:

```
var имя_кортежа = (список_литералов);
```

Контекстнозависимое служебное слово **var** «сообщает компилятору» о необходимости неявной типизации локальной переменной, «*имя_кортежа*». Список литералов определяет значения (и типы) элементов кортежа. При такой декларации кортежа его элементы доступны с помощью свойств с фиксированными именами: Item1, Item2, ...

Пример:

```
var cost = (4.5, "евро");  
Console.WriteLine($"Цена = {cost.Item1} {cost.Item2}");  
cost.Item2 = "$";  
Console.WriteLine($"Цена = {cost.Item1} {cost.Item2}");
```

Результат на экране:

Цена = 4,5 евро
Цена = 4,5 \$

Имена элементов кортежа при его инициализации можно декларировать явно, используя для каждого элемента конструкцию:

```
имя_элемента: значение
```

При такой инициализации фиксированные имена Item1, Item2 также сохраняются за элементами кортежа.

Пример:

```
var area = (w:1.5, h:43.2);  
Console.WriteLine($"Площадь = {area.Item1*area.h}");
```

Результат на экране:

Площадь = 64,8

Имена элементов кортежа можно декларировать в указании его типа, т. е. при явном объявлении типа переменной с типом кортежа. Покажем это на примере:

```
(double leg1, double leg2) legs = (3, 4);  
double hypotenuse = Math.Sqrt(Math.Pow(legs.leg1, 2)  
    + Math.Pow(legs.leg2, 2));  
Console.WriteLine($"Гипотенуза = {hypotenuse}");
```

Результат на экране:

Гипотенуза = 5

В ряде случаев возникает необходимость «распаковать» кортеж и получить набор переменных (с выбранными программистом именами), каждая из которых получает значение соответствующего элемента кортежа. Для указанной деконструкции кортежей возможны несколько способов.

Можно в объявлении типа кортежа после типа каждого элемента указать имя (идентификатор) переменной, которой присваивается значение, очередного элемента кортежа. Такую конструкцию для распаковки кортежа можно представить так:

```
(спецификация_переменных) = имя_кортежа;
```

Пример распаковки определенного ранее кортежа `legs`:

```
(double k1, double k2) = legs;  
Console.WriteLine($"Катет = {k1}");  
k1 *= 2;  
Console.WriteLine($"Катет = {k1}");  
Console.WriteLine($"leg1 = {legs.leg1}");
```

Результат на экране:

Катет = 3
Катет = 6
leg1 = 3

Обратите внимание, что переменная `k1` получает значение первого элемента кортежа, но не ссылается на него. Значение `legs.leg1` не изменилось, после увеличения вдвое `k1`.

Так как типы всех элементов «известны компилятору», то типы переменных, формируемые при деконструкции кортежа, могут указываться неявно, с помощью контекстно-зависимого служебного слова **var**.

Пример распаковки определенного ранее кортежа `area`:

```
(var x, var y) = area;  
Console.WriteLine($"W = {x}; H = {y}");
```

Результат на экране:

W = 1,5; H = 43,2

Тот же результат получим, если «доверим» компилятору распознавание сразу всех типов элементов кортежа:

```
var (x, y) = area;
```

Кортежи удобно применять в методах, которые должны возвращать несколько значений. В качестве примера приведем код метода, вычисляющего вещественные корни квадратного уравнения.

```
static public (double, double)
Quadratic(double a = 1, double b = 1, double c = -6)
{
    double disk = b * b - 4 * a * c;
    if (disk < 0)
    {
        Console.WriteLine("Отрицательный дискриминант!");
        return (0, 0);
    }
    double z1 = (b + Math.Sqrt(disk)) / 2 / a;
    double z2 = (b - Math.Sqrt(disk)) / 2 / a;
    return (z1, z2); // Кортеж
}
```

В качестве типа результата, возвращаемого статическим открытым-методом Quadratic(), указан тип кортежа (**double, double**). Параметры метода имеют умалчиваемые значения, поэтому к методу можно обращаться с пустым списком аргументов. При неотрицательном дискриминанте (переменная **double disk**) вычисляются корни уравнения и в операторе **return** создается экземпляр кортежа с двумя вещественными элементами.

Обращение к методу и вывод результатов:

```
var (x1, x2) = Quadratic();
Console.WriteLine("x1 = " + x1);
Console.WriteLine("x2 = " + x2);
```

Результат на экране:

```
x1 = 3
x2 = -2
```

Кортеж можно использовать в качестве параметра, аргумента и возвращаемого методом результата. Чтобы было понятно, как метод может вернуть ссылку на кортеж, приведем более простой пример — определим метод, который возвращает в точку вызова ссылку на элемент массива, имеющий максимальное значение.

```
// 09_Ref - метод возвращает ссылку
using System;
class Program
{
    static ref int F(int[] arr)
    {
```



```

    int index = 0, max = int.MinValue;
    for (int k = 0; k < arr.Length; k++)
        if (max < arr[k])
            { index = k; max = arr[k]; }
    return ref arr[index];
}
static void Main()
{
    int[] massiv = { 1, 2, 3, 4, 0 };
    Console.WriteLine("Max = " + F(massiv));
    F(massiv) = 23;
    Console.WriteLine("Max = " + F(massiv));
}
}

```

Результат на экране:

```

Max = 4
Max = 23

```

В заголовке метода `F()` одна особенность — модификатор `ref` перед типом возвращаемого значения. Параметр метода — целочисленный массив. В теле метода определяется индекс (переменная `index`) максимального элемента массива-параметра, и ссылка на этот элемент служит возвращаемым значением в операторе

```
return ref arr[index];
```

В методе `Main()` определен массив, ссылка на который (`massiv`) служит аргументом троекратного обращения к методу `F()`. При втором обращении вызов метода помещен слева от знака операции присваивания — это позволило изменить значение максимального элемента массива.

Контрольные вопросы и задания

1. Какие элементы входят в заголовок метода?
2. Что такое тело метода?
3. Назовите особенности метода-процедуры.
4. В каких случаях метод, возвращающий отличное от `void` значение, играет роль процедуры?
5. В каком случае в теле метода может отсутствовать оператор `return`?
6. Перечислите возможные источники данных, получаемых методом при его выполнении.
7. Какой метод является локальным?
8. Что такое метод, сжатый до выражения?
9. Назовите глобальные по отношению к методу объекты.
10. Перечислите модификаторы параметров методов.
11. Укажите область видимости параметра метода.

12. Назовите виды параметров.
13. В чем особенности статических методов?
14. Может ли статический метод играть роль процедуры?
15. Назовите требования к аргументам метода, заменяющим фиксированные параметры.
16. В чем отличия передачи параметров по значениям от передачи по ссылкам?
17. Какие ограничения накладываются на аргументы, заменяющие передаваемые по ссылке параметры?
18. В чем отличия модификаторов **out** и **ref**?
19. В каких случаях допустимо объявление переменных-аргументов «на лету»?
20. Для каких параметров допустимо «отбрасывание» аргументов?
21. Объясните синтаксис и возможности параметров со значениями по умолчанию.
22. Что такое именованные аргументы?
23. Может ли параметр с типом ссылки передаваться методу по значению?
24. Может ли параметр с типом ссылки снабжен модификатором **ref**?
25. Может ли аргумент с типом ссылки, замещающий передаваемый по значению параметр, изменить внешний для метода объект?
26. В каком случае можно подставить аргумент типа **long** вместо параметра типа **object**?
27. Какими средствами можно сделать доступным вне метода объект, созданный в теле метода?
28. Какой параметр представляет в теле метода список аргументов нефиксированной длины?
29. Как в теле метода выполняются обращения к аргументам, количество которых переменное?
30. Можно ли за счет выполнения метода изменить значения аргументов, представляемых в методе параметром с модификатором **params**?
31. Приведите примеры полиморфизма.
32. Что входит в сигнатуру метода при перегрузке?
33. Что такое перегрузка методов?
34. Какой метод называют рекурсивным?
35. В чем отличие косвенной рекурсии от прямой?
36. Назовите требования к корректному рекурсивному методу и правила удовлетворения этих требований.
37. Что такое кортеж и как он определяется?
38. Как выполняется распаковка (деконструкция) кортежа?
39. Перечислите способы задания имен кортежа.
40. Назовите особенности синтаксиса метода, возвращающего в точку вызова ссылку.

Глава 10

КЛАСС КАК СОВОКУПНОСТЬ СТАТИЧЕСКИХ ЧЛЕНОВ

10.1. Статические члены класса

Мы уже отмечали, что класс в общем случае представляет собой «трафарет» для создания объектов (этого класса) и «оболочку», в которую заключены статические члены (например, поля и методы). Остановимся подробнее на второй из названных ролей класса — рассмотрим класс как совокупность статических членов. При этом обратим внимание читателя, что в программе кроме класса, включающего метод `Main()`, может присутствовать любое число других классов. В этой главе будем определять классы с помощью объявлений такого формата:

```
class имя_класса  
тело_класса
```

Здесь **class** — служебное (ключевое) слово; *имя_класса* — идентификатор, выбранный программистом в качестве имени класса; *тело_класса* — заключенная в фигурные скобки последовательность объявлений (деклараций) статических членов класса (сразу же отметим, что статический характер членов класса — это совсем не обязательная их особенность, а ограничение, которого мы будем придерживаться в этой главе).

Имеется две разных возможности определить член класса как статический. С помощью модификатора **static** объявляются статические:

- поле (field);
- метод (method);
- свойство (property);
- событие (event);
- операция (operator);
- конструктор (constructor).

Кроме того, без модификатора **static** по умолчанию объявляются как статистические члены класса:

- константы;
- локализованные в классе типы (например, вложенные классы).

Для целей данной главы ограничимся статическими полями, статическими методами, статическими конструкторами и статическими

константами. Статические методы, которые мы подробно рассмотрели в предыдущей главе, будем считать хорошо знакомыми.

При объявлении члена класса он может быть снабжен одним или несколькими модификаторами. В этой главе нам предстоит познакомиться со следующими: **static**, **public**, **protected**, **internal**, **private**, **readonly**, **volatile**. Модификатор **static** мы уже использовали. Применялся в предыдущих программах и модификатор **public**. Его назначение — сделать член класса доступным извне объявления класса. Имеются и другие возможности влиять на доступность членов класса. Проще всего объяснить правила доступа к членам классов, рассмотрев вначале список модификаторов доступа.

Модификатор доступа	Описание
public (открытый)	доступен без ограничений всем методам всех сборок
protected (защищенный)	доступен в классе и в любом классе, производном от данного
internal (внутренний)	доступен везде в пределах сборки
protected internal (защищенный внутренний)	доступен в пределах сборки, в пределах класса и в любом классе, производном от данного
private (закрытый)	доступен только в классе и вложенных в него типах (классах).

В объяснении модификаторов доступа нам впервые встречается термин «сборка» (*assembly*). В Стандарте C# [1] сборка определяется как совокупность из одного или нескольких файлов, которые создает компилятор как результат компиляции программы. В зависимости от того, какие средства и режимы компиляции выбраны, можно создать однофайловую или многофайловую сборку. Сборка позволяет рассматривать группу файлов как единую сущность. Сейчас достаточно, чтобы читатель знал о существовании такого понятия.

Каждый член класса может быть снабжен одним и только одним модификатором доступа. По умолчанию (при отсутствии модификатора) члену приписывается статус доступа **private**, т. е. член класса недоступен вне класса, где он объявлен.

Для обращения к статистическому члену класса используется квалифицированное имя:

```
имя_класса.имя_члена
```

Так как мы уже использовали статические члены библиотечных классов, например, `Math.PI` и `Console.WriteLine()`, то нам уже знаком этот синтаксис.

Внутри класса, которому принадлежит статический член, разрешено использовать имя члена без указания имени класса.

Внимание: никакие операторы, отличные от объявлений, в классе недопустимы!

10.2. Поля классов (статические поля)

Как формулирует стандарт языка C#, поле — это член класса, который представляет переменную, ассоциированную с объектом или классом. Поле — это объявление данных, принадлежащих либо классу в целом, либо каждому из его объектов. Чтобы поле принадлежало не объекту, а классу, и могло использоваться до определения объектов этого класса, его объявляют статическим, используя модификатор **static**. Если в объявлении поля этот модификатор отсутствует, то поле является полем объектов. В этом случае полю выделяется память и оно становится доступным для использования только при создании объекта и после его создания. Статическое поле размещается в памяти в момент загрузки кода класса. Статическое поле (поле класса) доступно для использования и до, и после создания объектов класса.

Объявление статического поля имеет вид:

```
модификаторыopt static  
тип_поля список_объявлений;
```

Индекс «opt» указывает на необязательность модификаторов, отличных от **static**. В качестве этих необязательных модификаторов поля могут использоваться уже перечисленные модификаторы доступа **public**, **private**, **protected**, **internal**. Кроме них в объявление поля могут входить модификаторы:

- **new** — применяется при наследовании (см. гл. 13);
- **readonly** — запрещает изменять значение поля. Поле получает значение при его инициализации или за счет действий конструктора, и это значение сохраняется постоянно;
- **volatile** — модификатор поля, указывающий, что его значение может измениться независимо от операторов программы. Примером может быть переменная, значение которой сохраняет момент начала выполнения программы. Два последовательных запуска программы приведут к разным значениям этого поля. Второй пример — поле, значение которого в процессе выполнения программы может изменить операционная система или параллельно выполняемый процесс.

Зачастую список объявлений полей содержит только одно объявление. Каждое объявление в списке имеет следующий вид:

```
идентификатор инициализатор_поляopt
```

Индекс «opt» указывает на необязательность инициализатора поля.

Если в объявлении поля отсутствует инициализатор, то инициализация выполняется по умолчанию значением соответствующего типа. В соответствии с общей концепцией типов языка C# поля могут иметь

типы ссылок и типы значений. Возможны следующие формы инициализаторов полей:

- = *выражение*
- = *инициализатор_массива*
- = **new** *тип* (*аргументы_конструктора*)

В выражение инициализатора, в инициализатор массива и в выражения, используемые в качестве аргументов конструктора, могут входить константы—литералы, а также другие статические члены этого класса. Нестатические члены невозможно использовать в инициализирующем выражении статического поля.

Пример инициализации и использования статических полей:

```
// 10_01.cs – статические поля класса
using System;
class T
{
    int x; // поле объектов класса
    static int y = 3; // y = x + 5; – ошибка!
    public static int z = 5 - y; // эквивалент: z = 5 - T.y;
}
class Program
{
    static void Main()
    {
        // Console.WriteLine("T. y = " + T. y); // Error!
        int z = T.z; // z и T.z – разные переменные!
        Console.WriteLine("T.z = " + ++T.z); // изменяется T.z
        Console.WriteLine("z = {0}", z);
    }
}
```

Результат выполнения программы:

```
T.z = 3
z = 2
```

Следует отметить одну особенность инициализации статических полей. Вначале они все получают значения по умолчанию (для арифметических типов — нулевые). Затем последовательно (сверху вниз) выполняются инициализаторы. Тем самым, если в инициализирующее выражение входит в качестве операнда статическое поле, которое еще не инициализировано явно, то для него берется значение по умолчанию (например, нулевое). Для иллюстрации особенностей инициализации статических полей рассмотрим программу:

```
// 10_02.cs - статические поля - порядок инициализации
using System;
class T
{
    public static int x = y;
    static int y = 3;
```

```

    public static int z = y;
}
class Program
{
    static void Main()
    {
        Console.WriteLine("T.x = " + T.x);
        Console.WriteLine("T.z = " + T.z);
    }
}

```

Результат выполнения программы:

```

T.x = 0
T.z = 3

```

Обратите внимание, что поле `y` по умолчанию закрытое (имеет статус доступа **private**) и обращение к нему невозможно вне класса `T`.

Открытые статические члены разных классов одной программы могут использоваться в одном выражении. В следующей программе объявлены три класса со статическими полями, обращения к которым используются в инициализирующих выражениях разных классов.

```

// 10_03.cs - статические поля разных классов
using System;
class c1
{
    public static double orbit = 2 * c2.pi * c3.radius;
}
class c2
{
    public static double pi = double.Parse(c3.str);
}
class c3
{
    public static string str = "3,14159";
    public static double radius = 10;
}
class Program
{
    static void Main()
    {
        Console.WriteLine("c1.orbit = {0}", c1.orbit);
        c3.radius = 20;
        Console.WriteLine("c3.radius = {0}", c3.radius);
        Console.WriteLine("c1.orbit = {0}", c1.orbit);
    }
}

```

Результат выполнения программы:

```

c1.orbit = 62,8318
c3.radius = 20
c1.orbit = 62,8318

```

Инициализация статических полей выполняется только однажды. Поэтому значение поля `c1.orbit` не изменилось после изменения `c3.radius` в методе `Main()`.

10.3. Статические константы

Константы могут быть локализованы, например, в теле метода, а могут принадлежать классу. В последнем случае они по умолчанию без модификатора **static** являются статическими, т. е. недоступны через ссылку на объекты класса и доступны с помощью квалификации именем класса.

Объявление статической константы имеет вид:

```
модификаторыopt const тип_константы  
список_объявлений_констант
```

Необязательные модификаторы для констант — это при отсутствии наследования модификаторы доступа (**public**, **private**, **protected**, **internal**).

Объявление из *списка_объявлений_констант* имеет вид:

```
идентификатор = инициализирующее_выражение
```

Идентификатор служит именем статической константы. В инициализирующее выражение имеют право входить только константы-литералы и статические константы (порядок их размещения в объявлении класса безразличен). Каждая константа должна быть обязательно явно инициализирована. Значения по умолчанию для статических констант не предусмотрены.

Пример с классом статических констант:

```
// 10_04.cs - статические константы класса  
using System;  
class Constants  
{  
    public const double скорость_света = 299793; // км/сек  
    public const double радиус_электрона = 2.82e-13; // см  
}  
class Program  
{  
    static void Main()  
    {  
        Console.WriteLine("радиус_электрона = {0}",  
            Constants.радиус_электрона);  
    }  
}
```

Результат выполнения программы:

```
радиус_электрона = 2,82E-13
```


В отличие от статических полей константы инициализируются однажды и не принимают умалчиваемых значений. Поэтому последовательность размещения объявлений статических констант не важна. Для иллюстрации этой особенности приведем следующую программу:

```
// 10_05.cs - статические константы разных классов
using System;
class One
{
    public const double circle = 2 * pi * Two.radius;
    public const double pi = 3.14159;
}
class Two
{
    public const double radius = 20;
}
class Program
{
    static void Main()
    {
        Console.WriteLine("One.circle = {0}", One.circle);
    }
}
```

Результат выполнения программы:

```
One.circle = 125,6636
```

Следует обратить внимание на отличия статических констант от статических полей с модификатором **readonly** (только чтение). Статические константы получают значения при компиляции, а статические поля (даже снабженные модификатором **readonly**) инициализируются в процессе выполнения программы. Инициализация статических полей выполняется в порядке их размещения в тексте объявления класса, и до инициализации поле имеет умалчиваемое значение. В то же время инициализирующие выражения статических констант вычисляются в «рациональной» последовательности, независимо от порядка размещения объявлений. Эти особенности иллюстрируют объявления констант в классе One:

```
public const double circle = 2*pi*Two.radius;
public const double pi = 3.14159;
```

В инициализатор константы `circle` входят константы `pi` и `Two.radius`, объявления которых следуют за объявлением `circle`. Несмотря на это, значением `circle` будет 125,6636. Конечно, при инициализации нескольких констант недопустимо появление «зацикливания». В инициализатор константы `K1` не должна входить константа `K2`, в инициализатор которой входит константа `K1`.

Указанное «зацикливание» никогда не возникает при инициализации статических полей (даже имеющих модификатор **readonly**).

Поле F2, объявление которого размещено ниже, может входить в инициализатор поля F1 — оно имеет там значение по умолчанию. При этом поле F1, использованное в инициализаторе F2, будет иметь там уже конкретное значение. Замена **const** на **static readonly** в предыдущем примере:

```
public static readonly double circle = 2*pi*Two.radius;  
public static readonly double pi = 3.14159;
```

При такой инициализации значением `circle` будет 0.

10.4. Статические методы

Статический метод `Main()` мы уже используем, начиная с первой программы этой книги. Нам уже известно, что в теле метода `Main()` можно обращаться ко всем статическим членам того класса, где размещен метод `Main()`. Кроме того, в теле метода можно обращаться к методам, локализованным в нем. Локальные методы не могут объявляться с модификатором **static**, но каждый локальный метод, включенный в статический метод, по умолчанию является статическим.

Статические методы, отличные от `Main()`, мы уже подробно рассматривали, но ограничивались их размещением в том классе, где находится метод `Main()`. В этом случае для обращения к методу можно использовать выражение *имя_метода(список_аргументов)*. Если статический метод определен в другом классе, то для обращения к нему используется выражение вида:

```
имя_класса.имя_метода(список_аргументов)
```

По умолчанию статический метод имеет модификатор доступа **private**, т. е. метод доступен только внутри класса. В объявлении статического метода, который планируется вызывать вне класса, необходимо указать модификатор доступа. В зависимости от дальнейшего применения метода используются модификаторы **public** (открытый), **protected** — (защищенный), **internal** — (внутренний), **protected internal** — (и защищенный, и внутренний). Области видимости, которые обеспечены перечисленными модификаторами, те же что и для полей классов.

Статические методы применяются: для инициализации статических полей; для обращения к статическим полям с целью их изменения или получения значений; для обработки внешних данных, передаваемых методу через аппарат параметров; для вызова из других методов класса. Особо нужно отметить, что в теле статического метода недоступны нестатические члены того класса, которому принадлежит метод.

В следующей программе статические методы используются для инициализации статических полей и для обеспечения внешнего доступа к закрытому полю класса.

```
// 10_06.cs - статические методы класса
using System;
class newClass
{
    static int n = 5;
    static int x = f1(n);
    public static int y = fb(n) ? 10 : -10;
    public static int getX()
    { return x; }
    static bool fb(int a)
    { return a % 2 == 0 ? true : false; }
    static int f1(int d)
    { return d * d * d; }
} // End of class newClass
class Program
{
    static void Main()
    {
        Console.WriteLine("newClass.y = " + newClass.y);
        Console.WriteLine("newClass.getX() = " + newClass.getX());
    }
}
```

Результат выполнения программы:

```
newClass.y = -10
newClass.getX() = 125
```

Обратите внимание, что в классе `newClass` только два открытых члена — поле `int` `y` и метод `getX()`. Поля `x`, `n` и методы `fb()`, `f1()` недоступны вне класса. В методе `Main()` выполнено обращение к полю `newClass.y`. Это поле инициализировано значением тернарного выражения, в котором первым операндом служит обращение к методу `fb(n)`. Аргумент — статическая переменная, инициализированная константой 5. Так как `fb(n)` возвращает значение `false`, то переменная `y` получает значение `-10`. При обращении к методу `newClass.getX()` он возвращает значение закрытого статического поля `static int` `x`. Это поле инициализировано за счет обращения к статическому методу `f1()`, что и определяет результат, выведенный на консоль.

В теле любого метода могут быть объявлены локальные переменные и константы с типами ссылок и значений. Имена таких переменных имеют право совпадать с именами статических членов того класса, в котором размещен метод. В этом случае локальная переменная «экранирует» одноименную статическую переменную, т. е. имя без квалификатора «имя_класса.» при совпадении имен относится только к локальному объекту метода. В качестве примера рассмотрим следующий метод `Main()` из класса `Test_cs`:

```
// 10_07.cs - Статические поля и локальные данные методов
using static System.Console;
class Test_cs
```

```

{
    static int n = 10; // поле класса Test_cs
    static string line = new string('*', n); // поле класса Test_cs
    static double constant = 9.81; // поле класса Test_cs
    double pi = 3.14159; // поле объекта класса Test_cs
    static void Main()
    {
        const double PI = 3.14159; // локальная константа
        double circle; // локальная переменная
        // circle = 2 * pi * n; // ошибка - нет объекта класса Test_cs
        circle = 2 * PI * n;
        WriteLine("Длина окружности=" + circle.ToString());
        WriteLine(line);
        Test_cs.n = 20; // изменили значение поля класса
        line = new string('*', n); // изменили значение ссылки
        WriteLine(line);
        const double constant = 2.718282; // локальная константа
        WriteLine("Test_cs.constant=" + Test_cs.constant);
        WriteLine("constant=" + constant);
    }
}

```

Результат выполнения программы:

```

Длина окружности=62,8318
*****
*****
Test_cs.constant=9,81
constant=2,718282

```

В тексте метода Main():

PI — локальная именованная константа;

circle — локальная переменная;

n, Test_cs.n — два обращения к одному и тому же статическому полю класса;

line — статическая ссылка — поле класса Test_cs;

constant — локальная константа, имя которой совпадает с именем статического поля класса Test_cs.

Попытка обращения в теле метода Main() к нестатическому полю pi приводит к ошибке — пока не создан объект класса (объект, которому будет принадлежать поле pi) к его нестатическим полям обращаться нельзя, их нет.

Каждое статическое поле существует только в единственном экземпляре. Статические поля класса играют роль глобальных переменных для всех методов, у которых есть право доступа к этому классу.

Итак, пока не создан объект класса, существуют и доступны только статические члены класса.

10.5. Статический конструктор и статический класс

Назначение статического конструктора — инициализация статических полей класса. Статический конструктор вызывается средой испол-

нения приложений (CLR) перед первым обращением к любому статическому полю класса или перед первым созданием экземпляра (объекта) класса.

Конструкторы классов — статические и нестатические (последние рассмотрены в гл. 11) обладают рядом особенностей, отличающих их от других методов классов. Имя конструктора всегда совпадает с именем того класса, которому он принадлежит. Для конструктора не указывается тип возвращаемого значения (даже тип **void** для конструктора запрещен). В теле конструктора обычно нет необходимости, но допустимо использовать оператор **return**. Для статического конструктора нельзя использовать модификаторы доступа.

Класс может иметь только один статический конструктор. Для статического конструктора параметры не указываются — спецификация параметров должна быть пустой.

Формат объявления статического конструктора:

```
static имя_класса()  
    {операторы_тела_конструктора}
```

Статический конструктор невозможно вызвать непосредственно из программы — статический конструктор вызывается только автоматически. Следует обратить внимание, что статический конструктор вызывается *после* выполнения инициализаторов статических полей класса. Основное назначение статического конструктора — выполнять более сложные действия, нежели инициализаторы полей и констант. Для статического конструктора недоступны нестатические члены класса.

В качестве примера определим статический конструктор того класса, в котором размещен метод `Main()`. В том же классе определим несколько статических полей и выполним их инициализацию как с помощью инициализаторов, так и с применением статического конструктора.

```
// 10_08.cs - Инициализаторы и статический конструктор  
using System;  
class Program  
{  
    static int[] ar = new int[] { 10, 20, 30, 40 };  
    static int numb = n + ar[3] - ar[1];  
    // Статический конструктор:  
    static Program()  
    { numb /= n; n = ar[1] + n; }  
    static int n = 2;  
    static void Main()  
    {  
        Console.WriteLine("numb={0}, n = {1}", numb, n);  
    }  
}
```

Результат выполнения программы:

```
numb=10, n = 22
```

В классе определены статические поля: ссылка `ag`, связанная с целочисленным массивом из 4 элементов; целочисленные переменные `numb` и `n`. В теле статического конструктора переменным `numb` и `n` присваиваются значения, отличные от тех, которые они получили при инициализации. Текст программы и результаты ее выполнения иллюстрируют следующие правила, большинство из которых мы уже приводили.

Инициализация статических полей выполняется последовательно (сверху вниз) по тексту определения класса. Поля, для которых инициализация еще не выполнена, имеют умалчиваемые значения (для арифметических типов значение статического поля по умолчанию равно нулю). В инициализирующих выражениях статических полей допустимо использовать обращения к другим статическим членам классов. После выполнения инициализации статических полей выполняется статический конструктор, действия которого могут изменить значения статических полей.

Как рекомендует Стандарт C# [1], классы, которые не предназначены для создания объектов и которые содержат только статические члены, нужно объявлять статическими. В .Net Framework такими классами являются `System.Control` и `System.Environment`. Перечислим основные особенности статических классов.

В статических классах нет конструкторов экземпляров (объектов) и эти классы не могут служить базовыми классами при наследовании (наследование рассматривается в гл. 13.) Статические классы могут использоваться только с операцией **`typeof`** и средствами доступа к членам класса. В частности, статический класс не может использоваться в качестве типа переменной и не может выступать в качестве типа параметра.

В заголовок объявления статического класса входит модификатор **`static`**. В отличие от других классов статический класс нельзя объявить с модификаторами **`sealed`** и **`abstract`**. Однако, так как статический класс не участвует в иерархиях наследования, именно свойства классов с этими модификаторами присущи статическому классу.

В объявлении статического класса нельзя применять спецификатор базы. Кроме того, статический класс не может реализовывать интерфейсы (интерфейсам посвящена гл. 14). Статический класс имеет только один базовый класс `System.Object`.

Так как статический класс не может быть базовым, то для его членов запрещены модификаторы **`protected`** и **`protected internal`**. Для членов статического класса в качестве модификаторов доступа можно использовать только **`public`** и **`private`**.

Несмотря на наличие модификатора **`static`** в заголовке статического класса, все его члены, отличные от констант и вложенных типов (классов), должны быть объявлены с явно указанным модификатором **`static`**.

Статические классы удобно применять для логического объединения функционально близких методов.

Примеры статических классов приводить нет необходимости, так как многие классы этой главы включают только статические члены, и каждый из этих классов может быть снабжен заголовком

```
static class имя_класса
```

В заключение отметим еще раз, что класс нельзя объявить статическим, если в его объявлении присутствует хотя бы один нестатический член.

Контрольные вопросы и задания

1. Перечислите члены класса, которые могут быть объявлены статическими.
2. Какие члены класса являются статическими без применения модификатора **static**?
3. Перечислите модификаторы доступа.
4. Можно ли в объявлении члена класса использовать два модификатора доступа?
5. Приведите формат имени статического члена класса, используемого для обращения к нему извне класса.
6. Что такое поле класса?
7. Когда статическое поле размещается в памяти?
8. Можно ли в объявлении статического поля использовать модификатор доступа?
9. Что такое список объявлений полей?
10. Назовите формы инициализаторов полей.
11. Что разрешено использовать в инициализирующем выражении статического поля?
12. Как выполняется инициализация статических полей при отсутствии инициализаторов?
13. Объясните последовательность инициализации статических полей.
14. Какой статус доступа у статического поля при отсутствии модификатора доступа?
15. Как объявляются константы, принадлежащие классу?
16. Сформулируйте правила инициализации констант класса.
17. В чем отличие статических констант от статических полей с модификатором **readonly**?
18. Перечислите возможные применения и ограничения статических методов.
19. Что такое статический конструктор?
20. Сколько статических конструкторов в классе?
21. Какова спецификация параметров статического конструктора?
22. Как и когда вызывается статический конструктор?

23. Какие члены объявления класса доступны в теле статического конструктора?
24. Перечислите особенности статических классов.
25. Какие модификаторы не могут входить в объявление статического класса?
26. Какие модификаторы могут входить в объявления членов статического класса?
27. Может ли в статический класс входить нестатический рекурсивный метод?

Глава 11

КЛАССЫ КАК ТИПЫ

11.1. Объявление класса

Уже говорилось, что класс в языке C# играет две роли: это совокупность или «контейнер» статических членов (методов, полей...), и это «трафарет», позволяющий порождать конкретные объекты. Класс как контейнер нами уже подробно изучен.

Класс как трафарет — это библиотечный или определяемый программистом-пользователем тип данных. В английском языке для определенного программистом класса, играющего эту роль, используется краткое обозначение UDT — *user-defined type*. Сейчас будем рассматривать именно такие классы.

Класс, как определяемый пользователем тип, позволяет создавать конкретные объекты, состав и состояние каждого из которых задают нестатические поля класса, а поведение — его нестатические методы.

Достаточно общее определение (декларация, иначе объявление) класса имеет следующий формат:

```
модификаторы_классаopt  
class имя_класса спецификация_базы_классаopt  
тело_класса;opt
```

Как мы уже говорили, индекс «opt» (от английского *option* — альтернатива, выбор) после элемента декларации указывает на необязательность предшествующего ему элемента. Даже точка с запятой после тела класса не является обязательной.

Начнем изучение особенностей построения тех пользовательских классов, в определения которых входят только обязательные элементы и, возможно, модификаторы класса. С учетом названных ограничений минимальный состав объявления класса таков:

```
модификаторы_классаopt class имя_класса тело_класса
```

В декларации класса **class** — служебное слово, называемое ключом класса. *Имя_класса* — идентификатор, выбранный автором класса в качестве его названия. *Модификаторы_класса* — это:

- один из пяти уже неоднократно названных модификаторов доступа (**public**, **protected**, **internal**, **private**, **protected internal**);

- модификаторы, применяемые при вложении и наследовании классов (**new**, **abstract**, **sealed** — одновременное применение модификаторов **sealed** и **abstract** недопустимо);

- **static** — модификатор статического класса.

При использовании нескольких модификаторов они отделяются друг от друга пробелами.

Тело_класса — заключенная в фигурные скобки последовательность объявлений (деклараций) членов класса.

Объявление члена класса:

- объявление константы;
- объявление поля;
- объявление метода;
- объявление свойства;
- объявление события;
- объявление индексатора;
- объявление операции;
- объявление конструктора;
- объявление деконструктора (C# 7.0);
- объявление финализатора (деструктора);
- объявление статического конструктора;
- объявление типа.

Перечислив разновидности членов класса, будем вводить их в употребление по мере необходимости и тогда же объяснять назначение и возможности каждого из них. Применяя в рассмотренных ранее программах библиотечные классы, мы использовали их методы, конструкторы, поля, константы и свойства. На первом этапе знакомства с особенностями декларации пользовательских классов ограничимся именно такими членами.

11.2. Поля объектов

Поле — это член объявления класса, представляющий переменную, ассоциированную с классом или его объектом. Поля классов или статические поля мы уже рассмотрели. Сосредоточимся на особенностях нестатических полей, т. е. полей объектов. Как уже сказано, одно объявление может вводить одно или несколько полей одного типа. Формат такого объявления:

```
модификаторы_поляopt  
тип_поля объявление_переменных;
```

Обратите внимание, что обязательными элементами объявления являются *тип_поля* и объявление переменных.

Тип_поля определяет тип вводимых этим объявлением переменных.

Объявление_переменных — это либо одно объявление, либо список объявлений, разделенных запятыми. Каждое объявление имеет один из следующих двух форматов:

идентификатор

идентификатор = инициализатор_переменной

В свою очередь инициализатор переменной может быть либо выражением, в том числе и выражением, содержащим обращение к конструктору объектов, либо инициализатором массива. Инициализация переменных нестатических полей выполняется при создании объекта класса. Если в объявлении переменной инициализатор отсутствует, то переменная инициализируется по умолчанию значением, соответствующим ее типу.

В определении поля могут присутствовать в допустимых сочетаниях несколько модификаторов, которые в этом случае отделяются друг от друга пробелами. Мы уже использовали поля с модификатором **static**.

Кроме модификатора **static** уже рассмотрены модификаторы, определяющие доступность членов класса вне объявления класса (**public** — открытый, **protected** — защищенный, **private** — закрытый, **internal** — внутренний). Статус закрытого доступа получают все члены класса по умолчанию, когда в объявлении модификатор доступа отсутствует.

Как и для статических полей, для полей объектов могут применяться модификаторы: **readonly** — только для чтения; **volatile** — подвержен внешним воздействиям; **new** — применяется при наследовании (см. гл. 13).

Некоторый опыт применения библиотечных классов у нас уже имеется, поэтому следующий пример мини-программы с двумя классами, которые определил программист, не вызовет затруднений.

```
// 11_01.cs - поля объектов и класса
using System;
class Person
{
    public static int year = 2020; // текущий год
    public int age;                // возраст
    public string name;            // имя
}
class Program
{
    static void Main()
    {
        Person who;                // ссылка на объект класса
        who = new Person();        // объект класса
        who.name = "Юджин";        // имя
        who.age = 19;              // возраст
        Console.WriteLine("Имя: {0}, Год рождения: {1}",
            who.name, Person.year - who.age);
    }
}
```

Результат выполнения программы:

Имя: Юджин, Год рождения: 2001

Класс `Person` включает одно статическое поле `year` (сегодняшний год) и два поля объектов `age` (возраст) и `name` (имя). Если в классе нет объявления конструктора, то компилятор `C#` автоматически создает для этого класса открытый (**public**) конструктор без параметров. Таким образом, в класс `Person` добавляется конструктор с заголовком:

```
public Person().
```

В методе `Main()` класса `Program` создана ссылка (переменная) с именем `who` типа `Person`. В операторе `who = new Person();` присваивается ссылка на объект класса `Person` переменной `who`. Статическое поле `year` инициализируется значением `2020`, а поля объекта (`age` и `name`) принимают значения по умолчанию (`0` и `""`).

Так как поля класса `Person` открытые, то ссылка `who` дает возможность присваивать полям объекта значения и получать их. Результаты выполнения иллюстрируют приведенные пояснения.

Важной особенностью (значение которой в дальнейшем будет показано, например, при создании связанных списков, см. параграф 11.5) является возможность объявить в классе поле, имеющее тип ссылки на объекты того же класса. Класс с таким полем приведен в следующей программе:

```
// 11_02.cs - ссылка с типом класса как поле объекта
using System;
class A
{
    public int a = 2;
    public int b = 3;
    public A memb; // ссылка на объект класса A
}
class Program
{
    static void Main()
    {
        A one = new A();
        one.a = 12;
        one.b = one.a / one.b;
        Console.WriteLine("one.a={0}", one.a);
        Console.WriteLine("one.b={0}", one.b);
        A two = new A();
        two.memb = one;
        Console.WriteLine("two.memb.a={0}", two.memb.a);
        Console.WriteLine("two.memb.b={0}", two.memb.b);
    }
}
```

Результат выполнения программы:

```
one.a=12
one.b=4
two.memb.a=12
two.memb.b=4
```

В классе A декларировано и по умолчанию инициализировано значением **null** поле `memb` — ссылка типа A. В методе `Main()` определены две ссылки (переменные `one` и `two`) на объекты класса A и ассоциированные с ними объекты. Полям `one.a` и `one.b` явно присвоены значения. Полю `two.memb` присвоено значение ссылки `one` и тем самым поле `two.memb` «настроено» на объект, адресуемый ссылкой `one`. Квалифицированные имена `two.memb.a` и `two.memb.b` позволяют получить доступ к полям `one.a` и `one.b`. Это подтверждают результаты выполнения программы.

Ссылка на объект класса может быть объявлена как статическое поле этого же класса. Следующая программа иллюстрирует эту возможность.

```
// 11_03.cs - статическое поле с типом класса
using System;
class A
{
    public int a = 2;
    public int b = 3;
    static public A smemb; // ссылка на объект класса A
}
class Program
{
    static void Main()
    {
        A two = new A();
        A.smemb = two;
        Console.WriteLine("A.smemb.a={0}", A.smemb.a);
        Console.WriteLine("A.smemb.b={0}", A.smemb.b);
    }
}
```

Результат выполнения программы:

```
A.smemb.a=2
A.smemb.b=3
```

Программа очень похожа на предыдущую — отличие заключается в том, что поле `smemb` статическое, поэтому для обращений используются квалифицированные имена с префиксом "A".

11.3. Объявления методов объектов

Определение метода включает две части:

- заголовок_метода;
- тело_метода.

Достаточно общий формат заголовка метода:

```
модификаторы_методаopt
тип_возвращаемого_значения имя_метода
(спецификация_параметровopt)
```

В качестве модификаторов метода используются:

static — вводит член класса (а не объекта);

public, protected, internal, private — модификаторы доступа;

virtual — виртуальный метод, который может быть переопределен при наследовании;

new — метод переопределяет одноименный метод базового класса;

sealed — метод защищен от переопределения;

override — метод переопределяет виртуальный метод базового класса;

abstract — виртуальный метод без реализации;

extern — метод, который реализован вне класса и, возможно, на языке, отличном от C#.

Большинство из модификаторов методов нам понадобятся позднее, но уже рассмотренными модификаторами доступа мы будем активно пользоваться уже сейчас.

В определении одного метода могут присутствовать несколько модификаторов, которые в этом случае отделяются друг от друга пробелами. Мы уже использовали методы с модификатором **static**. Эта глава посвящена методам объектов, в объявлениях которых модификатор **static** отсутствует. В отличие от методов классов их называют нестатическими методами или методами объектов класса.

В декларацию метода модификаторы могут входить только в допустимых сочетаниях. Правила определяющие, какие модификаторы могут, а какие не могут совместно использоваться в декларации одного метода, Стандарт C# формулирует таким образом:

- в декларацию метода может входить только один модификатор доступа;
- ни один модификатор не может появиться в декларации более одного раза;
- декларация может включать только один из модификаторов **static, virtual, sealed, override**;
- декларация может включать только один из модификаторов **new** и **override**;
- декларация, содержащая модификатор **abstract**, не может включать ни одного из следующих модификаторов: **static, virtual, sealed, extern**;
- если декларация включает модификатор **private**, то она не может включать ни один из модификаторов **virtual, override, abstract**;
- если декларация содержит модификатор **sealed**, то она также включает модификатор **override**.

Если декларация представляет собой явную реализацию члена интерфейса (интерфейсам посвящена гл. 14), то декларация не должна включать никаких модификаторов кроме, возможно, модификатора **extern**.

В качестве типа возвращаемого значения метода указывается либо конкретный тип (значения либо ссылки) или **void** — отсутствие какого либо значения.

В качестве имен методов используют идентификаторы.

Спецификация параметров может отсутствовать, но если она есть, то содержит спецификации всех параметров метода. Спецификации параметров мы уже рассмотрели. Отметим, что при отсутствии параметров их спецификации нет, но круглые скобки обязательны.

Тело метода — это блок — последовательность операторов, заключенная в фигурные скобки, либо *пустой оператор*, обозначаемый только отдельным символом точка с запятой. Пустой оператор в качестве тела используется только для методов с модификаторами **abstract** и **extern**.

Сразу же отметим, что в выражениях и операторах тела нестатического метода могут использоваться непосредственные обращения (без квалификации имен) к переменным полей и методам того же класса, которому принадлежит метод.

11.4. Пример класса и его объектов

Приведем пример объявления класса и особенностей работы с его объектами, рассмотрев следующую задачу.

В технике широко распространены разнообразные цифровые счетчики, построенные по «кольцевой» схеме. В каждом из них задано предельное значение, по достижению которого счетчик «сбрасывается» в начальное состояние. Так обычно устроены счетчики километража (пробега) в автомобилях, счетчики потребления воды, электроэнергии и т. п. Требуется определить класс Counter, объект которого моделирует работу кольцевого счетчика. В классе декларировать статическое поле `maxCount` для представления предельного значения, допустимого для показаний счетчиков — отдельных объектов класса. Текущее показание конкретного счетчика будет представлять целочисленное поле объекта (нестатическое поле класса) с именем `count`. В классе определить: метод `increment()` для увеличения на единицу текущего показания счетчика, метод `getCount()` для считывания текущего показания и метод `display()` для вывода информации о состоянии и возможностях счетчика.

Один из возможных вариантов определения такого класса (программа 11_04.cs):

```
class Counter // Класс "кольцевой счетчик"
{ // Закрытые поля:
    static int maxCount = 100000; // предельное значение
    int count; // текущее показание
    // Метод - приращение показания:
    public void increment()
    {
        count += 1;
        if (count >= maxCount) count = 0;
    }
}
```

```

// Метод для получения текущего показания:
public int getCount() { return count; }
// Метод для вывода сведений о счетчике (об объекте):
public void display()
{
    Console.WriteLine("Reading: {0, -8} maxCount: {1, -8}",
        count, maxCount);
}
}

```

В объявлении полей **static int maxCount** и **int count**; модификаторы доступа не использованы, и по умолчанию переменные **maxCount** и **count** закрыты для доступа извне (имеют статус доступа **private**). Статическое поле **maxCount** инициализируется значением 100 000 при компиляции. При создании каждого объекта класса инициализируется поле **count** конкретного объекта. Если инициализатор переменной поля отсутствует, то, как мы знаем, выполняется ее инициализация значением по умолчанию. В нашем примере переменная **count** при создании объекта инициализируется целочисленным значением 0.

В классе явно определены два нестатических открытых (имеющих статус доступа **public**) метода.

Метод с заголовком

```
public void increment()
```

при каждом обращении к нему увеличивает на единицу значение показания (поля **count**), представляемого объектом класса **Counter**. У метода пустой список параметров и метод ничего не возвращает в точку вызова. Обратите внимание, что в теле метода изменяется закрытый член класса (поле **count**). т. е. метод непосредственно обращается к переменной закрытого поля, и это не требует уточнения его имени.

Метод с заголовком

```
public void display()
```

осуществляет вывод сведений о текущем показании счетчика, представляемого объектом класса **Counter** и предельном значении **maxCount**, которое уже не может представлять объект класса. Метод открытый, ничего не возвращает в точку вызова и имеет пустой список параметров. В теле метода выполняются обращения к закрытым полям **count** и **maxCount**, значения которых выводятся на консоль.

Чтобы привести пример создания и использования объектов класса **Counter**, напомним следующее. Класс является типом ссылок, т. е. переменная с типом пользовательского класса представляет собой всего-навсего ссылку на объект класса. При создании переменной класса соответствующий ей объект может еще не существовать, или эта переменная еще не связана ни с каким объектом. Для создания объекта класса используется выражение с операцией **new**, где в качестве операнда — обращение к конструктору объектов класса.

Возникает вопрос: а как же быть, если в нашем классе Counter нет явного определения конструктора? Как уже упомянуто, в языке C# принято, что при отсутствии конструктора в декларации класса, в этот класс компилятор автоматически встраивает открытый конструктор умолчания, т. е. конструктор, при обращении к которому не требуется задавать аргументы. В следующей программе определяются ссылки c1 и c2 типа Counter и создаются связанные с ними объекты. К объектам посредством ссылок применяются нестатические методы класса Counter.

```
static void Main()
{
    Counter c1 = new Counter(); // конструктор умолчания
    Counter c2 = new Counter();
    c1.display();
    Console.WriteLine("c1.ToString(): " + c1.ToString());
    Console.WriteLine("c1.GetType(): " + c1.GetType());
    for (int i = 0; i < 150000; i++)
    {
        c1.increment();
        if (i % 10 == 0) c2.increment();
    }
    Console.WriteLine("c1.getCount() = " + c1.getCount());
    c2.display();
}
```

Для краткости здесь только текст метода Main(). Предполагается, что и определение класса Counter и метод Main() принадлежат одному пространству имен, например, метод Main() является статическим методом класса Counter.

Результаты выполнения программы:

```
Reading: 0          maxCount: 100000
c1.ToString( ): Counter
c1.GetType( ): Counter
c1.getCount( ) = 50000
Reading: 15000      maxCount: 100000
```

В теле метода Main() оператор Counter c1 = **new** Counter(); объявляет переменную c1 — ссылку типа Counter. Выражение с операцией **new** представляет собой обращение к конструктору умолчания, неявно включенному компилятором в класс Counter. Тем самым создан объект класса Counter и с ним связана ссылка c1. Аналогично определяется второй объект и связанная с ним ссылка c2.

Оператор c1.display(); — это обращение к нестатическому методу display(), который вызывается для обработки объекта, связанного со ссылкой c1. В результате выполнения метода на консоль выводится строка, представляющая значения поля объекта и статического поля maxCount:

```
Reading: 0          maxCount: 100000
```

Вид представления определяет форматная строка метода `Console.WriteLine()`, вызываемого в теле метода `display()`. Так как при создании объекта использован конструктор умолчания `Counter()`, то переменные (поля) объекта получили значения за счет их инициализации (`count = 0` и `maxCount = 100000`).

Следующие два оператора метода `Main()` иллюстрируют применимость к объектам введенного программистом-пользователем класса `Counter`, методов, унаследованных этим классом от класса **object**. Как мы уже говорили, класс **object** является первичным базовым классом всех классов программ на C#. Применяемые к объектам пользовательского класса `Counter` методы `ToString()` и `GetType()` позволяют получить имя этого класса.

В цикле с заголовком `for (int i = 0; i < 150000; i++)` выполняются обращения к методу `increment()` для двух объектов класса, адресуемых ссылками `c1` и `c2`. Для первого из них выполняется 150 000 обращений, для второго — в 10 раз меньше. Метод при каждом обращении увеличивает текущее значение счетчика на 1, но «следит» за переполнением. После 99 999 обращений счетчик, связанный со ссылкой `c1` будет обнулен. Приведенные результаты иллюстрируют изменения объектов.

Прежде чем завершить пояснения особенностей определенного нами класса и возможностей работы с его объектами, отметим, что в методе `Main()` используются только открытые методы класса `Counter`. В объявлениях методов `increment()`, `getCount()` и `display()` явно использован модификатор **public**. Конструктор умолчания `Counter()` создается компилятором автоматически как открытый. Именно названные четыре метода класса `Counter` формируют его внешний интерфейс. Обращения к закрытым полям класса невозможны. Если попытаться использовать вне класса, например, такой оператор:

```
Console.Write ("count = {0}", c1.count);
```

то компилятор сообщит об ошибке:

```
Error 1 'Counter.count' is inaccessible due to its protection level
```

11.5. Ссылка **this**

Нестатические методы класса отличаются от статических наличием в первых ссылки **this**. Эта ссылка позволяет нестатическому методу «узнать», для какого объекта метод вызван и выполняется в данный конкретный момент. Значением ссылки **this** является ссылка на тот объект, обработка которого выполняется. Ссылку **this** нельзя и нет необходимости определять — она всегда автоматически включена в каждый нестатический метод класса и связана с тем объектом, для которого метод вызывается. Иногда ссылку **this** называют дополнительным параметром нестатического метода. Этот параметр представляет в методе

вызвавший его объект. Можно назвать следующие четыре вида возможных применений ссылки **this**.

1. Во избежание конфликта имен между параметрами метода и членами объекта.
2. Для организации связей между объектами одного класса.
3. Для обращения из одного конструктора класса к другому конструктору того же класса.
4. Для определения расширяющего метода.

Продemonстрируем первые две из перечисленных возможностей применения ссылки **this** на следующем примере.

Определим класс `Link`, объекты которого можно объединять в цепочку — в односвязный список. Начало списка, т. е. ссылку на первый объект, включенный в список, представим статическим полем класса (**static** `Link beg`). Когда список пуст, значением `beg` будет **null**.

Программа с указанным классом:

```
// 11_05.cs - this и связный список
using System;
class Link
{
    static Link beg;
    Link next;
    int numb;
    public void add(int numb)
    {
        this.numb = numb;
        if (beg == null) // список пуст
        { beg = this; return; }
        Link temp = beg;
        while (temp.next != null)
            temp = temp.next;
        temp.next = this;
    }
    static public void display()
    {
        Link temp = beg;
        while (temp != null)
        {
            Console.Write(temp.numb + " ");
            temp = temp.next;
        }
        Console.WriteLine();
    }
}
class Program
{
    static void Main()
    {
        Link a = new Link(), b = new Link(), c = new Link();
        a.add(7);
        b.add(-4);
```

```

        c.add(0);
        Link.display();
    }
}

```

Результат выполнения программы:

7 -4 0

Нестатическими полями класса являются `int numb` и `Link next`. Первое поле нужно нам только для иллюстрации, значением второго поля будет ссылка на следующий объект, подключенный к списку вслед за рассматриваемым. Последовательность формирования списка из объектов класса `Link` можно показать на рисунке.

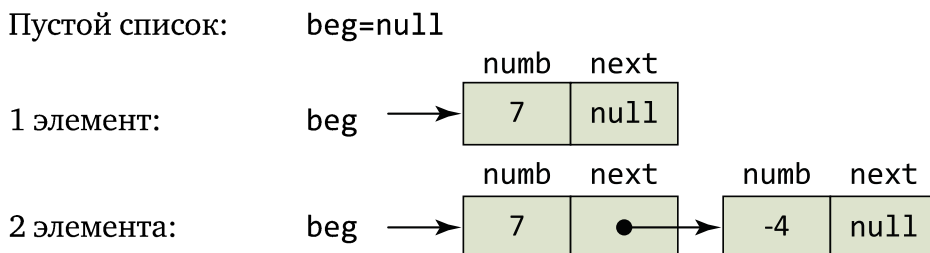


Рис. 11.1. Формирование списка объектов

Для создания объектов класса `Link` в программе использован конструктор без параметров, который автоматически встраивается в класс, когда в нем нет явно определенных конструкторов.

Для «подключения» объекта к списку введен нестатический открытый метод:

```

public void add(int numb)
{
    this.numb = numb;
    if (beg == null) // список пуст
    { beg = this; return; }
    Link temp = beg;
    while (temp.next != null)
        temp = temp.next;
    temp.next = this;
}

```

Особенность метода — имя его параметра совпадает с именем поля класса. Полное (квалифицированное) имя `this.numb` относится только к полю объекта. Первое действие метода — присваивание полю `this.numb` значения параметра с тем же именем. Применение ссылки `this` позволяет отличать поле объекта, для которого вызван метод `add()`, от одноименного параметра.

По умолчанию статическое поле `Link beg` и нестатическое поле `Link next` инициализируются значениями `null`. Пока список пуст, поле `beg` сохраняет это пустое значение. Полю `beg` присваивается ссылка на пер-

вый объект класса `Link`, для которого будет вызван метод `add()`. Если в списке уже есть элементы (объекты класса `Link`), то создается локальная переменная `Link temp`, инициализированная значением ссылки `beg`. Далее в цикле перебираются все элементы списка, пока не будет найден элемент, к которому никакой объект «не подключен». Его полю, доступ к которому обеспечивает квалифицированное имя `temp.next`, присваивается значение ссылки **this**, т. е. ссылка на очередной объект класса `Link`, для которого вызван метод `add()`.

Для последовательного перебора элементов списка и вывода значений полей `numb` определен статический метод:

```
static public void display()
{
    Link temp = beg;
    while (temp != null)
    {
        Console.Write(temp.numb + " ");
        temp = temp.next;
    }
    Console.WriteLine();
}
```

В теле метода `display()` цикл, в котором локальная переменная `temp` (ссылка на объекты класса `Link`) последовательно адресует объекты списка, начиная с того, который ассоциирован со статической переменной `beg`.

Для иллюстрации возможностей класса `Link` в классе `Program` определен метод `Main()`:

```
class Program
{
    static void Main()
    {
        Link a = new Link(), b = new Link(), c = new Link();
        a.add(7);
        b.add(-4);
        c.add(0);
        Link.display();
    }
}
```

В коде метода `Main()` определены три объекта класса `Link`, полям которых (`int numb`) с помощью обращений к методу `add()` присвоены числовые значения. Метод `add()` не только присваивает значение полю объекта (`int numb`), но и «прикрепляет» объект в конец цепочки, началом которой служит объект, адресуемый статической ссылкой **static** `Link beg`.

Начиная перебор звеньев цепочки со ссылки `beg`, статический метод `Link.display()` выводит значения всех целочисленных полей объектов, включенных в цепь (в односвязный список).

11.6. Конструкторы объектов класса

Недостаток рассмотренных выше классов Counter и Link — невозможность определять объекты, начальные значения полей которых задаются вне объявления класса. Конструктор умолчания умеет создавать только объекты с одними и теми же значениями полей, определенными их инициализацией. Нужные программисту начальные значения полей отдельно для каждого создаваемого объекта можно определять с помощью явно объявленного в классе конструктора.

Конструктор объектов (иначе экземпляров) класса — это метод класса, в котором реализованы все действия, необходимые для инициализации объекта класса при его создании. Для простоты выражение «конструктор объектов (иначе экземпляров) класса» обычно сокращают и говорят просто о конструкторе (обратите внимание, что этот термин не применяют к статическому конструктору).

Внешне любой конструктор — это метод класса, имя которого всегда совпадает с именем самого класса. Формат объявления конструктора:

```
модификаторы_конструктораopt  
имя_конструктора  
(спецификация_параметровopt)  
инициализатор_конструктораopt  
тело_конструктора
```

Имя конструктора — имя именно того класса, которому конструктор принадлежит, других имен у конструктора не может быть.

Обратите внимание, что у конструктора отсутствует тип возвращаемого значения, даже тип **void** в качестве типа возвращаемого значения для конструктора недопустим.

В качестве *модификаторов конструктора* используются **public**, **protected**, **internal**, **private**, **extern**

Первые четыре модификатора — это модификаторы доступа. Их назначение мы уже рассмотрели. О назначении модификатора **extern** мы уже упоминали в связи с методами класса.

Спецификация параметров конструктора может отсутствовать в круглых скобках после его имени. В этом случае мы получаем конструктор без параметров.

Тело конструктора это: либо блок — последовательность операторов, заключенная в фигурные скобки; либо *пустой оператор*, обозначаемый только отдельным символом точка с запятой. Пустой оператор в качестве тела используется только для конструкторов с модификатором **extern**. Мы будем в качестве тела конструктора использовать блок.

Назначение операторов тела конструктора — выполнить инициализацию создаваемого объекта класса.

Инициализатор конструктора — это специальное выражение, позволяющее до выполнения операторов тела конструктора обратиться к конструктору базового класса или к другому конструктору

этого же класса. Для обращения к конструктору базового класса используется выражение:

```
: base (список_аргументовopt)
```

Здесь **base** — служебное слово; необязательный список аргументов — список выражений, соответствующих параметрам вызываемого конструктора базового класса.

Инициализатор конструктора, выполняющий обращение к другому конструктору своего же класса, выглядит так:

```
: this (список_аргументовopt)
```

Здесь **this** — служебное слово; необязательный список аргументов — список выражений, соответствующих параметрам другого конструктора.

Хотя язык С# с помощью механизма сбора мусора (*garbage collection*) в достаточной мере защищен от таких проблем как «утечка памяти» и «висячие ссылки», однако в С# остается, например, задача глубокого копирования и важным вопросом в ряде случаев является приведение типов, введенных пользовательскими классами. Зачастую для квалифицированного решения названных проблем можно пользоваться конструкторами разных видов. В классе могут быть явно объявлены:

- конструктор умолчания (конструктор без параметров, либо с параметрами, имеющими умалчиваемые значения);
- конструктор копирования;
- конструктор приведения типов;
- конструктор общего вида.

При отсутствии в объявлении класса каких бы то ни было конструкторов, компилятор встраивает в класс открытый конструктор без параметров. При его использовании начальные значения полей создаваемого объекта определяются либо присутствующими инициализаторами полей, либо по умолчанию. Если автор класса желает по-особому инициализировать поля объектов, либо в классе наряду с другими конструкторами конструктор умолчания просто необходим — его требуется явно включать в объявления класса. Формат объявления конструктора без параметров:

```
class CLASS
{
    public CLASS()
    {
        операторы_тела_конструктораopt
    }
}
```

Роль конструктора умолчания может играть не только конструктор без параметров, но и конструктор, все параметры которого снабжены умалчиваемыми значениями аргументов (см. параграф 9.3).

Конструктор копирования — это конструктор с одним параметром, тип которого — ссылка на объект своего класса. Такой конструктор по умолчанию не создается, а зачастую нужно в программе иметь возможность создать точную копию уже существующего объекта. Присваивание ссылке с типом класса значения ссылки на существующий объект того же класса копии объекта не создает. Две ссылки начинают адресовать один и тот же объект. Примеры такой ситуации мы уже приводили, разбирая отличия ссылок от переменных с типом значений. В следующей программе объявлен класс с конструктором копирования.

```
// 11_06.cs - конструкторы умолчания и копирования
using System;
class CL
{
    public int dom = 6; // поле объекта
    public int[] ar;    // поле объекта
    public CL()
    { // конструктор умолчания
        ar = new int[] { 1, 2, 3 };
    }
    public CL(CL ob)
    { // конструктор копирования
        dom = ob.dom;
        ar = (int[])ob.ar.Clone();
    }
}
class Program
{
    static void Main()
    {
        CL one = new CL();
        CL two = new CL(one);
        two.dom = 5 * one.dom;
        Console.WriteLine("one.dom=" + one.dom + ", two.dom=" + two.dom);
        two.ar[0] *= 10;
        Console.WriteLine("one.ar[0]=" + one.ar[0]
            + ", two.ar[0]=" + two.ar[0]);
    }
}
```

Результат выполнения программы:

```
one.dom=6, two.dom=30
one.ar[0]=1, two.ar[0]=10
```

В классе CL конструктор без параметров объявлен явно. В классе CL два открытых поля — переменная `int dom` и ссылка на массив `int [] ar`. То есть, каждый объект класса включает в качестве данных целочисленную переменную и одномерный массив.

В методе `Main()` ссылка `one` связана с объектом, инициализированным конструктором без параметров. Объект, адресуемый ссылкой

two, — копия объекта one, но эти объекты независимы, что иллюстрируют результаты выполнения программы. Названная независимость объектов обеспечена конструктором копирования. В нем поля нового создаваемого объекта получают значения, копируемые из объекта-параметра. Для целочисленного поля `int dom` копирование выполняется с помощью присваивания. Копирование массива, ассоциированного с полем `int [] ar` (ссылка с типом массива), требует не присваивания, а глубокого копирования. Для этого используются возможности «библиотечного» нестатического метода `Clone()`:

```
ar = (int[])ob.ar.Clone();
```

Метод вызывается для того массива, который представлен ссылочным полем объекта-параметра (`ob.ar`) конструктора копирования. Так как метод `Clone()` возвращает значение типа **object**, то для присваивания результата переменной выполнено явное приведение типов.

В основном методе для иллюстрации независимости объектов данные одного из них изменяются за счет явных присваиваний. Вывод значений на консоль показывает, что поля второго объекта остаются неизменными.

Особенности обращений конструкторов одного класса друг к другу рассмотрим на примере класса, особым образом представляющего вещественные числа. В научной записи (в научной нотации) число записывается в виде произведения двух чисел: целой степени p числа 10 и числа m , удовлетворяющего условию $1.0 \leq m < 10.0$. Иногда p называют показателем, а m — мантиссой числа. Таким образом, запись каждого числа выглядит так: $m \cdot 10^p$ (при выводе значения числа возведение в степень будем обозначать символом $^$).

Примеры:

— постоянная Авогадро: $6.02486 \cdot 10^{23} \text{ (г*моль)}^{-1}$,

— заряд электрона: $-4.80286 \cdot 10^{-10} \text{ (СГСЭ)}$.

Определим класс `Real` для представления чисел в научной нотации. В классе определим метод `display()` для вывода на консоль значения объекта и метод `incrementM()` для увеличения на 1 значения мантиссы числа. При этом значение числа увеличивается, конечно, не на 1, а на 10^p . Для мантисс и показателей введем закрытые поля **double** `m` и **int** `p`. Определение класса может быть таким (программа `11_07.cs`):

```
// Класс чисел в научной нотации
class Real
{ // Закрытые поля:
    double m = 8.0; // мантисса — явная инициализация
    int p; // порядок — инициализация по умолчанию
    // Метод — приращение мантиссы:
    public void incrementM()
    {
        m += 1;
        if (m >= 10) { m /= 10; p++; }
    }
}
```

```

// Метод для вывода значения числа (объекта):
public void display(string name)
{
    string form = name + "\t = {0,8:F5}*10^{1, -3:D2}";
    Console.WriteLine(form, m, p);
}
// Конструктор общего вида:
public Real(double mi, int pi)
{
    m = mi;
    p = pi;
    reduce();
}
// Конструктор приведения типов:
public Real(double mi) : this(mi, 0)
{ }
// Конструктор копирования:
public Real(Real re) : this(re.m, re.p)
{ }
// Конструктор умолчания:
public Real() { }
// "Внутренний" для класса метод:
void reduce()
{ // Приведение числа к каноническому виду
    double sign = 1;
    if (m < 0) { sign = -1; m = -m; }
    for (; m >= 10; m /= 10, p += 1) ;
    for (; m < 1; m *= 10, p -= 1) ;
    m *= sign;
}

```

Среди методов класса нам сейчас важно рассмотреть явно определенные в классе конструкторы.

Конструктор общего вида:

```

public Real(double mi, int pi)
{
    m = mi;
    p = pi;
    reduce();
}

```

Параметры определяют значения мантиссы m и порядка p создаваемого объекта класса. В соответствии с правилами записи чисел в научной нотации для них необходимо соблюдение условия:

$$1.0 \leq m < 10.0.$$

Так как значение аргумента mi при обращении к конструктору может не удовлетворять этому условию, то в теле конструктора вызывается закрытый для внешних обращений метод **void** `reduce()`. Его задача — нужным образом преобразовать значения полей m и p .

Конструктор приведения типов:

```

public Real(double mi) : this(mi, 0) {}

```

Это частный случай конструктора общего вида с одним параметром. В нашем примере он формирует объект класса `Real` по одному значению типа **double**, использованному в качестве аргумента. Тем самым этот конструктор позволяет преобразовать числовое значение в объект класса `Real`. В конструкторе применен инициализатор, содержащий обращение `this(mi, 0)` к конструктору с заголовком `Real(double mi, int pi)`. Значение второго аргумента, определяющего значение поля `int p`, задано нулевой константой, что соответствует естественному для математики способу записи вещественного числа.

Конструктор копирования:

```
public Real(Real re) : this(re.m, re.p) {}
```

позволяет создать копию объекта. Еще раз обратим внимание на его отличие от операции присваивания, применение которой копирует только значение ссылки на объект. После присваивания ссылок на один объект начинают указывать несколько переменных (ссылок). Тело конструктора копирования в нашем примере не содержит операторов. Для присваивания значений полям создаваемого объекта используется инициализатор конструктора, содержащий обращение `this(re.m, re.p)` к конструктору общего вида. Вместо инициализатора можно было бы присваивать значения переменным `m` и `p` в теле конструктора (конструктор копирования по умолчанию не создается).

Конструктор умолчания, т. е. конструктор без параметров:

```
public Real() {}
```

При наличии явно определенных конструкторов (хотя бы одного) компилятор не встраивает в определение класса конструктор с пустым списком параметров. В тех случаях, когда конструктор умолчания нужен, он декларируется явным образом, что и сделано в классе `Real`.

Пример применения конструкторов класса (программа `11_07.cs`):

```
static void Main()
{
    Real number = new Real(303.0, 1); // констр. общего вида
    number.display("number");
    Real number1 = new Real(0.000321); // констр. приведения
    number1.display("number1");
    Real numCopy = new Real(number); // конструктор копирования
    number1 = number; // присваивание ссылок
    number.incrementM(); // изменение объекта
    number.display("number");
    number1.display("number1");
    numCopy.display("numCopy"); // копия сохранила значение
    Real numb = new Real(); // конструктор умолчания
    numb.display("numb");
}
```

Результат выполнения программы:

```
number = 3,03000*10^03  
number1 = 3,21000*10^-04  
number = 4,03000*10^03  
number1 = 4,03000*10^03  
numCopy = 3,03000*10^03  
numb = 8,00000*10^00
```

11.7. Деструкторы и инициализаторы объектов

Деструктор — это член класса, где запрограммированы все действия, которые необходимо выполнить для уничтожения объекта класса. Объявление деструктора:

```
externopt ~имя_класса()  
тело_деструктора
```

Как показано в формате, *имя_деструктора* — это имя класса с префиксом ~ (тильда). Других имен у деструкторов не бывает. Деструктор в классе может быть только один. Параметров у деструктора нет. Нет и возвращаемого значения. Тело деструктора — это блок либо пустой оператор, обозначаемый символом точка с запятой. Пустой оператор в качестве тела деструктора используется в том случае, если деструктор снабжен модификатором **extern**. В противном случае тело деструктора — это блок, включающий операторы, необходимые для уничтожения объекта. Практически тело деструктора аналогично телу метода без параметров с возвращаемым значением типа **void**.

Деструктор выполняется после того как соответствующий объект класса перестает использоваться в программе. Вызов деструктора выполняется автоматически. Момент вызова конкретно не определен. Явно вызвать деструктор из кода программы нельзя.

В следующей программе класс включает определение деструктора. Несмотря на то, что явно деструктор в программе не вызывается, его выполнение иллюстрирует результат следующей программы (11_08.cs).

```
// 11_08.cs деструктор  
using System;  
class A  
{  
    ~A()  
    {  
        Console.WriteLine("Destructor!");  
    }  
}  
class Test  
{  
    static void Main()  
    {  
        A b = new A();  
        b = null;  
    }  
}
```

Результат выполнения программы:

Destructor!

В архитектуре .NET деструкторы реализуются с помощью метода с названием `Finalize()`. Этот метод, называемый финализатором, подменяет в сборке реально использованный в коде деструктор. Программируя на уровне языка C#, можно не обращать на это внимание, но это важно понимать, если исследовать код на языке IL.

Деструктор нужен только в том классе, который требует для создаваемого объекта выделения неуправляемых ресурсов. Например, когда конструктор объекта связывает создаваемый объект с дескриптором файла или устанавливает сетевое соединение. Когда объект выходит из области определения, необходимы действия по освобождению дескриптора файла или сетевого соединения. Именно такие действия должен выполнять деструктор.

В версии C# 7.0 введена еще одна синтаксическая конструкция, относящихся к обработке объектов: *деконструктор*. Здесь о деконструкторе упомянем только чтобы предупредить, что его нельзя путать с деструктором. Деконструктор удобнее рассмотреть в следующей главе, «Средства взаимодействия с объектами». А сейчас остановимся на инициализаторах объектов.

Инициализаторы объектов позволяют присваивать значения открытым членам данных (полям и свойствам) непосредственно при создании объекта. Формат инициализатора объекта:

```
{имя_члена = выражение, ..., имя_члена = выражение}
```

Объявление объекта с применением инициализатора объекта выглядит так:

```
new Имя_класса (список_аргументов)opt инициализатор_объекта
```

Как отмечено в формате, список аргументов вместе с ограничивающими его скобками может быть опущен. Но это возможно только в случае, когда в классе присутствует конструктор умолчания (конструктор без параметров или с используемыми по умолчанию значениями аргументов).

Для применения инициализаторов объектов нет необходимости вносить какие-либо изменения или дополнения в декларацию класса.

Пример применения инициализаторов объектов:

```
// 11_09 - Инициализаторы объектов
using System;
class CalendarDate
{
    public string Dow = "Среда"; // День недели
    public string Mounth; // Название месяца
    public uint Date; // Дата (число в месяце)
}
```

```

class Program
{
    static void Main()
    {
        CalendarDate date = new CalendarDate()
        {
            Mounth = "Декабрь",
            Date = 12,
            Dow = "Четверг"
        };
        Console.WriteLine("date.Mounth = " + date.Mounth);
        Console.WriteLine("date.Date = " + date.Date);
        Console.WriteLine("date.Dow = " + date.Dow);
        CalendarDate nowDate = new CalendarDate
        {
            Date = 11,
            Mounth = date.Mounth
        };
        Console.WriteLine("nowDate.Mounth = " + nowDate.Mounth);
        Console.WriteLine("nowDate.Date = " + nowDate.Date);
        Console.WriteLine("nowDate.Dow = " + nowDate.Dow);
    }
}

```

Результаты выполнения программы:

```

date.Mounth = Декабрь
date.Date = 12
date.Dow = Четверг
nowDate.Mounth = Декабрь
nowDate.Date = 11
nowDate.Dow = Среда

```

В основной программе создаются и инициализируются два объекта класса *CalendarDate*. В первом из них значения всех полей определены инициализацией объекта. Для второго значение поля Dow ("Среда") выбрано из декларации этого поля в объявлении класса.

В приведенной программе компилятор автоматически добавил конструктор без параметров в класс *CalendarDate*. Других конструкторов в этом классе нет. При наличии в классе конструкторов с параметрами перед инициализатором объекта может размещаться список аргументов конструктора. С их помощью могут быть определены значения полей, отсутствующих в инициализаторе объекта.

При знакомстве с механизмом инициализаторов объектов может возникнуть вопрос об их необходимости. По сути, инициализатор объектов можно заменить обращением к конструктору с параметрами или явным присваиванием значений открытым членам данных объекта. В документации по C# отмечается, что наиболее полезны инициализаторы объектов при использовании LINQ-технологии.

Контрольные вопросы и задания

1. Назовите модификаторы класса, применяемые при отсутствии наследования.
2. Назовите разновидности членов класса.
3. Какие элементы являются обязательными в объявлении нестатического поля?
4. Когда выполняется инициализация нестатических полей?
5. Каков статус доступа нестатического поля при отсутствии в его объявлении модификаторов доступа?
6. Можно ли объявить статическое поле с типом класса, которому оно принадлежит?
7. В каком случае в классе могут одновременно присутствовать одноименные статический и нестатический методы?
8. В каких случаях телом нестатического метода может быть пустой оператор?
9. В каком случае конструктор умолчания (конструктор без параметров) создается автоматически?
10. Назовите возможные применения ссылки **this**.
11. В каких методах ссылка **this** отсутствует?
12. Опишите формат объявления нестатического конструктора.
13. Перечислите модификаторы конструктора.
14. Объясните назначение инициализатора конструктора.
15. Перечислите виды конструкторов.
16. Каков статус доступа у конструктора умолчания, встраиваемого в класс автоматически?
17. Что такое конструктор копирования?
18. Каким образом конструктор может обратиться к другому конструктору своего класса?
19. Объясните назначение деструктора.
20. Сколько деструкторов может быть в одном классе?
21. Что такое финализатор?
22. Какие члены класса можно инициализировать в инициализаторе объектов?

Глава 12

СРЕДСТВА ВЗАИМОДЕЙСТВИЯ С ОБЪЕКТАМИ

12.1. Принцип инкапсуляции и свойства классов

Объектно-ориентированное программирование базируется на трех принципах: **полиморфизм**, **инкапсуляция** и **наследование**. С проявлениями полиморфизма, а именно с перегрузкой методов и некоторых операций, мы уже познакомились.

Инкапсуляцию можно рассматривать как сокрытие особенностей реализации того или иного фрагмента программы от внешнего пользователя. Фрагмент должен быть доступен для обращений к нему только через внешний интерфейс фрагмента. Описание внешнего интерфейса должно быть достаточно для использования фрагмента. Если таким фрагментом является процедура (или функция), то нужно знать, как следует обратиться к процедуре, как передать ей необходимые входные данные и как получить результаты выполнения процедуры. Подробности внутренней реализации процедуры не должны интересовать того программиста, который использует процедуру. Именно принцип инкапсуляции лежит в основе запрета на использование в процедурах и функциях глобальных переменных. Ведь глобальная переменная определена вне процедуры и доступна внешним изменениям, не зависящим от обработки данных в ее теле.

Если фрагментом инкапсуляции является класс, то при его проектировании очень важно выделить в нем средства, обеспечивающие внешний интерфейс, и отделить их от механизмов реализации (внутреннего построения) класса. При этом нужно стремиться к достижению следующих трех целей:

- 1) возможности повторного использования объектов класса, например, в других программах (в этих других программах понадобится только знание внешнего интерфейса объектов класса);
- 2) возможности модифицировать внутреннюю реализацию класса без изменения тех программ, где применяются объекты этого класса;
- 3) обеспечению защиты объекта от нежелательных и непредсказуемых взаимодействий с ним других фрагментов программ, в которых он используется.

Для реализации в классах принципа инкапсуляции используется разграничение доступа к его членам. Основной принцип состоит в следующем. Доступ к данным (к полям) объектов должен быть возможен

только через средства внешнего интерфейса, предоставляемые классом. Если не рассматривать применений классов в цепочках наследования и в сборках (всему свое время, см. гл. 13), то реализацию класса определяют его закрытые члены (со спецификатором **private**), а внешний интерфейс — открытые (со спецификатором **public**). Обычная практика — закрывать все поля класса и открывать только те средства (например, методы), которых достаточно для работы с объектами класса.

Итак, поля классов рекомендуется определять как закрытые, а для обеспечения достаточно полного интерфейса вводить нужное количество открытых методов. Полнота внешнего интерфейса определяется требованиями тех программ, которые должны работать с классом и его объектами.

Объектно-ориентированный подход к программированию рекомендует для расширения внешнего интерфейса класса и его объектов вводить в класс специальные методы, позволяющие получать значения закрытых полей и позволяющие нужным способом задавать их значения. По-английски эти методы называют, соответственно, *get method* (*accessor*) — **метод доступа** и *set method* (*mutator*) — **метод изменения**. В зависимости от целей решаемых задач для каждого поля могут быть заданы или оба метода, или один из них.

Кроме традиционного для объектно-ориентированных языков применения специальных методов, обеспечивающих обращение к закрытым полям, язык C# позволяет получить к ним доступ с помощью механизма свойств. Однако свойство это не просто средство доступа к полям класса или его объекта. У свойств более широкие возможности. Дело в том, что в ряде случаев объекты класса могут иметь признаки, вторичные по отношению к значениям их полей. Например, для рассмотренного ранее класса чисел в научной нотации вторичными характеристиками каждого объекта можно сделать его значение в естественной форме вещественного числа или значение 10^p , где p — порядок числа в научной нотации, представленный полем объекта. Если в классе треугольников полями объектов сделать длины трех сторон треугольника, то такие характеристики как периметр или площадь можно объявить свойствами объектов класса.

Свойство — это член класса, который обеспечивает доступ к характеристикам класса или его объекта. С точки зрения внешнего пользователя свойства синтаксически не отличаются от полей класса. Но между свойствами и полями имеется принципиальное различие — в объекте *отсутствует явно ассоциированный со свойством участок памяти*. Если свойство представляет поле класса, то участок памяти выделяется для поля, а свойство, связанное с этим полем, представляет собой упрощенное по сравнению с методами средство для получения либо задания значения поля.

Для обращения к свойствам применяются их имена. Эти имена, как и имена полей, можно использовать в выражениях. Однако прежде

чем объяснять особенности применения свойств, рассмотрим правила их объявления в классе.

Декларация свойства состоит из двух частей. Первая часть подобна объявлению поля и подобна заголовку метода без параметров и окружающих их круглых скобок. Вторая часть представляет собой конструкцию особого вида, включающую заключенную в фигурные скобки пару особых «методов» с фиксированными именами: **get** и **set**. Эти специфические методы называют аксессорами. Общую форму объявления свойства можно представить так:

```
модификаторы_свойстваopt  
тип_свойства имя_свойства  
{  
    декларация get-аксессораopt  
    декларация set-аксессораopt  
}
```

В качестве модификаторов свойства используются: **new**, **public**, **protected**, **internal**, **private**, **static**, **virtual**, **sealed**, **override**, **abstract**, **extern**

В объявлении свойства могут присутствовать в допустимых сочетаниях несколько модификаторов, которые в этом случае отделяются друг от друга пробелами. Модификаторы, определяющие доступность членов вне объявления класса, мы уже рассмотрели в связи с полями и методами классов. Модификатор **static** позволяет отнести свойство к классу в целом, а не к его объектам. Остальные модификаторы до изучения наследования рассматривать не будем.

Тип_свойства — это тип той характеристики (того значения), которую представляет свойство.

Имя_свойства — идентификатор, выбираемый программистом для именования свойства.

Вторая часть объявления свойства (можно назвать ее телом объявления свойства) — это заключенная в фигурные скобки пара объявлений особых методов-аксессоров со специальными именами **get** и **set**.

Формат декларации аксессора доступа (get-аксессора):

```
модификаторы_аксессораopt  
get тело_аксессора
```

Формат декларации аксессора изменения (set -аксессора):

```
модификаторы_аксессораopt  
set тело_аксессора
```

Модификаторы аксессоров: **protected**, **internal**, **private**, **protected internal**, **internal protected**

Тело аксессора — это либо блок, либо пустой оператор, обозначаемый символом точка с запятой. Пустой оператор в качестве тела применяется для аксессоров тех свойств, которые объявлены с модификаторами **abstract** и **extern**. Сейчас такие свойства мы не рассматриваем.

Аксессор доступа (get-аксессор) подобен методу без параметров, возвращающему значение, тип которого определяется типом свойства. Достаточно часто аксессор доступа возвращает значение конкретного поля класса или его объекта. Для возврата значения из аксессора в его теле должен выполняться оператор

```
return выражение;
```

Аксессор изменения (set-аксессор) подобен методу с возвращаемым значением типа **void** и единственным неявно заданным параметром, значение которого определяет новое значение свойства. Тип параметра определяется типом свойства. Имя параметра, которое используется в теле аксессора изменений, всегда *value*.

В теле аксессоров свойства могут содержаться сложные алгоритмы обработки. Например, при изменении свойства можно контролировать диапазон допустимых значений. В теле аксессора доступа возвращаемое значение может вычисляться с учетом значений не одного, а разных полей, и т. д. Заметим, что свойство не вводит новых полей, а только управляет доступом к уже существующим в классе полям. Существует соглашение (не обязательное) начинать имена свойств с заглавных букв. Если свойство представляет «во внешнем мире» конкретное поле класса, то имя свойства повторяет имя поля, но отличается от него первой заглавной буквой. Например, если в классе объявлено поле `tempor`, то представляющее его свойство рекомендуется назвать `Tempor`.

Для примера еще раз обратимся к представлению чисел в научной нотации и изменим класс `Real`, введя в его декларацию объявление свойств (`12_01.cs`):

```
// Класс чисел в научной нотации
class Real
{
    // Закрытые поля:
    double m = 8.0; // мантисса — явно инициализирована
    int p; // инициализация по умолчанию
    // Свойство для получения значения мантиссы:
    public double Mantissa
    {
        get { return m; }
    }
    // Свойство для показателя:
    public int Exponent
    {
        get { return p; }
        set { p = value; }
    }
    // Свойство для значения числа:
    public double RealValue
    {
        get { return m * Math.Pow(10, p); }
        set { m = value; p = 0; reduce(); }
    }
}
```

```
// Приведение числа к каноническому виду:
void reduce()
{
    // "Внутренний" для класса метод
    double sign = 1; if (m < 0) { sign = -1; m = -m; }
    for (; m >= 10; m /= 10, p += 1) ;
    for (; m < 1; m *= 10, p -= 1) ;
    m *= sign;
}
}
```

В измененном классе Real уже рассмотренные закрытые члены: вспомогательный метод `reduce()`; поля: **double** `m` — мантисса, **int** `p` — показатель. Кроме того, объявлены три открытых свойства:

public double `Mantissa` — для получения значения мантиссы;

public int `Exponent` — для получения и изменения показателя;

public double `RealValue` — для получения числа в виде значения вещественного типа и для задания значений полей объекта по значению типа **double**.

В определении свойства `Mantissa` только один аксессор `get`, он позволяет получить значение поля **double** `m`.

Свойство `Exponent` включает два аксессора:

`set {p = value;}` — изменяет значение поля **int** `p`;

`get {return p;}` — возвращает значение того же поля.

Свойство `RealValue` позволяет обратиться к объекту класса Real как к числовому значению типа **double**. Аксессоры свойства:

```
get {return m * Math.Pow(10, p);}
set {m = value; p = 0; reduce();}
```

Аксессор `get` возвращает числовое значение, вычисленное на основе значений полей объекта.

Аксессор `set`, получив из внешнего обращения значение *value*, присваивает его переменной поля **double** `m`. При этом переменная **int** `p` получает нулевое значение. Затем для приведения числа к научной нотации в теле аксессора выполняется обращение к вспомогательному закрытому методу `reduce()`. Его мы уже рассмотрели в связи с обсуждением конструкторов.

Следующий фрагмент кода иллюстрирует применение свойств класса (программа 12_01.cs):

```
static void Main()
{
    Real number = new Real(); // конструктор умолчания
    string form = " = {0,8:F5} * 10^{1, -3:D2}";
    Console.WriteLine("number" + form,
        number.Mantissa, number.Exponent);
    number.Exponent = 2;
    Console.WriteLine("number" + form,
        number.Mantissa, number.Exponent);
    Console.WriteLine("number RealValue = "
        + number.RealValue);
}
```

```

number.RealValue = -314.159;
Console.WriteLine("number" + form,
    number.Mantissa, number.Exponent);
Console.WriteLine("number RealValue = "
    + number.RealValue);
}

```

В программе с помощью конструктора умолчания `Real()` определен один объект класса чисел в научной нотации и объявлена ссылка `number`, ассоциированная с этим объектом. Дальнейшие манипуляции с объектом выполнены с помощью свойств `Mantissa`, `Exponent`, `RealValue`. Для обращения к ним используются уточненные имена вида «ссылка_на_объект.имя_свойства».

Результат выполнения программы:

```

number = 8,00000 * 10^00
number = 8,00000 * 10^02
number RealValue = 800
number = -3,14159 * 10^02
number RealValue = -314,159

```

В первой строке результатов приведено изображение числа из объекта, созданного конструктором умолчания. Значения полей при выводе получены с помощью уточненных имен свойств `number.Mantissa`, `number.Exponent`.

Оператор `number.Exponent = 2`; через свойство `Exponent` изменяет значение поля показателя `int` `p`. Этим определяется вторая строка результатов выполнения программы.

В третьей строке — числовое значение объекта `number`, полученное с помощью свойства `RealValue`.

Оператор `number.RealValue = -314.159`; через свойство `RealValue` изменяет оба поля объекта `number`.

Результат изменения полей иллюстрирует предпоследняя строка результатов. В последней строке — значение свойства `RealValue`.

Аксессор `get` выполняется, когда из кода, внешнего по отношению к классу или его объекту, выполняется «чтение» значения свойства. При этом в точку вызова возвращается некоторое значение или ссылка на какой-то объект. Тип значения или ссылки соответствует типу в объявлении свойства. При этом возможны неявные приведения типов. Например, если `get`-аксессор возвращает значение типа `int`, а тип свойства `double`, то будет автоматически выполнено приведение типов. Аксессор `get` подобен методу без параметров, возвращающему значение или ссылку с типом свойства.

Если внешний по отношению к классу или его объекту код присваивает некоторое значение свойству, то вызывается `set`-аксессор этого свойства. В теле этого аксессора присвоенное свойству значение представлено специальной переменной с фиксированным именем `value`.

Тип этой переменной совпадает с типом свойства. У set-аксессуара возвращаемое значение отсутствует. Можно считать, что set-аксессуар функционально подобен методу с одним параметром. У этого параметра фиксированное имя *value* и тот же тип, что и тип свойства.

Можно использовать в объявлении свойства только один из аксессуаров. Это позволяет вводить свойства только для записи (изменения) и свойства только для чтения. Возникает вопрос, чем открытое свойство, обеспечивающее только чтение, отличается от открытого поля, объявленного с модификатором **readonly**. Основное отличие в том, что поле хранит некоторое значение, которое не может изменить процесс чтения из этого поля. При чтении значения свойства есть возможность выполнить заранее запланированные действия (вычисления), причем никаких ограничений на характер этих действий (вычислений) не накладывается. Результат вычислений свойства может зависеть, например, от состояния среды, в которой выполняется программа, или от влияния процессов, выполняемых параллельно. Пример поля с модификатором **readonly**: "дата рождения". Свойство «точный возраст» должно вычисляться с учетом и поля "дата рождения" и конкретного момента обращения к этому свойству.

Отметим, что свойства, доступные для записи, можно использовать в инициализаторах объектов наряду с открытыми полями.

12.2. Автореализуемые свойства и свойства, сжатые до выражений

Наиболее распространенное применение свойств — чтение и запись значений закрытых полей (поле, представляемое во внешнем мире свойством называют «поддерживающим» полем свойства). В этом случае имена полей не используются вне декларации класса и поля могут создаваться автоматически. Эту возможность обеспечивают свойства, специфицированные как автоматически реализуемые (*automatically implemented properties*). Покажем их декларацию на примере.

Пример кода, который готовит программист [1]:

```
public class Point
{
    public int X {get; set;}
    public int Y {get; set;}
}
```

В классе `Point` нет явно определенных полей, а два открытых свойства представляют собой объявления автореализуемых свойств. В автореализуемых свойствах отсутствуют тела аксессуаров, но каждый из аксессуаров может иметь модификаторы доступа. На основе такой декларации компилятор автоматически формирует эквивалентное ей объявление класса:

```
public class Point
{
    private int x;
    private int y;
    public int X {get {return x;} set{x = value;}}
    public int Y {get {return y;} set{y = value;}}
}
```

В автоматически построенном объявлении появились два закрытых поля, типы которых совпадают с типами свойств. Поле, отнесенное к автореализуемому свойству, называют *hidden backing field* — «скрытым полем заднего плана» («поддерживающим» полем). Именно с этими полями ассоциированы свойства X и Y, которые были декларированы программистом. Имена скрытых полей компилятор формирует по правилам, которые известны только ему. В нашем примере x и y — это только условные обозначения, выбранные для иллюстрации. Никакие прямые обращения к этим скрытым полям невозможны ни в классе, ни за его пределами. Доступ к этим полям обеспечивают только ассоциированные с ними автореализуемые свойства.

В приведенном примере скрытые поля класса Point доступны с помощью свойств X и Y как для чтения, так и для изменений. При объявлении автореализуемого свойства можно ограничить доступ к связанному с ним полю, например, разрешив извне класса только чтение. Сразу возникает вопрос: как задать значение такого поля, если имя его недоступно и неизвестно (известно только компилятору), а автореализуемое свойство разрешает только чтение? Одно из решений — явно объявить в классе с автореализуемыми полями конструктор и в нем через закрытое (для внешнего мира) свойство устанавливать значение скрытого поля (с неизвестным именем).

Второй вариант присваивания значения открытому только для чтения автореализуемому свойству — применение инициализатора свойства. Начиная с версии C# 6.0 в декларацию автореализуемого свойства разрешено добавлять инициализатор, похожий на инициализатор поля:

= *выражение*;

К инициализирующему выражению требования те же, что и к инициализатору поля.

Следующая программа иллюстрирует предлагаемые решения:

```
// 12_02.cs - автореализуемые свойства
using System;
public class Point
{
    public int X { get; private set; }
    public int Y
    { get; private set; } = 22 / 4;
    public Point(int xi, int yi)
    { X = xi; Y = yi; }
```

```

    public Point() { }
}
class Program
{
    static void Main()
    {
        Point a = new Point(12, 7);
        Console.WriteLine("a.Y = " + a.Y);
        Point g = new Point();
        Console.WriteLine("g.Y = " + g.Y);
    }
}

```

Результат выполнения программы:

```

a.Y = 7
g.Y = 5

```

Начиная с версии C# 7.0 в объявление свойств введена возможность сокращать запись аксессоров, включающих единственный оператор. Для этого применяется синтаксис, схожий с записью методов, сжатых до выражения. Речь идет о свойствах, сжатых до выражений. В их объявлениях после служебного слова `get` или `set` помещают конструкцию `=>`, затем выражение, определяющее получаемое или возвращаемое значение свойства. Фигурные скобки и `return` опускаются. В аксессоре `set` записывается выражение с операцией присваивания, либо создания нового объекта, либо обращения к методу, либо выражения с операцией инкремента или декремента.

Если для свойства нужно обеспечить и чтение, и запись, то декларация при использовании синтаксиса сжатия до выражений должна иметь вид:

```

модификаторыopt тип имя_свойства
{
    get => выражение;
    set => выражение;
}

```

Если определяется свойство только для чтения (в нем нет аксессора `set`), то формат объявления свойства, сжатого до выражения, будет еще проще:

```

модификаторыopt тип имя_свойства => выражение;

```

Например, в класс `Point` можно так добавить свойство для вычисления квадрата расстояния от начала координат до точки:

```

public double Norma => X * X + Y * Y;

```

Еще одно свойство, обеспечивающее и чтение, и запись, с использованием синтаксиса сжатия до выражений в тот же класс `Point` можно добавить так:


```
public double Z
{
    get => Z;
    set => Z = value / Norma;
}
```

В этом свойстве определены оба аксессора.

12.3. Индексаторы

Если в класс в качестве поля входит коллекция, например, массив элементов, то в ряде случаев удобно обращаться к элементам коллекции, используя индекс (или индексы, если массив многомерный). Для одномерной коллекции, принадлежащей объекту класса, обращение к ее элементу с помощью индекса будет таким:

```
ссылка_на_объект [индексное_выражение]
```

Возможность обращаться к элементу коллекции, принадлежащей объекту класса, с помощью индексирования обеспечивает специальный член этого класса, называемый *индексатором*.

Именно наличие в библиотечном классе **string** индексатора позволяет обращаться к отдельным символам строки, используя операцию индексации:

```
char буква = "Сообщение"[4];
```

В результате выполнения этого оператора символьная переменная «буква» будет иметь значение 'щ'.

Индексатор можно считать разновидностью свойства. Как и для свойства, возможности индексатора определяются аксессорами *get* и *set*. В отличие от свойства, у индексатора нет собственного уникального имени. При объявлении индексатор всегда именуется служебным словом **this**, т. е. ссылкой на тот конкретный объект, для которого используется индексатор. Тип возвращаемого значения для индексатора соответствует типу элементов коллекции, с которой ассоциирован индексатор.

Объявление индексатора имеет следующий формат:

```
модификаторы_индексатораopt
тип this [спецификация параметров]
{
    декларация get-аксессораopt
    декларация set-аксессораopt
}
```

Модификаторы_индексатора те же, что и для свойств за одним исключением — для индексаторов нельзя использовать модификатор **static**. Индексаторы не могут быть статическими и применимы только к объектам класса (не к классу в целом). Вслед за ключевым словом

this в квадратных скобках — спецификация параметров индексатора. Разрешена перегрузка индексаторов, т. е. в одном классе может быть несколько индексаторов, но они должны отличаться друг от друга спецификациями параметров. В этом индексаторы подобны методам. За квадратными скобками размещается код, который можно назвать телом индексатора. Это конструкция, подобная телу свойства, — заключенные в фигурные скобки объявления аксессоров `get` и `set`.

Операторы `get`-аксессора предназначены для получения значения элемента, соответствующего значению индекса. Операторы `set`-аксессора выполняют присваивание значения элементу, соответствующему значению индекса. Вызов индексатора, т. е. выражение

ссылка_на_объект [список аргументов]

может размещаться слева и справа от операции присваивания. В первом случае выполняется аксессор `set`, во втором — аксессор `get`. В теле аксессора `set` значение, которое нужно использовать для изменения элемента коллекции, всегда представлено контекстным служебным словом `value`.

В качестве примера класса с индексатором определим класс `Work_hours` для представления отработанных часов по дням недели. В массиве должно быть 7 элементов, с индексами от 0 (для понедельника) до 6 (для воскресенья). Значения элементов массива — количества отработанных часов по дням недели от 0 (не рабочий день) до 14 часов (больше работать запрещено). Для обращения к элементам массива в класс введен индексатор.

В данном примере нужно в теле индексатора запретить обращения к массиву с индексом, выходящим за пределы его граничной пары (от 0 до 6). Кроме того, предусмотрим в индексаторе защиту от неверных значений, присваиваемых элементам массива. В соответствии со смыслом определяемого класса значения элементов массива (количество отработанных часов) не должны быть отрицательными и не должны превышать некоторого предельного значения. Для конкретности в условии предположено, что работа одного дня не должна превышать 14 часов.

```
// 12_03.cs - индексатор - рабочие часы дней недели
using System;
class Work_hours
{
    int[] days; // часы по дням недели
    public Work_hours() // конструктор
    { days = new int[7]; }
    public int this[int d]
    { // индексатор
        get { return (d < 0 || d > 6) ? -1 : days[d]; }
        set
        {
            if (d < 0 || d > 6 || value < 0 || value > 14)
                Console.WriteLine("Ошибка: день={0}, часы={1}!", d, value);
        }
    }
}
```

```

        else days[d] = value;
    }
}
}
class Program
{
    static void Main()
    {
        Work_hours week = new Work_hours();
        week[0] = 7; // понедельник
        week[2] = 17; // недопустимые данные
        week[3] = 7; // четверг
        week[6] = 7; // воскресенье
        Console.WriteLine("Рабочие дни: ");
        for (int i = 0; i < 7; i++)
            if (week[i] > 0)
                Console.Write("day[{0}] = {1} ", i, week[i]);
        Console.WriteLine();
        Console.WriteLine("day[{0}] = {1}\t", 8, week[8]);
    }
}

```

Результат выполнения программы:

```

Ошибка: день=2, часы=17!
Рабочие дни:
day[0] = 7 day[3] = 7 day[6] = 7
day[8] = -1

```

Параметр индексатора и соответствующее ему индексное выражение индексатора не обязательно должны иметь целочисленный тип. В следующем примере рассмотрим класс Dictionary, объект которого может служить простейшим словарем. В класс Dictionary включим в качестве полей два массива строк — массив исходных слов, например, на английском языке, и массив переводных эквивалентов. Массив исходных слов будем заполнять в конструкторе при создании словаря-объекта. Элементам массива переводных эквивалентов будем присваивать значения с использованием индексатора. Параметр индексатора будет иметь тип **string**. Задавая в квадратных скобках английское слово в виде строки, получим доступ к элементу с его переводным эквивалентом. Определение класса может быть таким (программа 12_04.cs):

```

class Dictionary
{ // словарь
    string[] words; // слова
    string[] trans; // переводы
    public Dictionary(params string[] str)
    { // конструктор
        words = new string[str.Length];
        trans = new string[str.Length];
        int ind = 0;
        foreach (string s in str)
            words[ind++] = s; // заполнили массив слов
    }
}

```

```

    }
    int search(string str)
    { // поиск слова
        for (int i = 0; i < words.Length; i++)
            if (words[i] == str) return i;
        return -1;
    }
    public string this[string w]
    {
        set
        {
            int ind = search(w);
            if (ind == -1)
                Console.WriteLine("Слова Нет!");
            else trans[ind] = value;
        }
        get
        {
            int ind = search(w);
            if (ind == -1) return "Слова Нет!";
            else return trans[ind];
        }
    }
}

```

В классе Dictionary две ссылки words и trans на массив слов и на массив их переводных эквивалентов. Собственно, массивы как объекты создаются в конструкторе. У конструктора есть параметр с модификатором **params**, позволяющий использовать переменное число аргументов. Реальное количество аргументов определяется как str.Length. Это значение определяет размеры массивов, адресуемых ссылками words и trans. Строки-аргументы конструктора присваиваются элементам массива words[] в цикле **foreach**. Массив переводов trans[] остается незаполненным.

В классе определен вспомогательный закрытый метод search(), для поиска в словаре (в массиве words[]) слова, заданного с помощью аргумента. Метод вернет индекс слова, либо -1, если слово отсутствует. Метод search() используется в индексаторе. В аксессоре set определяется индекс ind того элемента массива trans[], которому нужно присвоить новое значение переводного эквивалента. Если поиск неудачен — выводится сообщение, иначе элементу trans[ind], присваивается значение переводного эквивалента. Аксессор get возвращает значение trans[ind] либо строку с сообщением, что слова нет в словаре.

Для иллюстрации возможностей класса dictionary и его индексатора приведем следующий фрагмент кода:

```

static void Main()
{
    Dictionary number = new Dictionary("zero", "one", "two");
    number["zero"] = "нуль";
    number["one"] = "один";
}

```

```

number["two"] = "2";
Console.WriteLine("number[\"one\"]: " + number["one"]);
Console.WriteLine("number[\"three\"]: "
    + number["three"]);
Console.WriteLine("number[\"two\"]: " + number["two"]);
}

```

В методе Main() создан один объект класса Dictionary. В объекте-словаре всего три слова, у которых вначале нет переводных эквивалентов. Для задания переводов используются выражения с индексами. При обращении к отсутствующему слову значением выражения number["three"] будет строка "Слова Нет!".

Результат выполнения программы:

```

number["one"]: один
number["three"]: Слова Нет!
number["two"]: 2

```

Для индексаторов, допускающих *только чтение* (не включающих аксессуара set), объявление можно сократить, используя сжатие до выражения:

```

модификаторыopt тип_результата
this [тип_индекса индекс] => выражение_с_индексом;

```

Пример индексатора, декларированного с использованием сжатия до выражения, приведем в программе следующего параграфа.

Для программиста-пользователя обращение с помощью индексатора к объекту чужого класса выглядит как обращение к элементу массива. Но массива как такового в объекте может и не быть. Дело в том, что индексатор можно определить в классе, где контейнер (например, массив) отсутствует. В этом случае индексатор просто-напросто заменяет метод. Отличие состоит в синтаксисе обращения.

В качестве примера рассмотрим класс, представляющий две температурные шкалы. Температура T° по абсолютной шкале, введенной Вильямом Томсоном (лордом Кельвином), связана с температурой t° по шкале Цельсия соотношением: $T^{\circ} = t^{\circ} + 273.16^{\circ}$.

Определим класс Temperature с индексатором, позволяющий получать значение T° по величине t° , которую будет задавать параметр индексатора. Так как температура по Кельвину не может быть отрицательной, то примем, что при $t^{\circ} < -273.16^{\circ}$ индексатор будет возвращать значение -1. Декларация класса:

```

class Temperature
{ // Температура по Кельвину и Цельсию
    public double this[double t] =>
        (t < -273.16)? -1 : t + 273.16;
}

```

В классе нет массива, и нет полей, которым нужно присваивать значения с помощью обращений к индексатору, поэтому нет смысла

в индексаторе определять аксессор `set`. Это обстоятельство и возможность выразить действия индексатора с помощью единственного оператора **`return`** позволяют использовать синтаксис сжатия до выражения. В отличие от предыдущих примеров параметр индексатора и возвращаемое им значение имеют тип **`double`**. Конструктор задается неявно. Применение индексатора иллюстрирует следующий фрагмент кода:

```
static void Main()
{
    Temperature TK = new Temperature();
    double t = 43;
    Console.WriteLine("TK[{0}] = {1:f2}", t, TK[t]);
    t = -400;
    Console.WriteLine("TK[{0}] = {1:f2}", t, TK[t]);
    t = -273;
    Console.WriteLine("TK[{0}] = {1:f2}", t, TK[t]);
}
```

Результат выполнения:

```
TK[43] = 316,16
TK[-400] = -1,00
TK[-273] = 0,16
```

12.4. Расширяющие методы и деконструкторы

Расширяющий метод позволяет дополнять существующий класс (или другой тип: структуру, интерфейс) методами объектов, не изменяя декларации класса. Это особенно удобно в тех случаях, когда у программиста нет доступа к коду декларации класса. Несмотря на то, что мы еще не рассмотрели полностью синтаксис объявления классов, расширяющие методы мы уже можем определять, так как каждый из них является просто по-особому декларированный статический метод статического класса. Особенности декларации две. Первая состоит в том, что первый параметр расширяющего метода должен иметь тип того класса, для которого выполняется расширение. Вторая особенность — первый параметр должен быть снабжен модификатором **`this`**.

Для иллюстрации возможностей расширяющих методов предположим, что нужен метод, формирующий случайные целые числа с четными значениями, не превышающие заданной величины. Средства для программной генерации случайных чисел определены в библиотечном классе `System.Random`. Для получения целых чисел из диапазона `[0; Max)` используется нестатический метод с заголовком:

```
int Next(int Max).
```

Метода для получения только четных случайных чисел в классе `Random` нет. Для дополнения объектов класса `Random` соответствующим методом определим такой класс:

```

public static class RandomExpansion
{
    public static int EvenNumber(this Random rd, int max)
    {
        if (max < 2) return 0;
        while (true)
        {
            int res = rd.Next(max);
            if (res % 2 == 0) return res;
        }
    }
} // class RandomExpansion

```

Первый параметр метода передает в его тело ссылку `rd` на объект класса `Random`. С помощью этой ссылки выполняется обращение к методу `Next` объекта класса `Random` и формируется случайное целое число. Если оно четное, то его значение — результат метода `EvenNumber()`.

К расширяющему методу `EvenNumber()` можно обращаться так, как будто это метод объектов класса `Random`:

```

class Program
{
    static void Main()
    {
        Random rnd = new Random();
        for (int k = 0; k < 10; k++)
            Console.Write(rnd.EvenNumber(12) + " ");
    }
}

```

Результат выполнения кода:

2 10 8 10 4 6 4 6 2 8

В методе `Main()` создан объект класса `Random`, ссылка на который позволяет вызвать метод `EvenNumber()` с единственным аргументом.

Так как метод `EvenNumber()` декларируется как статический метод класса `RandomExpansion`, то его можно вызывать и как обычный статический метод. Например, так:

```
int stat = RandomExpansion.EvenNumber(new Random(), 20);
```

В таком обращении к расширяющему методу требуется явно указать первый аргумент.

Еще одна возможность получить доступ к членам данных (к полям и свойствам) экземпляра класса — применение деконструктора.

Деконструктор — это метод класса со специальным названием `Deconstructor()`, имеющий *не менее двух параметров* с модификатором **out**. Назначение деконструктора — «извлечение» данных из объектов. Таким образом, деконструктор выполняет работу, в некотором смысле

противоположную той, которую обычно выполняет конструктор экземпляров класса.

С помощью параметра деструктора можно вернуть не только значение поля или свойства, но и достаточно произвольное значение, вычисляемое на основе данных объекта. По существу, деструктор — это обычный нестатический метод экземпляров типа, но для его вызова введен специальный удобный синтаксис. Именно этот синтаксис требует, чтобы у деструктора было не менее двух параметров с модификаторами **out**. Если обращаться к деструктору с помощью уточненного имени, квалифицированного ссылкой на объект, то можно определить в классе и применять деструктор с одним выходным параметром.

Для иллюстрации особенностей декларации деструкторов приведем объявление класса правильных многоугольников. Известно, что периметр P правильного n -угольника, описанного около окружности радиуса r , равен $2 \cdot n \cdot r \cdot \tan(\pi/n)$, площадь этого многоугольника S равна $n \cdot r \cdot r \cdot \tan(\pi/n)$. Используя эти сведения, так определим класс правильных многоугольников:

```
public class Polygon
{ // Класс многоугольников
  int Numb { get; set; } // Число сторон
  double Radius { get; set; } // Радиус вписанной окружности
  public Polygon(int n = 3, double r = 1)
  { // конструктор
    Numb = n;
    Radius = r;
  }
  public double Perimeter
  { // Периметр многоугольника
    get
    { // аксессор свойства
      double term = Math.Tan(Math.PI / Numb);
      return 2 * Numb * Radius * term;
    }
  }
  // Площадь многоугольника:
  public double Area => Perimeter * Radius / 2;
  // Деструктор с двумя параметрами:
  public void Deconstruct(out int n, out double r)
  { r = Radius; n = Numb; }
  // Деструктор с тремя параметрами:
  public void Deconstruct(out int n, out double r,
    out double ambit)
  { r = Radius; n = Numb; ambit = Perimeter; }
}
```

В классе *Polygon* два автореализуемых свойства (*Numb* и *Radius*) задают, соответственно, число сторон и радиус вписанной в многоугольник окружности. Свойства, определяющие периметр и площадь конкретного многоугольника (*Perimeter* и *Area*), доступны только для

чтения, причем в декларации свойства *Area* используется синтаксис сжатия до выражения.

В классе *Polygon* конструктор общего вида с умалчиваемыми значениями параметров и два деконструктора. Первый из них — с двумя параметрами:

```
public void Deconstruct(out int n, out double r)
{r = Radius; n = Numb;}
```

Параметры этого деконструктора возвращают значения автореализуемых свойств объекта класса.

Список параметров второго деконструктора дополнен третьим параметром (**out double ambit**), позволяющим получить значение свойства, вычисляющего периметр многоугольника, представленного объектом класса.

Синтаксис языка C# допускает несколько вариантов организации вызова деконструктора. Так как деконструктор определяется как открытый нестатический метод класса, то и обращаться к нему можно непосредственно, используя ссылку на объект класса:

```
Polygon poly = new Polygon();
poly.Deconstruct(out int n, out double r);
Console.WriteLine("Numb = " + n + "; Radius = " + r);
```

При создании объекта, адресуемого ссылкой *Polygon poly*, используются умалчиваемые значения параметров конструктора. В результате выполнения деконструктора определяемые налету переменные (**int n** и **double r**) получают значения автореализуемых свойств. Результат выполнения такого фрагмента программы:

```
Numb = 3; Radius = 1
```

Для упрощения вызова деконструктора в C# 7.0 введен специальный синтаксис:

```
(список_аргументов) = ссылка_на_объект;
```

В списке аргументов перечисляются имена переменных, типы и последовательность которых должны полностью соответствовать спецификации выходных (с модификаторами **out**) параметров деконструктора. Как и при непосредственном обращении к деконструктору, переменные, играющие роль аргументов, могут быть объявлены «на лету», т. е. в списке аргументов. Однако (в отличие от списка аргументов прямого вызова деконструктора) аргументы (даже при их объявлении «на лету») не снабжаются модификаторами **out**.

Пример обращения к деконструктору с тремя параметрами, когда типы аргументов объявлены «на лету»:

```
(int nv, double rv, double pv) = poly;
Console.WriteLine("Numb = " + nv +
    "; Radius = " + rv + "; Perimeter = " + pv);
```

Результат выполнения такого фрагмента программы:

```
Numb = 3; Radius = 1; Perimeter = 10,3923048454133
```

Тот же результат будет получен, если заменить один оператор обращения к деконструктору таким кодом:

```
int nv;  
double rv, pv;  
(nv, rv, pv) = poly;
```

Так как спецификация параметров деконструктора однозначно определяет требования к аргументам, то возможно применение неявной типизации. Пример обращения к деконструктору с двумя параметрами, когда типы аргументов выводит компилятор:

```
(var num, var rad) = poly;  
Console.WriteLine("Numb = " + num + "; Radius = " + rad);
```

Результат выполнения приведенного фрагмента программы:

```
Numb = 3; Radius = 1
```

Допустимо и такое сокращение записи вызова деконструктора:

```
var(num, rad) = poly;
```

Если при обращении к деконструктору не нужны некоторые из значений, то вместо соответствующих им аргументов используют подстановку: «_». Например, чтобы получить только значение периметра, используют такой код:

```
var (_, _, perimeter) = poly;  
Console.WriteLine("Perimeter = " + perimeter);
```

Результат выполнения приведенного фрагмента программы:

```
Perimeter = 10,3923048454133
```

До сих пор мы рассматривали возможности деконструкторов на примере класса, определенного программистом, когда деконструктор можно декларировать непосредственно в объявлении класса.

Расширяющие методы позволяют создавать деконструкторы и для структур и тех классов, код которых программисту недоступен. В качестве примера определим в виде расширяющих методов деконструкторы для библиотечной структуры `Complex`, предназначенной для работы с комплексными числами. Особенности структур будут рассмотрены в следующих главах, но расширяющие методы для структур и обращения к свойствам структур внешне не отличаются от расширяющих методов для классов, поэтому пример будет понятен.

Примечание. Библиотечная структура `Complex` принадлежит пространству имен `System.Numerics` и находится в сборке `System.Numerics.dll`.

Для подключения сборки к проекту необходимо в обозревателе решений «развернуть» список проекта, правой кнопкой мыши активировать пункт «ссылки». Из развернувшегося меню выбрать пункт «Добавить ссылку...», на панели «Менеджер ссылок» активировать пункт «System.Numerics» и нажать клавишу «ОК».

Чтобы сборка стала доступна в коде, потребуется директива:

```
using System.Numerics; // Пространство имен для Complex
```

В структуре `Complex` четыре свойства:

`Imaginary` — мнимая часть комплексного числа;

`Magnitude` — модуль (абсолютное значение) комплексного числа;

`Phase` — аргумент (фаза) комплексного числа;

`Real` — вещественная часть комплексного числа

Чтобы получать из экземпляра структуры типа `Complex` значения перечисленных свойств, определим статический класс `ComplexHelper` с методом-деконструктором, расширяющим возможности типа `Complex`. Текст программы с этим классом:

```
using System;
using System.Numerics; // Для типа "Complex"...
public static class ComplexHelper
{
    // Деконструктор (вернет все свойства экземпляра):
    public static void Deconstruct(this Complex cm,
        out double magnitude, out double phase,
        out double real, out double image)
    {
        magnitude = cm.Magnitude;
        phase = cm.Phase;
        real = cm.Real;
        image = cm.Imaginary;
    }
} // class ComplexHelper
class Program
{
    static void Main()
    {
        Complex cmp = new Complex(4, 6);
        (double module, double argument, _, _) = cmp;
        Console.WriteLine("module = " + module);
        Console.WriteLine("argument = " + argument);
    }
}
```

Результаты выполнения программы:

```
module = 7,21110255092798
argument = 0,982793723247329
```

У деконструктора пять параметров. Первый с модификатором **this** соотносит метод с экземпляром структуры `Complex`. Остальные четыре

параметра возвращают соответствующие названиям значения свойств комплексного числа. В методе `Main()` создан экземпляр структуры `Complex` и представляющая этот экземпляр ссылка (`Complex ptr`). При обращении к конструктору в качестве аргументов указаны два аргумента — значения вещественной и мнимой частей комплексного числа. Они известны из кода вызова конструктора, поэтому при обращении к деконструктору явно декларированы на лету только переменные для аргументов, получающих значения модуля и аргумента комплексного числа. Результаты выполнения программы иллюстрируют получаемые значения.

Контрольные вопросы и задания

1. Объясните принципы инкапсуляции и ее применения к классам.
2. В чем отличия свойств от полей?
3. Приведите формат объявления свойства.
4. Какие модификаторы используются в объявлении свойства?
5. Что такое тип свойства?
6. Что такое тело аксессуара в объявлении свойства?
7. Какие модификаторы допустимы в объявлении аксессуара?
8. Каким идентификатором представлено в `set`-аксесоре новое значение свойства?
9. Как можно задать значение свойства, пригодного только для чтения?
10. Объясните назначение механизма автореализуемых свойств.
11. В каком случае для декларации свойства можно применять синтаксис сжатия до выражения?
12. Что такое скрытые поля?
13. Объясните роль служебного слова **this** в индексаторе.
14. Может ли в одном классе быть несколько индексаторов?
15. Какой тип допустим для параметра индексатора?
16. В каком случае для индексатора разрешен синтаксис сжатия до выражения?
17. Назовите особенности декларации расширяющего метода.
18. Какими способами можно обращаться к расширяющим методам?
19. Что такое деконструктор?
20. Какие ограничения наложены на параметры деконструктора?
21. В чем особенность вызова деконструктора?
22. Как при обращении к деконструктору игнорировать получение значения аргумента?

Глава 13

ВКЛЮЧЕНИЕ, ВЛОЖЕНИЕ И НАСЛЕДОВАНИЕ КЛАССОВ

13.1. Включение объектов классов

В соответствии с основной задачей, решаемой при проектировании программы, входящие в нее классы могут находиться в разных отношениях. Наиболее простое — отношение независимости классов, т. е. независимости порождаемых ими объектов. Более сложное — отношение включения, для которого используют названия «имеет» (*has-a*) и «включает», иначе «является частью» (*is-part-of*).

В теории объектно-ориентированного анализа различают две формы отношения включения — композицию и агрегацию.

При отношении **композиции** объекты одного класса или нескольких разных классов входят как поля в объект другого (включающего) класса. Таким образом, включенные объекты не существуют без включающего их объекта.

При отношении **агрегации** объект одного класса объединяет уже существующие объекты других классов, т. е. и включающий объект, и включаемые в него объекты существуют в некотором смысле самостоятельно. При уничтожении включающего объекта входившие в него объекты сохраняются.

Рассмотрим на примерах особенности реализации на языке C# отношений композиции и агрегации.

Определим (программа 13_01.cs) класс «точка на плоскости»:

```
class Point
{
    double x, y;
    public double X {get {return x;} set {x = value;}}
    public double Y {get {return y;} set {y = value;}}
}
```

В классе закрытые поля **double** *x*, *y* определяют координаты точки. Свойства *X* и *Y* обеспечивают удобный доступ к координатам точки, представленной объектом класса *Point*. В классе *Point* нет явно определенного конструктора, и компилятор добавляет конструктор умолчания — открытый конструктор без параметров. Координаты создаваемой точки по умолчанию получают нулевые значения.

Объекты класса Point можно по-разному включать в более сложные классы. Возьмем в качестве такого включающего класса класс Circle, объект которого представляет «окружность на плоскости». Объект класса Point (точку) будем использовать в качестве центра окружности.

Начнем с *композиции классов* и, отложив объяснения, дадим такое определение класса:

```
class Circle
{ // Закрытые поля:
  double rad; // радиус окружности
  Point centre = new Point(); // центр окружности
  // Свойство для радиуса окружности:
  public double Rad
  {
    get { return rad; }
    set { rad = value; }
  }
  // Свойство для значения длины окружности:
  public double Len { get { return 2 * rad * Math.PI; } }
  // Свойство для центра окружности:
  public Point Centre
  {
    get { return centre; }
    set { centre = value; }
  }
  public void display()
  {
    Console.WriteLine("Centre: X={0}, Y={1};"
      + " Radius={2}, Length = { 3, 6:f2}",
      centre.X, centre.Y, this.rad, Len);
  }
}
```

В классе Circle два закрытых поля: **double** rad — радиус окружности и Point centre — ссылка на объект класса Point. Для инициализации этой ссылки используется выражение с операцией **new**, в котором выполняется явное обращение к конструктору класса Point. Тем самым при создании каждого объекта «окружность» всегда создается в виде его поля объект «точка», определяющий центр окружности.

Три открытых свойства обеспечивают доступ к характеристикам объекта-окружности: Rad — радиус окружности, Len — длина окружности, Centre — центр окружности.

В классе определен открытый метод display(), выводящий координаты центра и значения других характеристик объекта, представляющего окружность.

Так как в классе Circle нет явно определенных конструкторов, то неявно создается конструктор без параметров, и поля определяемого с его помощью объекта получают значения по умолчанию.

В следующей программе создан объект класса Circle. Затем с помощью свойств классов Circle и Point изменены значения его полей. Метод display() выводит сведения о характеристиках объекта.

```
static void Main()
{
    Circle rim = new Circle();
    rim.Centre.X = 10;
    rim.Centre.Y = 20;
    rim.Rad = 3.0;
    rim.display();
}
```

Результат выполнения программы:

Centre: X=10, Y=20; Radius=3, Length= 18,85

Основное, на что следует обратить внимание, — в программе нет отдельно существующего объекта класса Point. Именно это является основным признаком композиции классов. Объект класса Point явно создается только при создании объекта класса Circle.

На рис. 13.1 приведена диаграмма классов, находящихся в отношении композиции. Конструкторы умолчания, которые добавлены в объявления классов автоматически, на схемах классов не показаны. Тот факт, что ссылка на объект класса Point является значением поля centre объекта класса Circle, никак явно не обозначен.

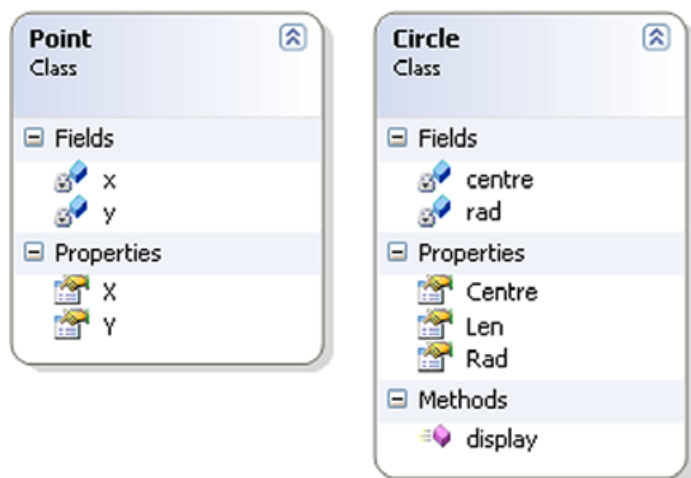


Рис. 13.1. Диаграмма композиции классов

Не изменяя класс Point, можно следующим образом построить класс «окружность на плоскости», используя агрегацию классов (программа 13_2.cs):

```
class Circle
{ // Закрытые поля:
    double rad; // радиус окружности
    Point centre; // ссылка без инициализации
    public Circle(Point p, double rd) // конструктор
    {
        centre = p;
        rad = rd;
    }
    public double Rad {...}
```

```

public double Len {...}
public Point Centre {...}
public void display() {...}
}

```

В тексте нового класса Circle показаны полностью только объявления полей и конструктор, первый параметр которого — ссылка на объект класса Point. Свойства и метод display() остались без изменений.

При таком измененном определении класса Circle для создания его объекта необходимо, чтобы уже существовал объект класса Point, с помощью которого в объекте класса Circle будет определено значение поля centre.

Диаграмма классов, находящихся в отношении агрегации (рис. 13.2), практически та же, что и диаграмма композиции. Только в классе Circle явно присутствует конструктор общего вида.

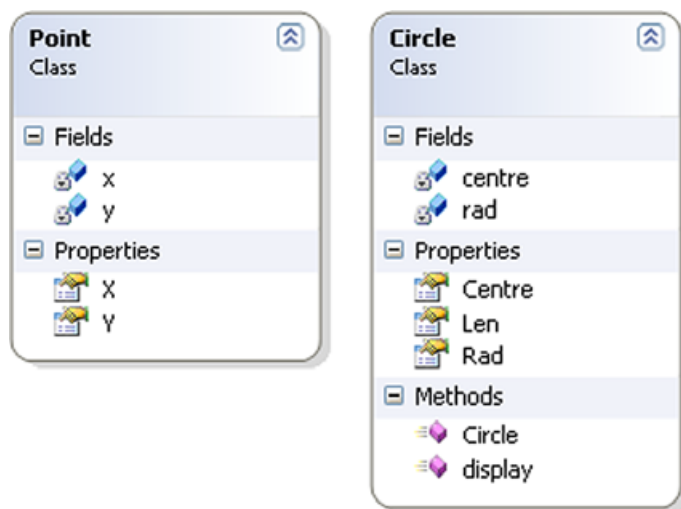


Рис. 13.2. Диаграмма агрегации классов

В следующей программе (в методе Main) создан объект класса Point. Затем с помощью свойств X и Y изменены значения его полей. На основе этого объекта конструктор класса Circle формирует объект «окружность на плоскости». Метод display() выводит сведения о характеристиках построенного объекта.

```

static void Main()
{
    Point pt = new Point();
    pt.X = 10;
    pt.Y = 20;
    Circle rim = new Circle(pt,10);
    rim.display();
}

```

Результат выполнения программы:

Centre: X=10, Y=20; Radius=10, Length=62,83

В отличие от композиции, при агрегации в классе Circle нет явной инициализации поля centre. Для обеспечения включения объекта класса Point в объект класса Circle в классе Circle явно определен конструктор, одним из параметров которого служит ссылка на объект класса Point. В теле конструктора значение этой ссылки присваивается полю centre класса Circle.

13.2. Вложение классов

В объявление класса в качестве его члена может войти объявление типа. Таким типом может быть класс. Этот внутренний класс называют **вложенным** классом, а включающий его класс — **внешним**. Особенностью вложенного класса является то, что ему доступны все члены внешнего класса, даже если эти члены закрытые (**private**). Обычно вложенный класс вводится только для выполнения действий внутри внешнего класса и «не виден» вне включающего его класса. Однако вложенный класс может быть объявлен с модификатором **public** и тогда он становится доступен везде, где виден внешний класс. Если открытый класс Nested вложен в класс Outside, то для внешнего обращения к вложенному классу следует использовать квалифицированное имя Outside.Nested.

Продемонстрируем синтаксис вложения классов на примере класса окружности (Circle), центр которой представляет объект вложенного класса Point. Чтобы возможности внешнего класса Circle были близки к возможностям уже рассмотренных классов, реализующих композицию и агрегацию, сделаем вложенный класс Point открытым. В следующей программе объявлен указанный класс Circle с вложенным классом Point.

```
// 13_03.cs - вложение классов
using System;
class Circle
{ // Закрытые поля:
    double rad; // радиус окружности
    Point centre = new Point(); // центр окружности
    // свойство для радиуса окружности:
    public double Rad
    {
        get { return rad; }
        set { rad = value; }
    }
    // свойство для значения длины окружности:
    public double Len { get { return 2 * rad * Math.PI; } }
    // свойство для центра окружности:
    public Point Centre
    {
        get { return centre; }
        set { centre = value; }
    }
    public void display()
```

```

{
    Console.WriteLine("Centre: X={0}, Y={1}; " +
        "Radius={2}, Length={3,6:f2}",
        centre.X, centre.Y, this.rad, Len);
}
public class Point
{
    double x, y;
    public double X { get { return x; } set { x = value; } }
    public double Y { get { return y; } set { y = value; } }
} // end of Point
} // end of Circle
class Program
{
    static void Main()
    {
        Circle rim = new Circle();
        rim.Centre.X = 100;
        rim.Centre.Y = 200;
        rim.Rad = 30.0;
        rim.display();
    }
}

```

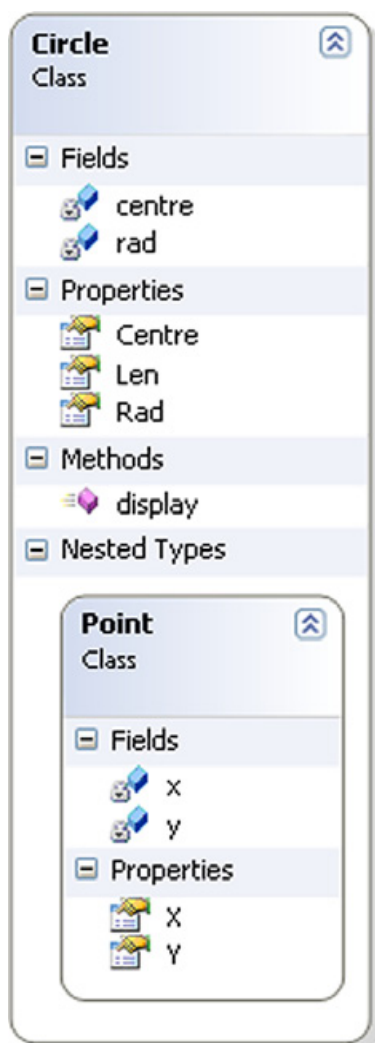


Рис. 13.3. Диаграмма вложения классов

Результат выполнения программы:

Centre: X=100, Y=200; Radius=30, Length=188,50

В классе Circle для инициализации поля center используется конструктор умолчания класса Point. Префикс «Circle.» при этом не требуется применять, хотя его использование не приведет к ошибке. То же самое относится и к обозначению типа свойства Centre. Нет явного определения конструкторов и во внешнем классе. Поэтому в методе Main() для создания объекта, ассоциированного со ссылкой rim, используется конструктор умолчания Circle(). Для обращения к свойствам, определяющим центр окружности, потребовалось, как и при композиции, использовать имена с двойной квалификацией: rim.Centre.X и rim.Centre.Y.

В отличие от композиции и агрегации, при вложении классов внутренний класс (не только объект, но и сам внутренний класс) не существует независимо от внешнего. На диаграмме классов вложенный класс изображается именно внутри внешнего (см. рис.13.3).

13.3. Наследование классов

Наиболее богатым в отношении повторного использования кода и полиморфизма является отношение наследования классов. Для него используют название «является» (*is-a*). При наследовании объект производного класса служит частным случаем или специализацией объекта базового класса. Например, велосипед является частным случаем транспортного средства. Не проводя сравнительного анализа всех тонкостей возможных отношений между классами (этим нужно заниматься в специальном курсе, посвященном объектно-ориентированной методологии, см., например, [2]), покажем на примере рассматриваемых классов «точка на плоскости» и «окружность с заданным точкой центром», как реализуется наследование в языке C#. Итак, класс Point будет базовым классом, а класс Circle сделаем его наследником, иначе говоря, производным от него.

Для объявления класса, который является наследником некоторого базового класса, используется следующий синтаксис:

```
модификаторыopt class  
    имя_производного_класса : имя_базового_класса  
{операторы_тела_производного_класса}
```

Конструкция «: имя_базового_класса» в стандарте C# называется спецификацией базы класса.

Таким образом, класс Circle как производный от базового класса Point, можно определить таким образом (программа 13_04.cs):

```

class Circle : Point // класс "окружность на плоскости"
{ // Закрытое поле:
    double rad; // радиус окружности
    // свойство для радиуса окружности:
    public double Rad
    {
        get { return rad; }
        set { rad = value; }
    }
    // свойство для значения длины окружности:
    public double Len { get { return 2 * rad * Math.PI; } }
    // свойство для центра окружности:
    public Point Centre
    {
        get
        {
            Point temp = new Point();
            temp.X = X;
            temp.Y = Y;
            return temp;
        }
        set { X = value.X; Y = value.Y; }
    }
    public void display()
    {
        Console.WriteLine("Centre: X={0}, Y={1}; "
            + "Radius={2}, Length={3,6:f2}",
            X, Y, rad, Len);
    }
}

```

По сравнению с предыдущими примерами класс Point не изменился. Он так же содержит два закрытых поля, задающих координаты точки, и два открытых свойства X, Y, обеспечивающих доступ к этим полям. В классе Point конструктор добавлен компилятором. В производном классе Circle явно определены: вещественное поле rad, три уже рассмотренных свойства Rad, Len Centre и метод display(). Явного определения конструктора нет и в классе Circle. Поэтому объекты класса Circle можно создавать только с используемыми по умолчанию значениями полей.

При наследовании производный класс «получает в наследство» все члены (поля, свойства, индексаторы и методы) базового класса, за исключением конструктора — конструктор базового класса не наследуется. Получив от базового класса его члены, базовый класс может по-разному «распорядиться с наследством». Доступ к полям базового класса для членов и объектов производного класса разрешен не всегда. Закрытые члены базового класса недоступны для членов и объектов производного класса.

Открытые члены базового класса доступны для членов и объектов производного класса. В нашем примере класс Point имеет два открытых свойства (X и Y), которыми можно пользоваться как внутри

класса `Circle`, так и во «внешнем мире», обращаясь к этим свойствам с помощью ссылок на объекты класса `circle`. В методе `display()`, а также в аксессорах свойства `Center` производного класса выполняется непосредственное обращение к унаследованным свойствам `X`, `Y`.

Особое внимание в нашем примере с наследованием нужно обратить на свойство `Circle.Centre`. При агрегации и композиции классов `Point` и `Circle` значением этого свойства служит ссылка на непосредственно существующий объект класса `Point`. В случае наследования в объекте класса `Circle` нет объекта класса `Point` — наследуются поля и свойства класса `Point`. Поэтому для объявления в классе `Circle` свойства `Centre` объект класса `Point` приходится «реконструировать»:

```
public Point Centre
{
    get
    {
        Point temp = new Point();
        temp.X = X;
        temp.Y = Y;
        return temp;
    }
    set {X = value.X; Y = value.Y;}
}
```

В `get`-аксессоре явно создается временный объект класса `Point`, его полям с помощью унаследованных свойств присваиваются значения полей, унаследованных классом `Circle` от базового класса `Point`. Ссылка на этот временный объект возвращается как значение свойства `Circle.Centre`. `Set`-аксессор свойства `Circle.Centre` очень прост — непосредственно используются унаследованные свойства `X`, `Y` класса `Point`.

Следующий фрагмент программы (см. `13_04.cs`) демонстрирует возможности класса `Circle`.

```
class Program
{
    static void Main()
    {
        Circle rim = new Circle();
        rim.X = 24;
        rim.Y = 10;
        rim.Rad = 2;
        rim.display();
        rim = new Circle();
        rim.display();
    }
}
```

В методе `Main()` создан объект класса `Circle`. Он ассоциирован со ссылкой `rim`, и с ее помощью осуществляется доступ как к свойствам и методам объекта класса `Circle`, так и к свойствам объекта базового класса `Point`, созданного в аксессоре `get` свойства `Circle.Centre`. Следую-

щие результаты выполнения программы дополняют приведенные объяснения:

```
Centre: X=24, Y=10; Radius=2, Length=12,57
Centre: X=0, Y=0; Radius=0, Length=0,00
```

В языке UML предусмотрено специальное обозначение для отношения наследования классов. Два класса (рис. 13.3) изображаются на диаграмме отдельными прямоугольниками, которые соединены сплошной линией со стрелкой на одном конце. Стрелка направлена на базовый класс.

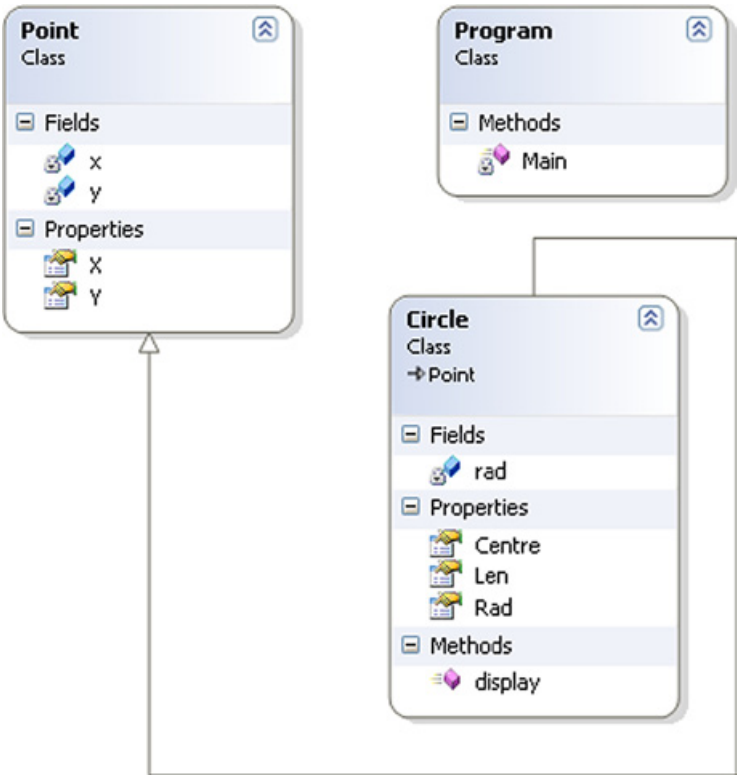


Рис. 13.4. Диаграмма к программе 13_04.cs с наследованием классов

13.4. Доступность членов класса при наследовании

Члены базового класса, имеющие статус доступа **private**, как были недоступны для внешнего по отношению к объявлению базового класса мира, так и остаются закрытыми для членов и объектов производного класса. Члены базового класса, имеющие модификатор **public**, открыты для членов и объектов производного класса.

В ряде случаев необходимо, чтобы члены базового класса были доступны (открыты) для членов производного класса, но в то же время были закрыты (недоступны) для объектов производного класса. В этом случае в базовом классе эти члены должны быть **защищенными**, т. е. объявлены с модификатором **protected**. Сразу же отметим, что если класс рассматривается вне наследования, то защищенные члены ничем

не отличаются от закрытых. К защищенным членам класса нельзя обращаться извне объявления класса.

В производном классе обычно вводятся новые члены, определяющие новое поведение и дополнительные характеристики объектов производного класса. Для новых (не унаследованных) членов производных классов имена выбираются произвольно.

Если в производном классе объявлен член, имя которого совпадает с именем какого-либо члена базового класса, то для их различения в производном классе используются уточненные имена:

```
this.имя_члена_производного_класса  
base.имя_члена_базового_класса
```

При внешних обращениях одноименные члены базового и производного класса различаются по типам объектов, для которых эти обращения выполнены.

Рассмотрим пример. Определим класс «круг» и производный от него класс «кольцо». Предположим, что у нас нет необходимости создавать объекты класса «круг», и он будет использоваться только как базовый для определения других классов. Тогда его определение может быть таким (Программа 13_05.cs):

```
class Disk // класс "круг"  
{  
    protected double rad; // радиус круга  
    protected Disk(double ri) {rad = ri;} // конструктор  
    protected double Area  
    {  
        get {return rad * rad * Math.PI;}  
    }  
}
```

В классе Disk одно поле rad, задающее значение радиуса круга, конструктор общего вида и свойство Area, позволяющее получить значение площади круга. Параметр конструктора явно использован в теле конструктора, где он задает значение поля rad, т. е. определяет радиус круга. Все члены класса объявлены с модификатором **protected**. При таком определении класс вне наследования ни к чему не годен. Невозможно создать объект класса Disk — его конструктор защищенный (**protected**). Если убрать явно определенный конструктор, компилятор добавит открытый конструктор умолчания. Но и в этом случае пользы не видно — создав объект, нельзя будет обратиться к его полю или свойству.

Используем класс Disk в качестве базового в следующем объявлении:

```
class Ring : Disk // класс "кольцо"  
{  
    new double rad; // радиус внутренней окружности  
    // конструктор:  
    public Ring(double Ri, double ri)
```

```

        : base(Ri) { rad = ri; }
    public new double Area
    { get { return base.Area - Math.PI* rad *rad; } }
    public void print()
    {
        Console.WriteLine("Ring: Max_radius={0:f2}, "
            + "Min_radius={1:f2}, Area={2:f3}",
            base.rad, rad, Area);
    }
}

```

В производном классе Ring поле **new double** rad определяет значение внутренней окружности границы кольца. Радиус внешней границы определяет одноименное поле **double** rad, унаследованное из базового класса. Оба поля вне объявления класса Ring недоступны. Конструктор производного класса Ring объявлен явно, как открытый метод класса. У этого конструктора два параметра, позволяющие задавать значения радиусов границы кольца. В теле конструктора второй параметр **double** ri определяет внутренний радиус. В инициализаторе конструктора **:base(Ri)** выполнено явное обращение к конструктору базового класса Disk. Параметр конструктора Ri служит аргументом в этом обращении. Отметим, что для простоты кода не используются никакие проверки допустимости значений параметров.

Обратите внимание, что в объявление поля rad производного класса Ring входит модификатор **new**. Появление **new** обусловлено следующим соглашением языка C#. Имя члена производного класса может совпадать (вольно или по ошибке) с именем какого-либо члена базового класса. В этом случае имя члена производного класса **скрывает** или, говорят, **экранирует** соответствующее имя члена базового класса. При отсутствии модификатора **new** компилятор выдает сообщение с указанием совпадающих имен и предложением «*Use the new keyword if hiding was intended*» — «Используйте служебное слово new, если экранирование запланировано». Именно для того, чтобы удостоверить компилятор в преднамеренном совпадении имен, радиус внутренней окружности объявлен с модификатором **new**. То же самое сделано и при объявлении в классе Ring свойства Area, позволяющего получить площадь кольца. Кроме того, класс Ring унаследовал свойство с тем же именем из базового класса. В get-аксессоре свойства Area из класса Ring выполнено явное обращение **base.Area** к свойству базового класса Disk. Открытый метод print() позволяет вывести сведения об объекте класса Ring. Выводятся значения полей **base.rad**, rad и свойства Area. Отметим, что принадлежность членов rad и Area классу Ring можно подчеркнуть, если задать аргументы метода WriteLine() в таком виде:

```

Console.WriteLine("Ring: Max_radius={0:f2}, "
    + "Min_radius={1:f2}, Area={2:f3}",
    base.rad, this.rad, this.Area);

```


В качестве иллюстрации возможностей класса Ring рассмотрим такой фрагмент программы:

```
class Program
{
    static void Main()
    {
        Ring rim = new Ring(10.0, 4.0);
        rim.print();
    }
}
```

Результат выполнения программы:

Ring: Max_radius=10,00, Min_radius=4,00, Area=263,894

Для иллюстрации доступности членов классов при наследовании удобно использовать схему, приведенную на рис. 13.5. Так как при изложении материала мы не затронули понятие сборки, то члены с модификатором **internal** на схеме не показаны. Стрелки на схеме обозначают возможность обращения (доступность) к членам с разными модификаторами.

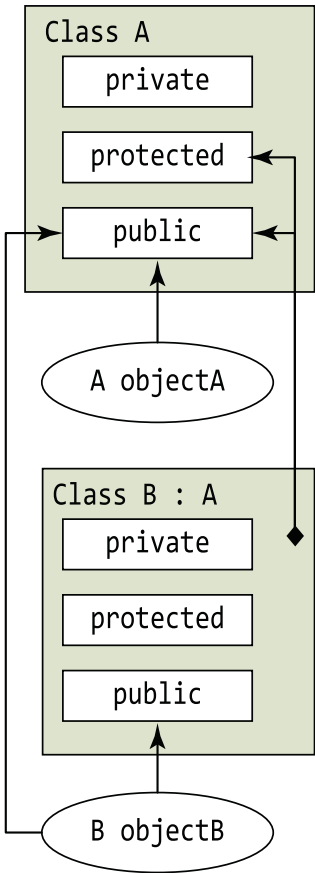


Рис. 13.5. Доступность членов при наследовании классов

Если класс В является наследником (производным от) класса А, то для объекта класса В (objectB на рис. 13.5) доступны открытые члены классов А и В. В то же время для членов производного класса В доступны

открытые и защищенные члены класса A (само собой, доступны и все члены самого класса B). Объект класса A (objectA на схеме) никогда не имеет доступа к членам производного класса B.

13.5. Методы при наследовании

О конструкторах. В приведенном примере наследования с классами «круг» и «кольцо» конструктор производного класса Ring явным образом обращается к конструктору базового класса Disk с помощью выражения `base(Ri)`. Обращение происходит до выполнения операторов тела конструктора — в инициализаторе конструктора:

```
public Ring(double Ri, double ri) : base(Ri) {rad = ri;}
```

Так как в нашем примере поле `rad` (с модификатором *protected*) базового класса Disk доступно для методов производного класса, то программист может попытаться так записать определение конструктора:

```
public Ring(double Ri, double ri) {base.rad = Ri; rad = ri;}
```

Внешне все выглядит правильно, но если не менять определения базового класса Disk, то компилятор выдаст следующее сообщение об ошибке:

```
'Disk' does not contain a constructor that takes '0' arguments  
(Disk не содержит конструктора, который имеет '0' аргументов)
```

Чтобы принять такое решение, компилятор использовал два правила. Первое из них мы уже приводили — если в определении класса присутствует объявление хотя бы одного конструктора, то конструктор без параметров автоматически в класс не добавляется. Правило второе относится к наследованию. Уже говорилось, что, в отличие от других членов базового класса, конструкторы не наследуются. **Конструктор базового класса необходимо явно вызвать из инициализатора конструктора производного класса.** Если этого не сделано, то компилятор по умолчанию самостоятельно дополнит объявление конструктора (точнее, дополнит его инициализатор) обращением к конструктору базового класса без параметров. Так как в классе Disk конструктор без параметров отсутствует, то компиляция завершается приведенным сообщением об ошибке.

Экранирование методов базового класса. Тот факт, что в производном классе могут быть определены новые методы, имена которых отличны от имен методов базового класса, наверное, нет необходимости пояснять. А вот на одноименных методах базового и производного классов остановимся подробно.

Во-первых, для методов возможна перегрузка (*overload*). В этом случае одноименные методы базового и производного классов должны иметь разные спецификации параметров. Во-вторых, разрешено пере-

определение (иначе экранирование или сокрытие, *hiding*) методом производного класса одноименного метода базового класса (спецификации параметров совпадают). В третьих, метод базового класса может быть объявлен как виртуальный (*virtual*), и тогда при его переопределении (*overriding*) в производных классах обеспечивается полиморфизм.

В случае перегрузки методов при наследовании никаких нововведений нет.

При экранировании метода стандарт рекомендует снабжать метод производного класса модификатором **new**. При его отсутствии компиляция проходит успешно, но выдается предупреждение (*Warning*). В нем программисту указывают, что он выполнил переопределение метода базового класса, возможно, по оплошности. Если при переопределении методов необходимо из методов производного класса обращаться к методу базового класса, то используется уточненное имя **base.имя_метода_базового_класса**.

Покажем на примере переопределение (экранирование) методов. Определим класс «фигура на плоскости с заданными размерами вдоль координатных осей». Такой класс будет служить базовым для классов «прямоугольник», «эллипс», «треугольник» и т. д. Для простоты будем считать, что у прямоугольника стороны параллельны координатным осям, у эллипса оси параллельны координатным осям, у треугольника сторона и перпендикулярная ей высота параллельны координатным осям.

```
// 13_06.cs – экранирование методов при наследовании
class Figure // базовый класс
{
    protected double dx, dy; // размеры вдоль осей
    public void print()
    {
        Console.WriteLine("Габариты: dx={0:f2}, dy={1:f2}",
            dx, dy);
    }
}
// Производный класс - прямоугольник:
class Rectangle : Figure
{
    public Rectangle(double xi, double yi)
    { dx = xi; dy = yi; }
    public new void print()
    {
        Console.Write("Прямоугольник! \t");
        base.print();
    }
}
// Производный класс - треугольник:
class Triangle : Figure
{
    public Triangle(double xi, double yi)
    { dx = xi; dy = yi; }
    public new void print()
    {
```

```

        Console.Write("Треугольник! \t");
        base.print();
    }
}

```

В каждом из производных классов есть свой метод `print()`, который экранирует одноименный метод базового класса. В методе `print()` производного класса выполнено обращение к методу `print()` базового класса.

Применение декларированных классов иллюстрирует следующий код:

```

class Program
{
    static void Main()
    {
        Rectangle rec = new Rectangle(3.0, 4.0);
        rec.print();
        Triangle tre = new Triangle(5.0, 4.0);
        tre.print();
        Figure fig = new Figure();
        fig.print();
    }
}

```

Результат выполнения программы:

```

Прямоугольник! Габариты: dx=3,00, dy=4,00
Треугольник! Габариты: dx=5,00, dy=4,00
Габариты: dx=0,0, dy=0,0

```

Отметим, что при экранировании метод может иметь другой тип возвращаемого значения. Для тех, кто помнит, что в сигнатуру при перегрузке методов тип возвращаемого значения не входит, этот факт не окажется неожиданностью.

Виртуальные методы и полиморфизм. Ссылке с типом базового класса можно присвоить значение ссылки на объект производного класса. После такого присваивания ссылка не обеспечивает доступа к обычным (не виртуальным) методам производного класса. Рассмотрим следующий фрагмент программы с несколькими ссылками базового класса `Figure`, адресуящими объекты и базового, и производных классов `Triangle`, и `Rectangle` (программа 13_07.cs).

```

static void Main()
{
    Figure fig1 = new Rectangle(3.0, 4.0);
    Figure fig2 = new Triangle(5.0, 4.0);
    Figure fig3 = new Figure();
    fig1.print();
    fig2.print();
    fig3.print();
}

```

Три ссылки, имеющие тип `Figure`, ассоциированы с объектами разных классов. Затем с помощью этих ссылок выполнены обращения к методу `print()`. Во всех случаях вызывается метод базового класса.

Результат выполнения программы:

Габариты: `dx=3,00, dy=4,00`

Габариты: `dx=5,00, dy=4,00`

Габариты: `dx=0,00, dy=0,00`

Адресаты обращений в выражениях `fig1.print()`, `fig2.print()`, `fig3.print()` определяются объявленным типом ссылок, а не типом значения, которое ассоциировано с каждой ссылкой в текущий момент. Ссылки `fig1`, `fig2`, `fig3` имеют тип базового класса `Figure`, поэтому все три обращения отнесены к методу `print()` базового класса. Связано это с неvirtualностью методов `print()`.

Ссылка с типом базового класса может обеспечить доступ к виртуальному методу того производного класса, объект которого в этот момент адресован этой ссылкой. Для определения в базовом классе виртуального метода в его заголовок нужно добавить модификатор **virtual**. В производном классе для переопределения виртуального метода используется модификатор **override**. Заметим, что виртуальный метод не может быть закрытым (**private**).

В нашем примере изменения будут минимальными. Заголовок метода в базовом классе примет вид (программа `13_08.cs`):

```
public virtual void print()
```

В каждом из производных классов заголовок переопределяющего метода примет вид:

```
public override void print()
```

Обратите внимание, что модификатор **new** заменен в производном классе для виртуального метода модификатором **override**.

Результат выполнения того же фрагмента кода с определением и использованием ссылок `fig1`, `fig2`, `fig3` будет таким:

Прямоугольник! Габариты: `dx=3,00, dy=4,00`

Треугольник! Габариты: `dx=5,00, dy=4,00`

Габариты: `dx=0,00, dy=0,00`

Здесь дважды выполнено обращение к методам `print()` производных классов, и последним выполнен вызов виртуального метода `print()` базового класса.

Обратим внимание, что спецификации параметров и, конечно, имена виртуального метода и переопределяющего метода производного класса должны совпадать.

Виртуальным может быть не только метод, но и индексатор, событие (см. в разд. 17.6) и свойство, в объявление каждого из которых входит

модификатор **virtual**. Все описанные возможности виртуальных методов распространяются также и на эти члены.

Массивы ссылок с типом базового класса позволяют продемонстрировать возможности виртуальных методов и свойств. Как уже показано, ссылке с типом базового класса можно присвоить значение ссылки на объект любого производного класса. Если в производных классах переопределены виртуальные методы базового класса, то с помощью одной ссылки с типом базового класса можно обращаться к методам объектов разных производных классов. В качестве примера рассмотрим следующую программу:

```
// 13_09.cs - массив ссылок с типом базового класса
using System;
class A
{
    public virtual string record()
    {
        return "Базовый класс!";
    }
}
class B : A
{
    public override string record()
    {
        return "Производный B!";
    }
}
class C : A
{
    public override string record()
    {
        return "Производный C!";
    }
}
class Program
{
    static void Main()
    {
        A[] arrA = new A[] { new A(), new B(), new C(), new B() };
        foreach (A rec in arrA)
            Console.WriteLine(rec.record());
    }
}
```

Результат выполнения программы:

```
Базовый класс!
Производный B!
Производный C!
Производный B!
```

Если параметром метода служит ссылка с типом базового класса, то вместо нее в качестве аргумента можно использовать ссылку на объ-

ект производного класса. Эту возможность демонстрирует следующая программа:

```
// 13_10.cs - ссылка с типом базового класса как параметр
using System;
class Aclass { }
class Bclass : Aclass { }
class Cclass : Aclass { }
class Program
{
    static void type(Aclass par)
    {
        Console.WriteLine(par.ToString());
    }
    static void Main()
    {
        type(new Aclass());
        type(new Bclass());
        type(new Cclass());
    }
}
```

Результат выполнения программы:

```
Aclass
Bclass
Cclass
```

Если тип возвращаемого значения метода есть тип базового класса, то метод может вернуть значение ссылки на объект производного класса.

Пример (программа 13_11.cs):

```
static Aclass type(int m)
{
    if (m == 0) return new Bclass();
    if (m == 1) return new Cclass();
    return new Aclass();
}
static void Main()
{
    for (int i = 0; i < 3; i++)
        Console.WriteLine(type(i).GetType());
}
```

Результат:

```
Bclass
Cclass
Aclass
```

О возможности ссылки с типом базового класса представлять виртуальные члены производных классов говорят, используя термин «*динамическое связывание*». Эта возможность основана на наличии у ссылки

двух типов. Тип, получаемый в декларации ссылкой на объекты базового класса, является ее **объявленным** (статическим) типом. Если этой ссылке присваивается значение ссылки на объект производного класса, то ссылка дополнительно получает **тип времени исполнения** (динамический тип). При обращении к неvirtуальным членам учитывается статический тип ссылки, что обеспечивает доступ к членам базового класса. При обращении к virtуальным членам используется динамический тип ссылки и вызываются члены объекта производного класса.

13.6. Абстрактные методы и абстрактные классы

В ряде случаев класс создается только как базовый, и его автор не предполагает, что кто-то будет создавать объекты этого класса вне наследования. Например, объекты уже рассмотренного в примерах класса фигур с заданными габаритами вряд ли пригодны для метода, который вычисляет площадь или периметр фигуры. Эти характеристики нельзя определить, если о фигуре известны только ее габариты. Однако в базовом классе можно «запланировать» проведение этих вычислений в производных классах. Для этого в базовый класс добавляют так называемые абстрактные методы, и класс объявляют абстрактным. Абстрактные методы задают «прототипы» реальных методов, которые должны быть реализованы в классах, производных от абстрактного.

Абстрактный метод может быть объявлен только в абстрактном классе. В заголовке абстрактного метода указывается модификатор **abstract**. У абстрактного метода после скобки, ограничивающей спецификацию параметров, помещается символ «точка с запятой». У абстрактного метода не может быть тела в виде блока операторов в фигурных скобках. Абстрактный метод по умолчанию является виртуальным. Таким образом, добавлять модификатор **virtual** не требуется.

Чтобы класс был определен как абстрактный, в его заголовок помещают модификатор **abstract**. Создавать объекты абстрактных классов невозможно. Если в абстрактном классе объявлены несколько абстрактных методов, а производный класс содержит реализацию не всех из них, то производный класс в свою очередь становится абстрактным. В абстрактном классе могут быть определены любые не абстрактные члены (методы, поля, свойства и т. д.). Кроме того, класс может быть объявлен абстрактным даже в том случае, если в нем отсутствуют абстрактные члены.

Продemonстрируем особенности использования абстрактных классов и абстрактных методов на примере классов, производных от класса «фигура на плоскости». Превратим этот базовый класс в абстрактный, добавив в него абстрактный метод для вычисления площади фигуры. Метод вывода сведений об объекте класса также сделаем абстрактным. Для иллюстрации определим в этом же классе не виртуальный метод (*compress*), выполняющий сжатие (или увеличение) габаритных размеров фигуры в заданное число раз. Текст программы с таким классом:


```

// 13_12.cs - абстрактные методы в абстрактном классе
using System;
abstract class Figure // абстрактный базовый класс
{
    protected double dx, dy; // размеры вдоль осей
    public abstract void print();
    public void compress(double r) { dx *= r; dy *= r; }
    abstract public double square();
}
class Rectangle : Figure
{
    public Rectangle(double xi, double yi)
    { dx = xi; dy = yi; }
    public override void print()
    {
        Console.Write("Площадь прямоугольника={0:f2}. \t",
            square());
        Console.WriteLine("Габариты: dx={0:f2}, dy={1:f2}",
            dx, dy);
    }
    public override double square()
    { return dx * dy; }
}
class Triangle : Figure
{
    public Triangle(double xi, double yi)
    { dx = xi; dy = yi; }
    public override void print()
    {
        Console.Write("Площадь треугольника={0:f2}. \t",
            square());
        Console.WriteLine("Габариты: dx={0:f2}, dy={1:f2}",
            dx, dy);
    }
    public override double square()
    { return dx * dy / 2; }
}
class Program
{
    static void Main()
    {
        Figure fig = new Rectangle(3.0, 4.0);
        fig.print();
        fig = new Triangle(5.0, 4.0);
        fig.print();
        fig.compress(2.0);
        fig.print();
        Triangle tri = new Triangle(8.0, 4.0);
        tri.print();
        tri.compress(0.25);
        tri.print();
    }
}

```

Результат выполнения программы:

Площадь прямоугольника=12,00. Габариты: dx=3,00, dy=4,00
Площадь треугольника=10,00. Габариты: dx=5,00, dy=4,00
Площадь треугольника=22,50. Габариты: dx=7,50, dy=6,00
Площадь треугольника=16,00. Габариты: dx=8,00, dy=4,00
Площадь треугольника=1,00. Габариты: dx=2,00, dy=1,00

В методе Main() определены две ссылки, fig и tri. Первая имеет тип базового класса Figure и адресует вначале объект производного класса Rectangle, затем производного объект класса Triangle. Ссылка fig обеспечивает обращения к перегруженным методам print() производных классов Rectangle и Triangle. Ссылка tri может адресовать только объекты класса Triangle и способна обеспечить вызов метода print() только этого же производного класса. В то же время для этой ссылки (и для ссылки fig) доступен метод compress() базового класса, унаследованный производными классами. Результаты выполнения программы иллюстрируют сказанное.

Завершая рассмотрение наследования классов, повторим, что в производном классе метод по отношению к методу базового класса может быть *новым, перегруженным, унаследованным, скрывающим (экранирующим)* метод базового класса и *реализующим виртуальный* или *абстрактный* метод базового класса.

13.7. Опечатанные классы и члены классов

C# разрешает определять методы, свойства и классы, для которых невозможно наследование. В объявление такого метода, свойства или класса входит модификатор **sealed** (опечатанный, герметизированный). Опечатанный класс нельзя использовать в качестве базового класса. Опечатанный метод и опечатанное свойство при наследовании нельзя переопределить.

В базовой библиотеке классов .NET Framework много опечатанных классов. Программисты, работающие с библиотекой в качестве пользователей, не могут создавать собственные производные классы на основе опечатанных классов .NET Framework. Таким опечатанным библиотечным классом является класс **string**.

13.8. Применение абстрактных классов

Мы уже отметили, что нельзя создать объект абстрактного класса. Однако можно объявить ссылку с типом абстрактного класса. Такая ссылка может служить параметром метода и возвращаемым значением метода. В обоих случаях ссылка предназначена для представления объекта того конкретного класса, который является производным от абстрактного.

В качестве примера обратимся еще раз к абстрактному классу `Figure` и производным от него классам `Rectangle` и `Triangle`. В этих классах определены методы `square()`, возвращающие значения площадей объектов конкретных классов (не абстрактных). Следующий метод позволяет вывести значение площади любого объекта классов `Triangle` и `Rectangle` (программа 13_13.cs).

```
static void figSq(Figure f)
{ Console.WriteLine("Площадь = " + f.square()); }
```

Можно определять массивы ссылок с типами абстрактных классов. Значениями этих ссылок должны быть «адреса» объектов конкретных классов, реализующих базовый. Следующий метод возвращает ссылку с типом абстрактного класса:

```
static Figure figRec(Figure[] ar, int n)
{ return ar[n]; }
```

Параметры метода — ссылка на массив ссылок с типом абстрактного класса `Figure` и целое `n`, определяющее индекс элемента этого массива. Возвращаемое значение метода — значение элемента массива. При обращении к методу в качестве аргумента используется ссылка на массив с типом элементов `Figure`. Каждому элементу массива-аргумента должно быть присвоено значение ссылки на объект конкретного класса, производного от `Figure`.

В следующем фрагменте программы 13_13.cs определен массив ссылок с типом `Figure`. Его элементам присвоены ссылки на объекты классов `Rectangle` и `Triangle`.

```
class Program
{
    static void figSq(Figure f)
    { Console.WriteLine("Площадь = " + f.square()); }
    static Figure figRec(Figure[] ar, int n)
    {
        return ar[n];
    }
    static void Main()
    {
        Figure[] arF = new Figure[4];
        arF[0] = new Rectangle(3.0, 4.0);
        arF[1] = new Triangle(5.0, 4.0);
        Triangle tri = new Triangle(8.0, 4.0);
        arF[2] = tri;
        Rectangle rec = new Rectangle(1.0, 3.0);
        arF[3] = rec;
        for (int i = 0; i < arF.Length; i++)
        {
            Figure f = figRec(arF, i);
            figSq(f);
        }
        Console.WriteLine("Типы элементов массива: ");
    }
}
```

```

        foreach (Figure g in arF)
            Console.WriteLine(g.GetType());
    }
}

```

Результат выполнения программы:

```

Площадь = 12
Площадь = 10
Площадь = 16
Площадь = 3
Типы элементов массива:
Rectangle
Triangle
Triangle
Rectangle

```

В программе элементы массива `arF` обрабатываются в двух циклах. В цикле с заголовком **for** переменной `Figure f` присваиваются значения, возвращаемые методом `figRec()`. Затем `f` используется в качестве аргумента метода `figSq()`. Тем самым выводятся значения площадей всех фигур, «адресованных» элементами массива `arF`.

В цикле с заголовком **foreach** перебираются все элементы массива `arF` и к каждому значению элемента, которое присвоено переменной `Figure g`, применяется метод `GetType()`. Результат — список названий классов, реализовавших абстрактный класс `Figure`.

Возможность присваивать элементам массива с типом абстрактного класса ссылки на объекты любых классов, реализовавших абстрактный, является очень сильным средством полиморфизма. Не меньшие возможности обеспечивает применение ссылок с типом абстрактного класса в качестве параметров и возвращаемых методами значений.

Контрольные вопросы и задания

1. Объясните различие между агрегацией и композицией классов.
2. Какого типа параметр должен быть у конструктора класса, находящегося в отношении агрегации с включаемым классом?
3. Какие члены внешнего класса доступны для вложенного класса?
4. Какой статус доступа должен иметь вложенный класс, чтобы он был доступен там, где виден внешний класс?
5. Как обратиться к члену вложенного класса вне внешнего класса?
6. В чем отличия вложения классов от агрегации и композиции?
7. Сколько прямых базовых классов допустимо для производного класса?
8. Какова роль инициализатора конструктора в конструкторе производного класса?
9. Что такое спецификация базы класса?
10. Какие члены базового класса наследуются производным классом?

11. Объясните правила доступа к членам базового класса из методов производного класса.
12. Объясните правила доступа к членам базового класса для объектов производного класса.
13. Что такое защищенный член класса?
14. Как различаются при внешних обращениях одноименные члены базового и производного классов?
15. Как различаются одноименные члены базового и производного классов в обращениях из производного класса?
16. Каково назначение модификатора **new** в производном классе?
17. Как и где вызывается конструктор базового класса из конструктора производного класса?
18. Какие действия выполняются автоматически при отсутствии в конструкторе производного класса обращения к конструктору базового класса?
19. В каком отношении могут находиться одноименные методы базового и производного классов?
20. Что такое экранирование при наследовании классов?
21. Должны ли совпадать типы возвращаемых значений при экранировании методов?
22. Что такое виртуальный метод?
23. В каком случае ссылке с типом базового класса доступен метод производного класса?
24. В каком случае применяется модификатор **override**?
25. Какой статус доступа должен быть у виртуального метода?
26. Может ли быть виртуальным свойство?
27. Объясните различия между динамическим и статическим связыванием.
28. Что такое статический и динамический типы ссылки?
29. Чем должно быть тело абстрактного метода?
30. Назовите особенности абстрактного метода.
31. Где должен быть объявлен абстрактный метод?
32. Что такое опечатанный класс?
33. Приведите примеры опечатанных классов из .NET Framework.
34. Каковы возможности массивов ссылок с типом абстрактного класса?

Глава 14

ИНТЕРФЕЙСЫ

14.1. Два вида наследования в ООП

Методология ООП рассматривает два вида наследования — наследование реализации и наследование специфицированной функциональности (контракта).

Наследование реализации предполагает, что производный тип получает члены базового типа и использует их наряду со своими членами, добавленными в его объявлении. При выделении памяти для объекта производного класса память отводится и для полей базового класса и для полей, явно объявленных в теле производного класса. Унаследованные функциональные члены базового класса либо входят в арсенал средств производного класса, либо переопределяются в его объявлении. Наследование реализации обеспечено в С# наследованием классов.

Наследование специфицированной функциональности означает, что все типы, построенные на основе одного и того же базового, имеют одинаковую функциональность, и эта функциональность определена спецификацией базового типа.

Наследование специфицированной функциональности может быть реализовано либо на основе абстрактных классов, не включающих полей, либо на основе **интерфейсов**.

Итак, интерфейс в С# (и, например, в языке Java) это механизм, предназначенный для определения правил поведения объектов еще не существующих классов. Интерфейс описывает, какие действия нужны для экземпляров типа, но не определяет, как эти действия должны выполняться. В объявление интерфейса входят декларации (прототипы) **методов, свойств, индексаторов и событий**. Прототип метода не содержит тела, в нем только заголовок метода, завершенный точкой с запятой. Прототипы других членов рассмотрим позднее.

В отличие от классов, с помощью интерфейсов нельзя определять объекты. На основе интерфейса объявляются новые классы. Говорят, что класс, построенный на базе интерфейса, реализует данный интерфейс. Таким образом, интерфейс — это только описание тех возможностей, которые будут у класса, когда он реализует интерфейс. Другими словами, интерфейс определяет средства взаимодействия с внешним миром объектов реализующего его класса.

На основе одного интерфейса могут быть созданы несколько разных классов, и у каждого из этих классов будет набор всех средств, объявленных в интерфейсе. Однако каждый из классов, реализующих один и тот же интерфейс, может по-своему определить эти средства. Зная, **какие методы, свойства, индексаторы и события** декларированы в интерфейсе, программист знает средства взаимодействия с объектами класса, реализовавшего данный интерфейс. Таким образом, объекты разных классов, реализующих один интерфейс, могут обрабатываться одинаково. Это, наряду с перегрузкой методов и операций, еще один пример проявления полиморфизма.

Более того, интерфейсы позволяют реализовать в языке C# одну из фундаментальных посылок методологии объектно-ориентированного программирования — **принцип подстановки Барбары Лисков** (см. [2]). В соответствии с этим принципом объекты разных классов, реализующих один и тот же интерфейс, могут заменять друг друга в ситуации, где от них требуется функциональность, специфицированная интерфейсом.

14.2. Объявления интерфейсов

Будем рассматривать интерфейсы, объявления которых имеют следующий формат:

```
модификатор_интерфейсаопт interface  
имя_интерфейса  
спецификация_базы_интерфейсаопт  
тело_интерфейсаопт
```

Необязательные модификаторы интерфейса это: **new**, **public**, **protected**, **internal**, **private**. Все перечисленные модификаторы нам уже знакомы. Обратите внимание, что для интерфейса нельзя использовать модификатор **static**.

interface — служебное слово, вводящее объявление интерфейса;

Имя_интерфейса — идентификатор, выбираемый автором интерфейса. Принято начинать имя интерфейса с заглавной буквы I, за которой помещать осмысленное название, которое тоже начинается с заглавной буквы.

Спецификация_базы_интерфейса — предваряемый двоеточием список интерфейсов, производным от которых является данный интерфейс. В отличие от наследования классов, где может быть только один базовый класс, базовых интерфейсов может быть любое количество.

Тело_интерфейса — заключенная в фигурные скобки последовательность деклараций (описаний или прототипов) членов интерфейса. Ими могут быть (как мы уже отметили):

- декларация метода;
- декларация свойства;

- декларация индексатора;
- декларация события.

В любую из перечисленных деклараций членов интерфейса может входить только модификатор **new**. Его роль та же, что и в декларациях членов класса. Другие модификаторы в декларации членов интерфейса не входят. По умолчанию все члены интерфейса являются открытыми, т. е. им приписывается статус доступа, соответствующий модификатору **public**. Формат простейшего объявления интерфейса (без спецификации базы):

```
interface имя_интерфейса
{
    тип имя_метода(спецификация_параметров);
    тип имя_свойства{get; set;}
    тип this [спецификация_индекса] {get; set;}
    event тип_делегата имя_события;
}
```

Как видно из формата, ни один из членов интерфейса не содержит операторов, задающих конкретные действия. В интерфейсе только прототипы методов, свойств, индексаторов, событий. Прежде чем давать другие пояснения, приведем пример объявления интерфейса (с прототипами методов и свойств):

```
interface IPublication
{ // интерфейс публикаций
    void write(); // готовить публикацию
    void read(); // читать публикацию
    string Title {set; get;} // название публикации
}
```

В данном примере интерфейс с именем IPublication специфицирует функциональность классов, которые могут представлять публикации — такие объекты как статья, доклад, книга. В соответствии с данным интерфейсом публикации можно писать — у реализующих интерфейс классов должен быть метод write(). Публикации можно читать — в интерфейсе есть прототип метода read(). Свойство Title должно обеспечивать получение и задание в виде строки названия публикации.

Больше никаких возможностей (никакой функциональности) интерфейс IPublication не предусматривает. Данный интерфейс не может ничего предполагать о таких характеристиках публикаций как фамилия автора, год издания, число страниц и т. п. Эти сведения могут появиться только у конкретных объектов тех классов, которые реализуют интерфейс IPublication.

В объявлении интерфейса IPublication отсутствует модификатор доступа — по умолчанию этот интерфейс доступен в том пространстве имен, которому принадлежит его объявление. Для интерфейса IPublication не указана спецификация базы, т. е. IPublication не является наследником никакого другого интерфейса.

Членами интерфейса `IPublication` являются прототипы двух методов `write()`, `read()`. Если сравнить их с объявлениями абстрактных методов в абстрактном классе, то следует отметить отсутствие модификаторов. По существу, метод интерфейса, например `write()`, в `IPublication` играет роль абстрактного метода класса, но модификатор **abstract** для прототипа метода в интерфейсе не нужен (и будет ошибочен). Чуть позже мы покажем, что интерфейс является ссылочным типом и можно объявлять ссылки-переменные с типом интерфейса. Такие ссылки позволяют получать доступ к реализациям членов интерфейса. В этом отношении прототип, определяемый членом интерфейса, выступает в роли виртуального функционального члена базового класса. Однако модификатор **virtual** нельзя использовать в декларации члена интерфейса. Так как допустимо наследование интерфейсов (мы его пока не рассматривали), то следует обратить внимание на невозможность появления в прототипе члена интерфейса и модификатора **override** (модификатор **new** допустим).

14.3. Реализация интерфейсов

Прежде чем рассмотреть реализацию интерфейса, уточним отличия интерфейсов от абстрактных классов. Наиболее важное отличие состоит в том, что при наследовании классов у каждого класса может быть только один базовый класс — множественное наследование классов в языке `C#` невозможно. При построении класса на основе интерфейсов их может быть любое количество. Другими словами, класс может реализовать сколько угодно интерфейсов, но при этом может иметь только один базовый класс. В интерфейсе не определяются поля и не может быть конструкторов. В интерфейсе нельзя объявлять статические члены.

Как и для абстрактных классов, невозможно определить объект с помощью интерфейса.

Чтобы интерфейсом можно было пользоваться, он должен быть реализован классом или структурой. В этой главе рассмотрим реализацию интерфейсов с помощью классов. Синтаксически отношение реализации интерфейса обозначается включением имени интерфейса в спецификацию базы класса. Напомним формат объявления класса со спецификацией базы (для простоты не указаны модификаторы класса):

```
class имя_класса спецификация_базы
{
    объявления_членов_класса
}
```

Спецификация базы класса в этом случае имеет вид:

```
: имя_базового_классаopt , opt список_интерфейсовopt
```

Имя базового класса (и следующая за ним запятая) могут отсутствовать. В списке интерфейсов через запятые помещаются имена тех интерфейсов, которые должен реализовать класс. В спецификацию базы класса может входить только один базовый класс и произвольное число имен интерфейсов. При этом должно выполняться обязательное условие — класс должен реализовать все члены всех интерфейсов, включенных в спецификацию базы. Частный случай — класс, реализующий только один интерфейс:

```
class имя_класса: имя_интерфейса
{
    объявления_членов_класса
}
```

Реализацией члена интерфейса является его полное определение в реализующем классе, *снабженное модификатором доступа public*.

Сигнатуры и типы возвращаемых значений методов, свойств и индексаторов в реализациях и в интерфейсе должны полностью совпадать.

Покажем на примерах, как при построении класса на основе интерфейса класс реализует его методы, свойства и индексаторы. Реализацию событий мы рассмотрим позднее в главе, посвященной событиям.

В следующей программе (14_01.cs) интерфейс IPublication реализуется классом Item — «заметка в газете».

```
// 14_01.cs – Интерфейс и его реализация
using System;
interface IPublication
{ // интерфейс публикаций
    void write(); // готовить публикацию
    void read(); // читать публикацию
    string Title { set; get; } // название публикации
}
class Item : IPublication
{ // заметка в газете
    string newspaper = "Известия"; // название газеты
    string headline; // заголовок статьи
    public string Title
    { // реализация свойства
        set { headline = value; }
        get { return headline; }
    }
    public void write()
    { // реализация метода
        /* операторы, имитирующие подготовку статьи */
    }
    public void read()
    { // реализация метода
        /* операторы, имитирующие чтение статьи */
        Console.WriteLine(@"Прочел в газете "{0}" статью "{1}"",
            newspaper, Title);
    }
}
```

```

}
class Program
{
    static void Main()
    {
        Console.WriteLine("Publication!");
        Item article = new Item();
        article.Title = "О кооперации";
        article.read();
    }
}

```

Результат выполнения программы:

Publication!

Прочел в газете "Известия" статью "О кооперации".

Класс Item кроме реализаций членов интерфейса включает объявления закрытых полей: newspaper (название газеты) и headline (заголовков статьи). Для простоты в класс не включен конструктор и только условно обозначены операторы методов write() и read(). Реализация свойства Title приведена полностью — аксессоры get и set позволяют получить и задать название статьи, представляемой конкретным объектом класса Item. В методе Main() нет ничего незнакомого читателю — определен объект класса Item и ссылка article на него. С помощью обращения article.Title задано название статьи.

В UML для изображения интерфейсов применяется та же символика, что и для классов (см. рис. 14.1). Конечно, имеется отличие — в верхней части, после имени интерфейса IPublication помещается служебное слово **Interface**. Тот факт, что класс реализует интерфейс, отображается с помощью специального символа и указанием имени интерфейса над прямоугольником, представляющим класс.

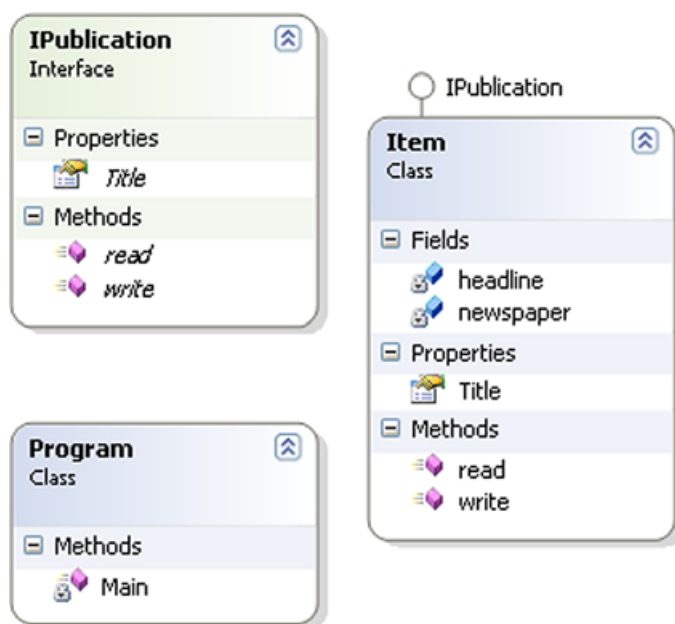


Рис. 14.1. Диаграмма классов с интерфейсом для программы 14_01.cs

В качестве второго примера рассмотрим интерфейс, реализация членов которого позволит получать целочисленные значения членов числовых рядов [9]:

```
interface ISeries
{
    void setBegin(); // восстановить начальное состояние
    int GetNext {get;} // вернуть очередной член ряда
    int this[int k] {get;} // вернуть k-й член ряда
}
```

Поясним роли прототипов, входящих в этот интерфейс. В приведенном объявлении: `setBegin()` — прототип метода; `GetNext` — имя свойства, позволяющего получить значение очередного члена ряда и настроить объект на следующий член. Индексатор в этом примере позволяет получить значение не очередного, а произвольного k -го члена ряда и перевести объект в состояние, при котором свойство `GetNext` вернет значение $(k+1)$ -го члена.

Те функциональные возможности, которые приписаны членам интерфейса `ISeries`, при реализации в конкретных классах могут быть изменены. Но в этом случае нарушается общий принцип применения интерфейсов. Поэтому в примерах конкретных классов будем придерживаться описанных соглашений о ролях членов интерфейса `ISeries`.

Интерфейс `ISeries` можно реализовать для представления разных числовых рядов. Вот, например, регулярные числовые последовательности, которые можно представить классами, реализующими интерфейс `ISeries`:

1, 1, 2, 3, 5, 8, 13, ... — ряд Фибоначчи: $a_i = a_{i-2} + a_{i-1}$, где $i > 2$; $a_1 = 1$, $a_2 = 1$;

1, 3, 4, 7, 11, 18, 29, ... — ряд Лукаса: $a_i = a_{i-2} + a_{i-1}$, где $i > 2$; $a_1 = 1$, $a_2 = 3$;

1, 2, 5, 12, 29, 70, ... — ряд Пелла: $a_i = a_{i-2} + 2 * a_{i-1}$, где $i > 2$; $a_1 = 1$, $a_2 = 2$.

В следующей программе на основе интерфейса `ISeries` определен класс, представляющий ряд Пелла (см. диаграмму на рис 14.2).

```
// 14_02.cs – Интерфейс и его реализация
using System;
interface ISeries // интерфейс числовых рядов
{
    void setBegin(); // задать начальное состояние
    int GetNext { get; } // вернуть очередной член ряда
    int this[int k] { get; } // вернуть k-й член ряда
}
class Pell : ISeries // Ряд Пелла: 1, 2, 5, 12, ...
{
    int old, last; // два предыдущих члена ряда
    public Pell() { setBegin(); } // конструктор
    public void setBegin() // задать начальное состояние
    { old = 1; last = 0; }
    public int GetNext // вернуть следующий после last
```

```

{
    get
    {
        int now = old + 2 * last;
        old = last; last = now;
        return now;
    }
}
public int this[int k] // вернуть k-й член ряда
{
    get
    {
        int now = 0;
        setBegin();
        if (k <= 0) return -1;
        for (int j = 0; j < k; j++)
            now = GetNext;
        return now;
    }
}
public void seriesPrint(int n)
{ // вывести n членов, начиная со следующего
    for (int i = 0; i < n; i++)
        Console.Write(GetNext + "\t");
    Console.WriteLine();
}
}
class Program
{
    static void Main()
    {
        Pell pell = new Pell();
        pell.seriesPrint(9);
        Console.WriteLine("pell[3] = " + pell[3]);
        pell.seriesPrint(4);
        pell.seriesPrint(3);
    }
}

```

Результат выполнения программы:

```

1 2 5 12 29 70 169 408 985
pell[3] = 5
12 29 70 169
408 985 2378

```

Кроме реализации членов интерфейса `ISeries` в классе `Pell` объявлен метод `seriesPrint()`. Он выводит значения нескольких членов ряда, следующих за текущим. Количество членов определяет аргумент метода `seriesPrint()`. После выполнения метода состояние ряда изменится — текущим членом станет последний выведенный член ряда. Обратите внимание, что при реализации индексатора нумерация членов ряда начинается с 1 (а не с нуля).

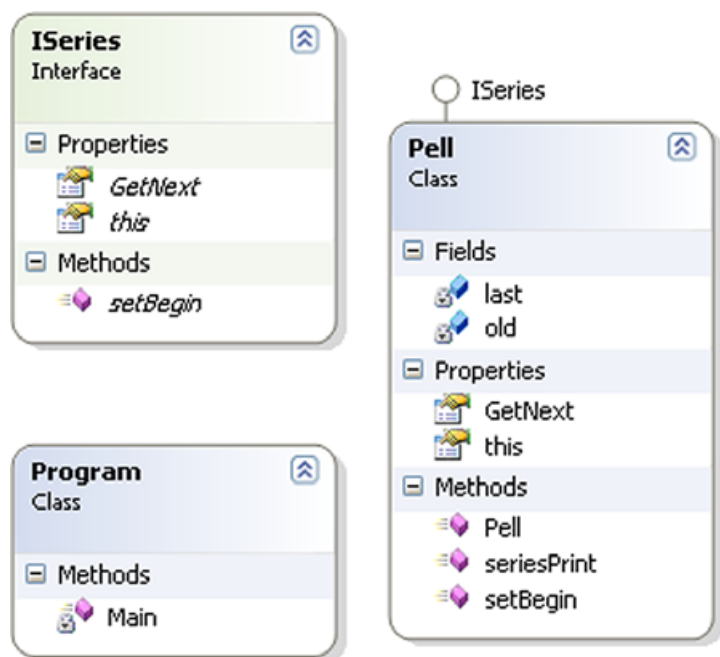


Рис. 14.2. Диаграмма классов с интерфейсом для программы 14_02.cs

Обратите внимание, что в реализации интерфейса заголовок метода должен полностью совпадать с заголовком прототипа этого метода в интерфейсе (за исключением появления модификатора доступа). Соответствие должно быть и при реализации других членов интерфейса.

В наших примерах интерфейсы и реализующие их классы размещены в одном файле. Если интерфейс объявлен не в том файле, где выполняется его реализация, то объявление интерфейса необходимо снабдить соответствующим модификатором доступа.

В приведенных выше программах классов `Item` и `Pell` была использована **неявная реализация** членов интерфейсов. Термин «неявная» употребляется для обозначения того, что в объявлении класса, реализующего интерфейс, не применяются квалифицированные имена членов интерфейса и их реализации снабжаются обязательным модификатором **public**.

Когда один класс реализует несколько интерфейсов, возможны совпадения имен членов из разных интерфейсов. Для разрешения такой конфликтной ситуации в классе, реализующем интерфейс, используется квалифицированное имя члена интерфейса. Здесь существует ограничение — такая реализация члена называется явной и она не может быть открытой, т. е. для нее нельзя использовать модификатор **public**. Подробнее об особенностях явной реализации интерфейсов можно узнать, например, из работ [1, 5].

14.4. Интерфейс как тип

Несмотря на то, что нельзя создавать экземпляры (объекты) интерфейсов, но интерфейс, как и класс, в языке C# является ссылочным типом. Подобно тому, как можно объявлять ссылку, имеющую тип

абстрактного класса, разрешено объявлять ссылку с типом интерфейса. Такая ссылка может быть равноправно связана с объектом любого класса, который реализовал данный интерфейс. С помощью такой ссылки можно получить доступ ко всем членам класса, реализующим соответствующие члены интерфейса. Однако ссылка с типом интерфейса не позволяет получить доступ к членам класса, которые отсутствовали в интерфейсе, а были добавлены в класс, реализующий интерфейс.

Интерфейс как тип может специфицировать параметр метода, может определять тип возвращаемого методом значения, может определять тип элементов массива. Рассмотрим эти возможности на иллюстративных примерах. Определим интерфейс, специфицирующий средства обработки геометрических фигур:

```
interface IGeo
{ // Интерфейс геометрической фигуры
  void transform(double coef); // преобразовать размеры
  void display();             // вывести характеристики
}
```

В приведенном интерфейсе нет сведений о конкретных особенностях тех геометрических фигур, классы которых могут реализовать IGeo. Например, ничто не ограничивает размерность пространства, в котором определены фигуры. Возможности преобразований фигур с помощью реализаций метода transform() могут быть достаточно произвольными. Самыми разными могут быть сведения о фигурах, выводимые реализациями метода display().

Определим два класса, реализующих интерфейс IGeo: Circle — круг и Cube — куб. Поле **double** rad в классе Circle — это радиус круга. Поле **double** rib в классе Cube — ребро куба. Реализация метода transform() в обоих классах изменяют линейные размеры (rad и rib) в заданное параметром метода число раз.

Чтобы продемонстрировать применимость интерфейсной ссылки в качестве параметра, определим статическую функцию report(), из тела которой выполняется обращение к реализации метода display(). Указанным соглашениям соответствует следующая программа:

```
// 14_03.cs - интерфейсные ссылки
using System;
interface IGeo
{ // интерфейс геометрической фигуры
  void transform(double coef); // преобразовать размеры
  void display();             // вывести характеристики
}
class Circle : IGeo
{ // круг
  double rad = 1; // радиус круга
  public void transform(double coef) { rad *= coef; }
  public void display()
  {
```

```

        Console.WriteLine("Площадь круга: {0:G4}",
            Math.PI * rad * rad);
    }
}
class Cube : IGeo
{ // куб
    double rib = 1; // ребро куба
    public void transform(double coef) { rib *= coef; }
    public void display()
    {
        Console.WriteLine("Объем куба: {0:G4}",
            rib * rib * rib);
    }
}
class Program
{
    public static void report(IGeo g)
    {
        Console.WriteLine("Данные объекта класса {0}:", g.GetType());
        g.display();
    }
    public static void Main()
    {
        Circle cir = new Circle();
        report(cir);
        Cube cub = new Cube();
        report(cub);
        IGeo ira = cir;
        report(ira);
    }
}

```

Результаты выполнения программы:

Данные объекта класса Circle:

Площадь круга: 3,142

Данные объекта класса Cube:

Объем куба: 1

Данные объекта класса Circle:

Площадь круга: 3,142

Обратим внимание на статистический метод `report()`. Его параметр `g` — ссылка с типом интерфейса `IGeo`. Выражение `g.GetType()` в теле метода `report()` позволяет получить имя класса, ссылка на который используется в качестве аргумента при обращении к методу `report()`. Оператор `g.display()` обеспечивает вызов той реализации метода `display()`, которая соответствует типу аргумента. Использование интерфейсной ссылки в качестве параметра позволяет применять метод `report()` для обработки объектов любых классов, которые реализовали интерфейс `IGeo`. Код метода `Main()` иллюстрирует сказанное. В нем определены два объекта классов `Circle` и `Cube` и ассоциированные с ними ссылки `cir`, `cub`. Их использование в качестве аргументов

метода `report()` приводит к вызовам методов `display()` из соответствующих классов. Это подтверждают результаты выполнения программ.

Обратите внимание на последние две строки тела функции `Main()`. Определена ссылка `ira` с типом интерфейса `IGeo`, которой затем присвоено значение ссылки на объект класса `Circle`. Это очень важная особенность — интерфейсной ссылке можно присвоить ссылку на объект любого класса, реализующего данный интерфейс. В теле метода `report()` аргумент в этом случае воспринимается как имеющий тип класса `Circle`, реализовавшего интерфейс `IGeo`. Остановимся на этом подробнее. Ссылка `IGeo ira` при объявлении имеет тип интерфейса, а после присваивания `ira = cir` ссылка `ira` получает тип класса `Circle`. Таким образом, наша программа иллюстрирует различие между **объявленным типом ссылки** и ее **типом времени исполнения** (о таком различии говорят как о **статическом** и **динамическом** типах одной и той же переменной).

Несмотря на то, что параметр метода `report()` специфицирован как ссылка типа `IGeo` и ссылка `ira` типа `IGeo` использована в качестве аргумента, вызов базируется на типе времени исполнения и выполняется обращение к объекту класса `Circle`. Такая возможность называется **поздним**, иначе **динамическим**, **связыванием**. Решение о том, какой метод вызывать, принимается при позднем связывании на основе типа времени исполнения. В противоположность этому, стратегия использования объявленного типа называется **ранним**, или **статическим**, **связыванием**.

Применение в качестве параметров и аргументов интерфейсных ссылок и ссылок с типом базового класса (при наличии виртуальных членов) обеспечивает позднее (динамическое) связывание. Позднее связывание — одно из проявлений **полиморфизма** в языке C#.

Чтобы продемонстрировать другие возможности применения интерфейсных ссылок, используем без изменений еще раз интерфейс `IGeo` и реализующие его классы `Circle` и `Cube`. В класс `Program` добавим статический метод:

```
public static IGeo mapping(IGeo g, double d)
{
    g.transform(d);
    return g;
}
```

Первый параметр — ссылка с типом интерфейса. Метод, получив в качестве первого аргумента ссылку на конкретный объект, изменяет его линейные размеры и возвращает в качестве результата ссылку на измененный объект. Второй параметр — коэффициент изменения линейных размеров. Используем методы `mapping()` и `report()` следующим образом (программа 14_04.cs):

```
public static void Main()
{
    IGeo ira = new Circle(); // единичный радиус
```

```

    report(ira);
    ira.transform(3);
    report(ira);
    ira = mapping(new Cube(), 2);
    report(ira);
}

```

Результаты выполнения программы:

```

Данные объекта класса Circle:
Площадь круга: 3,142
Данные объекта класса Circle:
Площадь круга: 28,27
Данные объекта класса Cube:
Объем куба: 8

```

В методе Main() ссылка ira с объявленным типом IGeo связана с объектом класса Circle, который она «представляет» в обращениях к статическому методу report() и в вызове нестатического метода transform(). Первый аргумент метода mapping() — вновь созданный объект класса Cube. По умолчанию у этого объекта-куба ребро равно 1. Результат выполнения статического метода mapping() присваивается ссылке ira, после чего она ассоциирована с измененным объектом класса Cube. Во всех использованиях интерфейсной ссылки ira проявляется ее динамический тип, и этот тип не остается постоянным во время исполнения программы. Результаты выполнения программы дополняют сказанное.

Так как интерфейс является типом, то можно определять массивы с элементами, имеющими тип интерфейса. Элементам такого массива можно присваивать как значения интерфейсных ссылок, так и значения ссылок на объекты любых классов, реализующих данный интерфейс. В следующей программе (14_05.cs) определен массив типа IGeo[] и именуемая его ссылка iarray. Элементам массива присваиваются значения ссылок на объекты классов Circle и Cube. Код без объявлений интерфейса, классов и статических методов report() и mapping():

```

public static void Main()
{
    IGeo[] iarray = new IGeo[4];
    IGeo ira = new Circle();
    iarray[0] = ira;
    ira.transform(3);
    iarray[1] = ira;
    ira = mapping(new Cube(), 2);
    iarray[2] = ira;
    iarray[3] = new Circle();
    foreach (IGeo obj in iarray)
        report(obj);
}

```

Результаты выполнения программы:

```

Данные объекта класса Circle:
Площадь круга: 28,27

```

Данные объекта класса Circle:
Площадь круга: 28,27
Данные объекта класса Cube:
Объем куба: 8
Данные объекта класса Circle:
Площадь круга: 3,142

В методе Main() элементам массива присвоены ссылки на объекты разных классов. Затем в операторе **foreach** с помощью ссылки типа IGeo перебираются значения всех элементов массива, и для каждого из них вызывается статический метод report(). Обратим внимание на выводимые результаты и последовательность присваивания значений элементам массива. Ссылка IGeo ira при объявлении адресует объект класса Circle, и ее значение присвоено элементу iarray[0]. Затем оператор ira.transform(3) изменяет объект, связанный со ссылкой ira, и ее значение присваивается элементу iarray[1]. Таким образом, значения элементов iarray[0] и iarray[1] равны, и оба элемента адресуют уже измененный объект класса Circle. Оператор ira = mapping(new Cube(), 2); присваивает интерфейсной ссылке ira адрес модифицированного объекта класса Cube. После этого присваивается значение элементу iarray[2]. Наконец элементу iarray[3] присваивается ссылка на новый объект класса Circle. Таким образом, в программе определены 3 объекта, адресуемые четырьмя элементами массива.

14.5. Интерфейсы и наследование

До сих пор мы рассматривали отдельные интерфейсы и их реализацию с помощью классов. Отмечали (но не иллюстрировали) возможность реализации одним классом нескольких интерфейсов. Интерфейсы С# могут наследоваться независимо от классов. Однако наследование интерфейсов отличается от наследования классов. Рассмотрим, что такое наследование интерфейсов и в чем его отличия от наследования классов.

Напомним, что в объявление интерфейса может входить спецификация базы интерфейса, формат которой можно представить так:

: список_базовых_интерфейсов_{опт}

В свою очередь интерфейс, входящий в список базовых, может быть наследником других интерфейсов. Тем самым формируются цепочки или иерархии наследования интерфейсов. Естественное ограничение — среди базовых интерфейсов не может присутствовать определяемый интерфейс.

При реализации интерфейса, который является наследником других интерфейсов, класс должен реализовать все члены всех интерфейсов, входящих в иерархию. Другими словами, все члены иерархии интерфейсов объединяются в единый набор членов, каждый из которых дол-

жен быть реализован конкретным классом (или конкретной структурой). Для иллюстрации приведем следующую программу:

```
// 14_06.cs - наследование интерфейсов
using System;
interface IPublication // интерфейс публикаций
{
    void write();           // готовить публикацию
    void read();           // читать публикацию
    string Title { set; get; } // название публикации
}
interface IBook : IPublication // интерфейс книг
{
    string Author { set; get; } // автор
    int Pages { set; get; }     // количество страниц
    string Publisher { get; }   // издательство
    int Year { get; set; }      // год опубликования
}
class Book : IBook
{
    string title;           // название книги
    string author;         // автор
    int pages;             // количество страниц
    string publisher;       // издательство
    int year;              // год опубликования
    public string Title
    {
        set { title = value; }
        get { return title; }
    }
    public string Author
    {
        set { author = value; }
        get { return author; }
    }
    public int Pages
    {
        set { pages = value; }
        get { return pages; }
    }
    public string Publisher
    {
        set { publisher = value; }
        get { return publisher; }
    }
    public int Year
    {
        set { year = value; }
        get { return year; }
    }
    public void write() { /* операторы метода */ }
    public void read() { /* операторы метода */ }
}
class Program
{
```

```

public static void Main()
{
    Book booklet = new Book();
    booklet.Author = "Л. Н. Волгин";
    booklet.Title = @""""Принцип согласованного оптимума"""";
    Console.WriteLine("Автор: {0}, Название: {1}.",
        booklet.Author, booklet.Title);
}
}

```

Результаты выполнения программы:

Автор: Л. Н. Волгин, Название: "Принцип согласованного оптимума".

Здесь интерфейс IBook построен на основе интерфейса IPublication. Класс Book реализует интерфейс IBook и в нем реализованы члены обоих интерфейсов. Чтобы не усложнять пример, реализации методов IPublication.write() и IPublication.read() приведены не полностью. Но удалить эти методы из класса Book нельзя — класс должен реализовывать все члены всех интерфейсов, на основе которых он построен. В методе Main() выполняются простейшие действия с объектом класса Book.

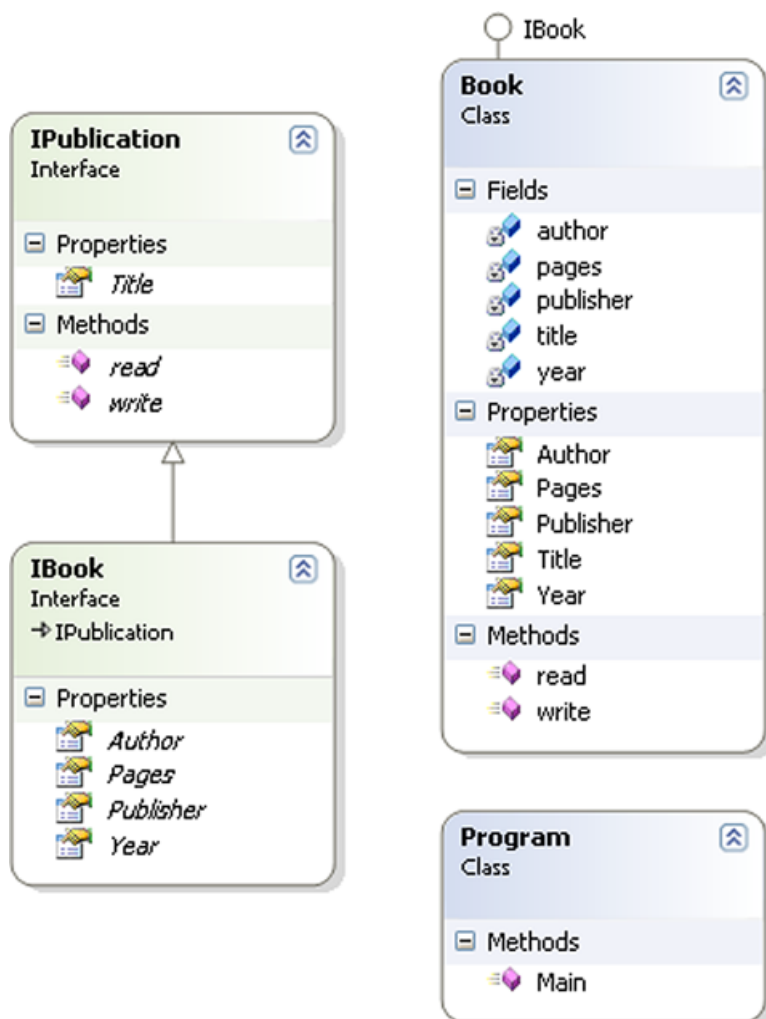


Рис. 14.3. Наследование интерфейсов в программе 14_06.cs

Для изображения наследования интерфейсов в UML используется та же нотация, что и для наследования классов. Производный интерфейс соединяется с базовым стрелкой, острие которой направлено на изображение базового интерфейса. На рис. 14.3 показано отношение интерфейсов IPublication и IBook, а над классом Book отображен тот факт, что класс Book реализует интерфейс IBook.

При наследовании интерфейсов разрешено множественное наследование, при этом возможно и построение ромбовидных решетчатых иерархий. Так как в интерфейсах любой член представляет только спецификацию еще не существующего метода, свойства, индексатора или события, то никаких коллизий не возникает. Покажем и поясним это на условном примере. В следующей программе построена ромбовидная иерархия интерфейсов:

```
// 14_07.cs - множественное наследование интерфейсов
using System;
interface IBase
{
    void write();
}
interface IN1 : IBase { }
interface IN2 : IBase { }
interface INt : IN1, IN2 { }
class Class : INt
{
    public void write() { /* операторы метода */ }
}
class Program
{
    public static void Main() { }
}
```

Два интерфейса IN1 и IN2 построены на базе интерфейса IBase. У интерфейса INt базовыми являются IN1 и IN2. Класс с именем Class реализует интерфейс INt и тем самым должен реализовать и все остальные интерфейсы. Самое важное — тот факт, что класс получает для реализации только один экземпляр члена write(). Несмотря на то, что write() как член базового класса наследуется двумя интерфейсами IN1 и IN2, в интерфейс INt он входит только один раз и только один раз должен быть реализован в классе Class. Сказанное иллюстрирует (рис. 14.4) диаграмма классов программы.

При построении иерархии интерфейсов возможно совпадение имен и сигнатур членов разных интерфейсов. Если сигнатуры методов не совпадают, т. е. прототипы методов интерфейсов заведомо разные, то ничего предпринимать не нужно — класс должен реализовать все одноименные члены, имеющие разные сигнатуры. В случае совпадения сигнатур член производного интерфейса экранирует (скрывает) одноименный член базового интерфейса. В этом случае компилятор потребует подтверждения правильности экранирования. Для этого

член производного интерфейса нужно снабдить модификатором **new**. При экранировании класс не должен реализовать член базового интерфейса (достаточно реализации экранирующего члена производного интерфейса).

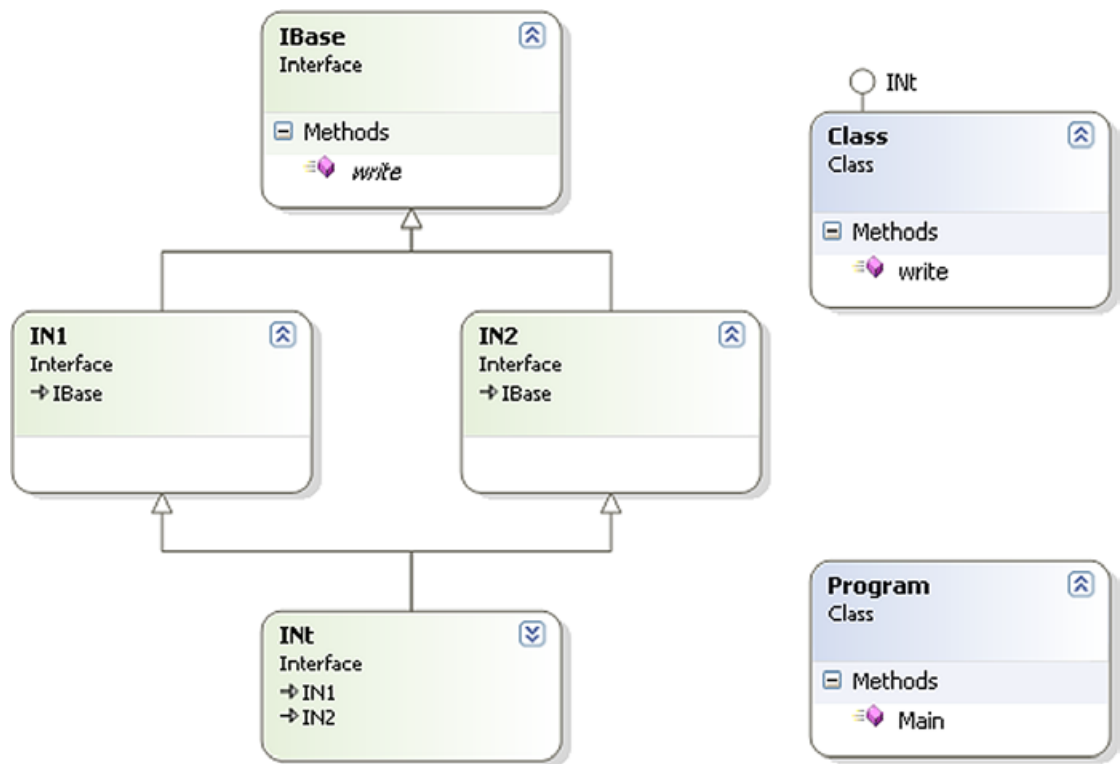


Рис. 14.4. Множественное наследование интерфейсов в программе 14_07.cs

Следующая программа иллюстрирует приведенные правила:

```
// 14_08.cs - множественное наследование интерфейсов
using System;
interface IBase
{
    void write();
}
interface IN1 : IBase
{
    new void write();
}
interface IN2 : IBase
{
    int write(int r);
}
interface INT : IN1, IN2 { }
class Class : INT
{
    public void write()
    { Console.WriteLine("IN1"); }
    public int write(int r)
    { Console.WriteLine("IN2 " + r); return 1; }
}
class Program
```

```

{
    public static void Main()
    {
        Class dot = new Class();
        dot.write();
        dot.write(23);
    }
}

```

Результаты выполнения программы:

```

IN1
IN2 23

```

В базовом интерфейсе IBase есть прототип метода write(). В производных интерфейсах IN1, IN2 декларированы одноименные методы с разной сигнатурой. Прототип метода из IN1 экранирует прототип метода базового интерфейса. Прототип метода из IN2 существует независимо от других одноименных прототипов. Класс Class, реализующий всю иерархию интерфейсов, реализует как перегруженные (overloaded) два метода, декларированные в интерфейсах IN1, IN2. Результаты выполнения программ и диаграмма классов и интерфейсов (рис. 14.5) дополняют приведенные объяснения. Обратите внимание на обозначение метода write() в изображении класса Class. В подписи указано, что существует две реализации: (+ 1 overloaded).

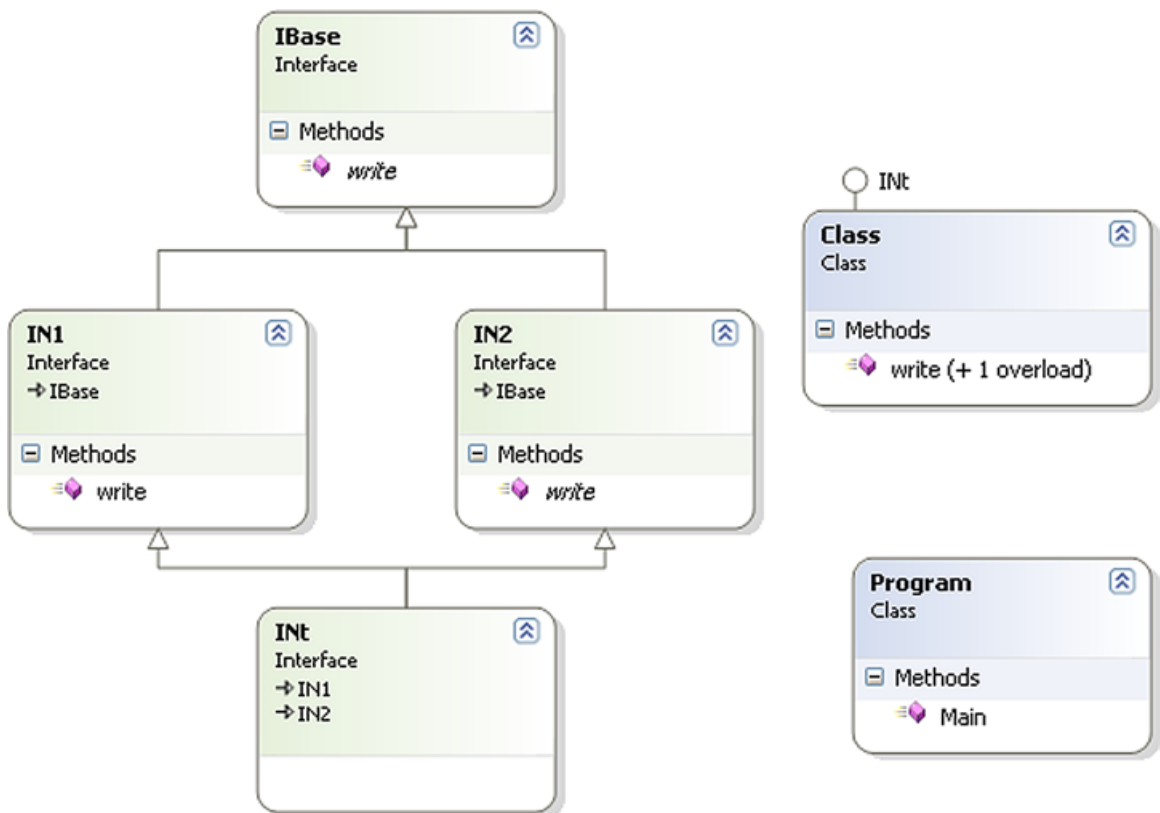


Рис. 14.5. Множественное наследование интерфейсов с перегрузкой и экранированием членов интерфейсов

Контрольные вопросы и задания

1. Что такое наследование реализации?
2. Что такое наследование специфицированной функциональности?
3. Какие механизмы C# обеспечивают реализацию наследования специфицированной функциональности?
4. Что такое интерфейс?
5. Какие объявления могут входить в декларацию интерфейса?
6. В чем отличия интерфейса от абстрактного класса?
7. Как проявляется принцип полиморфизма при использовании интерфейсов.
8. Что такое прототип метода и где прототипы используются?
9. Назовите правила реализации классом интерфейса.
10. Что такое принцип подстановки Барбары Лисков?
11. Можно ли объявить интерфейс с модификатором **static**?
12. Что такое спецификация базы интерфейса?
13. Какие модификаторы допустимы для члена интерфейса?
14. Какой статус доступа имеют члены интерфейса?
15. Приведите формат объявления свойства в интерфейсе.
16. Какие поля допустимы в объявлении интерфейса?
17. В чем различия прототипа метода в абстрактном классе от прототипа метода в интерфейсе?
18. В чем различия и сходства интерфейса и абстрактного класса?
19. Что такое реализация члена интерфейса?
20. Является ли интерфейс типом?
21. К какому виду типов относится интерфейс?
22. Доступ к каким членам класса, реализующего интерфейс, обеспечивает ссылка с типом интерфейса?
23. Как с помощью интерфейсов обеспечивается динамическое связывание?
24. Что такое наследование интерфейсов?

Глава 15

ПЕРЕЧИСЛЕНИЯ И СТРУКТУРЫ

15.1. Перечисления

В предыдущих главах рассматривались только ссылочные пользовательские типы — *классы* и *интерфейсы*. Однако программист может объявлять и использовать собственные типы значений. Эту возможность предоставляют *перечисления* и *структуры*. Напомним, что переменная с типом значения однозначно представляет конкретные данные. Самое главное для программиста — невозможность связать с одним участком памяти (т. е. с одним элементом данных) несколько переменных с типом (или типами) значений (вопрос о том, что для переменных с типами значений память выделяется в стеке, мы уже обсуждали). Итак, и перечисления, и структуры дают возможность объявлять пользовательские типы значений. Начнем с перечислений.

Перечисление — особый тип значений, определяющий набор именованных целочисленных констант. В отличие от других типов значений, в перечислении не существует методов, свойств и событий. Формат объявления перечисления:

```
модификаторы_перечисленияопт enum имя_перечисления  
: базовый_тип  
{ список_перечисления }
```

Имя_перечисления — выбранный программистом идентификатор, который становится именем нового типа.

Список_перечисления — список неповторяющихся имен констант, для каждой из которых указано или задано по умолчанию инициализирующее выражение (значения инициализирующих выражений могут совпадать).

enum — служебное слово.

Конструкция `":базовый_тип"` может быть опущена, и тогда константы списка имеют тип **int**. В качестве базового типа явно можно использовать любой целый тип (**int**, **long**, ...), кроме **char**.

В качестве модификаторов перечислений могут использоваться **new**, **public**, **protected**, **internal**, **private**.

В качестве примера введем тип перечисления с именем *НомерПланеты*, объявляющее константы, значениями которых служат номера наиболее известных малых планет.

```
enum НомерПланеты : uint
{
    Церера = 1, Паллада = 2, Юнона = 3, Веста = 4,
    Эрос = 433, Гидальго = 944, Ганимед = 1036,
    Амур = 1221, Икар = 1566
}
```

Номера планет (по астрономическому каталогу) — это целые положительные числа, поэтому в качестве базового типа выбран тип констант `uint`. В списке перечисления *НомерПланеты* имена констант — это названия планет, и каждой константе приписано конкретное значение — номер планеты по каталогу.

Если для констант перечисления не заданы инициализирующие выражения, то первой константе присваивается нулевое значение, а каждая следующая получает значение на 1 больше. Если в списке перечисления отсутствует инициализация для не первой константы, то она получает значение на 1 больше, нежели предыдущая. Операндами инициализирующего выражения могут быть только константы. Значение инициализирующего выражения должно принадлежать области допустимых значений базового типа перечисления.

В инициализирующее выражение могут входить другие константы перечисления. Например, тот же список перечисления *НомерПланеты*, можно определить так:

```
enum НомерПланеты : uint
{
    Церера = 1, Паллада, Юнона=Веста-Церера, Веста=4,
    Эрос = 433, Гидальго = 944, Ганимед = 1036,
    Амур = 1221, Икар = 1566
}
```

Константы перечислений подобны статическим членам классов. Внутри списка перечисления к ним можно обращаться по именам констант. Обращение к константе перечисления вне списка выполняется с помощью квалифицированного имени:

```
имя_перечисления.имя_константы_из_списка
```

Как многократно повторялось, все типы языка C# являются производными от базового класса `System.Object`. Это относится и к перечислениям. Поэтому к ним применимы уже рассмотренные нами методы класса `object`. Однако непосредственным (прямым) базовым классом для перечислений служит класс `System.Enum`, который построен на базе класса `object`. В классе `System.Enum` методы класса `object` переопределены с тем, чтобы максимально приблизиться к задачам, решаемым с помощью перечислений. Например:

```
string str = НомерПланеты.Икар.ToString();
```

Значением `str` будет строка "Икар" (а вовсе не "1566", что соответствовало бы значению константы).

Хотя константы перечисления имеют целочисленные значения, однако их типом служит то конкретное перечисление, которому они принадлежат. Если применить к константе перечисления метод `GetType()`, то строка-результат будет иметь вид:

```
"имя_класса+имя_перечисления"
```

Здесь указывается имя того класса, в котором определено перечисление. Обратите внимание, что разделяет или соединяет имена знак «+». Например, если перечисление `НомерПланеты` объявлено в классе `Program` (`15_01.cs`), то значением выражения

```
НомерПланеты.Веста.GetType()
```

будет строка

```
"Program+НомерПланеты"
```

При конкатенации со строкой квалифицированное имя

```
имя_перечисления.имя_константы_из_списка
```

трактруется как строка вида «имя_константы_из_списка». Например, значением выражения:

```
"имя планеты: " + НомерПланеты.Амур
```

будет строка:

```
"имя планеты: Амур"
```

Чтобы получить при конкатенации со строкой целочисленное значение константы перечисления, необходимо явное приведение типа. Например, значением выражения:

```
НомерПланеты.Юнона + " имеет номер " + (int)НомерПланеты.Юнона
```

будет строка:

```
"Юнона имеет номер 3"
```

В тоже время при использовании констант перечисления в арифметических выражениях они воспринимаются как целочисленные значения. Например:

```
long res = НомерПланеты.Ганимед - НомерПланеты.Гидальго;
```

значением переменной `res` будет 92.

Так как **enum** вводит тип, то разрешено определять переменные этого типа. Такая переменная имеет право принимать значения именованных целочисленных констант, введенных соответствующим перечислением. В этом случае переменная приобретает свойства этих констант. Например, рассмотрим последовательность операторов:

```

НомерПланеты number = НомерПланеты.Амур;
string str = number.ToString();
Console.WriteLine("str: " + str);
long res = number - НомерПланеты.Веста;

```

Значением str будет "Амур", значением переменной res будет 1217.

К константам перечисления и переменным с типом перечисления применимы: операции сравнения (`==`, `!=`, `<`, `>`, `<=`, `>=`), бинарная арифметическая операция `-`, поразрядные операции (`^`, `&`, `|`, `~`) и `sizeof`. К переменным с типом перечисления применимы унарные операции `++`, `--`.

Обратите внимание, что к константам перечисления нельзя применять бинарные арифметические операции `+`, `/`, `*`, `%` и унарные операции `-` и `+`. Таким образом, нельзя выполнить суммирование или деление двух констант перечисления.

Поэтому для использования значений констант в арифметических выражениях необходимо выполнить явное приведение их типов. Например, так:

```

number = (НомерПланеты)((int) НомерПланеты.Церера +
(int) НомерПланеты.Юнона);

```

После выполнения этого оператора переменная number примет значение константы перечисления со значением 4 и именем "Веста".

Чтобы присвоить переменной с типом перечисления значение (даже того типа, который является для перечисления базовым), необходимо явное приведение типов. Например, так:

```

number = (НомерПланеты)944;

```

Переменная number примет значение константы Гидальго.

Если переменной с типом перечисления присвоить значение, которое отсутствует в списке именованных констант, то имя представляемой переменной константы совпадет с ее значением. Например:

```

number = (НомерПланеты)999;

```

Переменная number примет значение константы перечисления с именем «999» и значением 999.

Операции автоизменений (декремент `--` и инкремент `++`) позволяют так изменять значение переменной перечисления, что она принимает и значения, отсутствующие среди именованных констант перечисления. Принимая такие значения, переменная перечисления играет роль литерала — ее имя совпадает с представленным значением.

В следующей программе введена переменная number с типом перечисления НомерПланеты, значением которой в цикле становятся константы Церера, Паллада, Юнона, Веста. Затем переменная number принимает значения, отсутствующие в перечислении.

```

// 15_04.cs - переменная перечисления

```

```

using System;
class Program
{
    enum НомерПланеты : uint
    {
        Церера = 1, Паллада = 2, Юнона = 3, Веста = 4,
        Эрос = 433, Гидальго = 944, Ганимед = 1036,
        Амур = 1221, Икар = 1566
    }
    static void Main()
    {
        НомерПланеты number;
        Console.WriteLine("Номера планет: ");
        for (number = НомерПланеты.Церера;
            number <= НомерПланеты.Веста + 2; number++)
            Console.WriteLine(number + " = " + (int)number);
    }
}

```

Результат выполнения программы:

```

Номера планет:
Церера = 1
Паллада = 2
Юнона = 3
Веста = 4
5 = 5
6 = 6

```

Члены (константы) перечислений и переменные с типами перечислений можно использовать в качестве аргументов методов и в метках переключателей. В метке переключателя член перечисления применяется в качестве константного выражения. В следующей программе статический метод `данныеОпланете()` имеет параметр с типом перечисления `НомерПланеты`. Метод выводит в консольное окно некоторые сведения о планетах с конкретными номерами.

```

// 15_05.cs - переменная перечисления в переключателе
using System;
class Program
{
    enum НомерПланеты : uint
    {
        Церера = 1, Паллада = 2, Юнона = 3, Веста = 4,
        Эрос = 433, Гидальго = 944, Ганимед = 1036,
        Амур = 1221, Икар = 1566
    }
    static void данныеОпланете(НомерПланеты p)
    {
        switch (p)
        {
            case НомерПланеты.Амур:
                Console.WriteLine("Номер: {0}, название: {1}, "
                    + "диаметр: {2}", (int)p, p, 1.5);

```

```

        break;
    case НомерПланеты.Веста:
        Console.WriteLine("Номер: {0}, название: {1}, "
            + "диаметр: {2}", (int)p, p, 380);
        break;
    // Про остальные планеты из экономии ничего не пишем...
}
}
static void Main()
{
    НомерПланеты number;
    number = НомерПланеты.Веста;
    данныеОпланете(number);
    данныеОпланете(НомерПланеты.Амур);
}
}

```

Результат выполнения программы:

```

Номер: 4, название: Веста, диаметр: 380
Номер: 1221, название: Амур, диаметр: 1,5

```

В функции Main() определена переменная number с типом перечисления НомерПланеты. Ей присвоено значение константы перечисления Веста и переменная служит аргументом при обращении к методу данныеОпланете(). При втором обращении к тому же методу аргумент — квалифицированное имя другой константы перечисления.

15.2. Базовый класс перечислений

Основой перечислений служит системный класс Enum, из пространства имен System. Методы этого класса позволяют в ряде случаев существенно упростить работу с перечислениями. Рассмотрим только некоторые статические методы:

public static Type GetUnderlyingType (Tun_Перечисления);
возвращает базовый тип перечисления. Аргументом должен быть объект класса System.Type, представляющий перечисления.

public static string GetName (Tun_Перечисления, значение);
возвращает имя константы перечисления, соответствующей второму параметру. Второй параметр может быть либо переменной с типом перечисления, либо литеральным представлением значения константы. Если в списке перечисления несколько констант с одинаковыми значениями, то выбирается имя самой левой или явно инициализированной. Если значение отсутствует в списке — возвращается пустая строка.

public static string Format (Tun_Перечисления, значение, формат);
Второй параметр — переменная с типом перечисления либо значение константы. В зависимости от формата возвращается либо имя, либо значение константы. Если формат «G» или «g», то метод подобен GetName — возвращает имя константы. Если формат «X» или «x» — воз-

возвращается шестнадцатеричное представление значения константы. При формате «D» или «d» возвращается десятичное представление.

public static Array GetValues (*Tun_Перечисления*);

Метод возвращает массив значений всех констант перечисления. Константы, независимо от их размещения в перечислении, в массиве размещаются по возрастанию значений. Обратите внимание, что тип результата — System.Array.

public static string [] GetNames (*Tun_Перечисления*);

Метод возвращает массив имен констант перечисления в порядке возрастания значений констант.

public static bool IsDefined (*Tun_Перечисления, значение*);

возвращает **true**, если в перечислении присутствует константа со значением, равным второму параметру.

В каждом из приведенных методов класса Enum первый параметр должен представлять тип перечисления (а не его имя). Поэтому в обращениях к этим методам в качестве первого аргумента нужно использовать объект класса System.Type, представляющий тип перечисления. Зная имя перечисления, его тип можно получить с помощью операции **typeof()**.

Следующая программа иллюстрирует особенности и возможности применения методов базового класса Enum.

```
// 15_06.cs - перечисления, System.Enum
using System;
class Program
{
    enum НомерПланеты : uint
    {
        Церера = 1, Паллада = 2, Юнона = 3, Веста = 4,
        Эрос = 433, Гидальго = 944, Ганимед = 1036,
        Амур = 1221, Икар = 1566
    }
    static void Main()
    {
        Console.WriteLine("Базовый тип перечисления: " +
            Enum.GetUnderlyingType(typeof(НомерПланеты)));
        Console.WriteLine("Значения констант перечисления:");
        foreach (uint i in Enum.GetValues(typeof(НомерПланеты)))
            Console.Write(" " + i);
        Console.WriteLine("\nИмена констант: ");
        foreach (string s in Enum.GetNames(typeof(НомерПланеты)))
            Console.WriteLine("\n\t" + s);
        Console.WriteLine("\nЗначение 433u имеет константа "
            + Enum.Format(typeof(НомерПланеты), 433u, "G"));
    }
}
```

Результат выполнения программы:

Базовый тип перечисления: System.UInt32
Значения констант перечисления:

1 2 3 4 433 944 1036 1221 1566

Имена констант:

Церера

Паллада

Юнона

Веста

Эрос

Гидальго

Ганимед

Амур

Икар

Значение 433и имеет константа Эрос

В программе нужно обратить внимание на операцию **typeof**, которая формирует по имени типа НомерПланеты объект класса System.Type, представляющий тип операнда.

Так как каждое перечисление это тип, то его декларация может помещаться в том же пространстве, где размещены определения других типов, например, класса с методом Main(). Более того, к перечислениям применима директива импорта типов **using static**. Покажем эти возможности на примере:

```
// 15_Test - Перечисление как импортируемый тип
using static System.Console;
using static НомерПланеты;
enum НомерПланеты : uint
{
    Церера = 1, Паллада = 2, Юнона = 3, Веста = 4,
    Эрос = 433, Гидальго = 944, Ганимед = 1036,
    Амур = 1221, Икар = 1566
}
class Program
{
    static void Main()
    {
        WriteLine("{0} = {1}", Ганимед, (int)Ганимед);
    }
}
```

Результат выполнения программы:

Ганимед = 1036

В приведенной программе директива **using static** используется дважды — для импорта системного консольного класса и для импорта перечисления «НомерПланеты». Это позволяет в методе Main() обратиться к статическому методу WriteLine() консольного класса без указания имени класса и обозначать элементы перечисления, опуская имя перечисления:

```
WriteLine("{0} = {1}", Ганимед, (int)Ганимед);
```

15.3. Структуры

Структуры во многом похожи на классы, но имеют и существенные отличия. Как и класс, структура вводит тип. Однако, в отличие от классов, структуры не могут участвовать в наследовании, и поэтому в декларации структуры отсутствует спецификация базы.

Определение (иначе декларация или объявление) структуры (структурного типа) имеет следующий формат (указаны не все части полного формата):

```
модификаторы_структурыopt  
struct имя_структурного_типа интерфейсы_структурыopt  
тело_структуры
```

В декларации структуры **struct** — служебное слово, *имя_структурного_типа* — выбранный программистом идентификатор, определяющий имя структурного типа. *Тело_структуры* — заключенная в фигурные скобки последовательность объявлений (деклараций) членов структуры. Модификаторов структуры меньше чем модификаторов класса. Могут использоваться: **new**, **public**, **protected**, **internal**, **private**.

Модификаторы определяют статус доступа структурного типа, и они практически все уже рассмотрены в связи с классами.

Структуры не участвуют в наследовании, однако могут служить реализацией интерфейсов, перечисленных в декларации после имени структурного типа. Элемент декларации *интерфейсы_структуры_{opt}* — это список интерфейсов, перед которым помещено двоеточие.

Членами структуры могут быть: явно декларируемые константы, поля, методы, свойства, индексаторы, события, операции, конструкторы экземпляров с параметрами, статический конструктор, деструкторы и вложенные типы. В отличие от классов, членом структуры не может быть финализатор (деструктор).

Структурные типы всегда наследуются от `System.ValueType`, который, в свою очередь, является производным классом от `System.Object`. Поэтому в структурах присутствуют уже рассмотренные ранее открытые и защищенные методы класса **object**. Напомним основные из них: `ToString()`, `GetHashCode()`, `Equals()`, `GetType()`.

Существенным отличием структур от классов является то, что класс вводит тип ссылок, а структурный тип — тип значений. Таким образом, каждой переменной структурного типа соответствует конкретный экземпляр структуры. И, присвоив новой переменной значение другой переменной, уже связанной с экземпляром структурного типа, мы получим еще одну копию экземпляра структуры.

Прежде чем привести пример, отметим еще некоторые отличия структур от классов. Во-первых, в объявлении поля структурного типа нельзя использовать инициализатор. Во-вторых, в каждое объявление структурного типа автоматически встраивается конструктор умолчания без параметров. Назначение этого конструктора — инициализи-

ровать все члены структуры нулевыми значениями, которые соответствуют их типам по умолчанию. В-третьих, в структурном типе нельзя явно объявить конструктор умолчания без параметров. В-четвертых, в структурных типах недопустимы виртуальные члены и члены с модификатором **protected**. В-пятых, конструктор с параметрами должен явно присвоить значения *всем полям* экземпляра структуры.

В качестве иллюстрации возьмем класс «точка на плоскости» PointC и эквивалентную ему структуру (структурный тип) PointS:

```
// 15_07.cs – структуры и классы "точка на плоскости"
using System;
class PointC // класс
{
    double x = 10, y = 20;
    public PointC() { y -= 5; }
    public double X { get { return x; } set { x = value; } }
    public double Y { get { return y; } set { y = value; } }
}
class Program
{
    static void Main()
    {
        PointC pc = new PointC();
        PointC pc1 = pc;
        pc1.X = 10.2;
        Console.WriteLine("Из объекта: X={0}; Y={1}", pc.X, pc.Y);
        PointS ps = new PointS();
        PointS ps1 = ps;
        ps1.X = 10.2;
        Console.WriteLine("Из структуры: X={0}; Y={1}", ps.X, ps.Y);
    }
}
struct PointS // структурный тип
{
    // double x = 10, y = 20; // Error!
    // public PointS() {y -= 5;} // Error!
    double x, y;
    public double X { get { return x; } set { x = value; } }
    public double Y { get { return y; } set { y = value; } }
}
```

Результат выполнения программы:

```
Из объекта: X=10,2; Y=15
Из структуры: X=0; Y=0
```

В классе PointC выполняется явная инициализация полей x, y. Явно определенный конструктор умолчания PointC() изменяет значение поля y.

В структурах инициализация полей невозможна и недопустимо присутствие явного определения конструктора умолчания. Попытки нарушить эти правила в структурном типе PointS отмечены в тексте программы комментариями *// Error!*

В программе `ps` — имя ссылки, ассоциированной с объектом класса `PointC`. В противоположность этому `ps` — имя переменной, значением которой является экземпляр (структура) типа `PointS`. Объявив еще одну ссылку `ps1` типа `PointC` и присвоив ей значение `ps`, мы связываем две ссылки с одним объектом класса `PointC`. Квалифицированные имена `ps.X` и `ps1.X` обеспечивают доступ к одному и тому же свойству одного и того же объекта класса `PointC`.

Выполнение присваивания `PointS ps1 = ps;` приводит к созданию копии структуры, представляемой переменной `ps`. Таким образом, после выполнения присваивания квалифицированные имена `ps.X` и `ps1.X` именуют свойства независимых друг от друга экземпляров одного структурного типа `PointS`.

Поясним отсутствие в структурах финализатора (деструктора). Финализатор — это метод, который автоматически вызывается всякий раз, когда сборщик мусора убирает более не используемый в программе объект из той части памяти, которую называют кучей. Так как для объектов с типами значений память выделяется не в куче, а в стеке, а сборка мусора на стек не распространяется, то финализатор для структур не нужен.

Как сказано, в каждый структурный тип неявно автоматически включается конструктор без параметров, инициализирующий нулевыми значениями все поля создаваемого экземпляра структуры. Поэтому при создании экземпляров структур удобно использовать инициализаторы экземпляров. Форма такого инициализатора та же, что и для классов:

```
new имя_типа(){список_инициализаторов_членов},
```

где каждый из инициализаторов члена записывается в таком виде:

```
имя_члена = инициализирующее_выражение
```

Обычно инициализируют поля и свойства структур. Для инициализации член структуры должен быть доступен в точке определения экземпляра структуры (т. е. должен быть объявлен с модификатором **public**).

Для типа `PointS` из предыдущего примера объявление экземпляра структуры с его инициализацией может быть, например, таким:

```
PointS ps = new PointS(){X = 66, Y = 99};
```

Копирование структуры выполняется не только при выполнении присваивания переменных структурного типа, но и при их использовании в качестве передаваемых по значениям параметров методов. Следующая программа иллюстрирует особенности применения структур в методах.

```
// 15_08.cs - структуры и методы
using static System.Console;
struct PointS
```

```

{ // структура
  double x, y;
  public double X { get { return x; } set { x = value; } }
  public double Y { get { return y; } set { y = value; } }
}
class Program
{
  static PointS reverse(PointS dot)
  {
    dot.X = -dot.X;
    dot.Y = -dot.Y;
    return dot;
  }
  static void Main()
  {
    PointS one = new PointS() { X = 12, Y = 8 };
    PointS two = reverse(one);
    WriteLine("one.X={0}; one.Y={1}", one.X, one.Y);
    WriteLine("two.X={0}; two.Y={1}", two.X, two.Y);
    ReadLine();
  }
}

```

Результат выполнения программы:

```

one.X=12; one.Y=8
two.X=-12; two.Y=-8

```

В программе объявлен структурный тип `PointS`. В нем два закрытых поля (`x`, `y`), два ассоциированных с этими полями открытых свойства (`X`, `Y`). В классе `Program` — два статических метода. Первый из них, `reverse()`, получает в качестве передаваемого по значению параметра структуру, изменяет с помощью свойств значения ее полей `x` и `y` и возвращает структуру как результат в точку вызова метода.

В методе `Main()` объявлены две переменные, `PointS one` и `PointS two`. Первая из них связана с инициализированной структурой. Вторая получает в качестве значения результат, возвращаемый методом `reverse(one)`.

Очень важен тот факт, что после выполнения метода `reverse()`, аргументом которого служит объект, ассоциированный со ссылкой `one`, значение этого объекта не изменилось. Это подтверждает тот факт, что при использовании структуры в качестве передаваемого по значению параметра или возвращаемого методом значения создается копия структуры, в дальнейшем полностью независимая от оригинала.

Если в рассмотренной программе заменить структуру на класс (т. е. заменить `struct PointS` на `class PointC`), то результат выполнения программы будет таким:

```

one.X=-12; one.Y=-8
two.X=-12; two.Y=-8

```

Тот же самый результат будет получен, если параметр структурного типа будет передаваться по ссылке. Для этого в нашей программе нужно следующим образом изменить заголовок метода:

```
static PointS reverse(ref PointS dot).
```

Соответствующим образом изменится вызов метода:

```
two = reverse(ref one);
```

Как уже сказано, в объявлении структурного типа нельзя инициализировать его поля. Кроме того, нельзя явным образом определить конструктор без параметров. Все поля объектов структурного типа без вмешательства извне инициализируются значениями по умолчанию, соответствующими типам полей. Например, для арифметических типов это нули, для ссылок — **null**. Например, объявим ссылку на массив структур и определим массив:

```
PointS[] ar = new PointS[50];
```

Элементами массива в этом примере станут структуры, для которых поля *x* и *y* будут иметь нулевые значения.

Переменную с типом структуры можно создать без явного обращения к конструктору и без применения операции **new**. Однако такая переменная в этом случае создается как неинициализированная и ей нужно присвоить подходящее ей значение (структуру того же типа).

Следующая программа иллюстрирует приведенные сведения о структурах.

```
// 15_09.cs - структуры, конструктор умолчания
using static System.Console;
struct PointS
{ // структурный тип
    double x, y;
    public double X { get { return x; } set { x = value; } }
    public double Y { get { return y; } set { y = value; } }
}
class Program
{
    static void Main()
    {
        PointS[] ar = new PointS[50];
        WriteLine("ar[0].X={0}; ar[0].Y={1}", ar[0].X, ar[0].Y);
        PointS three = new PointS() { Y = 2.34 };
        WriteLine("three.X={0}; three.Y={1}", three.X, three.Y);
        PointS four; // неинициализированная переменная!
        four = three;
        WriteLine("four.X={0}; four.Y={1}", four.X, four.Y);
    }
}
```

Результат выполнения программы:

```
ar[0].X=0; ar[0].Y=0  
three.X=0; three.Y=2,34  
four.X=0; four.Y=2,34
```

Обратим внимание, что в инициализаторе экземпляра структуры не обязательно инициализируются все члены структуры. Для иллюстрации этого правила посмотрите в программе на следующее объявление структуры:

```
PointS three = new PointS() {Y = 2.34};
```

Так как структура вводит тип значений, то использование структур вместо классов в ряде случаев повышает быстродействие программ и уменьшает объем занимаемой памяти. Связано это с тем, что доступ к объекту класса выполняется косвенно, а обращение к структуре — непосредственно. Имя экземпляра структуры является именем переменной, значением которой служит объект структурного типа.

В то же время при использовании структуры в качестве параметра, передаваемого по значению, или возвращаемого методом значения необходимы временные затраты на копирование всех полей структуры.

15.4. Упаковка и распаковка

Когда значение структурного типа преобразуется к типу **object** или приводится к типу того интерфейса, который реализован структурой, выполняется операция упаковки (*boxing*). Эта операция выполняется автоматически и не требует вмешательства программиста.

Упаковкой называют процесс явного преобразования из типа значений в тип ссылок. При упаковке создается и размещается в куче объект, которому присваивается значение объекта с типом значения. Возвращаемым значением при выполнении упаковки служит ссылка на объект кучи.

Обратной операцией является *распаковка* (*unboxing*), при которой значение объекта присваивается переменной с типом значения.

Автоматическая упаковка выполняется в тех случаях, когда выполняется допустимое присваивание ссылке на объект ссылочного типа переменной с типом значения. Так как все классы языка C# имеют общий базовый класс **object**, то ссылке типа **object** можно присвоить значение экземпляра структуры. В этом случае выполняется автоматическая упаковка, не требующая вмешательства программиста. Обратная процедура — распаковка — автоматически не выполняется. Для распаковки необходимо применять операцию приведения типов.

Сказанное относится не только к присваиванию, но и к передаче параметров и к возвращаемому методом результату. Рассмотрим статический метод с таким заголовком:

```
static object reDouble(object obj)
```

Метод принимает ссылку на объект типа **object** и возвращает значение того же типа. Внешне ничто не препятствует применению в обращении к этому методу в качестве аргумента ссылки на объект любого типа. Однако в теле метода необходимо учитывать конкретный тип аргумента и формировать возвращаемый результат в соответствии с этим типом. В следующей программе определен структурный тип Struct1 с полем x типа **double** и метод с приведенным выше заголовком.

```
// 15_10.cs - структуры, упаковка, распаковка
using System;
struct Struct1 // структурный тип
{
    double x;
    public double X { get { return x; } set { x = value; } }
}
class Program
{
    static object reDouble(object obj)
    {
        if (obj is Struct1)
        {
            Struct1 st = (Struct1)obj;
            st.X = 2 * st.X;
            return st;
        }
        else
            Console.WriteLine("Неизвестный тип!");
        return obj;
    }
    static void Main()
    {
        Struct1 one = new Struct1();
        one.X = 4;
        Struct1 two = (Struct1)reDouble(one);
        Console.WriteLine("one.X={0}; two.X={1}", one.X, two.X);
        Console.WriteLine("(int)reDouble(55)={0}", (int)reDouble(55));
    }
}
```

Результат выполнения программы:

```
one.X=4; two.X=8
Неизвестный тип!
(int)reDouble(55)=55
```

Метод reDouble() обрабатывает только аргументы типа Struct1, хотя ему можно передать аргумент любого типа. Если аргумент имеет тип Struct1, метод reDouble() выполняет его распаковку и удваивает значение поля **double** x. Если тип аргумента отличен от Struct1, то тип распознается как неизвестный и аргумент возвращается в точку вызова в «упакованном» виде. Для примера в методе Main() обращение к reDouble() выполнено дважды с аргументами разных типов.

Понимание процедур упаковки и распаковки необходимо для применения таких библиотечных средств как коллекции. К ним относится класс `ArrayList` (массив-список) из пространства имен `System.Collections`. Объект класса `ArrayList` во многих случаях «ведет себя» как массив. Например, к нему применима индексация. В отличие от массивов, производных от класса `Array`, объекты класса `ArrayList` могут расти в процессе выполнения программы. Количество их элементов увеличивается, как только в этом возникает необходимость, причем рост выполняется автоматически без вмешательства программиста.

Но не все просто. Рассмотрим объявление такого растущего массива-списка:

```
using System.Collection;  
...  
ArrayList dinamo = new ArrayList(3);
```

В данном примере объявлена ссылка `dinamo` и ассоциированный с нею объект класса `ArrayList`. В обращении к конструктору `ArrayList()` указан начальный размер массива-списка. Можно предположить, что теперь можно обращаться к элементам массива-списка, используя индексы со значениями 0, 1, 2. Однако следующая попытка будет ошибочной:

```
dinamo [1] = 45.3; // ошибка времени исполнения
```

Даже если в вызове конструктора указан размер массива-списка, первоначально элементов в создаваемом массиве-списке НЕТ! Отличие объекта класса `ArrayList` от традиционного массива состоит в том, что в массив-список элементы должны быть вначале занесены с помощью нестатического метода `Add()` класса `ArrayList`.

Заголовок метода:

```
public virtual int Add(object value)
```

Метод добавляет в конец массива-списка элемент, заданный аргументом. Возвращаемое значение — порядковый номер (индекс) добавленного элемента. Нумерация элементов начинается с нуля.

Так как тип параметра **object**, то в качестве аргумента можно использовать значение любого типа. Следовательно, в один массив-список можно помещать элементы разных типов. Процедура упаковки, необходимая для аргументов с типами значений, выполняется автоматически. А вот при получении значения элемента массива-списка нужно явно выполнить распаковку, узнав предварительно, какой тип имеет элемент.

В следующей программе создан массив-список, представляемый ссылкой `ArrayList` `dinamo`. Затем в этот массив-список добавлены элементы со значениями типов **double**, **int** и `PointS`, где `PointS` — пользовательский тип, объявленный в программе как структурный. Текст программы:

```
// 15_11.cs - структуры и массив-список типа ArrayList
using System;
using System.Collections; // Для ArrayList
class Program
{
    static void Main()
    {
        ArrayList dinamo = new ArrayList();
        dinamo.Add(4.8);
        dinamo.Add(new PointS());
        dinamo.Add(100);
        PointS ps = new PointS();
        ps.X = 10.2;
        dinamo.Add(ps);
        dinamo[1] = 1.23;
        foreach (object ob in dinamo)
        {
            if (ob is PointS)
                Console.WriteLine("Struct: X={0}; Y={1}",
                    ((PointS)ob).X, ((PointS)ob).Y);
            else
                if (ob is Double)
                    Console.WriteLine("Double: Value={0}",
                        ((double)ob).ToString());
        }
    }
}
struct PointS // структурный тип
{
    double x, y;
    public double X { get { return x; } set { x = value; } }
    public double Y { get { return y; } set { y = value; } }
}
```

Результат выполнения программы:

```
Double: Value=4,8
Double: Value=1,23
Struct: X=10,2; Y=0
```

В методе Main() в массив-список `dinamo` занесены четыре разнотипных элемента. Обратите внимание, как с помощью выражения с индексацией выполнено присваивание второму элементу массива-списка:

```
dinamo[1] = 1.23;
```

Перед этим присваиванием значением элемента `dinamo[1]` был объект структуры `PointS`.

После размещения в массиве-списке `dinamo` четырех элементов эти элементы перебираются в цикле **foreach**. Параметр цикла `ob` имеет тип **object**. Ему последовательно присваиваются ссылки на разнотипные элементы массива-списка. Непосредственно использовать параметр типа **object** для доступа к объектам разных типов невозможно — у каждого типа своя архитектура, т. е. свои члены, свои поля. В теле цикла условные операторы, предназначенные для распознавания типов. Рас-

познаются только типы **double** и **PointS**. В качестве проверяемого условия в операторах **if** используется выражение с операцией **is**.

15.5. Реализация структурами интерфейсов

Применение одного базового класса с виртуальными членами позволяет единообразно (но, возможно, по-разному) обрабатывать объекты разных типов, каждый из которых является производным от базового и переопределяет его виртуальные члены.

Такой возможности для экземпляров разнотипных структур нет, так как структуры не наследуются. Однако структуры могут реализовывать интерфейсы. Если несколько структурных типов реализуют один и тот же интерфейс, то к объектам этих разных структур применимы методы (а также свойства, индексаторы и события) интерфейса. Тем самым интерфейсы позволяют единообразно обрабатывать объекты разных структурных типов. Для этого ссылка на интерфейс используется в качестве параметра, вместо которого могут подставляться аргументы структурных типов, реализующих данный интерфейс. Второй пример — применение коллекций (частный случай — массивов), элементы которых имеют разные структурные типы. Создание такого массива возможно, если он объявлен с типом интерфейса, реализованного всеми структурными типами.

В качестве примера определим интерфейс с прототипами свойств:

```
interface IShape
{
    double Volume {get;} // объем
    double Area {get;}   // площадь
}
```

Реализовать такой интерфейс можно с помощью разных классов и структур. Например, параллелепипед имеет площадь поверхности (**Area**) и объем (**Volume**). Те же свойства имеются у любой трехмерной геометрической фигуры. Реализовать тот же интерфейс можно и в классе или структуре двумерных фигур. В этом случае объем будет равен, например, нулю.

Определим статический метод с заголовком:

```
static void information (IShape sh)
```

Его назначение — вывести данные об объекте, ссылка на который используется в качестве аргумента. Тип параметра — это интерфейс, поэтому метод сможет обрабатывать объекты любых типов, реализующих интерфейс **IShape**.

В следующей программе определены два структурных типа, реализующих интерфейс **IShape**. Первый из них **Circle** — представляет круги с заданными значениями радиусов, второй **Sphere** — сферы с задан-

ными значениями радиуса. Метод `information()` выводит сведения об аргументе. Текст программы:

```
// 15_12.cs - структуры и интерфейсы
using System;
interface IShape
{ // интерфейс
    double Volume { get; } // объем
    double Area { get; } // поверхность
}
struct Circle : IShape // Круг
{
    public double radius;
    public Circle(double radius) // конструктор
    { this.radius = radius; }
    public double Area
    { get { return Math.PI * radius * radius; } }
    public double Volume { get { return 0; } } // объем
}
struct Sphere : IShape // Сфера
{
    public double radius;
    public Sphere(double radius) // конструктор
    { this.radius = radius; }
    public double Area // поверхность
    { get { return 4 * Math.PI * radius * radius; } }
    public double Volume // объем
    { get { return 4 * Math.PI * radius * radius * radius / 3; } }
}
class Program
{
    static void information(IShape sh)
    {
        Console.Write(sh.GetType());
        Console.WriteLine(":\\t Area={0,5:f2};\\t "
            + "Volume={1,5:f2}", sh.Area, sh.Volume);
    }
    static void Main()
    {
        Circle ci = new Circle(25);
        information(ci);
        Sphere sp = new Sphere(4);
        information(sp);
    }
}
```

Результат выполнения программы:

```
Circle: Area=1963,50; Volume= 0,00
Sphere: Area=201,06; Volume=268,08
```

В методе `Main()` созданы объекты структур `Circle` и `Sphere`, которые в качестве аргументов передаются методу `information()`. Обратите внимание на отсутствие приведения типов в теле метода `information()`.

Если бы его параметр имел тип **object**, то необходимо было бы выполнять преобразование к типу конкретного аргумента, т. е. в методе необходимо было бы знать тип аргумента и использовать его имя в операции приведения типов.

Как и классы, структура может реализовать одновременно несколько интерфейсов. Покажем на примере, как можно использовать эту возможность. Сначала объявим интерфейс такого вида:

```
interface IImage
{
    void display();
    double Measure {get;}
    double BaseSize {set;}
}
```

Члены этого интерфейса могут быть реализованы по-разному, т. е. им можно придать самый разный смысл. Пусть свойство `Measure` — это максимальный линейный размер («размах») геометрической фигуры; свойство `BaseSize` — базовый линейный размер; `display()` — прототип метода, который выводит сведения о типе, реализовавшем интерфейс, и значения свойств `Measure`, `BaseSize` конкретного объекта. Такими типами в нашем примере будут структурные типы `Cube` и `Square`, представляющие, соответственно, объекты «куб» и «квадрат». Для куба базовый размер — ребро куба, максимальный размер — наибольшая диагональ. Для квадрата базовый размер — сторона квадрата, максимальный размер — его диагональ.

Предположим, что экземпляры этих структур мы хотим разместить в массиве и упорядочить элементы массива по убыванию значений свойства `Measure`. Для сортировки элементов массива можно применить метод `Array.Sort()`. Этот метод предполагает, что элементы массива сравниваются друг с другом с помощью метода `CompareTo()`. Прототип этого метода размещен в интерфейсе `IComparable` из пространства имен `System`. Метод `CompareTo()` уже определен для таких типов как **int**, **char**, **string** и т. д. Однако для пользовательских типов, которыми будут структурные типы `Cube` и `Square`, этот метод нужно определять явно. Поэтому реализуем в указанных структурах интерфейс `IComparable`. Тем самым в каждом из этих структурных типов с необходимостью появится такой нестатический метод:

```
public int CompareTo (object obj)
{
    if (Measure < ((IImage)obj).Measure) return +1;
    if (Measure == ((IImage)obj).Measure) return 0;
    else return -1;
}
```

Обратите внимание, что тип **object** параметра `obj` приводится к типу интерфейса `IImage`, который должны иметь элементы массива.

Напомним, что в коде метода `Array.Sort()` выполняются многократные обращения к методу `CompareTo()`, где при каждом обращении сравниваются характеристики двух элементов сортируемого массива. Если характеристика (в нашем примере свойство `Measure`) вызывающего элемента-объекта находится в «правильном» отношении к характеристике объекта-параметра, то метод `CompareTo()` должен возвращать отрицательное значение. При нарушении «порядка» элементов возвращается положительное значение. Для элементов, одинаковых по характеристике сравнения, возвращается значение 0.

Программа с указанными структурами может быть такой:

```
// 15_13.cs - структуры и интерфейсы
using System;
interface IImage
{
    void display();
    double Measure { get; }
    double BaseSize { set; }
}
struct Cube : IImage, IComparable // Куб
{
    double rib; // ребро – базовый размер
    public double Measure // максимальный линейный размер
    { get { return Math.Sqrt(3 * rib * rib); } }
    public double BaseSize { set { rib = value; } }
    public void display()
    {
        string form = "Размеры куба: ребро={0,7:f3}; размах={1,7:f3}";
        Console.WriteLine(form, rib, Measure);
    }
    public int CompareTo(object obj)
    {
        if (Measure < ((IImage)obj).Measure) return +1;
        if (Measure == ((IImage)obj).Measure) return 0;
        else return -1;
    }
}
struct Square : IImage, IComparable // Квадрат
{
    double side; // сторона – базовый размер
    public double Measure // максимальный размер
    { get { return Math.Sqrt(2 * side * side); } }
    public void display()
    {
        string form = "Размеры квадрата: сторона={0,7:f3};
размах={1,7:f3}";
        Console.WriteLine(form, side, Measure);
    }
    public double BaseSize { set { side = value; } }
    public int CompareTo(object obj)
    {
        if (Measure < ((IImage)obj).Measure) return +1;
        if (Measure == ((IImage)obj).Measure) return 0;
    }
}
```

```

        else return -1;
    }
}
class Program
{
    static void Main()
    {
        Cube cube = new Cube();
        cube.BaseSize = 5;
        Square sq = new Square();
        sq.BaseSize = 5;
        Cube cube1 = new Cube();
        cube1.BaseSize = 7;
        IImage[] arIm = new IImage[] { cube, sq, cube1 };
        Array.Sort(arIm);
        foreach (IImage memb in arIm)
            memb.display();
    }
}

```

Результаты выполнения программы:

Размеры куба: ребро= 7,000; размах= 12,124

Размеры куба: ребро= 5,000; размах= 8,660

Размеры квадрата: сторона= 5,000; размах= 7,071

В методе Main() определены два экземпляра структуры Cube и один экземпляр структуры Square. С помощью свойства BaseSize заданы значения базовых размеров структур. Объявлен и инициализирован массив типа IImage[]. Ссылка на него arIm использована в качестве аргумента метода Array.Sort(). Цикл **foreach** перебора элементов коллекции (в нашем примере перебираются элементы массива) последовательно обращается через ссылку memb типа IImage ко всем элементам упорядоченного массива. Для каждого элемента вызывается метод display().

Контрольные вопросы и задания

1. Как можно определить свой тип значений?
2. Приведите формат объявления перечисления.
3. Что такое базовый тип перечисления?
4. Что такое список перечисления?
5. Как инициализируются константы перечисления?
6. Приведите правила обращения к константам перечисления.
7. Какой тип имеет константа перечисления?
8. Когда константа перечисления воспринимается как значение с базовым типом перечисления?
9. Назовите операции, применимые к константам перечислений.
10. Назовите операции, не применимые к константам перечислений.
11. Где допустимо применять константы перечисления?

12. Назовите статические методы типов перечислений.
13. Как можно получить тип перечисления?
14. В чем различия структурных типов и классов?
15. Назовите допустимые модификаторы членов структурных типов.
16. Назовите допустимые модификаторы структурных типов.
17. Почему в структурных типах отсутствует финализатор?
18. Объясните особенности копирования структур.
19. Что называют упаковкой?
20. Когда выполняется упаковка при работе со структурами?
21. Объясните особенности и возможности класса ArrayList.
22. К каким структурам применимы одинаковые методы?
23. Что определяет интерфейс, реализованный структурным типом?
24. В каком интерфейсе размещен прототип метода CompareTo()?
25. Какой метод используется по умолчанию в методе ArraySort()
для сравнения элементов сортируемого массива?

Глава 16

ИСКЛЮЧЕНИЯ

16.1. 0 механизме исключений

Язык C#, как и некоторые предшествующие ему языки программирования, включает механизм генерации и обработки исключений.

Прежде чем объяснить, что такое исключения и каковы средства для работы с ними, необходимо обратить внимание на отличие ошибок в программе от особых ситуаций, которые могут возникнуть при ее выполнении. Ошибка в программе — это производственный брак, допущенный при проектировании или кодировании программы. Синтаксические ошибки позволяет выявить компилятор, и до их устранения программа неработоспособна. Семантические или логические ошибки выявляются в процессе отладки и тестирования исполняемой программы. И синтаксические, и логические ошибки должны быть устранены до завершения разработки программы. Механизм исключений к ним не имеет никакого отношения.

Однако такое событие, как выход индекса за граничную пару при попытке обращения к элементу массива, зачастую предугадать (выявить) на этапе компиляции нельзя. Подобным образом невозможно предсказать, что пользователь при работе с готовой программой вместо числа введет последовательность нечисловых символов. И в первом, и во втором случае программа не может продолжать правильное исполнение и оба случая непредсказуемы (не выявляются) на этапе компиляции. Но при разработке программы можно предусмотреть в ней возможность реагирования на такие особые события. Эту возможность обеспечивает механизм исключений.

Прежде чем перейти к рассмотрению этого механизма, обратим внимание на тот факт, что особые ситуации, которые могут возникнуть в процессе выполнения программы, делятся на две группы: **синхронные** и **асинхронные**.

Асинхронные ситуации возникают за счет внешних воздействий на программу, происходящих в моменты времени, никак не связанные с каким-либо участком кода программы. Примерами могут служить отключение питания компьютера, команда на перезагрузку операционной системы, вредоносные воздействия на код программы со стороны параллельных процессов и т. д. Исключения не позволяют защитить программу от воздействия асинхронных событий (асинхронных ситуаций).

Синхронные особые ситуации создаются, точнее, могут возникнуть, только при выполнении тех или иных фрагментов кода программы. Каждый такой фрагмент кода представляет собой последовательность конкретных операторов текста программы. Самое важное то, что при разработке программы зачастую известно, в каком участке текста программы может возникнуть синхронная особая ситуация.

Например, при вводе данных пользователь может по ошибке ввести текстовую информацию вместо числовой. При записи данных в файл может не хватить места на внешнем носителе информации. В первом случае источник возникновения особой ситуации — оператор чтения данных из входного потока, во втором — процедура записи информации в файл.

Именно для реагирования на синхронные особые ситуации, возникающие в процессе выполнения программы, предназначен механизм исключений. Так как асинхронные особые ситуации мы рассматривать не будем, то термин «синхронные» больше не будем добавлять и применительно к исключениям говорить будем просто об особых ситуациях.

Особых ситуаций, если они потенциально возможны, избежать нельзя. Но программист может предусмотреть в программе операторы для распознавания особой ситуации и для реакции на нее.

До появления механизма исключений реакция на особые ситуации зачастую была самой жесткой — программа выдавала сообщение об ошибке и завершалась аварийно. В лучшем случае, если особая ситуация возникла по вине пользователя, ему предлагалось, например, повторно и правильно ввести исходные данные, и программа повторяла выполнение тех операторов, где возникла и была распознана особая ситуация...

Исключения позволяют отделить распознавание особой ситуации от реакции на нее. В механизме исключений есть два этапа — генерация исключения и обработка исключения. Генерация исключения выполняется в том месте, где возникла и обнаружена особая ситуация, а обработка — там, где это удобно в конкретном случае.

Если бы современные программы создавались по старинке, без использования стандартных или специализированных библиотек (подпрограмм, процедур, функций, классов), то без механизма исключений можно было бы обойтись. Более того, прародитель С-образных языков — язык С обходится без исключений. Но не будем упрекать в этом его авторов. При создании языка С не был еще внедрен в практику программирования объектно-ориентированный подход.

В настоящее время автор библиотечной функции или метода класса обязательно предусматривает операторы, распознающие особую ситуацию. Однако он не может предугадать, как будет использована эта функция или метод программистом-пользователем.

Например, в библиотечном методе чтения данных из стандартного входного потока нельзя при ошибке в данных просто выдать сообщение пользователю «повтори ввод!». Входной поток может быть настроен

на текстовый файл и программа при этом выполняется без ввода данных с клавиатуры. Пользователь в этом случае не участвует в диалоге с программой.

Таким образом, распознавание особой ситуации в библиотечном методе должно быть «оторвано» от действий по устранению причин ее появления. Эту возможность обеспечивают исключения.

16.2. Системные исключения и их обработка

В языке C# исключение — это объект класса `System.Exception` или производного от него класса. В пространство имен `System` входят кроме `Exception` еще несколько классов исключений, которыми можно пользоваться в программах. В табл. 16.1 приведены исключения из пространства `System`, соответствующие особым ситуациям в программах на C#.

Таблица 16.1

Классы системных исключений

Имя исключения	Описание ситуации
<code>ArgumentException</code>	Недопустимое значение аргумента при вызове метода
<code>ArithmeticException</code>	Особые ситуации при выполнении арифметических операций. Например, <code>DivideByZeroException</code> или <code>OverflowException</code>
<code>ArrayTypeMismatchException</code>	Попытка присвоить элементу массива значение, тип которого не совместим с типом элементов массива
<code>DivideByZeroException</code>	Попытка деления на нуль целочисленного значения
<code>FormatException</code>	Несоответствие параметров спецификации форматирования
<code>IndexOutOfRangeException</code>	Выход индекса массива за пределы граничной пары (индекс отрицателен или больше верхней границы)
<code>InvalidCastException</code>	Неверное преобразование от базового типа или базового интерфейса к производному типу
<code>NullReferenceException</code>	Попытка применить ссылку со значением null для обращения к объекту
<code>OutOfMemoryException</code>	Неудачная попытка выделить память с помощью операции new (недостаточно свободной памяти)
<code>OverflowException</code>	Переполнения в арифметических выражениях, выполняемых в контексте, определенном служебным словом checked
<code>RankException</code>	Несоответствие размерностей параметра и аргумента при вызове метода

Имя исключения	Описание ситуации
StackOverflowException	Переполнение рабочего стека. Возникает, когда вызвано слишком много методов, например, при очень глубокой или бесконечной рекурсии
TypeInitializationException	Необработанное исключение в статическом конструкторе.

Исключения определены не только в пространстве имен System. В других пространствах имеются исключения, которые относятся к соответствующим разделам библиотек среды исполнения .NET. Например, в пространстве System.Drawing.Printing имеются исключения, соответствующие особым ситуациям вывода на печать и т. д.

Исключение как объект создается с помощью специального оператора генерации (посылки) исключения:

```
throw выражение;
```

Однако этот оператор мы рассмотрим позже. Гораздо важнее сейчас научиться распознавать появление исключений и обрабатывать эти исключения. Дело в том, что при выявлении особых ситуаций методы библиотечных классов, которыми мы уже пользуемся в программах, посылают исключения, и нужно уметь их распознавать и обрабатывать.

Достаточно часто начинающий программист наталкивается на исключения, возникающие из-за неверного ввода данных при выполнении совершенно правильной программы. В качестве примера рассмотрим такой фрагмент кода (программа 16_01.cs):

```
double x;  
Console.Write("x = ");  
x = double.Parse(Console.ReadLine());  
Console.WriteLine("res = " + x);
```

Если при европейской настройке операционной среды пользователь в ответ на приглашение "x =" введет, например, 3.3, т. е. допустит ошибку, отделив дробную часть вещественного числа не запятой, а точкой, или введет вместо цифры букву, то программа завершится аварийно и выдаст такое сообщение о необработанном исключении:

```
Необработанное исключение: System.FormatException:  
Входная строка имела неверный формат в System.Double.Parse(String s)
```

В этом сообщении об исключении указано, что его источник — метод System.Double.Parse(string s).

Для перехвата и обработки исключений используется конструкция **try/catch/finally** . Она состоит из двух или трех частей. Первая обязательная часть — это оператор **try**, называемый блоком контроля за возникновением исключений. Оператор **try** представляет собой заключенную в фигурные скобки последовательность операторов языка C#,

перед которой размещается служебное слово **try**. Непосредственно за фигурной скобкой, закрывающей блок контроля за исключениями, размещается последовательность **catch**-блоков — ловушек (иначе называемых обработчиками) исключений, а также необязательный блок завершения (**finally**-блок). Каждый обработчик исключений вводится служебным словом **catch**. Имеются следующие формы **catch**-блока:

```
catch (тип_исключения имя) фильтр_исключений {операторы}
catch (тип_исключения имя) {операторы}
catch (тип_исключения) {операторы}
catch {операторы}
```

Входящий в ловушку исключений блок операторов предназначен для выполнения действий по обработке захваченного исключения.

Блок завершения имеет такой формат:

```
finally {операторы}
```

Операторы блока завершения выполняются после выхода из набора **catch**-блоков. Точнее, блок **finally** выполняется всегда независимо от того, выполнены или нет в конструкции **try/catch/finally** обработчики событий.

Определены три формы конструкции **try/catch/finally**:

- блок контроля, за которым следуют **catch**-обработчики (один или несколько);
- блок контроля, за которым непосредственно следует блок завершения (**finally**-блок);
- блок контроля, за которым следуют **catch**-обработчики (один или несколько), за которыми размещен блок завершения (**finally**-блок).

В качестве примера применения конструкции **try/catch** дополним приведенную выше последовательность операторов, которая аварийно завершается при неверно введенных данных, средствами для обработки исключений типа `System.FormatException`. Код станет таким:

```
double x;
while (true)
{
    try
    {
        Console.Write("x = ");
        x = double.Parse(Console.ReadLine());
        Console.WriteLine("res = " + x);
    }
    catch (FormatException)
    {
        Console.WriteLine("Ошибка в формате данных!");
        continue;
    }
    break;
}
```

В программу добавлен цикл **while**, выход из которого возможен только при достижении в конце тела цикла оператора **break**. В теле цикла — блок контроля за возникновением исключений, где размещены три приведенные выше оператора:

```
Console.Write("x = ");  
x = double.Parse(Console.ReadLine());  
Console.WriteLine("res = " + x);
```

Во втором из них возможна генерация исключения `System.FormatException`. Если оно появляется, то управление передается за пределы блока контроля (третий оператор пропускается). Ловушка (обработчик) исключений с заголовком `catch(FormatException)` настроена на обработку исключений типа `FormatException`. В теле обработчика два оператора. Первый из них выдает на консоль сообщение, оператор **continue**; передает управление на начало следующей итерации цикла. Цикл не будет завершен, пока пользователь не введет значение `x` без ошибок. Диалог пользователя с программой может быть, например, таким:

```
x = 3.5<ENTER>  
Ошибка в формате данных!  
x = 3d5<ENTER>  
Ошибка в формате данных!  
x = 3,5<ENTER>  
res = 3,5
```

Обратите внимание, что в **try**-блоке при возникновении исключения оператор `Console.WriteLine("res = " + x);` не выполняется.

Порядок действий в конструкции **try/catch/finally** следующий. Выполняются операторы блока контроля за исключениями. Если исключений не возникло, то все **catch**-обработчики пропускаются. При возникновении исключения управление незамедлительно передается набору **catch**-блоков. Обработчики исключений просматриваются последовательно до тех пор, пока не будет обнаружен обработчик, «настроенный» на переданное исключение. После выполнения операторов этого обработчика выполняется блок завершения (если он есть) и только затем заканчивается исполнение конструкции **try/catch/finally**. Отметим, что среди набора блоков обработки выполняется только один или не выполняется ни один.

Обратите внимание, что обработка исключения не прекращает выполнения программы. В блоках **catch** могут быть запрограммированы действия по устранению причин появления исключений и программа продолжает выполняться.

16.3. Свойства исключений

Каждое исключение — это объект либо класса `System.Exception`, либо производного от него. В классе `Exception` есть свойства, которые можно использовать при обработке исключений. Вот два из них:

- `Message` — текстовое описание ситуации, при которой создано исключение;

- `Source` — имя объекта или приложения, пославшего исключение.

Прежде чем привести пример использования этих свойств, еще раз обратимся к предыдущей программе, где использован **catch**-блок для исключения `FormatException`. Если пользователь введет синтаксически правильное числовое значение, лежащее вне диапазона представимых в системе чисел, то возникает ситуация, при которой будет сгенерировано исключение, отличное от `FormatException`. Например, в результате ввода:

```
x = 1e999<ENTER>
```

произойдет аварийное завершение программы с такой выдачей сообщения:

```
Необработанное исключение: System.OverflowException:  
Значение было недопустимо малым или недопустимо большим для Double.
```

Чтобы защитить программу от аварийного завершения при возникновении исключений этого типа, можно включить в нее еще один обработчик исключений с заголовком:

```
catch (OverflowException)
```

Более общим решением является дополнение программы **catch**-блоком, настроенным на перехват всех исключений типа `ArithmeticException`, относящихся к обработке арифметических данных.

Чтобы получить доступ к свойствам перехваченного исключения, необходимо в заголовке **catch**-блока в скобках вслед за типом исключения поместить имя, которое будет представлять исключение в блоке обработки. Предыдущая программа с учетом сказанного может быть такой:

```
try  
{  
    Console.Write("x = ");  
    x = double.Parse(Console.ReadLine());  
    Console.WriteLine("res = " + x);  
}  
catch (FormatException ex)  
{  
    Console.WriteLine("ex.Message=" + ex.Message);  
    Console.WriteLine("ex.Source=" + ex.Source);  
    continue;  
}  
catch (ArithmeticException ex)  
{  
    Console.WriteLine("ex.Message=" + ex.Message);  
    Console.WriteLine("ex.Source=" + ex.Source);  
    continue;  
}
```

После **try**-блока в коде два обработчика. Первый «настроен» только на исключения типа `FormatException`. Второй перехватывает исключения типа `ArithmeticException` к которым относится и `OverflowException`. В каждом из обработчиков выводятся значения свойств `Message` и `Source`, а затем управление с помощью операторов **continue** передается следующей итерации цикла. Результаты выполнения программы могут быть такими:

```
x = 1e999<ENTER>
ex.Message=Значение было недопустимо малым или
недопустимо большим для Double.
ex.Source=mcorlib
x = qwer<ENTER>
ex.Message=Входная строка имела неверный формат.
ex.Source=mcorlib
x = 4.0<ENTER>
ex.Message=Входная строка имела неверный формат.
ex.Source=mcorlib
x = 4,0<ENTER>
res = 4
```

Обратите внимание, что сообщения (значения свойства `Message`) различны для разных типов исключений. Источник генерации исключений во всех примерах один — базовая библиотека Microsoft (`mcorlib`).

Примечание. При анализе исключения в **catch**-блоке полезно вывести значение выражения `ex.ToString()`. В нем содержится информация как о самом исключении, так и о точке его генерации в коде программы

Еще одна форма заголовка **catch**-блока, в котором есть имя захваченного исключения, предусматривает «фильтрацию» перехваченных исключений. Для этого в заголовок помещают фильтр, вводимый контекстозависимым словом **when**. Возможности фильтров исключений продемонстрируем в параграфе 16.5, рассмотрев там оператор генерации исключений.

16.4. Исключения в арифметических выражениях

При вычислениях значений арифметических выражений возникают особые ситуации, которые без специального вмешательства программиста обрабатываются по разному. Рассмотрим следующий фрагмент кода.

```
int x = 111111, y = 111111, z = 0;
double a = x/0.0; // результат: "бесконечность"
double b = x/0;   // ошибка компиляции
double c = x/z;   // исключение DivideByZeroException
double d = x * y; // результат: -539247567
```


Как отмечено в комментариях, все приведенные выражения приводят к возникновению особых ситуаций. В первых трех случаях программист по сообщениям компилятора или по результатам выполнения программы может явно распознать ситуацию. Значением переменной $a = x/0.0$ является бесконечно большая величина. В случае $b = x/0$ компилятор выдает сообщение о попытке целочисленного деления на нулевую константу. Выражение $c = x/z$ не смущает компилятор, но приводит к генерации исключения `System.DivideByZeroException` на этапе выполнения программы. В случае вычисления $x*y$ происходит целочисленное переполнение, но никаких признаков особой ситуации нет. Правда, переменной `d` присваивается некоторое отрицательное значение после умножения двух положительных величин. В нашем простом примере этот результат может служить сигналом об особой ситуации. Однако в сложных арифметических выражениях целочисленное переполнение может остаться незамеченным, но приведет к неверному результату.

Для отслеживания таких ошибок в арифметических выражениях следует использовать служебное слово **checked**. Это слово играет две роли, оно обозначает операцию и вводит специальный блок «наблюдения» за переполнениями при вычислениях выражений:

```
checked (выражение)
checked {операторы}
```

В первом случае отслеживаются возникновения переполнений в заключенном в скобки выражении. Во втором случае контролируются переполнения во всех операторах блока. В обоих случаях при возникновении переполнения генерируется исключение `System.OverflowException`.

Наш пример можно дополнить оператором

```
double e = checked(x*y);
```

В этом случае переполнение при вычислении выражения $x*y$ не будет игнорироваться, а приведет к генерации названного исключения. Обработка такого исключения может быть организована обычным образом с помощью конструкции **try/catch/finally**.

Чтобы при выполнении блока операторов или при вычислении выражения никогда не посылались исключения переполнения в арифметических операциях, можно использовать служебное слово **unchecked**.

В тех случаях, когда следить за арифметическими переполнениями необходимо во многих местах программы, удобно вместо служебного слова **checked** использовать специальный режим компиляции. Для этого компиляция программы выполняется с флагом **/checked**. В таком режиме отслеживаются возможные переполнения во всех арифметических операциях программы. Если реагировать на арифметические переполнения не нужно в конкретном фрагменте кода, то соответству-

ющее выражение или блок операторов помещают в контекст ключевого слова **unchecked**.

16.5. Генерация исключений

До сих пор речь шла об обработке исключений, посылаемых методами библиотечных классов. Чтобы глубже разобраться в механизме генерации и обработки исключений и научиться создавать и посылать исключения в любой заранее предусмотренной ситуации, возникшей при выполнении программы, нужно ясно понять и усвоить, что *исключения* — это объекты некоторых специальных классов. Все системные классы исключений являются производными от класса `System.Exception`. Этот класс имеет несколько замечательных свойств, два из которых (`Message` и `Source`) мы уже применяли. При создании объектов классов исключений наиболее важными являются `Message` и `InnerException`. Оба эти свойства в обязательном порядке используются всеми исключениями. Оба свойства предназначены только для чтения:

- `Message` — это свойство типа **string**, значение которого представляет собой текст сообщения о причине возникновения исключения.
- `InnerException` имеет тип `Exception` и представляет собой ненулевую ссылку на то исключение, которое является причиной возникновения данного исключения.

Значения этих свойств можно задать с помощью параметров конструктора при создании объекта класса `System.Exception`. Нам могут быть полезны следующие конструкторы:

- `Exception()` — инициализирует умалчиваемыми значениями данных новый экземпляр класса `Exception`.
- `Exception(string)` — инициализирует новое исключение (новый объект). Параметр задает значение свойства `Message`.
- `Exception(string, Exception)` — параметр типа **string** определяет значение свойства `Message`, а второй параметр представляет собой ссылку на исключение, которое явилось причиной данного исключения.

Зная конструкторы, создать объект-исключение, применяя операцию **new**, мы уже сможем. Осталось выяснить, каким образом послать исключение и как определить условие, при выполнении которого это нужно делать. Определение условия не относится к механизму обработки исключений, а зависит от той конкретной задачи, для решения которой создается программа. Для генерации (для посылки) исключения из выбранной точки кода программы используется оператор, имеющий одну из следующих двух форм:

```
throw выражение;  
throw;
```

Оператор **throw** без выражения можно использовать только в **catch**-блоке для ретрансляции исключения в **catch**-блок следующего уровня.

Значением выражения, использованного в операторе **throw**, должна быть ссылка на объект класса исключений. Именно этот объект посылается в качестве исключения. Если значение выражения равно **null**, то вместо него посылается исключение `System.NullReferenceException`.

Одной из наиболее важных задач, решаемых с применением исключений, является проверка корректности данных, получаемых программой или отдельным методом. Мы уже показали на нескольких примерах, как использовать исключения, автоматически формируемые при чтении данных от клавиатуры.

В следующей программе продемонстрируем, как программировать генерацию исключений. Пусть требуется вычислить значение электрической мощности, рассеиваемой на сопротивлении в 100 Ом при включении его в сеть с напряжением 110, или 127, или 220 вольт. Пользователь может ввести только одно из указанных значений напряжения. Ошибаться при вводе можно только два раза. При третьей ошибке программа должна завершить работу без вычисления мощности.

Ошибки ввода в данном случае могут быть двух видов — лексические и семантические. Лексическая ошибка — это неверная запись вещественного числа при его наборе на клавиатуре. Семантическая ошибка — отличие введенного числа от трех допустимых условиями задачи значений (110, 127, 220). Для преобразования введенной с клавиатуры последовательности символов в числовое значение применим метод `TryParse()`. Тогда послать исключение при лексической ошибке можно с помощью следующих операторов:

```
string input = Console.ReadLine();
if (!double.TryParse(input, out U))
    throw new Exception("Ошибка в формате данных!");
```

В данном фрагменте `U` — имя переменной типа **double**, которая будет представлять в программе введенное пользователем значение напряжения.

Для генерации исключения при семантической ошибке используем такой код:

```
if (U != 110 & U != 127 & U != 220)
    throw new Exception(input+" – недопустимое значение!");
```

Все приведенные операторы разместим в **try**-блоке, вслед за которым поместим обработчик исключений (**catch**-блок), где будем подсчитывать их количество. Всю конструкцию **try/catch** поместим в цикл. Тем, кто забыл раздел физики, посвященный электричеству, напомним, что мощность, потребляемая сопротивлением R при его включении в сеть с напряжением U , вычисляется по формуле U^2/R . Программа в целом может быть такой:

```
// 16_05.cs - исключения для проверки вводимых данных
using System;
class Program
```

```

{
    static void Main()
    {
        double U, R = 100;
        int counter = 0; // Счетчик исключений
        while (true)
        {
            Console.Write("Введите напряжение (110, 127, 220): ");
            try
            {
                string input = Console.ReadLine();
                if (!double.TryParse(input, out U))
                    throw new Exception("Ошибка в формате данных!");
                if (U != 110 & U != 127 & U != 220)
                    throw new Exception(input + " – недопустимое значение!");
            }
            catch (Exception ex)
            {
                Console.WriteLine(ex.Message);
                if (counter++ >= 2)
                { Console.WriteLine("Трижды ошиблись!"); return; }
                continue;
            }
            break;
        } // end of while
        Console.WriteLine("Потребляемая мощность: {0,5:f}", U * U / R);
    }
}

```

Результат выполнения программы:

```

* Первый сеанс:
Введите напряжение (110, 127, 220): 120<ENTER>
120 – недопустимое значение!
Введите напряжение (110, 127, 220): 110f<ENTER>
Ошибка в формате данных!
Введите напряжение (110, 127, 220): asd<ENTER>
Ошибка в формате данных!
Трижды ошиблись!
* Второй сеанс:
Введите напряжение (110, 127, 220): 127<ENTER>
Потребляемая мощность: 161,29

```

Как реагировать на особые ситуации в теле метода, рассмотрим на примере функции, вычисляющей скалярное произведение двух векторов многомерного пространства. Векторы будем передавать методу с помощью двух параметров — ссылок на массивы.

При обращении к методу могут быть по ошибке использованы аргументы, представляющие массивы с разным количеством элементов. Предусмотрим защиту от такой ошибки, посылая исключение в случае неравенства размеров массивов-аргументов. Тогда первые строки кода метода могут быть такими:

```
static double scalar(double[] x, double[] y)
{
    if(x.Length != y.Length)
        throw new Exception ("Неверные размеры векторов!");
}
```

Обработку исключений, формируемых методом при ошибке в его аргументах, обычно выполняют в коде, из которого выполнено обращение к методу (там, где заданы конкретные аргументы). В следующей программе обращения к методу `scalar()` размещены в **try**-блоке. Первое обращение корректно, во втором допущена ошибка — размеры массивов-аргументов не совпадают. Текст программы:

```
// 16_06.cs - исключения в теле метода
using System;
class Program
{
    static double scalar(double[] x, double[] y)
    {
        if (x.Length != y.Length)
            throw new Exception("Неверные размеры векторов!");
        double result = 0;
        for (int i = 0; i < x.Length; i++)
            result += x[i] * y[i];
        return Math.Sqrt(result);
    }
    static void Main()
    {
        try
        {
            Console.WriteLine("scalar1=" +
                scalar(new double[] { 1, 2, 3, 4 },
                    new double[] { 1, 2, 3, 4 }));
            Console.WriteLine("scalar2=" +
                scalar(new double[] { 1, 2, 3, 4 },
                    new double[] { 1, 2, 3 }));
        }
        catch (Exception ex)
        {
            Console.WriteLine(ex.Message);
        }
    }
}
```

Результат выполнения программы:

```
scalar1=5,47722557505166
Неверные размеры векторов!
```

Ознакомившись с оператором генерации исключений, вспомним о возможностях их фильтрации в **catch**-блоках.

Как уже сказано, имеется следующая форма **catch**-блока:

```
catch (тип_исключения имя) фильтр_исключений {операторы}
```

где фильтр исключений:

when (логическое_выражение)

Логическое выражение должно зависеть от получаемого данным **catch**-блоком исключения, поэтому в выражение входит имя исключения, указанное в круглых скобках после служебного слова **catch**. Когда **catch**-блок перехватил исключение, вычисляется значение логического выражения фильтра. Если результат **true**, выполняются операторы **catch**-блока, в противном случае исключение передается следующему **catch**-блоку.

В качестве иллюстрации приведем такой код:

```
// _16_When - фильтр исключений
using System;
class Program
{
    static void Main()
    {
        try
        {
            throw new Exception("9");
        }
        catch (Exception ex) when (ex.Message == "****")
        { Console.WriteLine("Hello ****"); }
        catch (Exception ex) when (ex.Message == "9")
        { Console.WriteLine("Hello 9"); }
    }
}
```

Результаты выполнения программы:

Hello 9

В **try**-блоке только один оператор **throw**, генерирующий исключение с простейшим сообщением. Два **catch**-блока «настроены» на перехват исключений типа **Exception**. Фильтры **catch**-блоков анализируют значение свойства **ex.Message**. Выполняются операторы второго **catch**-блока, что иллюстрирует результат, выводимый на консольный экран.

16.6. Пользовательские классы исключений

В предыдущих программах при генерации исключений был использован системный тип **Exception**. Кроме него в среде .NET имеется большое количество специализированных классов исключений. Часто этих системных исключений вполне достаточно для реакции на те события, которые могут произойти в вашей программе. Однако программист-пользователь может определять собственные классы исключений. Их можно создавать либо на базе класса **Exception**, либо используя в качестве базовых более специализированные системные классы исключений.

В стандартной библиотеке определены два класса исключений, каждый из которых является прямым наследником класса `System.Exception`. Класс `System.SystemException` соответствует исключениям, порождаемым средой, в которой выполняется прикладная программа. Класс `System.ApplicationException` предназначен для создания объектов-исключений, относящихся к выполнению прикладных программ. Создавая собственные классы исключений, стандарт языка C# рекомендует делать их наследниками класса `System.ApplicationException`, а не класса `System.Exception`.

Определяя собственный класс исключений, рекомендуется объявить в этом классе три конструктора: конструктор умолчания, конструктор с параметрами типа **string** и конструктор с параметрами **string** и `Exception`. В каждом из этих конструкторов нужно вызвать конструкторы базового класса. В конструкторе умолчания при обращении к базовому конструктору ему передается текст сообщения, уникального для создаваемого класса исключений. В конструкторе с параметром типа **string** базовому конструктору передается сообщение об ошибке. В третьем конструкторе базовому конструктору передается сообщение об ошибке и объект внутреннего исключения.

Контрольные вопросы и задания

1. Что такое исключение?
2. В чем различия синхронных и асинхронных ситуаций?
3. Для обработки каких ситуаций применяется механизм исключений?
4. Назовите классы системных исключений.
5. Объясните назначение **try**-блока и приведите его форматы.
6. Перечислите форматы обработчиков (ловушек) исключений.
7. Когда выполняется блок завершения обработки исключений?
8. Какими средствами могут обрабатываться ошибки времени исполнения программ?
9. В чем отличие исключения от прерывания?
10. Какими средствами поддерживается перехват исключений?
11. Что происходит в случае, если исключение не перехвачено?
12. Какими средствами могут обрабатываться ошибки времени исполнения программ?
13. Каким образом можно перехватывать все исключения?
14. Каким образом можно перехватить конкретное исключение?
15. Почему возникает необходимость в генерировании исключений в коде программы?
16. Может ли исключение генерироваться повторно, после того, как оно было перехвачено?
17. Каким образом отображается трассировка событий, предшествовавших возникновению исключения?

18. В каком случае обработка исключения может прекратить выполнение программы?
19. Назовите свойства класса `System.Exception`, которые полезны при обработке исключений.
20. Объясните назначение и возможности операции **checked**.
21. Перечислите конструкторы класса `Exception`.
22. Объясните правила применения двух форм оператора **throw**.
23. В каких случаях вычисляется логическое выражение фильтра исключений?

Глава 17

ДЕЛЕГАТЫ И СОБЫТИЯ

17.1. Синтаксис делегатов

Напомним, что в качестве типов в языке C# выступают: класс, структура, перечисление, интерфейс и делегат. Рассмотрим делегаты, назначение которых — представлять в программе методы (функции). Делегаты используются в .NET механизмом обработки событий.

Стандарт языка C# выделяет три этапа применения делегатов: определение (объявление) делегата как типа, создание экземпляра делегата (инстанцирование), обращение к экземпляру делегата (вызов делегата). Обратите внимание, что определение делегата не создает его экземпляр, а только вводит тип, на основе которого впоследствии могут быть созданы экземпляры делегата.

В литературе (см., например, [5]) отмечается связанная с делегатами терминологическая проблема. Если типом является класс, то экземпляр этого типа называют объектом. Для делегатов и тип, и его экземпляр зачастую называют одним термином. Чтобы избежать неоднозначного истолкования, нужно учитывать контекст, в котором термин «делегат» использован. При возможных неоднозначностях будем использовать термин «тип делегата» или «делегат-тип» для обозначения типа, а экземпляр этого типа будем называть экземпляром делегата или просто делегатом. По нашему мнению неудачно называть экземпляр делегата объектом. Объект в программировании по установившейся традиции принято наделять участком памяти для данных, что не свойственно экземпляру делегата.

Подобно тому, как тип конкретного массива, например, **long[]**, создается на основе базового класса **Array**, каждый делегат-тип является наследником системного класса **System.Delegate**. От этого базового класса каждый делегат-тип наследует некоторые члены, о которых чуть позже.

Синтаксис определения (объявления) делегата-типа:

```
модификаторыopt delegate тип_результата  
имя_делегата_типа (спецификация_параметров);
```

Необязательные модификаторы объявления делегата: **new**, **public**, **protected**, **internal**, **private**.

delegate — служебное слово, вводящее тип делегата.

тип_результата — обозначение типа результатов, возвращаемых теми методами, которые будет представлять делегат.

имя_делегата_типа — выбранный программистом идентификатор для обозначения конкретного типа делегатов.

спецификация_параметров — список спецификаций параметров тех методов, которые будет представлять делегат.

Тип_результата и *спецификация_параметров* в декларации делегата-типа определяют *протокол*, которому должны соответствовать методы, представляемые каждым экземпляром этого делегата-типа.

Делегат-тип может быть объявлен как локальный тип класса или структуры либо как декларация верхнего уровня в единице компиляции. Модификатор **new** применим только для делегатов, локализованных в классе.

Примеры определений делегатов-типов:

```
public delegate int[] Row(int num);  
public delegate void Print(int [] ar);
```

Эти два определения вводят два разных типа делегатов. Тип с именем *Row* и тип с именем *Print*. Экземпляр делегата-типа *Row* может представлять метод с целочисленным параметром, возвращающий ссылку на одномерный целочисленный массив. Экземпляр делегата-типа *Print* может представлять метод, параметр которого — ссылка на целочисленный массив. Метод, представляемый экземпляром делегата-типа *Print*, не может возвращать никакого значения (тип результата **void**).

Так как делегат это тип ссылок, то, определив делегат, можно применять его для создания переменных-ссылок. Синтаксис определения ссылок традиционен:

```
имя_делегата_типа имя_ссылки;
```

Используя введенные делегаты-типы, можно так определить ссылки:

```
Row delRow;      // Ссылка с типом делегата Row  
Print delPrint; // Ссылка с типом делегата Print
```

После такого определения ссылки *delRow*, *delPrint* имеют неопределенные значения (**null**). С каждой из этих ссылок можно связать экземпляр соответствующего делегата-типа. Экземпляр делегата создается конструктором делегата, обращение к которому выполняется из выражения с операцией **new**. При обращении к конструктору в качестве аргумента необходимо использовать имя метода, соответствующего *протоколу*, декларированному типом делегата (у метода должны быть те же тип возвращаемого значения и спецификация параметров, которые указаны в определении делегата-типа). Этот метод в дальнейшем будет доступен для вызова через ссылку на экземпляр делегата.

Обратите внимание, что в объявлении делегата-типа нет необходимости определять его конструктор. *Конструктор в определение деле-*

гата-типа компилятор встраивает автоматически. При обращении к конструктору в качестве аргумента можно использовать:

- метод класса (имя метода, уточненное именем класса);
- метод объекта (имя метода, уточненное именем объекта);
- ссылку на уже существующий в программе экземпляр делегата.

Экземпляр делегата может представлять как метод класса, так и метод объекта. Однако в обоих случаях метод должен соответствовать по типу возвращаемого значения и по спецификации параметров объявлению делегата-типа.

Как уже упомянуто, декларация делегата-типа может размещаться в пространстве имен наряду с другими объявлениями классов и структур. В этом случае говорят о внешнем размещении определения делегата. Кроме того, декларации делегатов-типов могут входить в объявления классов и структур. В этом случае имеет место локализация типа делегата.

Приведем пример использования введенных выше делегатов-типов и соответствующих ссылок.

В следующей программе нет защиты от неверных данных. Например, нельзя допускать, чтобы аргумент метода `series()` оказался отрицательным, но это условие нигде не проверяется.

```
// 17_01.cs - Делегаты. Их внешнее определение...
using System;
public delegate int[] Row(int num); // делегат-тип
public delegate void Print(int[] ar); // делегат-тип
public class Example
{
    // Метод возвращает массив цифр целого числа-параметра
    static public int[] series(int num)
    {
        int arLen = (int)Math.Log10(num) + 1;
        int[] res = new int[arLen];
        for (int i = arLen - 1; i >= 0; i--)
        {
            res[i] = num % 10;
            num /= 10;
        }
        return res;
    }
    // Метод выводит на экран значения элементов массива
    static public void display(int[] ar)
    {
        for (int i = 0; i < ar.Length; i++)
            Console.WriteLine("{0}\t", ar[i]);
    }
} // End of Example

class Program
```

```

{
    static void Main()
    {
        Row delRow;      // Ссылка на делегат
        Print delPrint;  // Ссылка на делегат
        delRow = new Row(Example.series); // Экземпляр делегата
        delPrint = new Print(Example.display); // Экземпляр
        // Вызов метода через делегата
        int[] myAr = delRow(13579);
        // Вызов метода через делегата
        delPrint(myAr);
        int[] newAr = { 11, 22, 33, 44, 55, 66 };
        delPrint(newAr); // Вызов метода через делегата
        Example.display(myAr); // Явное обращение к методу
    }
}

```

В одном пространстве имен объявлены два делегата-типа, класс Program с методом Main() и класс Example с двумя статическими методами. Метод **int[] series(int num)** может быть представлен экземпляром делегата Row, а метод **void display(int[] ar)** соответствует делегату Print. В функции Main() определены ссылки delRow и delPrint, которые затем «настраиваются» на экземпляры делегатов Row и Print. При создании экземпляров делегатов в качестве аргументов конструкторов использованы уточненные имена статических методов Example.series и Example.display. Теперь ссылка delRow представляет метод example.series(), а ссылка delPrint — метод Example.display(). Самое важное то, что к названным методам можно обращаться через соответствующие им ссылки на экземпляры делегатов. Именно это демонстрируют следующие ниже операторы метода Main().

Результат выполнения программы:

```

1 3 5 7 9
11 22 33 44 55 66
1 3 5 7 9

```

Обратите внимание, что последняя строка результатов получена за счет явного (минуя делегат) обращения к методу Example.display().

Кроме непосредственного обращения к конструктору делегата можно для «настройки» ссылки, имеющей тип конкретного делегата, на метод (соответствующий протоколу типу делегата) использовать присваивание.

Программа не изменится, если таким образом присвоить значения ссылкам:

```

delRow = Example.series;    // Имя метода
delPrint = Example.display; // Имя метода

```

Приведенные сведения о делегатах и разобранный пример не иллюстрируют особых достоинств делегатов и необходимости их присут-

ствия в языке. Мы даже не объяснили, какие члены есть в каждом делегате.

Каждое определение делегата-типа вводит новый тип ссылок, производный, как мы уже упоминали, от единого базового класса `System.Delegate`. От этого базового класса каждый делегат-тип наследует ряд полезных методов и свойств. Среди них отметим:

public MethodInfo Method {get;} — свойство, представляющее заголовок метода, на который в текущий момент «настроен» экземпляр делегата.

public object Target {get;} — свойство, позволяющее определить, какому классу принадлежит тот объект, метод которого представлен экземпляром делегата. Если значение этого свойства есть **null**, то экземпляр делегата в этот момент представляет статический метод класса.

Добавим в приведенную выше программу операторы:

```
Console.WriteLine("delRow is equals {0}.", delRow.Method);  
Console.WriteLine("delPrint is equals {0}.", delPrint.Method);
```

Результат выполнения этих операторов:

```
delRow is equals Int32[] series(Int32).  
delPrint is equals Void display(Int32[]).
```

Обратите внимание, что вместо типа **int** при выводе указано его системное обозначение `Int32`, а вместо **void** — `Void`.

Экземпляры делегатов представляют в нашем примере статические методы, поэтому выводить значения `delRow.Target` и `delPrint.Target` нет смысла — они равны **null**, и при выводе это значение заменяется пустой строкой.

17.2. Массивы делегатов

Делегаты, точнее ссылки на экземпляры делегатов, можно объединять в массивы. Такая возможность позволяет программисту задавать наборы действий (обращений к методам), которые затем будут автоматически выполнены в указанной последовательности.

В качестве примера рассмотрим модель перемещения робота по плоской поверхности. Пусть робот умеет выполнять четыре команды (вперед, назад, направо, налево), каждая из которых изменяет его положение на один шаг. Система управления роботом должна формировать последовательность команд, которые робот выполняет автоматически. После выполнения полученной последовательности команд робот должен сообщить о достигнутом местоположении. Конкретную последовательность можно формировать в виде массива ссылок на экземпляры делегата. Перебор элементов массива позволит выполнить всю цепочку команд.

Так как нашей целью является только иллюстрация возможностей массивов делегатов, максимально упростим задачу. Определим класс роботов `Robot` с четырьмя уже названными методами (командами) управления и методом для выдачи информации о местоположении робота. Вне класса роботов определим тип делегатов `Steps`, предназначенных для представления методов управления роботом. В главной программе создадим объект класса роботов и именующую его ссылку `rob`. Массив делегатов `trace` будем формировать, присваивая его элементам ссылки на безымянные экземпляры делегата, адресующие разные методы объекта класса роботов. Этот массив будет имитировать последовательность команд, управляющих перемещением конкретного объекта-робота. После формирования массива переберем его элементы в цикле и с их помощью последовательно вызовем все запланированные методы, управляющие перемещением робота.

```
// 17_02.cs - массив делегатов
using System;
class Robot // класс для представления робота
{
    int x, y; // положение робота на плоскости
    public void right() { x++; } // направо
    public void left() { x--; } // налево
    public void forward() { y++; } // вперед
    public void backward() { y--; } // назад
    public void position() // сообщить координаты
    {
        Console.WriteLine("The Robot position: x={0}, y={1}", x, y);
    }
}
delegate void Steps(); // делегат-тип
class Program
{
    static void Main()
    {
        Robot rob = new Robot(); // конкретный робот
        Steps[] trace = {new Steps(rob.backward),
            new Steps(rob.backward), new Steps(rob.left)};
        for (int i = 0; i < trace.Length; i++)
        {
            Console.WriteLine("Method={0}, Target={1}",
                trace[i].Method, trace[i].Target);
            trace[i]();
        }
        rob.position(); // сообщить координаты
    }
}
```

В цикле перебора элементов массива (`trace`) со ссылками на экземпляры делегата помещен вывод на консоль заголовка метода, адресованного очередным элементом массива. Тут же выводится имя того класса, которому принадлежит этот нестатический метод. Для получение

ния заголовка и имени класса используются уже упомянутые свойства класса делегатов (Method и Target).

Результат выполнения программы:

```
Method=Void backward(), Target=Robot  
Method=Void backward(), Target=Robot  
Method=Void left(), Target=Robot  
The Robot position: x=-1, y=-2
```

При создании объекта класса Robot его координаты по умолчанию равны нулю ($x = 0$, $y = 0$). Именно из этой точки начинается в нашем примере перемещение — два шага назад и один влево, что и определяет достигнутую позицию ($x = -1$, $y = -2$).

17.3. Многоадресные экземпляры делегатов

До сих пор мы рассматривали делегаты, каждый экземпляр которых представляет только один конкретный метод. Однако делегат (его экземпляр) может содержать ссылки сразу на несколько методов, соответствующих типу делегата. При однократном обращении к такому экземпляру делегата автоматически организуется последовательный вызов всех методов, которые он представляет.

Существование многоадресных делегатов конструктивно обеспечено наличием в каждом делегате списка тех методов, которые в каждый момент времени представляет экземпляр делегата. Указанный список методов называют *списком вызовов*.

Для поддержки многоадресности используются методы из базового класса MulticastDelegate, который в свою очередь является наследником класса System.Delegate. Рассмотрим основные из них.

public static Delegate Combine(Delegate a, Delegate b) — статический метод объединяет (группирует) два экземпляра делегата, создавая многоадресный экземпляр делегата. Аргументами при обращении должны быть две ссылки на экземпляры делегатов. Возвращаемое значение — многоадресный экземпляр делегата. Обращение к методу Combine() в языке C# заменяют последовательностью операций «+» и «+=» или операций «+=»:

```
ссылка_на_делегат_1 += ссылка_на_делегат_2;
```

public static Delegate Remove(Delegate source, Delegate value) — метод удаляет из многоадресного экземпляра делегата, заданного первым параметром, конкретную ссылку, заданную вторым параметром-делегатом. Метод открытый и статический, т. е. принадлежит типу делегатов. Аргументами при обращении должны быть две ссылки на экземпляры-делегаты. Первый аргумент представляет (обычно многоадресный) экземпляр делегата, второй представляет экземпляр делегата, значение которого (ссылку на метод) нужно удалить из списка

вызовов делегата. Обращение к методу Remove() в языке C# заменяют последовательностью операций «-» и «=» или эквивалентной операцией «-=»:

```
Многоадресный_делегат -= ссылка_на_делегат;
```

На примере с классом, моделирующим движения робота, мы уже показали, как с помощью массива делегатов можно составлять и использовать цепочки вызовов методов. Применение многоадресных экземпляров делегатов позволяет в рассмотренном выше примере расширить систему команд управления роботом, не изменяя его внутренней структуры. В следующей программе (17_03.cs) тот же класс Robot и тот же тип делегатов Steps по-другому используются в методе Main():

```
static void Main()
{
    Robot rob = new Robot(); // конкретный робот
    Steps delR = new Steps(rob.right);    // направо
    Steps delL = new Steps(rob.left);     // налево
    Steps delF = new Steps(rob.forward);  // вперед
    Steps delB = new Steps(rob.backward); // назад
    // Шаги по диагоналям:
    Steps delRF = delR + delF;
    Steps delRB = delR + delB;
    Steps delLF = delL + delF;
    Steps delLB = delL + delB;
    delLB();           // перемещение влево и назад
    rob.position();    // сообщить координаты
    delRB();           // перемещение вправо и назад
    rob.position();    // сообщить координаты
}
```

В программе определена ссылка rob, именуемая объект класса Robot, и определены экземпляры делегата (ссылки: delR, delL, delF, delB), именующие методы покоординатных перемещений робота по плоскости. Затем определены четыре многоадресных (двухадресных) экземпляра делегата Steps (ссылки: delRF, delRB, delLF, delLB), применение которых позволяет перемещать робот по диагоналям.

Результат выполнения программы:

```
The Robot position: x=-1, y=-1
The Robot position: x=0, y=-2
```

Обращение к экземпляру многоадресного делегата приводит к последовательному исполнению списка методов, которые представлены делегатом. Результат выполнения списка методов делегатов — то значение, которое возвращает последний метод. Стандарт не определяет порядка вызова методов, адресованных многоадресным делегатам. Нельзя быть уверенным, что вызов будет происходить в том порядке, в котором экземпляры делегатов добавлялись к многоадресному делегату. Отметим, что наиболее часто многоадресные делегаты

применяются для представления методов, каждый из которых возвращает значение типа **void**.

Если любой из методов списка вызовов многоадресного делегата пошлет исключение, то обработка методов списка прерывается и выполняется поиск подходящего обработчика исключений.

Если методы из списка вызовов многоадресного делегата принимают параметры по ссылке, то изменения таких параметров за счет операторов тела метода, вызванного через делегат, воспринимаются следующими методами списка вызовов.

17.4. Делегаты и обратные вызовы

В программировании достаточно часто возникает необходимость создавать фрагменты кода (например, классы, подпрограммы, функции), которые можно было бы каким-либо образом настраивать при конкретных применениях. Наиболее интересна и важна настройка, позволяющая изменять алгоритм выполнения кода. Классическим примером удобства и полезности такой настройки служат методы `Sort()` и `BinarySearch()` класса `Array`. Первый из них выполняет сортировку элементов массива, второй позволяет осуществлять бинарный поиск нужного значения в упорядоченном массиве. Применение каждого из названных методов требует, чтобы программист-пользователь «сообщил» при вызове, что он понимает под порядком, в котором должны разместиться сортируемые элементы, или что означает равенство значений элемента массива и эталона поиска. Указанные критерии сортировки и поиска оформляются в виде вспомогательных методов. Сведения об этих методах передаются методам `Sort()` и `BinarySearch()` в качестве аргументов вместе с тем массивом, который нужно обработать. Благодаря этому в программе пользователя появляется возможность, по-разному программируя вспомогательные методы, достаточно произвольно задавать правила сортировки и поиска, т. е. изменять поведение библиотечных методов `Sort()` и `BinarySearch()`.

О концепции построения методов, вызывающих при исполнении вспомогательные функции, определенные (позже) в точке вызова (например, в коде пользователя), говорят, используя термин «обратный вызов» (*callback*). В примере с `Sort()` и `BinarySearch()` методы обратного вызова параметризуют названные библиотечные методы.

Еще один пример применения методов обратного вызова — процедура вывода на экран дисплея графика функции $f(x)$, математическое описание которой заранее не известно. При обращении к такой процедуре могут быть заданы (например) пределы изменения аргумента $x_{\min}$, $x_{\max}$ и ссылка на программную реализацию конкретной функции $f(x)$.

Третий пример — процедура вычисления определенного интеграла функции $f(x)$. Так как существует много методов численного интегриро-

вания, то при использовании процедуры можно с помощью параметров задать: программную реализацию метода интегрирования, реализацию (метод) подинтегральной функции $f(x)$, пределы интегрирования и требуемую точность вычислений. В данном случае процедура вычисления определенного интеграла параметризуется двумя методами обратного вызова.

Итак, обратным вызовом называют обращение из исполняемого метода к другому методу, который зачастую определяется не на этапе компиляции, а в процессе выполнения программы. В языках C и C++ для реализации обратных вызовов в качестве параметра в процедуру передается указатель на ту функцию, к которой нужно обращаться при исполнении программы. Мы не рассматриваем механизм указателей, но заметим, что указатель на функцию — это адрес участка основной памяти, в котором размещен код функции. Опасность и ненадежность обращения к функции по ее адресу состоит в том, что зная только адрес, невозможно проверить правильность обращения. Ведь нужно убедиться, что количество аргументов и их типы соответствуют параметрам функции, что верен тип возвращаемого значения и т. д.

Возможность указанных проверок в языке C# обеспечивают делегаты.

При разработке средств обратного вызова в языке C# было решено обеспечить программистов не только возможностью контролировать сигнатуру методов, но и отличать методы классов от методов объектов. Каждый делегат включает в качестве полей ссылку на объект, для которого нужно вызвать метод, и имя конкретного метода. Если ссылка на объект равна **null**, то имя метода воспринимается как имя статического метода.

Как мы уже говорили, делегат — это тип, экземпляры которого представляют методы с конкретной спецификацией параметров и фиксированным типом возвращаемого значения. Делегаты дают возможность рассматривать методы как сущности, ссылки на которые можно присваивать переменным (соответствующего типа) и передавать в качестве параметров.

Покажем на простом примере, как ссылка на экземпляр делегата может использоваться в качестве параметра для организации обратных вызовов.

Определим класс, поля которого задают предельные значения $x_{\min}$, $x_{\max}$ отрезка числовой оси и количество n равноотстоящих точек на этом отрезке. В классе определим метод, выводящий на экран значения функции $f(x)$ в точках числовой оси, определенных значениями $x_{\min}$, $x_{\max}$, n . Конкретная функция $f(x)$, передается методу с помощью параметра-делегата. Определения делегата-типа и класса:

```
public delegate double Proc(double x); // делегат-тип
public class Series
{
    int n;
```

```

double xmi, xma;
public Series(int ni, double xn, double xk) // конструктор
{ n = ni; xmi = xn; xma = xk; }
public void display(Proc fun)
{
    Console.WriteLine("Proc: {0}, xmi={1:f2}, " +
        "xma={2:f2}, n={3}.", fun.Method, xmi, xma, n);
    for (double x = xmi; x <= xma; x += (xma - xmi) / (n - 1))
        Console.Write("{0:f2}\t", fun(x));
}
}

```

Делегат-тип Proc определен как внешний тип в глобальном пространстве имен. Там же определен класс Series. Для нас интересен его метод с заголовком

```
public void display(Proc fun)
```

Параметр метода — ссылка на экземпляр делегата Proc. В теле метода display() выводится заголовок того метода, который будет представлен аргументом-делегатом. Затем выводятся n значений адресованного аргументом метода, представляющего функцию $f(x)$. Значения выводятся в цикле, параметром которого служит аргумент функции $f(x)$.

Имея такие определения типа-делегата и класса, можно вводить разные методы, соответствующие делегату Proc, и определять разные объекты класса Series. Затем для объекта класса Series можно вызвать метод display(), передать ему в качестве аргумента имя метода, представляющего нужную математическую функцию $f(x)$, и получить заголовок метода, представляющего функцию $f(x)$, и набор значений этой функции. Указанные действия выполняет следующий фрагмент программы:

```

class Program
{
    static double mySin(double x) // Математическая функция
    { return Math.Sin(x); }
    static double myLine(double x) // Математическая функция
    { return x * 5; }
    static void Main()
    {
        Series sequence = new Series(7, 0.0, 1.0); // Объект класса
        sequence.display(mySin);
        Console.WriteLine();
        sequence.display(myLine);
        Console.WriteLine();
    }
}

```

В классе Program определены два статических метода, соответствующих делегату-типу Proc. В методе Main() определен именуемый ссылкой sequence объект класса Series. Поля этого объекта ($x_{\min}$, $x_{\max}$, n)

определяют набор точек оси аргумента, в которых будут вычисляться значения функции при обращении к методу `display()`.

Результат выполнения программы:

```
Proc: Double mySin(Double), xmi=0,00, xma=1,00, n=7.  
0,00 0,17 0,33 0,48 0,62 0,74 0,84  
Proc: Double myLine(Double), xmi=0,00, xma=1,00, n=7.  
0,00 0,83 1,67 2,50 3,33 4,17 5,00
```

Обратите внимание, что в данном примере нет явного создания экземпляра делегата `Proc`. Нет явного определения и ссылок на экземпляры делегатов. Вместо этого при обращении к методу `display()` параметр `fun` заменяется именем конкретного метода. Все дальнейшие действия выполняются неявно — создается экземпляр делегата-типа `Proc` и ссылка на него используется в теле метода.

17.5. Анонимные методы и лямбда-выражения

В том месте, где экземпляр делегата должен представлять конкретный метод, можно объявить безымянный метод. Такой метод называют анонимным. Протокол этого метода (спецификация параметров и тип возвращаемого значения) определяет тип делегата. Тело анонимного метода подобно телу соответствующего именованного метода, но имеются некоторые отличия (об отличиях чуть позже).

Формат декларации анонимного метода:

```
delegate (спецификация_параметров)  
    {операторы_тела_метода};
```

Компилятор, «встретив» декларацию анонимного метода, формирует экземпляр делегата, и настраивает его на выполнение операторов тела анонимного метода. Применение анонимных методов рекомендуется в тех случаях, когда метод используется однократно или его вызовы локализованы в одном блоке кода программы.

В следующей программе используются два анонимных метода, ассоциированных со ссылками, имеющими тип делегата (`Cast`), определенного в глобальном пространстве имен. Методы выполняют округление положительного вещественного числа: один до ближайшего целого и второй до большего целого.

```
// 17-05.cs - анонимные методы...  
using System;  
delegate int Cast(double x); // Объявление делегата-типа  
class Program  
{  
    static void Main()  
    {  
        double test = 15.3;  
        Cast cast1 = delegate (double z) // ближайшее целое
```

```

        { return (int)(z + 0.5); };
    Cast cast2 = delegate (double z) // большее целое
    { return ((int)z == z ? (int)z : (int)(z + 1)); };
    Console.WriteLine("cast1(test)={0}, cast2(test)= {1}",
        cast1(test), cast2(test));
    Console.WriteLine("cast1(44.0)={0}, cast2(44.0)= {1}",
        cast1(44.0), cast2(44.0));
}
}

```

Результаты выполнения программы:

```

cast1(test)=15, cast2(test)= 16
cast1(44.0)=44, cast2(44.0)= 44

```

Делегат-тип **Cast** предназначен для представления методов с параметром типа **double**, возвращающих значение типа **int**. В методе **Main()** объявлены два экземпляра делегата **Cast**, ассоциированные со ссылками **cast1** и **cast2**. Для инициализации ссылок вместо явного обращения к конструктору делегата **Cast** используются анонимные методы, каждый из которых вводится с помощью служебного слова **delegate**. Анонимный метод, связанный со ссылкой **cast1**, получив в качестве параметра вещественное значение, возвращает ближайшее к нему целое число. Анонимный метод, представляемый ссылкой **cast2**, возвращает целое число, не меньшее значения параметра.

Более удобным (с точки зрения краткости и выразительности) средством для кодирования методов, представляемых экземплярами делегатов, являются лямбда-выражения. Лямбда-выражения появились в языке **C#** позже анонимных методов, но практически полностью их заменяют, так как обладают большими выразительными возможностями и краткостью, нежели анонимные методы.

Общая форма записи лямбда-выражений:

(параметры) => блок операторов

Здесь *параметры* — это спецификация параметров, соответствующая требованиям к спецификации параметров в типе делегата.

Лексему «**=>**» называют лямбда-операцией. Именно эта операция является неотъемлемой частью лямбда-выражения.

Вслед за лямбда-операцией размещен блок операторов (тело лямбда-выражения). К этому блоку требования те же, что и к телу метода. Если протокол, декларируемый типом делегата, предусматривает возвращение результата, отличного от **void**, то для выхода из тела лямбда-выражения используются операторы **return** с выражениями соответствующего типа. Таким образом, возвращаемое из тела лямбда-выражения значение и типы параметров соответствуют протоколу, определяемому делегатом.

Для формирования экземпляра делегата **delegate int Cast (double x)**; можно записать, например, такое лямбда-выражение:

```

(double z) => {return (int)(z + 0.5);}

```

Возможны следующие варианты упрощения записи лямбда-выражений:

- 1) список параметров можно использовать без указания их типов;
- 2) если параметр один, то круглые скобки можно опустить;
- 3) если в блоке операторов только один оператор, то фигурные скобки можно опустить;
- 4) если в блоке операторов только один оператор и это оператор **return** *выражение*, то и служебное слово **return** (и фигурные скобки) можно опустить. В этом случае тело лямбда-выражения упрощается до отдельного выражения.

С учетом перечисленных сокращений приведенное выше лямбда-выражение принимает такой вид:

```
z => (int)(z + 0.5)
```

Обратим внимание, что соответствие между параметрами лямбда-выражения и спецификацией параметров делегата-типа устанавливается по их взаимному расположению (т. е. позиционно, имена параметров не учитываются).

Продemonстрируем применение лямбда-выражения с двумя параметрами, не возвращающего результат в точку вызова.

```
// 17_a - Pair - Делегаты и лямбда-выражения
using System;
public delegate void Pair(string s, int d); // делегат-тип
class Program
{
    static void Main()
    {
        Pair birthday = (month, date) =>
            Console.WriteLine("Month = {0}; Date = {1}",
                month, date);
        Pair person = (name, year) =>
        {
            DateTime moment = DateTime.Now;
            int age = moment.Year - year;
            Console.WriteLine("Name: {0}; Age: {1}", name, age);
        };
        person("Charley", 1967);
        birthday("December", 22);
    }
}
```

Результаты выполнения программы:

```
Name: Charley; Age: 52
Month = December; Date = 22
```

Делегат-тип *Pair* «настроен» на представления методов с двумя параметрами типов **string** и **int**, не возвращающих результат в точку вызова.

Инициализация ссылок *birthday* и *person*, имеющих тип делегата-типа *Pair*, выполнена с помощью лямбда-выражений.

В параметрах лямбда-выражения для переменной `person` задаются имя и год рождения некой персоны. В теле этого лямбда-выражения несколько операторов. Обращением к свойству `DateTime.Now.Year` определяется числовое значение текущего года и вычисляется возраст персоны в настоящий момент. Затем имя и возраст персоны выводятся на экран.

В теле лямбда-выражения для переменной `birthday` один оператор. Он выводит число и месяц какого-то события (наверное, это день рождения Чарли).

Анонимные методы и лямбда-выражения удобно применять для «настроек» библиотечных методов, предусматривающих применение обратного вызова. В гл. 9, посвященной методам C#, рассмотрено применение функций обратного вызова в обращениях к библиотечному методу сортировки элементов массива. Там мы использовали имена методов в качестве аргументов метода `Array.Sort()`. Если обратиться к документации, то увидим, что заголовок этого метода включает параметр с типом делегата. Поэтому замещать этот параметр можно и анонимными методами, и лямбда-выражениями, разместив их объявления непосредственно в обращениях к методу `Array.Sort()`.

Приведем программу, в которой элементы целочисленного массива упорядочиваются по убыванию значений, а затем сортируются по их четности. Для задания правил упорядочения применим в обращениях к методу `Array.Sort()` в качестве аргументов лямбда-выражения:

```
// 17_06.cs - лямбда-выражения и Array.Sort()
using System;
class Program
{
    static void Main()
    {
        int[] ar = { 4, 5, 2, 7, 8, 1, 9, 3 };
        Array.Sort(ar, (int x, int y) => // по убыванию
        {
            if (x < y) return 1;
            if (x == y) return 0;
            return -1;
        }
        );
        foreach (int memb in ar)
            Console.Write("{0} ", memb);
        Console.WriteLine();
        Array.Sort(ar, (int x, int y) => // по четности
        {
            if (x % 2 != 0 & y % 2 == 0) return 1;
            if (x == y) return 0;
            return -1;
        }
        );
        foreach (int memb in ar)
            Console.Write("{0} ", memb);
    }
}
```

```

        Console.WriteLine();
    }
}

```

Результат выполнения программы:

```

9 8 7 5 4 3 2 1
2 4 8 1 3 5 7 9

```

Так как каждый из методов используется в данной программе только один раз, то нет необходимости объявлять тип делегатов отдельно и определять ссылки с типом делегата. Лямбда-выражения представляют конкретные методы непосредственно в аргументах метода `Array.Sort()`.

Напомним, что делегат — это тип и поэтому может не только специфицировать параметры методов. В следующем примере делегат определяет тип свойства класса. В решаемой далее задаче нужно объявить делегат-тип для представления методов, вычисляющих значения логических функций двух переменных. Далее нужно определить класс, объект которого заполняет таблицу истинности для функции, представленной в объекте полем, имеющим тип делегата.

Для обращения к полю определить открытое свойство с типом делегата. (При заполнении таблицы истинности объект «не знает» для какой логической функции строится таблица; функция передается объекту в точке обращения к его методу.)

Итак, требуется определить класс со статическим методом для вывода на экран таблицы истинности логической функции двух переменных. При выводе заменять логическое значение ИСТИНА значением 1, а ЛОЖЬ — 0.

Конкретные функции передавать методу с помощью лямбда-выражений, определяемых в точке вызова. Ссылку с типом делегата, представляющую лямбда-выражение, присваивать свойству объекта.

В методе `Main()` определить два лямбда-выражения, представляющие логические функции, и построить для них таблицы истинности.

```

// 17_07.cs - свойство с типом делегата и анонимные методы
using System;
public delegate bool BoolDel(bool x, bool y); // Делегат-тип.
public class Create
{ // Класс таблиц истинности
    BoolDel specificFun; // поле, определяющее логическую функцию
    public BoolDel SpecificFun
    {
        set { specificFun = value; }
    }
    public bool[,] defne()
    { // Формирование логического массива
        bool[,] res = new bool[4, 3];
        bool bx, by;
        int k = 0;

```



```

    for (int i = 0; i <= 1; i++)
        for (int j = 0; j <= 1; j++)
        {
            bx = (i == 0 ? false : true);
            by = (j == 0 ? false : true);
            res[k, 0] = bx;
            res[k, 1] = by;
            res[k++, 2] = specificFun(bx, by);
        }
    return res;
}
}
public class Methods
{ // Класс с методами
    static public void printTabl(bool[,] tabl)
    {
        Console.WriteLine("A B F");
        for (int i = 0; i < tabl.GetLength(0); i++)
            Console.WriteLine("{0} {1} {2}",
                tabl[i, 0] ? 1 : 0, tabl[i, 1] ? 1 : 0, tabl[i, 2] ? 1 : 0);
    }
}
class Program
{ // Основной класс программы
    static void Main()
    {
        Create create = new Create();
        create.SpecificFun = (bool a, bool b) =>
            { return a || b; };
        Console.WriteLine("Таблица для (A || B):");
        Methods.printTabl(create.define());
        create.SpecificFun = (bool d, bool e) =>
            { return d && !e; };
        Console.WriteLine("\nТаблица для (A &&!B):");
        Methods.printTabl(create.define());
    }
}

```

Результаты выполнения программы:

Таблица для (A || B):

```

A B F
0 0 0
0 1 1
1 0 1
1 1 1

```

Таблица для (A &&!B):

```

A B F
0 0 0
0 1 0
1 0 1
1 1 0

```

Изучая лямбда-выражения и анонимные методы, следует обратить внимание на возможности применения в них внешних перемен-

ных. Как и для локальных методов, внешними считаются параметры и локальные переменные того метода, в котором определен анонимный метод или лямбда-выражение. Особенности использования внешних переменных в лямбда-выражениях, в локальных и анонимных методах совпадают, поэтому рассмотрим только лямбда-выражения.

Внешняя переменная, используемая в лямбда-выражении, называется *захваченной переменной*. Важно понимать, когда происходит «захват» значения, которое имеет захваченная переменная. Приведем пример:

```
// 17_b - захваченные переменные в лямбда-выражениях
using System;
class Program
{
    delegate void Pusto();
    static void Main()
    {
        string name = "Tom";
        Pusto output = () =>
            { Console.WriteLine($"Hello, {name}!"); };
        output();
        name = "Jack";
        output();
    }
}
```

Результаты, выводимые на экран:

```
Hello, Tom!
Hello, Jack!
```

Делегат-тип *Pusto* представляет методы без параметров, ничего не возвращающие в точку вызова. Переменная *output* с типом этого делегата инициализирована с помощью лямбда-выражения, в теле которого используется внешняя переменная *name*. При обращениях к экземпляру делегата с помощью выражения *output()* используется то значение захваченной переменной *name*, которое она имеет в момент обращения (а не в момент создания экземпляра делегата). Это правило иллюстрируют результаты выполнения программы.

Захваченная лямбда-выражением внешняя переменная продолжает существовать до тех пор, пока лямбда-выражение остается частью экземпляра делегата. Это происходит даже в том случае, когда переменная выходит из обычной области своего существования. Покажем эту особенность на примере:

```
// 17_c - расширение области существования переменных
using System;
class Program
{
    delegate void Pusto();
```

```

static void Main()
{
    Pusto output;
    {
        string report = "Сообщение из блока";
        output = () => Console.WriteLine($"{report}!");
    }
    output();
}
}

```

Результаты, выводимые на экран:

Сообщение из блока!

В методе `Main()` есть вложенный блок. Ссылка *output* определена вне этого блока, но ассоциирована с лямбда-выражением в теле внутреннего блока. В лямбда-выражение входит локальная переменная *report* внутреннего блока. Обращение к экземпляру делегата с помощью выражения *output()* выполнено за пределами внутреннего блока. Но используется значение захваченной переменной *report* из внутреннего блока, что иллюстрируют результаты выполнения программы.

Так как значение захваченной переменной определяется не в момент декларации лямбда-выражения, а в момент обращения к тому экземпляру делегата, в котором это выражение используется, то существует немного неожиданная ситуация, в том случае, когда лямбда-выражение захватывает параметр цикла.

Рассмотрим следующий фрагмент кода:

```

Pusto [] take = new Pusto[3];
for(char h = 'A'; h < 'D'; h++)
    take[h - 'A'] = () => Console.Write(h + " ");
foreach(Pusto d in take) d();

```

Декларирован массив, тип элементов которого определяет делегат *Pusto*. В параметрическом цикле каждый элемент массива инициализирован лямбда-выражением, которое захватывает переменную *h* — параметр цикла. После выхода из цикла **for** в цикле **foreach** выполняются обращения к элементам массива, каждый из которых выводит значение захваченного параметра. Так как обращения к элементам массива (к экземплярам делегата *Pusto*) выполняются после завершения параметрического цикла, то параметр цикла имеет то значение 'D', при котором цикл завершен. Результат на экране дисплея:

D D D

Лямбда-выражения могут изменять захваченные переменные практически так же, как могут изменять операторы блока внешние по отношению к блоку переменные.

17.6. События

Событие — средство, позволяющее объекту (или классу) послать во «внешний для объекта или класса мир» сообщение о переходе в некоторое новое состояние или о получении сообщения из внешнего источника. Так как на уровне программы все действия объектов и классов реализуются с помощью методов, то и посылка сообщения оформляется как оператор в теле некоторого метода. Синтаксически оператор посылки сообщения выглядит так:

```
имя_события (аргументы_для_делегата);
```

Разберем, о каком событии идет речь, и какую роль играет делегат, которому нужно предоставить аргументы.

Отметим, что объявление события может размещаться в классе, в структуре и в интерфейсе. Начнем с классов. Событие это член класса (или его объекта), вводимый объявлением:

```
модификаторыопт event имя_типа_делегата имя_события;
```

Модификатором может быть **abstract**, **new**, **override**, **static**, **virtual**, **public**, **protected**, **private**, **internal**.

event — служебное слово декларации события.

имя_события — идентификатор, выбираемый программистом в качестве названия конкретного члена, называемого переменной события. В практике программирования на C# принято начинать имена событий с префикса **on**.

имя_типа_делегата — имя того делегата-типа (его называют делегатом события или событийным делегатом), который должен представлять событию те методы, которые будут вызываться в ответ на посылку сообщения о событии.

Таким образом, событие — это член класса, имеющий тип делегата. Этот делегат-тип должен быть доступен в точке объявления события. Например, его определение должно находиться в том же файле. Событийный делегат-тип определяет для события сигнатуру и тип возвращаемого значения тех методов, которые могут быть вызваны в ответ на посылку сообщения о нем. Напомним, что сигнатура, и тип возвращаемого результата, вводимые типом делегата, определяют протокол для методов, представляемых делегатом.

В соответствии с этой сигнатурой определяются типы аргументов в операторе посылки сообщения о событии. В качестве типа возвращаемого значения в событийном делегате обычно используется **void**.

В качестве примера определим статический метод, посылающий через каждую секунду сообщение о событии. Чтобы такая возможность появилась, необходимо, чтобы в классе, которому принадлежит метод, было объявлено событие, и был доступен соответствующий этому событию делегат-тип. Соответствующие объявления могут быть такими:

```
delegate void TimeHandler(); // объявление делегата-типа  
static event TimeHandler onTime; // объявление события
```

Рекомендуется в название событийного делегата включать в качестве суффикса слово **Handler** (обработчик). Делегат **TimeHandler** в соответствии с его объявлением предназначен «представлять» методы без параметров, с возвращаемым значением типа **void** (ничего не возвращающие в точку вызова). Событие с именем **onTime** «настроено» на работу с экземплярами делегата **TimeHandler**.

В том же классе, где размещено объявление события и доступен делегат-тип, можно определить метод, через каждую секунду «генерирующий» посылку сообщений:

```
static void run()  
{ // процесс с событиями  
    Console.WriteLine("Для выхода нажмите Ctrl+C!");  
    while (true)  
    { // бесконечный цикл  
        onTime(); // посылка сообщения о событии  
        // задержка на секунду  
        System.Threading.Thread.Sleep(1000);  
    }  
}
```

В методе **run()** бесконечный цикл, в каждой итерации которого оператор **onTime()** посылает сообщение о событии, связанном с делегатом **TimeHandler**. Затем вызывается статический метод **Sleep()** класса **Thread** из пространства имен **System.Threading**. Назначение этого метода состоит в «задержке» процесса выполнения программы на количество миллисекунд, соответствующее значению аргумента. В данном случае задержка равна 1000 миллисекунд, т. е. одной секунде.

Метод **run()**, посылая сообщения о событиях, «ничего не знает» о том, кто будет получать эти сообщения, и как они будут обрабатываться. В технологии Windows-программирования принято говорить, что объект (в нашем примере не объект, а статический метод класса) *публикует события*, посылая сообщения о них. Другие объекты (в нашем примере это будут статические методы) могут *подписаться на события*.

Подписка на получение сообщений о событии в языке C# предусматривает следующие действия:

- создание экземпляра того делегата, на который настроено событие;
- подключение экземпляра делегата к событию.

Обычно эти два действия объединяют в одном операторе следующего формата:

```
имя_события += new имя_типа_делегата (имя_метода);
```

Условие применимости подписки на событие — наличие и доступность метода, который будет вызван для обработки события. Имя этого

метода используется в качестве аргумента конструктора делегата. Само собой, и делегат события, и имя события должны быть доступны в месте подписки.

При создании экземпляра делегата операцию **new** и явное обращение к конструктору можно заменить именем метода:

```
имя_события += имя_метода;
```

На одно событие могут быть подписаны несколько методов, для подключения каждого из которых нужно использовать свой оператор приведенного вида.

Предположим, что «принимать сообщения» о событиях в методе `run()` должен метод `Main()` того же класса, в котором определен метод `run()`. Пусть обработчиками сообщения о событии должны быть два метода с заголовками:

```
static void one()  
static void two()
```

Тогда метод `Main` может быть таким (программа 17_08.cs):

```
static void Main()  
{  
    onTime += new TimeHandler(one); // подписка на событие  
    onTime += new TimeHandler(two); // подписка для метода two  
    run(); // запуск процесса  
}
```

Остальное, т. е. функциональность программы в целом, зависит от возможностей и особенностей методов `one()` и `two()`. В следующей ниже программе метод `one()` выводит в консольное окно дату и посекундно изменяющееся значение времени. Метод `two()` в начале той же строки выводит порядковый номер события. Номер изменяется каждую секунду (при каждом обращении к методу).

```
// 17_08.cs - статические события и статические методы  
using System;  
delegate void TimeHandler(); // объявление делегата-типа  
class test_cs  
{  
    static event TimeHandler onTime; // объявление события  
    static void run()  
    { // процесс с генерацией событий  
        Console.WriteLine("Для выхода нажмите Ctrl+C!");  
        while (true)  
        { // бесконечный цикл  
            onTime(); // посылка сообщения о событии  
            // задержка на 1 секунду  
            System.Threading.Thread.Sleep(1000);  
        }  
    }  
    static void Main()
```

```

{
    onTime += new TimeHandler(one); // подписка на событие
    onTime += two;                  // для метода two
    run();                          // запуск процесса
}
static void one()
{ // приемник сообщения
    string newTime = DateTime.Now.ToString();
    Console.Write("\r\t\t{0}", newTime);
}
static int count = 0;
static void two()
{ // приемник сообщения
    Console.Write("\r{0}", count++);
}
}

```

Результат выполнения программы на 6-й секунде:

```

Для выхода нажмите Ctrl+C!
6                11.11.2018 12:59:49

```

В тексте программы обратим внимание на вывод указания пользователю:

```
Console.WriteLine("Для выхода нажмите Ctrl+C!");
```

Сочетание клавиш Ctrl и C приводит к незамедлительному прекращению выполнения программы. Чтобы не «затемнять» основную схему программы, иллюстрирующей только механизм работы с событиями, в нее не введены никакие средства диалога с пользователем.

В строке-аргументе метода консольного вывода Write() управляющая эскейп-последовательность \r обеспечивает при каждом обращении переход в начало строки дисплея. Тем самым вывод все время выполняется с начала одной и той же строки, изображение на которой обновляется каждую секунду.

В методе one() используется свойство Now класса Data.Time. Его назначение — вернуть текущее значение даты и времени. Применение метода ToString() позволяет представить эти значения в виде одной строки, которая затем выводится на дисплей.

Для подсчета событий (секунд) определена статическая переменная **int count**. Ее значение выводит и затем увеличивает на 1 метод two().

В рассмотренном примере делегат-тип объявлен вне классов, и все методы определены как статические — генерацию событий выполняет статический метод run(), подписаны на события два других статических метода. Таким образом, с помощью механизма событий взаимодействуют не объекты, а методы одного класса test_cs. Кроме того, в объявлении делегата-типа отсутствуют параметры. Поэтому при посылке сообщения о событии методы обработки не получают никакой информации из точки возникновения события.

Более общий случай — событие создается объектом, а в других объектах (в объектах других классов) имеется возможность реагировать на эти события. Как мы уже показали, к одному событию может быть «приписано» несколько обработчиков и все они будут вызваны при наступлении события.

Механизм работы с событиями предусматривает несколько этапов.

1. Объявление делегата-типа, задающего протокол для тех (еще неизвестных на данном этапе) методов, которые будут вызываться при обработке события.

2. Определение переменной события, имеющей тип делегата события.

3. Определение генератора события (средства посылки сообщения), с указанием аргументов, при необходимости информирующих получателей о состоянии объекта, пославшего сообщение.

4. Определение методов обработки события. Сигнатура и тип результата каждого метода должны соответствовать типу делегата события.

5. Создание экземпляра того делегата, на который «настроено» событие. Аргумент конструктора — имя метода обработки.

6. Подключение экземпляра делегата к переменной события.

Перечисленные этапы обычно относятся к разным классам программы. И этих разных классов по меньшей мере два. Класс, обрабатывающий события, должен содержать методы обработки или, по крайней мере, иметь доступ к этим методам. В нем реализуются этапы 4, 5, 6. Второй класс — это класс, генерирующий события, он реализует этапы 1, 2, 3.

Зачастую в программе присутствует третий класс, управляющий процессом на более высоком уровне. В разных задачах его называют супервизором, монитором, диспетчером и т. п. При наличии такого класса и двух подчиненных — класса генерации и класса обработки событий — схема работы супервизора сводится к следующим шагам:

1. Создать объект класса генерации событий.

2. Создать объект класса обработки событий (этого может не потребоваться, если метод обработки является статическим).

3. Создать экземпляр делегата, «настроив» его на метод класса обработки событий.

4. Подключить экземпляр делегата к переменной события (принадлежащей объекту класса генерации).

5. Передать управление объекту класса генерации событий (какому-либо из его методов, генерирующих события).

Далее все выполняется в соответствии с общими принципами событийного управления.

В качестве примера рассмотрим программу с делегатом и четырьмя классами. Класс `Sorting` содержит метод с двумя вложенными циклами, который сортирует в порядке возрастания одномерный целочисленный массив. В процессе сортировки подсчитывается количество перестановок значений элементов. При каждом завершении внутреннего цикла формируется событие, передающее «во внешний мир» количе-

ство выполненных перестановок, размер массива и счетчик итераций внешнего цикла. Для работы с событиями в классе объявлено событие `onSort`, имеющее тип внешнего делегата `SortHandler`.

Класс `View` содержит метод обработки события. Метод выводит на консоль значение счетчика перестановок.

Класс `Display` визуализирует динамику процесса сортировки — выводит на консоль имитацию элемента управления `ProgressBar`.

Метод `Main()` управляющего класса `Controller` в соответствии с общей схемой создает объект класса, генерирующего события, создает объект класса-обработчика (`View`). Затем подключает к переменной события два наблюдателя — два безымянных экземпляра делегата `SortHandler`. И, наконец, управление передается генератору событий — методу сортировки `sort()`, принадлежащего объекту класса `Sorting`.

```
// 17_09.cs – события и сортировка
using System;
using System.Text;
// Объявление делегата-типа:
public delegate void SortHandler(long cn, int si, int kl);
class Sorting
{ // класс сортировки массивов
    int size; // размер массива
    int[] ar; // ссылка на массив
    public long count; // счетчик обменов при сортировке
    public event SortHandler onSort; // объявление события
    public Sorting(int[] ls)
    { // конструктор
        size = ls.Length;
        count = 0;
        ar = ls;
    }
    public void sort()
    { // сортировка с посылкой извещений
        int temp;
        for (int i = 0; i < size - 1; i++)
        {
            for (int j = i + 1; j < size; j++)
                if (ar[i] > ar[j])
                {
                    temp = ar[i];
                    ar[i] = ar[j];
                    ar[j] = temp;
                    count++;
                }
            if (onSort != null)
                onSort(count, size, i); // генерация события
        }
    }
}

class View
```

```
{ // Обработчик событий в объектах:
    public void nShow(long n, int si, int kl)
    { Console.WriteLine("\r" + n); }
}

class Display
{ // Обработчик событий в этом классе
    static int len = 30;
    static string st = null;
    public static void barShow(long n, int si, int kl)
    {
        int pos = Math.Abs((int)((double)kl / si * len));
        string s1 = new string('\u258c', pos);
        string s2 = new string('-', len - pos - 1) +
            '\u25c4'; // unicode для треугольника;
        st = s1 + '\u258c' + s2; // '\u258c' - код прямоугольника
        Console.WriteLine("\r\t\t" + st);
    }
}

class Controller
{
    static void Main()
    {
        Random ran = new Random(55);
        int[] ar = new int[19999];
        for (int i = 0; i < ar.Length; i++)
            ar[i] = ran.Next();
        Sorting run = new Sorting(ar);
        View watch = new View(); // Создан объект
        run.onSort += new SortHandler(Display.barShow);
        run.onSort += new SortHandler(watch.nShow);
        run.sort();
        Console.WriteLine("\n");
    }
}
```

Результат выполнения программы:

100372610 ◀

Обратите внимание, что событийный делегат `SortHandler` и переменная события `onSort` должны быть одинаково доступны в месте подписки на событие.

При генерации события целесообразно проверять значение переменной события. Эта переменная остается равной **null**, если на событие нет ни одной подписки.

Обратите внимание также на тот факт, что генерация события, в отличие от генерации исключения, оформляется как обращение к методу. Благодаря этому после обработки события управление автоматически возвращается в точку, непосредственно следующую за оператором генерации события.

Контрольные вопросы и задания

1. В чем основное назначение делегата?
2. Назовите этапы создания и применения делегатов.
3. Члены каких видов могут присутствовать в делегате-типе?
4. Объясните назначение элементов объявления делегата-типа.
5. Как объявить ссылку с типом делегата?
6. Как создать экземпляр делегата?
7. Как аргументы можно использовать при обращении к конструктору делегата?
8. Где может размещаться объявление делегата-типа?
9. Назовите варианты инициализации переменной с типом делегата.
10. Каковы возможности свойств Method и Target?
11. Для чего применяются массивы делегатов?
12. Что такое многоадресный экземпляр делегата?
13. Какие средства поддерживают работу с многоадресными экземплярами делегатов?
14. Что такое механизм обратного вызова?
15. Как используются делегаты для организации обратных вызовов?
16. Что такое анонимный метод?
17. Как специфицируется сигнатура анонимного метода?
18. Приведите пример размещения анонимного метода в обращении к методу, требующему обратных вызовов.
19. Перечислите сходства и отличия лямбда-выражений и анонимных методов.
20. Объясните синтаксис записи лямбда-выражений.
21. Когда лямбда-выражение можно записать без параметров?
22. Что такое захваченная переменная?
23. В какой момент определяется значение захваченной переменной?
24. Что такое событие в языке C#?
25. Объясните синтаксис оператора отправки сообщения.
26. Приведите формат объявления события.
27. Что такое переменная события?
28. Что определяет делегат, указанный в объявлении события?
29. Какие действия предусматривает подписка на события?
30. Назовите этапы работы с событиями.

Глава 18

ОБОБЩЕНИЯ

18.1. Обобщения как средство абстракции

Код, записанный на каком-либо языке программирования, представляет собой наиболее детализированное и поэтому в общем случае сложное представление алгоритма, выбранного для решения конкретной задачи. Сложность кода объясняется тем обстоятельством, что в нем должны быть учтены все детали реализации алгоритма с помощью применяемого языка и ограничения той операционной среды, в которой должна выполняться программа. Сложность кода и трудоемкость его разработки делают задачи его повторного использования и автоматизации его генерации весьма важными и актуальными. В предыдущих главах рассмотрены средства, позволяющие программировать на основе объектно-ориентированного подхода. Этот подход позволяет создавать отдельные классы и библиотеки классов, пригодные для повторного использования. Применяя библиотеку классов, можно существенно снизить трудоемкость разработки программ, абстрагируясь от деталей кода, скрытых (инкапсулируемых) в используемых объектах библиотечных классов.

Следующий уровень абстракции — параметризация объявлений типов (например, классов) и методов. Прежде чем переходить к описанию этого механизма (реализованного в языке C# с помощью обобщений), поясним на примере его основные принципы.

Предположим, что для дальнейшего применения необходимо иметь класс, объект которого представляет собой отдельный элемент связанного списка, каждый из которых хранит значение типа **int**.

```
class ListInt
{
    public int data;
    public ListInt (int d)
    {data = d;}
    ...
}
```

Если в дальнейшем потребуется класс элементов связанного списка, объекты которого хранят значения типа **char**, то придется объявлять его, например, так:

```

class ListChar
{
    public char data;
    public ListInt (char d)
    {data = d;}
    ...
}

```

По структуре класс ListInt и ListChar подобны, единственное отличие — типы полей `data` и типы параметров конструкторов в одном случае `int`, а во втором `char`. Этот факт, а именно различие классов только в обозначениях типов, делает возможным создание обобщенного (параметризованного) класса, который будет служить шаблоном для автоматической генерации его частных случаев — классов с полями типов `int`, `char`, а при необходимости `long`, `double` и т. д.

В языке C# такой механизм автоматизированной генерации кодов конкретных объявлений существует. Определения (объявления) классов, структур, интерфейсов, делегатов и методов могут быть параметризованы типами тех данных, которые представлены этими конструкциями или обрабатываются с их помощью. О такой возможности параметризации объявлений говорят, используя термин «обобщения». Механизм обобщений языка C# схож с механизмом шаблонов (*templates*) классов и функций языка C++. Кроме того, как указывает стандарт C#, обобщения хорошо знакомы программистам, владеющим языками Эффель или Ада. Отметим, что в русскоязычной литературе, посвященной языкам Эффель и Ада обобщенные (*generic*) методы называют родовыми подпрограммами, а об обобщенном программировании говорят как о родовом программировании.

Параметризованные объявления классов и структур называют, соответственно, *объявлениями обобщенных классов* (*generic class declaration*) и *объявлениями обобщенных структур* (*generic struct declaration*). И классы и структуры в этих случаях параметризованы типами своих данных и типами тех данных, которые должны обрабатываться методами этих классов и структур. Интерфейсы могут быть параметризованы типами тех данных, которые обрабатываются функциональными членами (например, методами) интерфейсов после их реализации. Объявления таких параметризованных интерфейсов называют *объявлениями обобщенных интерфейсов* (*generic interface declaration*). Для того, чтобы создавать «обобщенные алгоритмы», выполняют параметризацию методов типами применяемых в них данных. Такие параметризованные методы называют *обобщенными методами* (*generic methods*). В объявлении *обобщенного делегата* типизирующие параметры определяют типы параметров и тип возвращаемого значения тех методов, которые должны представлять экземпляры делегата.

18.2. Декларации обобщенных классов

Рассмотрим формат декларации обобщенного (параметризованного) класса:

```
модификаторы_классаopt class имя_класса  
список_типизирующих_параметров  
база_классаopt  
ограничения_типизирующих_параметровopt  
тело_класса;opt
```

В приведенном формате обязательными являются: служебное слово **class**, *имя_класса* (а это, как мы знаем, — идентификатор), *список_типизирующих_параметров* и *тело_класса*. Основным признаком обобщенного класса служит наличие в его объявлении *списка_типизирующих_параметров*. Если в объявлении опустить этот *список* и, конечно, *ограничения_типизирующих_параметров*, то объявление превратится в определение обычного (непараметризованного) класса.

Список типизирующих параметров представляет собой заключенную в угловые скобки < > последовательность разделенных запятыми идентификаторов. Обратите внимание, что обобщенный класс может быть наследником базового класса или может реализовать интерфейс, причем и базовый класс, и реализуемый интерфейс, в свою очередь, могут быть параметризованными. В приведенном формате декларации обобщенного класса возможность наследования обозначена конструкцией «*база_класса*».

В стандарте C# для знакомства с обобщенными классами предлагается рассмотреть примерно такое объявление:

```
class Stack<ItemType>  
{ // стек для любых данных  
    static int stackSize = 100; // предельный размер стека  
    private ItemType[] items = new ItemType[stackSize];  
    private int index = 0; // номер свободного элемента  
    public void Push(ItemType data)  
    { // поместить в стек  
        if (index == stackSize)  
            throw new ApplicationException("Стек переполнен!");  
        items[index++] = data;  
    }  
    public ItemType Pop()  
    { // взять из стека  
        if (index == 0)  
            throw new ApplicationException("Стек пуст!");  
        return items[--index];  
    }  
}
```

Объявление вводит обобщенный класс (с именем *Stack*) для представления стека, т. е. такой структуры данных, которая позволяет запоминать последовательно передаваемые ей значения и извлекать

их в порядке, обратном их поступлению. Поступающие в такой стек значения запоминаются в одномерном массиве из 100 элементов с именем (ссылкой) `items`. Особенность объявления этого массива в том, что тип его элементов определяет типизирующий параметр с именем `ItemType`. Для работы с элементами стека в обобщенном классе определены два метода — `Push()` и `Pop()`. Первый из них получает через параметр некоторое значение и должен поместить его в массив, связанный со ссылкой `items`. Метод `Pop()` извлекает из массива последний из поступивших туда элементов и возвращает его значение в точку вызова. Самое важное здесь — использование типизирующего параметра `ItemType`. Он определяет тип элементов массива, тип параметра метода `Push()` и тип возвращаемого методом `Pop()` значения.

Основная идея обобщений состоит в том, что декларация обобщенного класса служит шаблоном для самых разнообразных уже не параметризованных классов, которые компилятор автоматически строит, исходя из декларации обобщенного класса и имеющихся в программе конкретных «обращений» к этой декларации. Конструкция, которую мы условно назовем «обращением к декларации обобщенного класса», имеет вид:

имя_обобщенного_класса <список_типизирующих_аргументов>

Здесь каждый типизирующий аргумент — имя конкретного типа, которое должно заменить в объявлении обобщенного класса все вхождения соответствующего типизирующего параметра. Соответствие между типизирующими параметрами в декларации обобщенного класса и типизирующими аргументами в обращении к нему устанавливается по их взаимному расположению. Здесь используется традиционная для обращений к методам, функциям и процедурам схема замещения параметров (формальных параметров) аргументами (фактическими параметрами).

Если в программе, содержащей декларацию обобщенного класса `Stack<ItemType>`, использовать конструкцию `Stack<char>`, то это вводит специализированный (инстанцированный, т. е. конкретный) тип, порождаемый из декларации обобщенного класса `Stack<ItemType>`. В этом специализированном типе, который существует только после компиляции программы, все вхождения типизирующего параметра `ItemType` заменены типом `char`. Элементы массива, связанного со ссылкой `items`, будут иметь тип `char`, метод `Pop()` будет возвращать символьные значения, параметр метода `Push()` будет символьного типа.

Для обозначения специализированного типа стандарт С# вводит термин «сконструированный тип» (*constructed type*). Имена сконструированных типов можно использовать как и обычные имена непараметризованных классов. Например, так:

```
Stack<char> symbols = new Stack<char>();
symbols.Push('A');
char ch = symbols.Pop();
```

В данном примере создан экземпляр (объект) класса `Stack<char>`, определена и связана с этим объектом ссылка `symbols` с типом того же класса. Затем в стек помещен символ 'A' и с помощью метода `Pop()` этот символ получен (извлечен) из стека.

Точно так же, как для хранения в стеке символьных данных определен класс `Stack<char>`, можно вводить сконструированные типы для стеков с элементами любых типов. Это могут быть как предопределенные типы языка, так и типы, введенные программистом. Например:

```
Stack<double>real = new Stack <double>();  
real.Push(3.141592);  
double pi = real.Pop();
```

Введя сконструированный тип, мы можем использовать его только для работы с теми данными, «на которые он настроен». С учетом предыдущего объявления переменной `symbol` следующий оператор неверен:

```
symbols.Push(2.7171); // ошибка компиляции
```

В декларации обобщенного класса может быть несколько типизирующих параметров. Ограничений на их количество нет. Естественное требование — число типизирующих аргументов должно быть равно количеству типизирующих параметров в декларации обобщенного класса.

Каждый типизирующий параметр определяет имя в пространстве декларации своего класса. Таким образом, имя типизирующего параметра не может совпадать: с именем другого типизирующего параметра своего же класса, с именем обобщенного класса, с именем члена этого класса.

Область существования типизирующего параметра включает базу класса, ограничения типизирующих параметров и тело класса. Типизирующие параметры не распространяются на производные классы. В области существования типизирующий параметр можно использовать как имя типа.

18.3. Ограничения типизирующих параметров

В приведенном выше формате объявления обобщенного класса указан один весьма важный раздел, который определяет ограничения, накладываемые на типизирующие аргументы. Он назван «ограничения_типизирующих_параметров». Рассмотрим его назначение.

Во многих случаях обобщенный класс не только хранит значения, тип которых определен типизирующим параметром, но и включает средства (обычно это методы класса или его объектов) для обработки этих значений. Предположим, что в экземплярах (в объектах) специализированных типов, порождаемых обобщенным классом `Stack<ItemType>`, нужно хранить только такие данные, для которых

выполняется определенное условие, истинность которого устанавливается с помощью метода `CompareTo()`. Проверку этого условия можно выполнять в методе `Push()`, определив его, например, таким образом:

```
public void Push (ItemType data)
{
    if (((IComparable)data).CompareTo(default(ItemType))<0)...
    return;
    ...
}
```

В теле метода `Push()` значение параметра `data` приводится к значению типа интерфейса `IComparable`. Только после этого к результату приведения можно будет применить метод `CompareTo()`, специфицированный в интерфейсе `IComparable`. Этот интерфейс определен в пространстве `System` и предназначен для сравнения объектов. Декларация интерфейса:

```
interface IComparable
{
    int CompareTo (object p);
}
```

Как видите, интерфейс содержит прототип только одного метода `CompareTo()`. Назначение метода — сравнивать значение того объекта, к которому он применен (для которого вызван этот метод), со значением объекта-аргумента. Этот аргумент заменяет параметр `p`, типом которого служит класс **object**. Так как **object** — общий базовый класс для классов библиотек и программ на C#, то аргумент может иметь любой тип. Предполагается, что целочисленный результат, возвращаемый методом `CompareTo()`, удовлетворяет следующим соглашениям.

Результат меньше нуля, если вызывающий объект нужно считать меньшим, нежели объект-параметр.

Результат больше нуля, если вызывающий объект нужно считать бóльшим, нежели объект-параметр.

Результат равен нулю, если вызывающий объект нужно считать равным объекту-параметру.

В методе `Push()` при обращении к методу `CompareTo()` в качестве аргумента использовано особое выражение механизма обобщений `default(ItemType)`. Результат его вычисления — принятое по умолчанию значение того типа, который будет замещать типизирующий параметр `ItemType` при создании специализации (инстанцировании, конкретизации) обобщенного класса `Stack<>`.

После такого изменения метода `Push()` на основе обобщенного класса `Stack<ItemType>` можно создавать сконструированные типы, используя только типизирующие аргументы, классы которых реализовали интерфейс `IComparable`. Предопределенные типы языка C#, такие как, например, **int** (`System.Int32`) или **double** (`System.Double`) реализуют интерфейс `IComparable`. При других типизирующих аргументах

на этапе исполнения программы будет генерироваться исключение `InvalidCastException`. Самое главное и плохое — допущенная ошибка в использовании неверного типизирующего аргумента не будет распознаваться на этапе компиляции.

Компилятор сможет идентифицировать неверное задание типизирующих аргументов только в том случае, если в декларацию обобщенного класса включить так называемый *список ограничений* (*list of constraints*) *типизирующих параметров*.

Элемент этого списка имеет вид:

```
where имя_типизирующего_параметра: список_ограничений
```

Здесь **where** — контекстно-зависимое служебное слово языка C#, за ним следует имя того типизирующего параметра обобщенного класса, для которого вводятся ограничения. Список допустимых ограничений представляет собой список условных обозначений тех ограничений, которые накладываются на типизирующие аргументы, которые заменяют соответствующий типизирующий параметр. Весь набор условных обозначений ограничений рассмотрим позже, а сейчас вернемся к примеру.

В нашем примере со стеком, помещать в который можно только значения, тип которых допускает применение метода `CompareTo()`, обобщенный класс можно определить так:

```
public class Stack<ItemType>
where ItemType : IComparable
{
    private ItemType[] item = new ItemType[100];
    public void Push(ItemType data)
    {
        if (data.CompareTo(default(ItemType)) < 0) return;
    }
    public ItemType Pop() {...}
}
```

На основе такой декларации обобщенного класса компилятор будет проверять все специализации и сообщать об ошибке в каждом случае, когда типизирующий аргумент не является классом, реализовавшим интерфейс `IComparable`. Обратите внимание, что теперь в методе `Push()` нет (не требуется) приведения типа параметра к типу интерфейса `IComparable`.

В данном примере обобщенного стека только один типизирующий параметр и для него введено только одно ограничение — соответствующий типизирующий аргумент должен быть именем класса, реализовавшего конкретный интерфейс `IComparable`.

В обобщении может быть несколько типизирующих параметров и для каждого из них может быть указано свое ограничение, вводимое контекстно-зависимым словом **where**. В то же время для одного пара-

метра в одном списке **where** можно указать несколько видов ограничений.

Таким образом, при определении обобщенного класса можно указать ограничения, которым должны соответствовать типизирующие аргументы при инстанцировании конкретного класса. Если в программе сделана попытка сформировать специализацию обобщенного класса с использованием типизирующих аргументов, которые не соответствуют заданным ограничениям, то результатом будет ошибка компиляции.

Определены шесть видов ограничений типизирующих параметров:

- **where T: struct** — типизирующий аргумент должен быть типом значений;
- **where T: class** — типизирующий аргумент должен быть типом ссылок. Ссылки могут относиться к любым классам, интерфейсам, делегатам и типам массивов;
- **where T: имя_интерфейса** — типизирующий аргумент должен быть типом, реализовавшим указанный интерфейс;
- **where T: new()** — типизирующий аргумент должен иметь конструктор без параметров. Когда для одного типизирующего параметра используется несколько ограничений, ограничение **new()** в списке должно быть последним;
- **where T: имя_класса** — типизирующий аргумент должен быть либо указанным классом, либо классом, производным от него;
- **where T: U**, где U — типизирующий параметр обобщения. При таком ограничении подставляемый вместо параметра T типизирующий аргумент должен быть либо тем же аргументом, который заменяет параметр U, либо должен быть именем производного от U класса. Этот вид ограничения называют *ограничением с помощью явного типа* (*naked type constraint*).

Список ограничений типизирующих параметров позволяет компилятору удостовериться, что операции и методы в теле обобщенного класса допустимы при тех типизирующих аргументах, которые будут применяться при создании специализаций обобщенного класса.

Типизирующие параметры, для которых ограничения не указаны, называют свободными (*unbounded*) типизирующими параметрами. Для таких параметров требуется соблюдение следующих требований:

- операции **!=** и **==** нельзя использовать, так как нет уверенности, что конкретные типизирующие аргументы будут поддерживать эти операции;
- они допускают приведение к типу **System.Object** или выведены из этого типа;
- они могут явно приводиться к некоторому интерфейсному типу;
- их можно сравнивать со значением **null**. Когда свободный типизирующий параметр сравнивается со значением **null**, то результат сравнения всегда **false**, если типизирующий аргумент является типом значений.

Задание ограничений с помощью явного указания типа полезно в тех случаях, когда член обобщенного класса имеет собственный типизирующий параметр, и необходимо, чтобы этот параметр соответствовал типизирующему параметру обобщенного класса. Пример из MSDN:

```
class List <T>
{
    void Add<U> (List< U > item) where U:T {...}
}
```

В данном примере обобщенный класс `List <T>` включает в качестве члена обобщенный метод `Add<U>` (обобщенным методам посвящен далее параграф 18.6). Для типизирующего параметра `U` ограничение задано с помощью указания явного типа `T`. Таким образом, `T` — ограничение (заданное указанием явного типа) в обобщенном методе `Add()`. В то же время `T` в контексте объявления обобщенного класса `List <T>` является свободным типизирующим параметром.

Ограничение с помощью указания явного типа может непосредственно использоваться в объявлении обобщенного класса. Пример из MSDN:

```
class SampleClass<T,U,V> where T:V {}
```

В данном примере требуется, чтобы аргумент, заменяющий параметр `T`, был типом, заменяющим параметр `V`, либо производным от него типом.

Полезность ограничений явным типом в объявлениях обобщенных классов наиболее велика в тех случаях, когда необходимо подчеркнуть отношение наследования между двумя типизирующими параметрами.

Перечисленные виды ограничений неравнозначны. Первичными ограничениями являются *имя_класса*, **class** и **struct**.

Вторичные ограничения: *имя_интерфейса*, *типизирующий_параметр* (ограничение с помощью указания явного типа).

Ограничения третьего уровня: **new()**.

Именно в таком порядке (первое, второе, третье) ограничения могут входить в список **where** для одного конкретного типизирующего параметра. При этом первичное ограничение может быть только одно, число вторичных ограничений может быть любым и ограничение третьего уровня, т. е. **new()**, может быть только одно. Ограничения в списке **where** отделяются друг от друга запятыми. Ограничения разных параметров (отдельные списки **where**) никак не разделяются (обычно их разделение обозначается пробельными символами).

При ограничении в виде имени класса:

- этот класс не должен быть запечатан (**sealed**);
- он не может иметь тип: `Array`, `Delegate`, `Enum`, и не может быть типом значений (`Value Type`);
- он не может иметь тип **object**.

Примеры объявлений с ограничениями типизирующих параметров:

```
class MyClass < T > where T: class...  
class newClass <T, F> where T: Stream  
where F: IComparable <int>, new()...
```

18.4. Обобщенные структуры

Правила объявления и использования обобщенных структур не отличаются от правил для обобщенных классов. Для обобщенных структур типизирующие параметры указываются в угловых скобках вслед за именем структурного типа. Для типизирующих параметров могут быть указаны ограничения, а правила объявления ограничений те же, что и для классов.

Так как у обобщенных структур много общего с обобщенными классами, то рассмотрим здесь только те правила, о которых не говорилось в связи с классами (хотя эти правила общие и для классов, и для структур).

Начнем с того, что обобщения и классов, и структур допускают *перегрузку*. Основой перегрузки является количество типизирующих параметров в объявлении одноименных типов. При перегрузке обобщенных типов конкретный тип определяется по числу аргументов, использованных при инстанцировании. Пример перегрузки обобщенных структур:

```
struct Element<D>  
{  
    public D memb;  
}  
struct Element<E, R>  
{  
    public E memb1;  
    public R memb2;  
}
```

В данном случае два открытых типа (два обобщенных структурных типа) имеют одинаковые имена, но разное количество типизирующих параметров. Имена этих параметров не имеют значения при перегрузке. Невозможно объявить в том же пространстве имен, например, обобщенную структуру с заголовком **struct** Element<T, F>.

Обобщенный тип называют открытым типом, а сконструированный на его основе полностью специфицированный тип — закрытым, подчеркивая, что на основе этого сконструированного типа уже нельзя объявлять другие типы.

Пример переменной и экземпляра структуры закрытого типа:

```
Element<int> ded = new Element<int>();  
Console.WriteLine("ded.memb = " + ded.memb);
```

Результат выполнения:

```
ded.memb = 0
```

Как всегда по умолчанию члены структур инициализируются умалчиваемыми значениями соответствующих типов. В нашем примере целочисленное поле структуры равно нулю.

Следует отметить, что обобщенный тип и одноименный с ним регулярный тип считаются разными типами. Например, наряду с обобщенным классом `Example<U>` в той же программе можно декларировать такой регулярный класс:

```
public class Example {}
```

Это не приведет к ошибочной ситуации.

Обобщенные типы могут включать *статические члены*. В этих случаях каждый статический член обобщенного типа при создании сконструированных типов «размножается», т. е. каждому закрытому типу принадлежит свой экземпляр статического члена обобщенного типа. Пример обобщенного типа структур со статическим членом (программа 18_02.cs):

```
using System;
struct Memb<T>
{
    public static int count;
    public T data;
    public Memb(T d)
    {
        data = d;
        count++;
    }
}
class Program
{
    static void Main()
    {
        Memb<char> mc1 = new Memb<char>('Z');
        Memb<char> mc2 = new Memb<char>('F');
        Memb<byte> mb = new Memb<byte>(12);
        Console.WriteLine("Memb<char>.count = " +
            Memb<char>.count);
        Console.WriteLine("Memb<byte>.count = " +
            Memb<byte>.count);
    }
}
```

Результаты выполнения программы:

```
Memb<char>.count = 2
Memb<byte>.count = 1
```

В обобщенный структурный тип `Memb<T>` входит счетчик экземпляров **`public static int count`**. По умолчанию он инициализируется нулевым значением. При каждом обращении к конструктору значение счетчика увеличивается на 1. Тип поля данных `data` и тип параметра

конструктора определяются типизирующим параметром обобщенной структуры, т. е. зависит от типизирующего аргумента. В основной программе созданы два объекта структуры `Memb<char>` и один экземпляр типа `Memb<byte>`. Результаты выполнения программы иллюстрируют правило «размножения» статических членов обобщенных типов. Статический член принадлежит не обобщенному типу, а в каждый из сконструированных на его основе типов входит свой собственный статический член (он в свою очередь может быть параметризован типизирующим параметром, но это не показано в приведенном примере).

В качестве примера обобщенного структурного типа со статическими методами рассмотрим такую программу:

```
// 18_03
using System;
struct GenStr<T, V>
{
    static GenStr()
    { // статический конструктор
        Console.WriteLine("{0}+{1}", typeof(T).Name, typeof(V).Name);
    }
    public static void method() { } // статический метод
}
class Program
{
    static void Main()
    {
        GenStr<string, int>.method();
    }
}
```

В обобщенном структурном типе `GenStr<T, V>` объявлены статический конструктор и статический метод `method()`, не имеющий параметров и «ничего не делающий». В основной программе конструируется закрытый структурный тип `GenStr<string, int>` и для него выполняется обращение к статическому методу `method()`. Тем самым неявно вызывается статический конструктор, в теле которого выводятся названия типов, использованных в качестве типизирующих аргументов при объявлении закрытого структурного типа. Результаты выполнения программы:

String+Int32

При вложении обобщенных типов рекомендуется для типизирующих параметров внешнего и вложенного обобщенных типов использовать разные идентификаторы. Предположим обратное, т. е. объявим такую обобщенную структуру, в теле которой объявлена другая обобщенная структура с тем же именем типизирующего параметра:

```
struct Memb<T>
{
```

```

...
struct into<T>
{
    T field;
}
}

```

Объявление корректно, но при создании закрытого типа, например, `Memb<int>`, вложенная структура продолжает иметь открытый тип `into<T>`, и конкретный тип поля `field` остается, по крайней мере, непонятным. В следующем примере показано корректное объявление вложенных обобщенных структур:

```

struct Memb<T>
{
    struct into<U>
    {
        T field1;
        U field2;
    }
}

```

Типизирующий параметр `T` внешнего обобщенного типа доступен и в нем, и во вложенном типе. В то же время параметр `U` доступен только во вложенном обобщенном типе. При таком объявлении закрытый тип `Memb<int>` определяет тип поля `field1`, но оставляет обобщенной структуру `into<U>` и не влияет на тип поля `field2`.

18.5. Обобщенные интерфейсы

Напомним, что членами интерфейса могут быть прототипы методов, свойств, индексаторов, событий. В обобщенном интерфейсе типизирующие параметры определяют типы возвращаемых значений и параметров членов интерфейса. Как и для классов и структур, признаком обобщенного интерфейса является список типизирующих параметров в его объявлении. Типизирующие параметры — это идентификаторы, разделенные запятыми и помещенные в угловые скобки. Пример:

```

public interface IExample<T>
{ // обобщенный интерфейс
    char this[T index] {get; set;} // прототип индексатора
    int add(T x, T y); // прототип метода
}

```

В обобщенном интерфейсе `IExample<T>` один типизирующий параметр. Он специфицирует тип параметра прототипа индексатора, а для прототипа метода — определяет типы параметров.

Обобщенный интерфейс может быть реализован как обобщенным, так и регулярным типом. Пример реализации обобщенного интерфейса обобщенным классом:


```
// реализация обобщенным классом
class real<U> : IExample<U>
{
    public char this[U r]
    {
        get {return r.ToString()[0];}
        set{}
    }
    public int add(U a, U b)
    {return a.GetHashCode() + b.GetHashCode();}
}
```

В реализации индексатор возвращает первый символ строкового представления индекса, использованного при обращении к индексатору. В качестве реализации аксессуора **set** использована «заглушка». Реализация метода **add(U a, U b)** выполняет суммирование хеш-кодов аргументов. Так как методы **ToString()** и **GetHashCode()** присущи всем классам языка **C#**, то в объявление обобщенного класса не потребовалось включать ограничения на типизирующий параметр.

Применение обобщенного класса **real<U>** иллюстрирует следующий код:

```
class Program
{
    static void Main()
    {
        var temp1 = new real<int>();
        var temp2 = new real<byte>();
        Console.WriteLine(temp1[655]);
        Console.WriteLine(temp1.add(950, 6));
        Console.WriteLine(temp2[155]);
        Console.WriteLine(temp2.add(34, 6));
    }
}
```

Результаты выполнения программы:

```
6
956
1
40
```

Регулярный тип (класс или структура) может реализовать несколько специализаций интерфейсов, сконструированных на основе одного обобщенного интерфейса. Общие принципы этого механизма иллюстрирует следующий код с тривиальными реализациями членов интерфейсов:

```
struct Realization : IExample<string>, IExample<double>
{
    public char this[string st]
    { get { return '#'; } set { } }
    public int add(string s1, string s2)
```

```

{ return -135; }
public char this[double d]
{ get { return (char)d; } set { } }
public int add(double d1, double d2)
{ return (int)(d1 - d2); }
}

```

В данном случае регулярная структура **struct** Realization реализует два интерфейса, сконструированных на основе обобщенного интерфейса **IExample<T>**. При такой реализации в структуре создаются перегруженные методы **add()** и перегруженные индексаторы. В следующем фрагменте кода создается один экземпляр структуры Realization и для него с аргументами разных типов вызываются метод **add()** и индексатор.

```

static void Main()
{
    Realization exemplar = new Realization();
    Console.WriteLine(exemplar.add(5.6, 3.3));
    Console.WriteLine(exemplar.add("123", "abc"));
    Console.WriteLine(exemplar[5.6]);
    Console.WriteLine(exemplar["screb"]);
}

```

В консольное окно выводятся следующие значения:

```

2
-135
♣
#

```

При реализации обобщенного интерфейса обобщенным типом необходимо следить, чтобы не появлялись такие комбинации типизирующих аргументов, которые приведут к дублированию интерфейса в этом типе.

Пример заголовка, который вызовет ошибку компиляции:

```

public class Example<N> : IExample<char>, IExample<N>

```

В этом классе предлагается реализовать сконструированный тип (интерфейс) **IExample<char>** и обобщенный интерфейс **IExample<N>** с типизирующим параметром **N**. Если при инстанцировании класса **Example<N>** вместо параметра **N** будет подставлен аргумент **char**, возникнет конфликт, так как класс должен будет реализовывать два одинаковых интерфейса, что невозможно.

18.6. Обобщенные методы

Обобщенными могут быть объявлены не только типы (класс, структура, интерфейс, делегат), но и методы, т. е. один из видов членов типов. Сразу же отметим, что среди всех членов типов обобщения возможны

только для методов. Недопустимы обобщения свойств, индексаторов, перегруженных операций, деструкторов. Среди типов недопустимы обобщения перечислений и событий.

Итак, в отличие от других обобщений методы являются не типами, а членами типов. Обобщенными могут быть объявлены методы: классов, структурных типов и интерфейсов. Можно создавать обобщенные методы экземпляров и статические методы, виртуальные методы и абстрактные методы. Объявлять обобщенными можно методы регулярных типов и методы обобщенных типов.

Декларация обобщенного метода обязательно включает список типизирующих параметров. Как обычно, это заключенный в угловые скобки список идентификаторов, разделенных запятыми. Размещается этот список непосредственно после имени метода перед круглыми скобками со спецификацией параметров метода. Таким образом, объявление обобщенного метода имеет два списка параметров — типизирующие параметры и параметры метода. Если для типизирующих параметров необходимо ввести ограничения (**where**), то они помещаются перед телом метода после закрывающей круглой скобки спецификации параметров (или перед точкой с запятой, если это обобщенный метод интерфейса).

Пример заголовка обобщенного метода, формирующего одномерный массив со случайными значениями элементов из интервала [*mi*, *ma*):

```
public static T[] randArray<T>(int n, T mi, T ma)
    where T : struct, IComparable<T>
```

Метод открытый, статический, возвращающий ссылку на массив с элементами, тип которых определяет типизирующий параметр *T*. Первый параметр метода **int** *n* определяет размер массива — количество его элементов. Второй и третий параметры задают диапазон случайных значений элементов создаваемого массива. На типизирующий параметр наложено ограничение — аргумент должен быть типом значений (ограничение **struct**), и этот тип должен реализовать стандартный обобщенный интерфейс *IComparable<T>*.

В качестве второго примера обобщенного метода приведем код метода для вывода в консольное окно значений элементов одномерного массива.

```
public static void printArray<T> (T[] a, string format = "{0} ")
{
    for (int i = 0; i < a.Length; i++)
        Console.Write(format, a[i]);
    Console.WriteLine();
} // printArray<T>()
```

В обобщенном методе *printArray<T>()* один типизирующий параметр *T*, специфицирующий тип первого параметра метода, т. е. тип элементов выводимого массива. Второй параметр метода определяет

форматную строку, которую при обращении к методу можно выбирать в некоторой степени независимо от типа первого аргумента (формат можно выбирать и соответствующий типу первого параметра, но соответствие выбирается программистом при обращении к методу). Вторым параметром имеет по умолчанию значение "{0} ". Таким образом, при отсутствии в обращении к методу второго аргумента значения элементов массива выводятся преобразованными к типу **string** и разделяются пробелами.

Для обращения к обобщенному методу необходимо в общем случае после его имени поместить список типизирующих аргументов и заменить аргументами параметры метода. При обработке такого обращения, которое происходит во время выполнения программы, создается конкретная версия обобщенного метода, в которой учтены реальные типизирующие аргументы, и именно эта версия метода выполняется. Как и в случае обобщенных типов, создание конкретной (сконструированной) версии обобщенного метода называют инстанцированием.

В качестве примера использования обобщенного метода рассмотрим следующий код:

```
static void Main()
{
    double[] dArray = new double[] {0, 1, 2, 3, 4};
    printArray<double>(dArray, "{0:E1} ");
    int[] testArray = new int[] {0, 10, 20, 30, 40,
                                50, 60, 70, 80, 90};
    printArray(testArray);
}
```

В программе определены и инициализированы вещественный и целочисленный массивы, ассоциированные со ссылками **double[]** dArray и **int[]** testArray. Обращение **printArray<double>(dArray, "{0:E1}");** обеспечивает создание варианта метода, настроенного на вывод вещественных элементов массива с использованием для каждого значения поля подстановки {0:E1}.

При втором обращении к обобщенному методу вместо полной формы вида **printArray<int>(testArray, "{0} ")** использована сокращенная форма его вызова **printArray(testArray)**. В ней отсутствует второй аргумент метода, и нет списка типизирующих аргументов. Вторым аргументом метода в этом случае получает умалчиваемое значение параметра "{0} ". А типизирующий аргумент компилятор автоматически «выводит», анализируя тип первого аргумента метода.

Результаты выполнения программы:

```
0,0E+000 1,0E+000 2,0E+000 3,0E+000 4,0E+000
0 10 20 30 40 50 60 70 80 90
```

Поясним правила определения выводимых типов, которыми пользуется компилятор. Типизирующий параметр не всегда может быть

использован для задания типов параметров метода. Он в этом случае может определять тип возвращаемого методом значения или специфицировать локальные переменные в теле метода. В этом случае, анализируя обращение к обобщенному методу, невозможно узнать, каков тип соответствующего аргумента. Если, как в нашем примере, типизирующий параметр специфицирует один или более параметров метода, то сделать вывод о реальном типе типизирующего параметра можно по типам аргументов, заменивших параметры в обращении к методу.

В качестве примера обобщенного метода с параметрами, передаваемыми по ссылкам, рассмотрим метод обмена значений аргументов:

```
public static void change <U> (ref U g, ref U s)
{
    U temp = g;
    g = s;
    s = temp;
}
```

Типизирующий параметр U специфицирует параметры метода, передаваемые по ссылкам. В теле обобщенного метода тот же типизирующий параметр определяет тип вспомогательной локальной переменной temp. В следующем фрагменте кода показано обращение к методу change(), не содержащее типизирующего аргумента:

```
static void Main()
{
    float d = 24F, r = 52F;
    change(ref d, ref r);
    Console.WriteLine("d = {0}, r = {1}", d, r);
}
```

Результат выполнения:

d = 52, r = 24

Полное обращение (с указанием типизирующего аргумента) к обобщенному методу нашего примера может быть и таким:

```
change<float>(ref d, ref r);
```

18.7. Обобщенные делегаты

Как и у обобщенных методов, у обобщенных делегатов имеется два списка параметров — список типизирующих параметров и список параметров методов, которые может представлять делегат. Но у делегата нет тела, и этим обобщенный делегат похож на прототип обобщенного метода.

В качестве примера приведем обобщенный делегат с одним типизирующим параметром:

```
delegate R DelMin<R> (R x, R y);
```

Типизирующий параметр **R** специфицирует типы параметров и тип возвращаемого значения того метода, который может представлять экземпляр реализации делегата.

В следующем классе **Program** два статических метода, каждый из которых возвращает минимальное из значений своих аргументов и может быть представлен обобщенным делегатом. Метод **minI()** соответствует делегату **DelMin<int>**, а метод **minL()** — делегату **DelMin<long>**.

```
class Program
{
    static int minI(int a, int b)
    {
        return a > b ? b : a;
    }
    static long minL(long a, long b)
    {
        return a > b ? b : a;
    }
    static void Main()
    {
        int first = 5, second = 6, third = 7;
        DelMin<int> dI2 = new DelMin<int>(minI);
        DelMin<long> dL2 = new DelMin<long>(minL);
        Console.WriteLine(dL2(dI2(first, dI2(second, third)), -3L));
    }
}
```

В методе **Main()** объявлены два экземпляра (**dI2** и **dL2**) конкретизаций **DelMin<int>** и **DelMin<long>** обобщенного делегата **DelMin<R>**. Экземпляры делегатов представляют методы **minI()** и **minL()**. В аргументе метода **Console.WriteLine()** выражение с вложенными обращениями к экземплярам делегатов вычисляет минимальное из значений трех переменных типа **int** и одной константы **-3L** типа **long**. Результат, конечно, равен **-3**. Обратите внимание, что возвращаемые значения метода **minI()** имеют тип **int**, а аргументы метода **minL()** должны иметь тип **long**. Соответствующие приведения типов при обращениях к этим методам с помощью делегатов выполняются по умолчанию (что не имеет отношения ни к делегатам, ни к их обобщениям, а связано с автоматическим преобразованием арифметических типов языка **C#**).

В библиотеку стандартных классов платформы **.NET Framework** включено семейство обобщенных типов делегатов, наиболее важными из которых для конечного пользователя являются **Func<>** и **Action<>**. Их назначение — обеспечить программиста обобщенными делегатами, которые можно применять в программах без предварительного явного объявления. Каждый из названных обобщенных делегатов перегружен, так что в пространстве имен **System** есть обобщенные типы делегатов с разным количеством типизирующих параметров. Для обобщенных делегатов **Func<>** можно использовать от 1 до 17 типизирующих параметров. В каждом из этих семнадцати обобщенных типов последний

из типизирующих параметров определяет тип возвращаемого значения тех методов, которые представляет соответствующий делегат. Объявления первых трех обобщенных типов делегатов `Func` из библиотеки `.NET` имеют такой вид:

```
delegate TResult Func<TResult> ();  
delegate TResult Func<T1, TResult> (T1 t1);  
delegate TResult Func<T1, T2, TResult> (T1 t1, T2 t2);  
...
```

Реализации первого из перечисленных обобщенных типов делегатов предназначены для представления методов без параметров, возвращающих значение типа, определяемого единственным типизирующим аргументом конкретного делегата. Например, метод с заголовком `double method()` может быть представлен экземпляром делегата `Func<double>`. Метод с заголовком `int count(int[] arr)` может быть представлен экземпляром делегата `Func<int[], int>` и т. д.

В отличие от делегатов `Func<>` делегаты `Action<>` предназначены для представления методов, не имеющих возвращаемого значения, т. е. методов, для которых в качестве типа возвращаемого результата указывается `void`. Таких обобщенных типов делегатов в библиотеке шестнадцать с количеством типизирующих параметров от 1 до 16:

```
delegate void Action<T1> (T1 t1);  
delegate void Action<T1, T2> (T1 t1, T2 t2);  
delegate void Action<T1, T2, T3> (T1 t1, T2 t2, T3 t3);  
...
```

Например, метод с заголовком `void printArray(float [] arr)` может быть представлен экземпляром делегата `Action<float []>`.

Зная о существовании этих двух семейств обобщенных типов делегатов и о том, что они известны в пространстве имен `System`, программист в ряде случаев может не включать в код собственное объявление типа делегата. Например, если в программе нужен тип

```
delegate int Processor (string line, char ch)
```

экземпляры которого могут представлять методы с параметрами типов `string` и `char`, возвращающие значение типа `int`, можно использовать `Func<string, char, int>`.

Следующая программа иллюстрирует применение библиотечных обобщенных типов делегатов `Func<>` и `Action<>`. Необходимо написать метод для формирования массива из заданного количества значений членов ряда Фибоначчи и обобщенный метод для вывода в консольное окно значений элементов одномерного массива. В основной программе объявить два экземпляра делегатов (`Func<>` и `Action<>`) для представления указанных методов и, используя эти экземпляры делегатов, вывести значения первых семи членов ряда Фибоначчи:

```
// Обобщенные делегаты .NET и обобщенный метод  
using System;
```

```

class Program
{
    static void printArray<T>(T[] ar, string format)
    {
        foreach (T z in ar)
            Console.Write(format, z);
        Console.WriteLine();
    } // printArray<T>()
    static int[] fib(int numb)
    {
        int[] ar = new int[numb];
        ar[0] = ar[1] = 1;
        for (int k = 2; k < numb; k++)
            ar[k] = ar[k - 1] + ar[k - 2];
        return ar;
    }
    static void Main()
    {
        Action<int[], string> act =
            new Action<int[], string>(printArray);
        Func<int, int[]> result = new Func<int, int[]>(fib);
        act(fib(7), "{0} ");
    }
}

```

Результаты выполнения программы:

1 1 2 3 5 8 13

Обобщенный метод `printArray<T>()` для вывода значений элементов массива мы уже приводили. В этой программе он немного изменен — исключено умалчиваемое значение второго параметра, а в теле метода для перебора элементов массива применен цикл **foreach**. Для представления такого метода можно использовать экземпляр реализации обобщенного делегата `Action<T1>`. Обратите внимание, что второй параметр обобщенного метода `printArray<T>()` не типизирован и имеет фиксированный тип **string**.

Метод для создания массива со значениями членов ряда Фибоначчи `fib(int numb)` особенностей не имеет. Параметр определяет число членов ряда, возвращаемый результат — ссылка на создаваемый массив. Для представления такого метода можно использовать экземпляр реализации обобщенного делегата `Func<T1, TResult>`.

В основной программе (в методе `Main`) ссылка `Action<int [], string> act` связана с экземпляром делегата, представляющим конкретизацию метода `printArray()`. Первый типизирующий аргумент `int []` обобщенного делегата `Action< >` определяет тип первого параметра метода

```
void printArray<T>(T[] ar, string format).
```

а второй типизирующий аргумент соответствует фиксированному типу второго параметра метода. Ссылка `Func<int, int[]> result` связана с экземпляром делегата, представляющим метод `int[] fib(int numb)`. После объявления указанных ссылок на экземпляры делегатов выраже-

ние `act(fib(7), "{0} ")` обеспечивает обращение к конкретизации метода `printArray<T>()`, который в свою очередь вызывает метод `fib(int numb)`.

Контрольные вопросы и задания

1. Назовите языки программирования, в которых тем или иным способом реализован механизм обобщений.
2. Какие типы языка C# могут объявляться как обобщенные?
3. Какие члены типов языка C# могут быть обобщенными?
4. Назовите обязательные элементы декларации (объявления) обобщенного класса.
5. Что такое типизирующий параметр?
6. Каково назначение типизирующих аргументов в обозначении специализированного типа?
7. Что такое сконструированный тип?
8. В чем отличия сконструированного типа от регулярного типа?
9. Назовите требования к именам типизирующих параметров.
10. Объясните назначение раздела ограничений типизирующих параметров в декларации обобщенного типа и обобщенного метода.
11. Какой формат имеет элемент списка ограничений типизирующих параметров?
12. Назовите виды ограничений типизирующего параметра.
13. Что такое свободные типизирующие параметры?
14. Перечислите требования к свободным типизирующим параметрам.
15. Когда применяется ограничение с помощью явного указания типа?
16. Назовите ранги и правила использования разных форм ограничений типизирующих параметров.
17. Объясните правила перегрузки обобщенных типов.
18. Почему обобщенный тип называют открытым типом, а сконструированный — закрытым?
19. Назовите особенности статических членов обобщенных типов.
20. Что определяют типизирующие параметры обобщенного интерфейса?
21. Как реализуется обобщенный интерфейс регулярным типом и обобщенным типом?
22. Как реализуется регулярным типом несколько сконструированных интерфейсов?
23. Назовите типы, не допускающие обобщений.
24. Приведите формат объявления обобщенного метода.
25. Что такое инстанцирование обобщенного метода?
26. Перечислите элементы объявления обобщенного типа делегата.
27. Что определяют типизирующие параметры обобщенного типа делегата?
28. Назовите имена обобщенных типов делегатов, входящих в библиотеку классов .NET.

Предметный указатель

- А**
автореализуемые свойства 211
аксессор 207
аксессор доступа 208
аксессор изменения 208
анонимный метод 321
аргумент 142
 именованный 147
 объявляемый «на лету» 146
 передача по значению 154
 подстановка "out_" 147
 приведение типов аргументов 151
аргумент деконструктора 222
 подстановка "_" 223
- Б**
база класса 339
базовые типы значений 30
библиотека классов FCL 60
блок
 завершения обработки исключения 298
 контроля за исключениями 297
 операторов 73, 188
- В**
выражение
 инициализирующее 30, 113, 271
 праводопустимое 115
 лямбда 322
- Г**
глубина рекурсии 157
- Д**
декларация
 аксессора доступа 207
 аксессора изменения 207
 свойства 207
 декларация обобщенного класса 339
 деконструктор 220, 222, 223
 деконструкция кортежа 164
 декремент 38, 274
 делегат 310
 вызов делегата 310
 массив делегатов 315
 определение типа делегата 310
 тип делегата 310
 экземпляр делегата 311
 делегаты пространства System
 Action 356
 Func 355
 деструктор 201
 имя деструктора 201
 десятичная переменная 52
 диаграмма
 агрегации классов 229
 вложения классов 231
 композиции классов 229
 наследования классов 235
 динамическое связывание 244, 262
 директива
 using 18
 using static 19, 278
 директива импорта
 пространства имен 18
 типа 18, 19
 дополнительный код 43
- З**
захваченная переменная 327
защищенные члены 235
- И**
идентификатор 14, 31
иерархия интерфейсов 264
имена элементов кортежа 163, 164

- именованные аргументы 147
- именованные константы 32
- импорт конкретного класса 21
- имя интерфейса 252
- индексатор 214
 - вызов индексатора 215
 - модификаторы индексатора 214
 - объявление индексатора 214
 - сжатие до выражение 218
 - тело индексатора 215
- индексирующее выражение 90
- инициализатор 93
 - автореализуемого свойства 212
 - конструктора 196, 239
 - объекта 202, 203
 - переменной 184
 - поля 171
 - свойства 212
 - экземпляра структуры 284
- инкапсуляция 205
 - принцип инкапсуляции 205
- инкремент 38, 274
- инстанцирование 310
- интерполируемая строка 123
- интерфейс 251, 252
 - Comparable 290
 - иерархия интерфейсов 267
 - как тип 260
 - наследование интерфейсов 264
 - неявная реализация интерфейса 259
 - реализация 288
 - спецификация базы 264
- исключения 294
 - System.FormatException 299
 - System.NullReferenceException 304
 - генерация исключения 295, 303
 - обработка исключения 295

К

- квалифицированное имя 169, 272
- класс 13, 14
 - Data.Time 332
 - Enum 276
 - object 24, 26
 - string 112
 - System.Delegate 310, 316
 - System.Exception 286, 308
 - System.Object 279
 - System.SystemException 308
 - System.Enum 272
 - Thread 330
 - абстрактный 245
 - агрегация классов 228
 - базовый 232
 - вложение классов 230
 - внутренний 232
 - генерации событий 333
 - множественное наследование 254
 - модификаторы класса 182
 - наследование классов 232
 - обработки событий 333
 - определение класса 182
 - отношения наследования классов 235
 - производный 232
 - спецификация базы 254
 - статические члены класса 168
 - статический 179
 - тело класса 168, 183
 - член класса 183
- композиция классов 227
- конкатенация
 - строк 17
 - строки и символа 56
- константа 173
 - именованная 32
 - объявление статической константы 173
 - перечисления 272
- конструктор 64, 113
 - базового класса 233, 239
 - делегата-типа 312
 - имя конструктора 195
 - инициализатор конструктора 195, 239
 - исключений 303
 - класса 178
 - конструктор объектов 195
 - копирования 196, 197, 200
 - общего вида 196, 199
 - приведения типов 196, 199
 - статический 177
 - структуры 279
 - тело конструктора 195
 - умолчания 190, 196, 200, 280

конструкция try/catch/finally 297
копирование
 поверхностное 108
 поразрядное 108
кортеж 162, 163
куча
 управляемая 26

Л

литерал 32
 буквальный строковый 112
 интерполируемой строки 113
 регулярный строковый 112
логические выражения
 булевы выражения 52
локальный метод 137, 139, 175
лямбда-выражение 322, 327
лямбда-операция 36

М

массив
 значения элементов по умолчанию 93
 копия массива 101
 массивов 104, 107
 массив-список 286
 непрямоугольный 104
 объект-массив 92
 одномерный 90
 размерность массива 101, 102
 ранг массива 103
 с ненулевым начальным индексом 91
 ссылка на массив 91
 ссылок на массивы 106
 тип массива 102
массивы
 многомерные 102
машинный нуль 51, 79
метка 72
 метка переключателя 87
 default 84
 шаблон типа 87
метод 16, 61
 Add() 286
 Array.Sort() 160, 290, 324
 Combine() 316
 CompareTo() 290

Format() 119, 276
get() 206
GetName() 276
GetNames() 277
GetUnderlyingType() 276
GetValues() 277
IsDefined() 277
Parse() 65
Remove() 316
set() 206
Sleep() 330
Sort() 160
ToString() 114
TryParse() 67
абстрактный 245
анонимный 321
виртуальный 241
заголовок метода 136
класса 135
класса Array 96
косвенно рекурсивный 156
локальный 175
метод-процедура 136
метод-функция 136
модификаторы метода 138, 187
нестатический 64, 187
обобщенный 351
объекта 135
опечатанный 247
определение метода 186
перегрузка методов 155
прототип метода 251
расширяющий 219
рекурсивный 156
сжатый до выражения 140, 141, 147
сигнатура метода 156
статический 64, 175
тело метода 136
экранирование методов 239
минимальная функциональность 61
модификатор 252
 extern 187
 new 237, 240, 253, 268
 out 67, 145
 override 242
 protected 235
 public 169

- ref 166
- ref 145
- sealed 247
- this 219
- virtual 242
- интерфейса 252
- перечислений 271
- модификаторы доступа 169
- Н**
- наследование
 - интерфейсов 267
 - множественное 267
 - реализации 251
 - функциональности 251
- О**
- обобщения 338
 - список ограничений 343
- обобщенные
 - вложенные типы 348
 - делегаты 338, 354
 - интерфейсы 349
 - классы 338
 - методы 338, 351
 - структуры 346
- обработчик исключений 298
- обратный вызов 318
 - методы обратного вызова 318
- общая система типов CTS 60
- общезыковая исполняющая среда
 - CLR 60
- объект 12
- объявление
 - делегата-типа 310
 - интерфейса 253
 - обобщенного интерфейса 338
 - обобщенного класса 338
 - обобщенной структуры 338
 - объекта класса 63
- ограничения типизирующих параметров 344
- оператор 71
 - break 80
 - continue 82
 - throw 303
 - безусловного перехода 72, 81
 - ветвлений 73

- встроенный 71
- выражение-оператор 71
- классификация операторов 71
- операторы цикла 74
- посылки сообщения 329
- присваивания 37
- пустой 72
- условный 73
- цикла foreach 95
- операции
 - is 288
 - автоизменений 274
 - арифметические бинарные 35
 - базовые (первичные) 34
 - индексирования 115
 - конкатенации 116
 - логические 52
 - логические поразрядные 36
 - отношений 53
 - перегрузка операций 155
 - поразрядные 36, 40
 - поразрядных сдвигов 35, 42
 - преобразования типов 46
 - приведения типов 46
 - присваивания 37
 - присваивания для строк 115
 - составные операции присваивания 38
 - сравнения на равенство 116
 - тернарная 36, 57
 - унарные 35
 - унарные операции ++ и -- 56
- опечатанное свойство 247
- опечатанный класс 247
- опечатанный метод 247
- особые ситуации 294
- отношение 53
- ошибки
 - логические 294
 - семантические 294
 - синтаксические ошибки 294
- П**
- параметр
 - виды параметров 137
 - выходной 137, 145
 - индексатора 216
 - передаваемый по значению 137, 142

- передаваемый по ссылке 137, 145
- с модификатором `params` 138, 152
- со значением по умолчанию 147
- спецификация параметров 137
- с типом ссылки 148
- параметризация
 - методов 337
 - типов 337
- перегрузка
 - методов 155
 - операций 54
- переключатель 83
 - метка переключателя 83, 84, 87
 - раздел (ветвь) переключателя 84
- переключение
 - по типу 88
- переменная 12
 - захваченная 327
 - инициализатор 184
 - локальная 176
 - объявление на лету 146
 - перечисления 274
 - предельные значения переменных 69
- переполнение 50
- перечисление 271
 - имя перечисления 271
 - объявление перечисления 271
 - список перечисления 271
- платформа .NET Framework 60
- поведение объекта 14
- позднее связывание 262
- поле 61, 170, 183
 - инициализатор полей 171
 - инициализация статических полей 171, 173, 179
 - класса 170
 - модификатор поля `readonly` 170
 - модификатор поля `volatile` 170
 - нестатическое 183
 - объектов 170
 - статическое 170, 183
 - тип поля 183
- полиморфизм 17, 155, 205, 252, 262
- пользовательские классы исключений 307
- поля подстановок 119, 120
- поразрядный сдвиг 44
- потеря значимости 51
- преобразования типов 49
- приведение типов 45
- принцип подстановки 252
- присваивание 37
- проект 19
- пространство имен 18
 - `System.Threading` 330
- протокол
 - декларированный типом делегата 311, 322
 - метода 321
 - представляемый делегатом 311
- прототип 252, 253
 - индексатора 253
 - метода 253
 - свойства 253
 - события 253
- процедура 135
- прямая рекурсия 156

Р

- распаковка 284
- расширяющее преобразование 48
- расширяющий метод 219
- реализация
 - интерфейса 254
 - члена интерфейса 255
- регулярный класс 347
- рекурсия 156
- решение 19

С

- свойство 61, 206
 - `InnerException` 303
 - `Length` 117
 - `Message` 300, 303
 - `Method` 314
 - `Now` 332
 - `Source` 300
 - `Target` 314
- автоматически реализуемое 211
- автореализуемое 211
- имя свойства 207
- объявления свойства 207
- опечатанное 247
- сжатое до выражения 213
- тело свойства 207

- тип свойства 207
- только для чтения 213
- сигнатура 155, 329
- символьное значение 55
- система типов унифицированная
 - CTS 60
- служебное (ключевое) слово
 - base 196
 - catch 298
 - checked 302
 - delegate 311
 - enum 271
 - event 329
 - interface 252
 - static 16
 - struct 279
 - this 196, 214
 - try 298
 - unchecked 302
 - where 343
- служебные (ключевые) слова 31
- событие 61, 329
 - генерация события 335
 - объявление события 329
 - подписка на событие 330
 - публикация события 330
- спецификатор размерности 102
- спецификация
 - CTS 61
 - базы интерфейса 252
 - базы класса 232
- среда исполнения CLR 60
- ссылка
 - this 191
 - копирование ссылок 107
 - с типом интерфейса 260
- статический член
 - обобщенного типа 347
- статическое связывание 262
- стек 26
- строка
 - буквальная 112
 - регулярная 112
 - форматирования 119
- структура 279
 - копирование структуры 281
 - модификатор структур 279

- объявление структуры 279
- члены структуры 279

Т

- таблица истинности 325
- тело
 - интерфейса 252
 - класса 16
 - метода 16
- тип 12
 - char 340
 - long 355
 - string 353
 - базовый 271
 - библиотечный 26
 - вещественный 28, 48
 - вложенный (локализованный) 168
 - времени исполнения 245
 - делегата 311
 - десятичный 28
 - динамический 262
 - закрытый 346
 - значений 24, 279
 - логический 28, 52
 - объявленный 245
 - объявленный тип ссылки 262
 - определенный программистом 26
 - открытый 346
 - пользовательский тип значений 271
 - предопределенный 26
 - символьный 28
 - системный 62
 - сконструированный 340
 - специализированный 340
 - ссылок 24
 - ссылочный пользовательский 271
 - статический 262
 - тип времени исполнения 262
 - целочисленный 28
 - числовой 28
 - явное приведение типов 273
- типизирующие параметры
 - ограничения 339, 341
 - свободные 344, 345
 - список ограничений 343
- типизирующий
 - аргумент 340
 - параметр 341, 354

У

- упаковка 284
 - автоматическая 284
- управляемая куча 26
- условная дизъюнкция 53
- условная конъюнкция 55

Ф

- фильтр исключений 298, 307
- финализатор 202, 281
- форматирование 119
 - в ToString() 122
- функция 135

Ц

- цикл
 - инициализатор цикла 77
 - параметрический 74
 - перебора элементов коллекции 75
 - с постусловием 74, 76
 - с предусловием 74, 75
 - тело цикла 75, 77

Ч

- члены типов CTS 61
- экземпляр
 - делегата 34, 310
 - класса 12
 - структуры 279

Э

- экранирование 237, 267
- эскейп-последовательность 29, 112

Литература и электронные ресурсы

1. Standard ECMA-334. C# Language Specification. 5-th Edition / December 2017, Geneva (ISO/IEC 23270).
2. Бадд Т. Объектно-ориентированное программирование в действии. СПб. : Питер, 1997.
3. Морган М. Java 2. Руководство разработчика. М. : Вильямс, 2000.
4. C# 5.0 и платформа .NET 4.5 для профессионалов / Нейгел К. [и др.]. М. : Вильямс, 2014.
5. Нэш Т. C# 2010: ускоренный курс для профессионалов. М. : Вильямс, 2010.
6. Рихтер Дж. CLR via C#. Программирование на платформе Microsoft .NET Framework 2.0 на языке C#. Мастер класс. 2-е изд. исправ. М. : Русская Редакция; СПб. : Питер, 2008.
7. Скит Дж. C# ; программирование для профессионалов. 2-е изд. М. : Вильямс, 2011.
8. Страуструп Б. Язык программирования C++. 3-е изд. М. : БИНОМ, 1999.
9. Шилдт Г. C# 4.0. Полное руководство. М. : Вильямс, 2013.
10. Подбельский В. В. Язык C#. Решение задач. М. : Финансы и статистика, 2014.
11. Подбельский В. В. Язык C#. Базовый курс. 2-е изд., перераб. и доп. М. : Финансы и статистика, 2013.
12. Solis Daniel, *Illustrated C# 2010*, Apress, 2010.
13. Solis Daniel, Schrottenboer Cal. *Illustrated C# 7*, Apress, 2018.
14. Албахари Дж., Албахари Б. C# 7.0. Карманный справочник. СПб. : Альфа-книга, 2017.
15. Албахари Дж., Албахари Б. C# 6.0. Справочник. Полное описание языка. М. : Вильямс, 2016.
16. Perkins B., Hammer J. V., Reid J. D. *Beginning C# 6 Programming with Visual Studio 2015*, John Wiley & Sons, 2016.
17. Farrell J. *Microsoft Visual C# 2010: An introduction to object-oriented programming, 4-th edition*, 2011.
18. McGee Pat. *C# A Beginner Guide*, McGraw-Hill Education, 2015.
19. Петцольд Ч. Программирование для Microsoft Windows 8. 6-е изд. СПб. : Питер, 2014.
20. Петцольд Ч. Программирование для Microsoft Windows на C#. В 2-х томах. — М. : Русская Редакция, 2002.
21. Троелсен, Э. Язык программирования C# 5.0 и платформа .NET 4.5, 6-е изд.: Пер. с англ. М. : Вильямс, 2013.

22. Troelsen Andrew and Japikse Philip. *C# 6.0 and the .NET 4.6 Framework, Seventh Edition*, Apress, 2015.

23. Предварительная спецификация C# 6.0. URL : <https://docs.microsoft.com/ru-ru/dotnet/csharp/language-reference/language-specification/index>

24. Справочник по C#. URL : <https://docs.microsoft.com/ru-ru/dotnet/csharp/language-reference/index>

Новинки издательства «Юрайт» по дисциплине «Программирование» и смежным дисциплинам

1. Бессмертный, И. А. Системы искусственного интеллекта : учеб. пособие для академического бакалавриата / И. А. Бессмертный. — 2-е изд., испр. и доп. — М. : Издательство Юрайт, 2018.
2. Демин, А. Ю. Информатика. Лабораторный практикум : учеб. пособие для прикладного бакалавриата / А. Ю. Демин, В. А. Дорофеев. — М. : Издательство Юрайт, 2019.
3. Дубров, Д. В. Система построения проектов Stake : учебник для магистратуры / Д. В. Дубров. — М. : Издательство Юрайт, 2019.
4. Жмудь, В. А. Моделирование замкнутых систем автоматического управления : учеб. пособие для академического бакалавриата / В. А. Жмудь. — 2-е изд., испр. и доп. — М. : Издательство Юрайт, 2019.
5. Зимин, В. П. Информатика. Лабораторный практикум в 2 ч. Часть 1 : учеб. пособие для вузов / В. П. Зимин. — М. : Издательство Юрайт, 2019.
6. Зимин, В. П. Информатика. Лабораторный практикум в 2 ч. Часть 2 : учеб. пособие для вузов / В. П. Зимин. — М. : Издательство Юрайт, 2018.
7. Казанский, А. А. Объектно-ориентированный анализ и программирование на Visual Basic 2013 : учебник для прикладного бакалавриата / А. А. Казанский. — М. : Издательство Юрайт, 2019.
8. Казанский, А. А. Прикладное программирование на Excel 2013 : учеб. пособие для прикладного бакалавриата / А. А. Казанский. — М. : Издательство Юрайт, 2019.
9. Казанский, А. А. Программирование на Visual C# 2013 : учеб. пособие для прикладного бакалавриата / А. А. Казанский. — М. : Издательство Юрайт, 2019.
10. Кувшинов, Д. Р. Основы программирования : учеб. пособие для вузов / Д. Р. Кувшинов. — М. : Издательство Юрайт, 2018.
11. Кудрина, Е. В. Основы алгоритмизации и программирования на языке C# : учеб. пособие для бакалавриата и специалитета / Е. В. Кудрина, М. В. Огнева. — М. : Издательство Юрайт, 2019.
12. Лаврищева, Е. М. Программная инженерия. Парадигмы, технологии и CASE-средства : учебник для вузов / Е. М. Лаврищева. — 2-е изд., испр. — М. : Издательство Юрайт, 2018.
13. Лебедев, В. М. Программирование на VBA в MS Excel : учеб. пособие для академического бакалавриата / В. М. Лебедев. — М. : Издательство Юрайт, 2019.

14. *Малявко, А. А.* Формальные языки и компиляторы : учеб. пособие для вузов / А. А. Малявко. — М. : Издательство Юрайт, 2019.
15. *Мамонова, Т. Е.* Информационные технологии. Лабораторный практикум : учеб. пособие для прикладного бакалавриата / Т. Е. Мамонова. — М. : Издательство Юрайт, 2019.
16. *Маркин, А. В.* Программирование на SQL в 2 ч. Часть 2 : учебник и практикум для бакалавриата и магистратуры / А. В. Маркин. — М. : Издательство Юрайт, 2019.
17. Методы оптимизации: теория и алгоритмы : учеб. пособие для академического бакалавриата / А. А. Черняк, Ж. А. Черняк, Ю. М. Метельский, С. А. Богданович. — 2-е изд., испр. и доп. — М. : Издательство Юрайт, 2019.
18. *Нагаева, И. А.* Программирование: Delphi : учеб. пособие для академического бакалавриата / И. А. Нагаева, И. А. Кузнецов. — М. : Издательство Юрайт, 2018.
19. *Огнева, М. В.* Программирование на языке C++ : практический курс : учеб. пособие для бакалавриата и специалитета / М. В. Огнева, Е. В. Кудрина. — М. : Издательство Юрайт, 2018.
20. *Соколова, В. В.* Вычислительная техника и информационные технологии. Разработка мобильных приложений : учеб. пособие для прикладного бакалавриата / В. В. Соколова. — М. : Издательство Юрайт, 2019.
21. *Трофимов, В. В.* Алгоритмизация и программирование : учебник для академического бакалавриата / В. В. Трофимов, Т. А. Павловская ; под ред. В. В. Трофимова. — М. : Издательство Юрайт, 2019.
22. *Тузовский, А. Ф.* Объектно-ориентированное программирование : учеб. пособие для прикладного бакалавриата / А. Ф. Тузовский. — М. : Издательство Юрайт, 2019.
23. *Федоров, Д. Ю.* Программирование на языке высокого уровня python : учеб. пособие для прикладного бакалавриата / Д. Ю. Федоров. — М. : Издательство Юрайт, 2018.
24. *Черпаков, И. В.* Основы программирования : учебник и практикум для прикладного бакалавриата / И. В. Черпаков. — М. : Издательство Юрайт, 2019.